Object Oriented Programming (Using Python)

- Top Programming Languages - 2023
- Objectives
- Outcomes
- Syllabus
- Text and Reference Books
- e-Resources
- Introduction to Python

Dr. KADIYALA RAMANA
Associate Professor, Artificial Intelligence & Data Science
Chaitanya Bharathi Institute of Technology, Hyderabad, Telangana, INDIA
www.cbitt.ac.in

Top Programming Languages:

TIOBE Index for March 2023

TIOBE Index for March 2023

https://www.tiobe.com/tiobe-index/

Mar 2023	Mar 2022	Change	Programming Language	Ratings	Change
1	1		Python	14.83%	+0.57%
2	2		C	14.73%	+1.67%
3	3		Java	13.56%	+2.37%
4	4		C++	13.29%	+4.64%
5	5		C#	7.17%	+1.25%

TIOBE Index for March 2023

https://www.tiobe.com/tiobe-index/

6	6		VB	Visual Basic	4.75%	-1.01%
7	7		JS	JavaScript	2.17%	+0.09%
8	10	↑	SQL	SQL	1.95%	+0.11%
9	8	↓	php	PHP	1.61%	-0.30%
10	13	↑	GO	Go	1.24%	+0.26%
11	9	↓	ASM	Assembly language	1.11%	-0.79%
12	15	↑		MATLAB	1.08%	+0.28%

TIOBE Index for March 2023

https://www.tiobe.com/tiobe-index/

13	12	⌄	Delphi/Object Pascal	1.06%	-0.06%
14	23	⌃⌃	Scratch	1.00%	+0.47%
15	17	⌃	Classic Visual Basic	0.98%	+0.38%
16	11	⌄⌄	R	0.93%	-0.44%
17	30	⌃⌃	Fortran	0.79%	+0.40%
18	16	⌄	Ruby	0.76%	+0.10%
19	26	⌃⌃	Rust	0.73%	+0.22%
20	14	⌄⌄	Swift	0.71%	-0.20%

Object Oriented Programming (Using Python)

COURSE OBJECTIVES:

1. Explore the concepts object-oriented programming like classes, constructors, Polymorphism, Inheritance, and File handling.
2. Prepare student for solving real-world problems using OOPs concepts.

Object Oriented Programming (Using Python)

COURSE OUTCOMES:

1. Understand the concepts of Object-Oriented features.
2. Apply OOPs concepts and different libraries to solve programming problems.
3. Understand the advanced concepts of Python.
4. Develop programs to access databases and web data.
5. Understand APIs and third-party libraries to be used with Python.

Object Oriented Programming (Using Python)

SYLLABUS:

UNIT I

Introduction to Object Oriented Programming Paradigms: Programming paradigms, advantages of OOP, comparison of OOP with Procedural Paradigms.

Classes and Objects: Prototyping, referencing the variables in functions, inline, static and friend functions. Memory allocation for classes and objects, arrays of objects, constructors.

SYLLABUS:

UNIT II

Polymorphism and Inheritance: Overriding methods, type conversions, base classes and derived classes, types of inheritance, various types of classes, invocation of constructors and destructors inheritance, aggregation, composition, classification hierarchies, metaclass/ abstract classes, unit testing and exceptions.

SYLLABUS:

UNIT III

Python Libraries: Basics of Open Source libraries for data pre-processing, modeling and visualization.

UNIT IV

Python to access Web Data: Regular Expressions, extracting data, sockets, using the Developer Console to Explore HTTP, Retrieving Web Page, and Passing Web Pages.

Object Oriented Programming (Using Python)

SYLLABUS:

UNIT V

Using Databases with Python: Using Databases, Single Table CRUD, Designing and representing a data model, reconstructing data with JOIN, many-to-many relationships.

SYLLABUS:

Text Books and References:

1. Allen Downey, Jeff Elkner, Chris Meyers, "How to Think Like a Computer Scientist: Learning with Python", SoHo Books, 2009.
2. R.S. Salaria , "Mastering Object-Oriented Programming", Khanna Book Publishing Co., Delhi
3. Jeeva Jose, "Introduction to Computing & Problem Solving with Python", Khanna Book Publishing, 2019.
4. https://www.coursera.org/specializations/python-3-programming#courses
5. Paul Barry , "Head First Python", O'Reilly, 2010

SYLLABUS:

NPTEL/SWAYAM Course:

1. Python for Data Science, Prof. Raghunathan Rengasamy, IIT Madras
2. The Joy of Computing using Python Prof. Sudarshan, Prof. Yayati Guptaingar, IIT Ropar, IIIT Dharwad.

Object Oriented Programming Lab

Course Objectives:

1. Master the concepts of Object Oriented Programming.
2. Explore the OOPs features of Python and build applications.

Course Outcomes:

1. Demonstrate the features of Object-Oriented Programming.
2. Understand APIs and third-party libraries to be used with Python.
3. Use Python libraries to solve real-world problems.
4. Write scripts to solve data science/machine leaning problems using NumPy and Pandas.
5. Develop applications by accessing web data and databases.

Object Oriented Programming Lab

List of experiments/programs:

1. Write a NumPy program to compute the cross product of two given vectors.
2. Write NumPy program to calculate the QR decomposition of a given matrix.
3. Write a Pandas program to convert a Panda Module Series to Python list and its type.
4. Write a Pandas program to convert a NumPy array to a Pandas series.
5. Create a Python project to get the citation from Google scholar using title and year of publication and volume and pages of journal.
6. Create a Python project to get total COVI-19 cases, total deaths due to Covid-19, total Covid-19 patients recovered in the world.

Object Oriented Programming Lab

Text Book:

1. Reema Thareja, "Python Programming", Oxford Press, 2017.

Online Resources:

1. https://vknight.org/cfm/labsheets/04-object-oriented-programming/
2. http://learning-python.com/class/Workbook/x-exercises.htm
3. https://inst.eecs.berkeley.edu/~cs61a/fa14/lab/lab06/#inheritance
4. https://anandology.com/python-practice-book/object_oriented_programming.html
5. http://stanfordpython.com/
6. https://docs.python.org/3/

Introduction to Python

Introduction to Python

OUTLINE:

- What is Python?
- History and versions of Python
- Features of Python
- Applications for Python
- Who uses Python Today

What is Python?

- **Guido van Rossum** created **Python** in the late **1980s** at the National Research Institute for Mathematics and Computer Science in the **Netherlands**.
- **Python** first version was released in **1991.**
- **Python** began as a competitor to **Java** in the industry, but it quickly grew in popularity.
- **Python** is currently quite popular among both **researchers** and **developers**.
- Today **python** interpreters are available for many operating systems incluidng **Windows, Linux/UNIX, macOS, Other**.

What is Python?

- **Python** is a programming language that lets you work more quickly and integrate your systems more effectively.
- You can learn to use **Python** and see almost immediate gains in productivity and lower maintenance costs.
- **Python** is powerful... and fast;

  plays well with others;

  runs everywhere;

  is friendly & easy to learn;

  is Open.

What is Python?

- **Python** is a high-level, interpreted, interactive and object-oriented scripting language.
- **Python** is designed to be highly readable. It uses English keywords frequently whereas the other languages use punctuations.

What is Python?

- **Python** is Interpreted: **Python** is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to **PERL** and **PHP**.
- **Python** is Interactive: You can actually sit at a **Python** prompt and interact with the interpreter directly to write your programs.
- **Python** is Object-Oriented: **Python** supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python** is a Beginner's Language: **Python** is a great language for the beginnerlevel programmers and supports the development of a wide range of applications from simple text processing to **WWW** browsers to games.

History and versions of Python:

- **Python** is derived from many other languages, including **ABC**, **Modula-3**, **C**, **C++**, **Algol-68**, **SmallTalk**, and **Unix shell** and other **scripting** languages.
- **Python** is copyrighted. Like **Perl**, **Python** source code is now available under the GNU General Public License (GPL).
- **Python** is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

History and versions of Python:

- **Python 1.0** was released in November 1991. **Python 1.5.2** in April 1999 & **Python 1.6.1** in September 2000.
- **Python 2.0.1** was released in june 2001. **Python 2.7.18** is the latest edition of **Python 2** was released on April 20, 2020.
- **Python 3.0.0** was released on Dec. 3, 2008.
- **Python 3.11.2** is the latest edition of **Python 3** was released on February 8th , 2023.

Features of Python:

- Automatic garbage collection
- Automatic memory management
- Beginners programming language
- Built-in tool set
- Component integration/embedded
- Developer activity
- Dynamically typed
- Ease of use
- Extensible
- Free and open source
- GUI Programming
- High level language
- Interactive
- Interpreted
- Large and comprehensive standard library
- Multi-paradigm
- Object oriented
- Platform independent
- Powerful
- Procedure oriented
- Program portability
- Programming becomes pleasure than work
- Ready-made stuff
- Scalable
- Simple and easy to learn
- Software quality
- Support libraries

Applications for Python:

Python is used in many application domains. Here's a sampling.

- Web and Internet Development
- Scientific and Numeric
- Database Access
- Desktop GUIs
- Software & Game Development
- Network Programming
- Education
- Business Applications

For more details visit: https://www.python.org/about/apps/

Who uses Python Today?

Many organizations use **Python** for various purporse. These include:

- Google: In web search systems
- YouTube: Video sharing service
- Bit-torrent: Peer to peer file sharing system
- Intel, HP, Seagate, IBM, Qualcomm: Hardware sharing
- Pixar, Industrial Light and Magic: Movie animation
- JP Morgan chase, USB: Financial market forecasting
- NASA, FermiLab: Scientific programming
- iRobot: Commercial robot vaccum cleaners
- NSA: Cryptographic and intelligence analysis
- IronPort: Email servers
- Drop Box, RospBerry PI, NETFLIX, Yahoo, Honeywell, Philips, United Space Alliance etc.,

Object Oriented Programming (Using Python)

UNIT- I (Part-1)

Introduction to Object Oriented Programming Paradigms:

- Programming paradigms
- Advantages of OOP
- Comparison of OOP with Procedural Paradigms

Dr. KADIYALA RAMANA
Associate Professor, Artificial Intelligence & Data Science
Chaitanya Bharathi Institute of Technology, Hyderabad, Telangana, INDIA
www.cbitt.ac.in

Programming Paradigms

Programming Paradigms

- Paradigm means organization principle, also known as model.
- Programming paradigm/model is a style of building the structure and elements of computer programs.
- The following are some of the programming models:
  1. Imperative Programming Model
  2. Functional Programming Model
  3. Procedural Programming Model
  4. Event-Driven Programming Model
  5. Logic Programming Model
  6. Object-Oriented Programming Model

Imperative Programming Model

- Imperative programming is a programming paradigm where the program describes a sequence of steps to perform a computation.
- This paradigm is based on the concept of statements that change the program's state.
- Examples of imperative programming languages include C, C++, and Java.

Imperative Programming Model

Strengths:

- **Control:** Imperative programming provides a high level of control over the program's behavior, allowing the programmer to precisely specify the steps to perform a computation.
- **Efficiency:** Imperative programming languages are often compiled directly to machine code, which can result in high performance.
- **Flexibility:** Imperative programming allows the programmer to use low-level features, such as direct memory access, to implement performance-critical algorithms.
- **Familiarity:** Imperative programming is a widely-used programming paradigm and is familiar to most programmers.

Imperative Programming Model

Weaknesses:

- **Complexity:** Imperative programming can be complex and difficult to read and understand, especially for large programs with many interacting components.
- **Maintenance:** Imperative programming can be difficult to maintain due to its tight coupling between data and behavior.
- **State management:** Imperative programming requires the programmer to manage the state of the program manually, which can lead to bugs and errors.
- **Lack of abstraction:** Imperative programming can be limited in terms of abstraction, making it difficult to write reusable code and maintain a high level of modularity.

Functional Programming Model

- Functional Programming Model divides a problem into a set of functions. These functions provide the main source of logic in the program.
- Functions take input parameters and produce outputs.
- Examples of functional programming languages include Haskell, Lisp, and Scala.
- **Python** provides functional programming techniques like lambda, map, reduce and filter.
- In this model computation is treated as evaluation of mathematical functions.

Functional Programming Model

- Example: To get factorial value of a number or n^{th} Fibonacci number, we can use the following functions:

factorial (n) = 1	if n == 0
factorial (n) = n * factorial (n-1)	if n > 0
fibo (n) = 0	if n = 0
fibo (n) = 1	if n = 1
fibo (n) = fibo (n-2) + fibo (n-1)	if n > 0

- The output value of a function depends only on its arguments, so calling a function with the same value for an argument always produces the same result. As a result, it is a good fit for parallel execution.

Functional Programming Model

- No function can have any side effects on other variables (state remains unaltered).
- Functional programming model is often called a ‘declarative’ programming paradigm as programming is done with expressions or declarations instead of statements.

Functional Programming Model

Strengths:

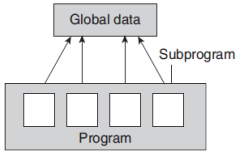
- **Immutability:** Functional programming encourages the use of immutable data structures, which can simplify program logic and make code easier to reason about.
- **Modularity:** Functional programming promotes modularity by allowing functions to be composed and reused across the codebase.
- **Parallelism:** Functional programming can be easily parallelized, allowing for efficient use of multi-core CPUs.
- **Declarative style:** Functional programming uses a declarative style, which focuses on the "what" rather than the "how" of computation. This can lead to more concise and understandable code.

Functional Programming Model

Weaknesses:

- **Complexity:** Functional programming can be complex and difficult to understand, especially for programmers who are used to imperative programming.
- **Performance:** Functional programming can have performance issues due to the need to create and manipulate many small objects.
- **Limited side effects:** Functional programming avoids side effects, which can limit the range of problems that can be solved using this paradigm.
- **Limited I/O:** Functional programming can be limited in terms of I/O capabilities, which can be a disadvantage for some software applications.

Procedural Programming Model



- Procedural programming model solves the problem by implementing one statement (a procedure) at a time. Thus it contains explicit steps that are executed in a specific order.
- It also uses functions, but these are not mathematical functions like the one used in functional programming.
- Functional programming focuses on expressions, where as procedural programming focuses on statements.
- The statements do not have values and instead modify the state of some conceptual machine.
- Examples of procedural programming languages include FORTRAN, Pascal, and C.

Procedural Programming Model

Strengths:

- **Simplicity:** Procedural programming is relatively simple and easy to understand, especially for programmers who are new to the field.
- **Performance:** Procedural programming can have good performance due to the lack of overhead associated with creating and manipulating objects.
- **Flexibility:** Procedural programming allows for flexible program structure and flow control.
- **Debugging:** Procedural programming can be easier to debug than other programming paradigms due to the lack of complex data structures.

Procedural Programming Model

Weaknesses:

- **Limited modularity:** Procedural programming can have limited modularity due to the lack of encapsulation and abstraction.
- **Limited reusability:** Procedural programming can have limited reusability due to the lack of encapsulation and abstraction.
- **Limited scalability:** Procedural programming can have limited scalability due to the lack of encapsulation and abstraction.
- **Limited expressiveness:** Procedural programming can be limited in terms of expressiveness, making it difficult to write complex software.

Event-Driven Programming Model

- Event-driven programming is a programming paradigm where the program responds to events such as user input, mouse clicks, and network requests.
- This paradigm is based on the concept of event handlers that are triggered by events.
- Examples of event-driven programming languages include JavaScript, Python, and Ruby.

Event-Driven Programming Model

Strengths:

- **Responsiveness:** Event-driven programming is highly responsive and can handle events as they occur in real-time.
- **Modular design:** Event-driven programming promotes a modular design, where different components of the program can respond to different events.
- **User-friendly:** Event-driven programming can be user-friendly because it can provide a graphical user interface (GUI) to interact with the program.
- **Scalability:** Event-driven programming can be highly scalable, especially in distributed systems where multiple events can be handled simultaneously.

Event-Driven Programming Model

Weaknesses:

- **Complexity:** Event-driven programming can be complex and difficult to understand, especially for programmers who are new to the paradigm.
- **Debugging:** Event-driven programming can be difficult to debug because of the non-linear flow of the program.
- **Performance:** Event-driven programming can have performance issues due to the need to constantly check for events and respond to them.
- **Difficulty in testing:** Event-driven programming can be difficult to test, especially for applications that require a high level of integration testing.

Logic Programming Model

- Logic programming is a programming paradigm where the program is composed of a set of logical rules and facts.
- In this paradigm, the program is based on the concept of inference rules that manipulate logical expressions.
- Examples of logic programming languages include Prolog, Mercury, and Oz.

Logic Programming Model

Strengths:

- **Declarative programming:** Logic programming is a declarative programming paradigm that allows programmers to focus on what they want to achieve rather than how to achieve it.
- **Natural language representation:** Logic programming languages such as Prolog use a natural language-like syntax that can make the programs easier to read and understand.
- **Non-determinism:** Logic programming can handle non-deterministic problems, where there are multiple possible solutions for a given problem.
- **Symbolic computation:** Logic programming can handle symbolic computation, which is useful for applications such as natural language processing and artificial intelligence.

Logic Programming Model

Weaknesses:

- **Limited applicability:** Logic programming is not well-suited for all types of problems, and may be less efficient than other programming paradigms for some tasks.
- **Limited expressiveness:** Logic programming can be limited in terms of expressiveness, making it difficult to write complex software.
- **Limited debugging:** Logic programming can be difficult to debug because the program execution is not always straightforward and can be influenced by the order in which rules are applied.
- **Limited performance:** Logic programming can have performance issues due to the need to perform extensive search operations to find solutions to problems.

Object-Oriented Programming Model

- This model mimics the real world by creating inside the computer a min-world of objects.
- A program divided into smaller units. These units are called classes and objects.
- In a University system objects can be Vice-Chancellor, Professor, non-teaching staff, students, courses, semesters, examinations, etc.
- Each object has a state (values) and behavior (interface/methods).
- Objects get state and behavior based on the class from which it created.
- Objects interact with one another by sending messages to each other, i.e. by calling each other's interface methods.
- Examples: C++, Java, Apex, Go, Move, Swift,.....etc.

Object-Oriented Programming Model

The striking features of OOP include the following

- The programs are data centered.
- Programs are divided in terms of objects and not procedures.
- Functions that operate on data are tied together with the data.
- Data is hidden and not accessible by external functions.
- New data and functions can be easily added as and when required.
- Follows a bottom-up approach for problem solving.

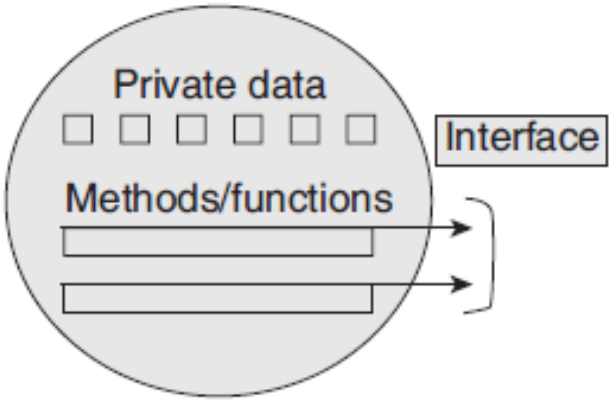
Classes, Objects, and Methods

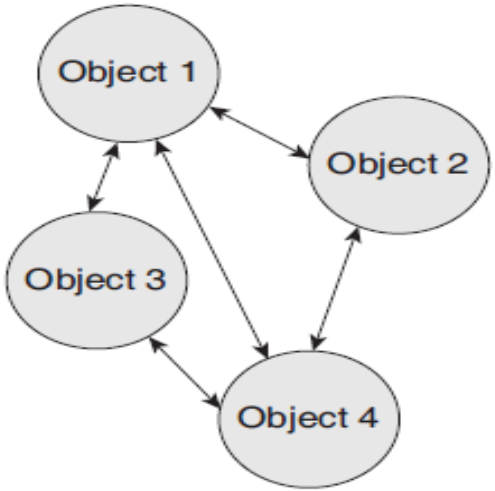
- A class is used to describe something in the world, such as occurrences, things, external entities, and so on.
- A class provides a template or a blueprint that describes the structure and behavior of a set of similar objects.
- Once we have the definition for a class, a specific instance of the class can be easily created.
- A class can have multiple instances or objects.
- Every object contains some data and procedures. They are also called methods.

Classes, Objects, and Methods

- A method is a function associated with a class. It defines the operations that the object can execute when it receives a message.
- In OOP language, only methods of the class can access and manipulate the data stored in an instance of the class (or object).
- Two objects can communicate with each other through messages. An object asks another object to invoke one of its methods by sending it a message.

Classes, Objects, and Methods





Object of a program interact by sending messages to each other

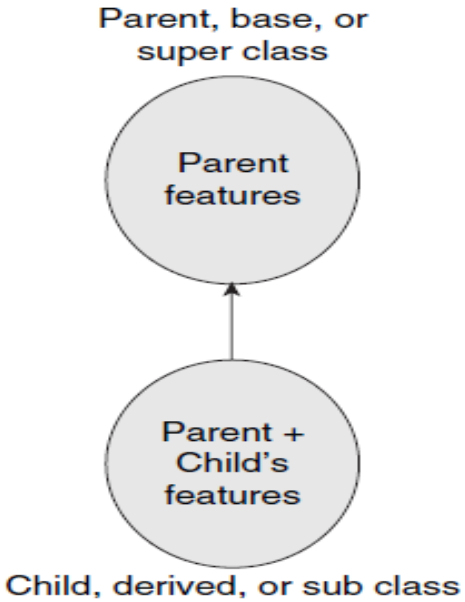
Inheritance

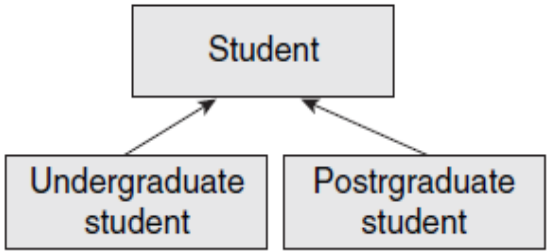
- Inheritance is a concept of OOP in which a new class is created from an existing class. The new class, often known as a sub-class, contains the attributes and methods of the parent class.
- The new class, known as sub-class or derived class, *inherits* the attributes and behavior of the pre-existing class, which is referred to as super-class or parent class.
- The inheritance relationship of sub- and super classes generates a hierarchy.
- Therefore, inheritance relation is also called 'is-a' relation.

Inheritance

- A sub-class not only has all the states and behaviors associated with the super-class but has other specialized features (additional data or methods) as well.
- The main advantage of inheritance is the ability to reuse the code.
- When we want a specialized class, we *do not have* to write the entire code for that class from scratch. We can inherit a class from a general class and add the specialized code for the sub-class.

Inheritance





Polymorphism, Containership and Reusability

- Polymorphism refers to having several different forms. It is related to methods.
- Polymorphism is a concept that enables the programmers to assign a different meaning or usage to a method in different contexts.
- Polymorphism can also be applied to operators.
- For example, we know that operators can be applied only on basic data types that the programming language supports.
- Therefore, `a + b` will give the result of adding `a` and `b`. If `a = 2` and `b = 3`, then `a + b = 5`. When we overload the `+` operator to be used with strings, then `Fraction1 + Fraction2` adds two fractional numbers and returns the result.

Polymorphism, Containership and Reusability

- Containership is the ability of a class to contain object(s) of one or more classes as member data.
- For example, class One can have an object of class Two as its data member. This would allow the object of class One to call the public functions of class Two. Here, class One becomes the container, whereas class Two becomes the contained class.
- Containership is also called composition.

Polymorphism, Containership and Reusability

- Reusability means developing codes that can be reused either in the same program or in different programs.
- Python gives due importance to building programs that are reusable.
- Reusability is attained through inheritance, containership, and polymorphism.

Data Abstraction and Encapsulation

- Data abstraction refers to the process by which data and functions are defined in such a way that only essential details are revealed and the implementation details are hidden.
- The main focus of data abstraction is to separate the interface and the implementation of a program.

Data Abstraction and Encapsulation

- Data encapsulation, also called data hiding, is the technique of packing data and functions into a single component (class) to hide implementation details of a class from the users.
- Users are allowed to execute only a restricted set of operations (class methods) on the data members of the class.
- Therefore, encapsulation organizes the data and methods into a structure that prevents data access by any function (or method) that is not specified in the class. This ensures the integrity of the data contained in the object.

Merits/Advantages of OOP Languages

Merits/Advantages of OOP Languages

- Elimination of redundant code through inheritance (by extending existing classes).
- Higher productivity and reduced development time due to reusability of the existing modules.
- Secure programs as data cannot be modified or accessed by any code outside the class.
- Real world objects in the problem domain can be easily mapped objects in the program.
- A program can be easily divided into parts based on objects.

Merits/Advantages of OOP Languages

- The data-centered design approach captures more details of a model in a form that can be easily implemented.
- Programs designed using OOP are expandable as they can be easily upgraded from small to large systems.
- Message passing between objects simplifies the interface descriptions with external systems.
- Software complexity becomes easily manageable.
- With polymorphism, behavior of functions, operators, or objects may vary depending upon the circumstances.

Merits/Advantages of OOP Languages

- Data abstraction and encapsulation hides implementation details from the external world.
- OOP enables programmers to write easily extendable and maintainable programs.
- OOP supports code reusability to a great extent.

Demerits/Disadvantages of OOP Languages

- Programs written using object oriented languages have greater processing overhead as they demand more resources.
- Requires more skills to learn and implement the concepts.
- Beneficial only for large and complicated programs.
- Even an easy to use software when developed using OOP is hard to be build.
- OOP cannot work with existing systems.
- Programmers must have a good command in software engineering and programming methodology.

Applications of OOP Languages

- Designing user interfaces such as work screens, menus, windows, and so on.
- Real-time systems
- Simulation and modelling
- Compiler design
- Client server system
- Object oriented databases
- Object oriented distributed database

Applications of OOP Languages

- Parallel programming
- Decision control systems
- Office automation systems
- Hypertext and hypermedia
- Computer-aided design (CAD) systems
- Computer-aided manufacturing (CAM) systems
- Computer animation

Applications of OOP Languages

- Developing computer games
- Artificial intelligence—expert systems and neural networks
- Networks for programming routers, firewalls, and other devices

Comparison of OOP with Procedural Paradigms

Comparison of OOP with Procedural Paradigms

Object-Oriented Programming (OOP)	Procedural-Oriented Programming (Pop)
Objects	Functions/Procedures
It is a bottom-up approach	It is a top-down approach
Program is divided into objects	Program is divided into functions
Makes use of Access modifiers ‘public’, private’, protected’	Doesn’t use Access modifiers
It is more secure	It is less secure
Object can move freely within member functions	Data can move freely from function to function within programs
It supports inheritance	It does not support inheritance
Modularity	Modular design through functions

OBJECT ORIENTED PROGRAMMING USING PYTHON

CLASSES & OBJECTS

OUTLINE

- Introduction
- Classes and Objects
- init method
- Constructors and Destructors

Classes and Objects: Introduction

- In all our programs till now, we have been using the procedure-oriented technique in which our programs written using functions or blocks of statements which manipulate data.
- However, another a better style of programming is called **object-oriented programming** in which data and functions are combined to form a **class**.
- Compared with other programming languages, **Python** has a very short and simple way to define and use **classes**.

Classes and Objects: Introduction

- The **class** mechanism supported by **Python** is actually a mixture of that found in **C++** and **Modula-3**.
- **Python** supports all the standard features of **Object-Oriented Programming**.

CLASSES AND OBJECTS

- **Classes** and **objects** are the two main aspects of **object oriented programming**.
- In fact, a **class** is the basic building block of **Python**.
- A **class** creates a new type and object is an instance(or variable) of the **class**.
- **Classes** provides a blueprint or a template using which objects are created.
- In fact, in **Python**, *everything is an object or an instance of some **class**.*

CLASSES AND OBJECTS

- For example, all integer variables that we define in our programs are actually instances of class **int**.
- Similarly, all string variables are objects of class **string**.
- We can find out the type of any object using the **type()** function.
- The **Python Standard Library** is based on the concept of classes and objects.

CLASSES AND OBJECTS

1.Defining Classes:

- A **class** is a user-defined blueprint or prototype from which objects are created.
- **Classes** provide a means of bundling data and functionality together.
- Creating a new class creates a new type of object.
- **Classes** are created by keyword **class**
- Attributes are the variables that belong to **class**.
- Attributes are always **public** and can be accessed using **dot (.)** operator.

```
Eg.: Myclass.Myattribute
```

CLASSES AND OBJECTS

1.Defining Classes:

A collection of variables (data members) and functions (methods) is called a **class**.

Syntax:

```
class class_name:
    list of data members or class variables
    list of methods or class methods
```

CLASSES AND OBJECTS

1.Defining Classes:

Example-1:

```
# To create a class with only variables
class test:
    a=10
    b=20
print(test.a+test.b)
```

OUTPUT:

```
30
```

CLASSES AND OBJECTS

1.Defining Classes:

Example-2:

```
# To create a class with only methods
class wish:
    def mng():
        print('good morning')
    def aft():
        print('good afternoon')
    def evng():
        print('good evening')
    def ngt():
        print('good night')
```

CLASSES AND OBJECTS

1.Defining Classes:

Example-2:

```
wish.mng()
wish.aft()
wish.evng()
wish.ngt()
```

OUTPUT:

```
good morning
good afternoon
good evening
good night
```

CLASSES AND OBJECTS

2. Creating Objects:

- Once a **class** is defined, next job is to create an object (or instances) of that class.
- The object can then access class variables and class methods using the dot operator(**.**).

The syntax to create an object is given as:

```
obect_name = class_name()
```

The syntax for accessing a class member through the class object is:

```
obect_name.class_member_name
```

CLASSES AND OBJECTS

2. Creating Objects:

Example: Program to access class variable using class object

```
class test:
    var = 10 # class variable
obj = test()
print(obj.var) # class variable is accessed
using class object
```

OUTPUT:

```
10
```

CLASSES AND OBJECTS

3. Data Abstraction and Hiding through Classes:

- **Data abstraction** refers to the process by which data and functions are defined in such a way that only essential details are provided to the outside world and the implementation details are **hidden**.
- In **Python** and other object oriented programming languages, **classes** provide methods to the outside world to provide the functionality of the object or to manipulate the object's data.

CLASSES AND OBJECTS

3. Data Abstraction and Hiding through Classes:

- Any entity outside the world does not know about the implementation details of the **class** or **class method**.
- **Data Encapsulation**, also called **Data Hiding**, organizes the data and methods into a structure that prevents data access by any function (or method) that is not specified in the class. This ensures the integrity of the data contained in the object.

CLASSES AND OBJECTS

3. Data Abstraction and Hiding through Classes:

- **Encapsulation** defines different access levels for data variables and member functions of the **class**. These access levels specifies the access rights.

Public access level:

- In **python**, by default, every member of class is **public**.
- Public members are available publicly. Means can be accessed from any where.
- These attributes we can access from any where either inside the class or from outside of the class.

CLASSES AND OBJECTS

3. Data Abstraction and Hiding through Classes:

Private access level:

- Private attributes can be accessed only within the class i.e. private attributes cannot access outside of the class.
- By convention, we can declare member as **private** type, we write **double underscore**(__) symbol before the identifiers.

CLASSES AND OBJECTS

3. Data Abstraction and Hiding through Classes:

Protected access level:

- Protected attributes can be accessed within the class anywhere but from outside of the class only in child classes.
- By convention, we can declare member as **protected** type, we write **single underscore**(_) symbol before the identifiers.

These attributes should only be used under certain conditions.

The __init()__ method (The Class Constructor)

- The **__init()__** function or method is a built-in function. and has a special significance in **Python classes**.
- The **__init()__** method is automatically executed when an object of a class is created.
- No need of explicit calling.
- The main purpose the **__init()__** method is to initialize the variables of class object.
- The **__init()__** method is prefixed as well as suffixed by **double underscore (__)**.

The __init()__ method (The Class Constructor)

Example-1: Program illustrating the use of __init()__ method

```
class test:
    def __init__(self,val):
        print("In class method!")
        self.val = val
        print("The value is:", val)
obj = test(20)
```

OUTPUT:

```
In class method!
The value is: 20
```

The __init()__ method (The Class Constructor)

Example-2: Program illustrating the use of __init()__ method

```
class person:
    def __init__(self,name):
        print("In class method!")
        self.name = name
    def display(self):
        print("Hello", self.name)
        return
p1 = person('Ram')
p1.display()
```

The __init()__ method (The Class Constructor)

Example-2: Program illustrating the use of __init()__ method

OUTPUT:

```
In class method!
The value is: 20
```

Constructors

- A **constructor** is a class function that instantiates an object to predefined values.
- A **constructor** is a special method. It begins with a **double underscore** (_ _).
- In **Python** constructors are implemented by using the **__init__()**
- The **__init__()** method is called the constructor and is always called when creating an object.

Constructors

Constructors are 2 types:

1. Default constructor
2. Parameterized constructor

Default constructor:

To create an object without parameters is called a **default constructor**.

Constructors

Default constructor: Example-1

```
# Program illustrating Default constructor
class test:
    def __init__(self):
        print('I am test class init
function..')
t1=test()
```

OUTPUT:

I am test class init function..

Constructors

Default constructor: Example-2

```
# Write a program to count no. of objects created for a class
class test:
    count = 0
    def __init__(self):
        test.count = test.count + 1
    def display():
        print('No. of objects = ',test.count)
t1=test()
t2=test()
t3=test()
t4=test()
t5=test()
t6=test()
test.display()
```

Constructors

Default constructor: Example-2

 # Write a program to count no. of objects created for a class

OUTPUT:

```
No. of objects =  6
```

Constructors

Parameterized constructor:

To create an object with parameters is called a **parameterized constructor**.

Constructors

Parameterized constructor: Example-1

```
# Write a program to illustrate parameterized constructor
class users:
    name = ""
    def __init__(self, name):
        self.name = name
    def sayHello(self):
        print("Welcome to Python Developers, " +
self.name)
user1 = users("RAM")
user1.sayHello()
```

Constructors

Parameterized constructor: Example-1

 # Write a program to illustrate parameterized constructor

OUTPUT:

```
Welcome to Python Developers, RAM
```

Constructors

Parameterized constructor: Example-2

```
class car:
    def __init__(self,x,y,z):
        self.color = x
        self.companyname = y
        self.model = z
    def display(self):
        print('Car color: ',self.color)
        print('Car company name: ',self.companyname)
        print('Car model name: ',self.model)
c1 = car('Red','KIA','SELTOS')
c1.display()
```

Constructors

Parameterized constructor: Example-2

OUTPUT:

```
Car color:  Red
Car company name:  KIA
Car model name:  SELTOS
```

Destructors

- **Destructors** are called when an object gets destroyed.
- In **Python**, destructors are not needed as much needed, in **C++** or **Java** , because **Python** has a garbage collector that handles memory management automatically.
- The **__del__()** method is a known as a **destructor** method in **Python**.

Destructors

Syntax of destructor declaration:

```
def __del__(self):
    body of destructor
```

Destructors

```
# Program to demonstrate destructor
class employee:
    def __init__(self):
        print('Employee created..')
    def __del__(self):
        print('Destructor called,Employee  deleted..')
e1 = employee()
del e1
```

OUTPUT:

```
Employee created..
Destructor called,Employee  deleted..
```

UNIT- I (Part-2) cont'd.

Classes and Objects:

- Prototyping
- Referencing the variables in functions
- Inline, static and friend functions
- Memory allocation for classes and objects
- Arrays of objects

Dr. KADIYALA RAMANA
Associate Professor, Artificial Intelligence & Data Science
Chaitanya Bharathi Institute of Technology, Hyderabad, Telangana, INDIA
www.cbitt.ac.in

Prototyping

- Prototyping in Python classes involves creating an initial version of a class that can be used to test and refine its design before finalizing it.
- This approach can help developers identify potential issues and make necessary changes in a timely manner, improving the overall quality of the code.

Prototyping

Here is an example of how to prototype a Python class:

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.speed = 0
    def accelerate(self):
        self.speed += 10
    def brake(self):
        if self.speed >= 10:
            self.speed -= 10
```

Prototyping

Example of how to prototype a Python class: cont'd.

```
    def current_speed(self):
        return self.speed
car1 = Car("TATA", "Nexon", 2023)
# Prototype testing
print(car1.make)  # Output: TATA
print(car1.model)  # Output: Nexon
print(car1.year)  # Output: 2023
car1.accelerate()
print(car1.current_speed())  # Output: 10
car1.brake()
print(car1.current_speed())  # Output: 0
```

Prototyping

Example of how to prototype a Python class: cont'd.

```
Output:
TATA
Nexon
2023
10
0
```

Prototyping

- In this example, we define a '**Car**' class with some basic attributes and methods for acceleration, braking, and reporting the current speed.
- We create an instance of the class (**'car1'**) and use it to test the prototype by calling its methods and attributes.
- Once we have tested and refined the prototype, we can finalize the class and use it in our application.

Referencing the variables in functions

- To reference variables in functions of classes in Python, you need to use the **self** keyword.
- The **self** keyword is used to reference the current instance of the class, allowing you to access and modify its variables and methods.

Referencing the variables in functions

Example of how to reference variables in functions of a class in Python

```
class MyClass:
    def __init__(self, my_var):
        self.my_var = my_var
    def my_function(self):
        print(self.my_var)
my_object = MyClass("Hello World!")
my_object.my_function()
```

Output:

```
Hello World!
```

Referencing the variables in functions

- In this example, we define a **MyClass** class with a constructor that takes a parameter **my_var**. We set **my_var** to an instance variable of the class using **self.my_var = my_var**.
- We also define a function **my_function** that uses **self.my_var** to access the instance variable **my_var** and print its value.
- Finally, we create an instance of the class **my_object** and call its **my_function** method, which prints the value of **my_var** ("Hello World!").

Referencing the variables in functions

- Note that you must use '**self**' to reference instance variables in any function within a class. If you do not use '**self**', Python will look for a local variable with that name and raise an error if it does not exist.

Inline, static and friend functions

- In Python, there are no inline, static or friend functions in classes like there are in some other programming languages like C++.
- Inline functions are used in C++ to optimize the performance of code by copying the function's code to the location where it is called instead of executing a function call.
- Python does not have inline functions because it uses dynamic typing, which means that the function's code needs to be evaluated at runtime, making inline functions unnecessary.

Inline, static and friend functions

- Static functions in C++ are used to group functions together in a class that do not need to access any of the class's instance variables or methods.
- In Python, functions can be defined outside of a class and do not need to be part of a class to be called.
- Friend functions in C++ are used to grant access to private members of a class to a function that is not a member of the class.
- Python does not have the concept of private members like C++, so there is no need for friend functions.

Inline, static and friend functions

- However, in Python, you can define methods and properties within a class using the **@staticmethod** and **@property** decorators, respectively, to achieve similar functionality.
- The **@staticmethod** decorator allows you to define a method that does not access any instance variables or methods and can be called directly on the class itself.

Inline, static and friend functions

Here is an example:

```
class MyClass:
    @staticmethod
    def my_static_method():
        print("This is a static method")


MyClass.my_static_method()
```

Output:

```
This is a static method
```

Inline, static and friend functions

The @property decorator allows you to define a method that is accessed like an attribute instead of a method call. Here is an example:

```
class MyClass:
    def __init__(self):
        self._my_var = "Hello World!"
    @property
    def my_var(self):
        return self._my_var
my_object = MyClass()
print(my_object.my_var)  # Output: Hello World!
```

Inline, static and friend functions

In this example, we define a class **MyClass** with a private instance variable **_my_var**. We define a method **my_var** with the **@property** decorator that returns the value of **_my_var**.

When we create an instance of the class **my_object** and call **my_object.my_var**, we get the value of **_my_var** ("Hello World!") without needing to call a method.

Memory allocation for classes and objects

- In Python, memory is dynamically allocated as needed during runtime. When a new object is created, memory is allocated for that object. For classes, memory is allocated when the class is defined.
- When you create an instance of a class, memory is allocated for that instance. The amount of memory allocated depends on the size of the object and the amount of data it contains. For example, an object that contains a large amount of data will require more memory than an object that contains only a few data elements.

Memory allocation for classes and objects

- Python also has a garbage collector that automatically frees memory that is no longer being used by the program. This means that you don't need to worry about manually deallocating memory in Python.

Here is an example of creating a class and an instance of that class:

```
class MyClass:
    def __init__(self, x, y):
        self.x = x
        self.y = y
my_instance = MyClass(1, 2)
```

Memory allocation for classes and objects

In this example, memory is allocated for the **MyClass** class when it is defined. When an instance of the class is created with **my_instance = MyClass(1, 2)**, memory is allocated for **my_instance**, which contains the **x** and **y** attributes.

Memory allocation for classes and objects

- It's worth noting that Python uses a reference counting mechanism for memory management.
- When an object is no longer referenced by any part of the program, its memory is automatically freed. However, in some cases, circular references between objects can prevent the reference counting mechanism from freeing up memory. To handle this, Python uses a garbage collector that periodically checks for and cleans up circular references.

Arrays of objects

- In Python, you can create arrays of objects of a class by using a list.
- A list in Python is a collection of objects that are ordered and changeable.
- To create a list of objects of a class, you can instantiate the class multiple times and add each instance to the list.

Arrays of objects

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
person_list = []
person1 = Person("Ram", 25)
person2 = Person("Bob", 30)
person3 = Person("Charlie", 35)
person_list.append(person1)
person_list.append(person2)
person_list.append(person3)
```

Arrays of objects

Example:

In this example, we create a **Person** class with **name** and **age** attributes. We then create an empty list called **person_list**. We create three instances of the **Person** class and add them to the **person_list** using the **append()** method.

You can access the objects in the list using their index. For example, to access the first object in the **person_list**, you can use **person_list[0]**.

Arrays of objects

You can also iterate over the list of objects using a for loop.

Here is an example:

```
for person in person_list:
    print(person.name, person.age)
```

This will print out the name and age of each person in the **person_list**.

Arrays of objects

- Arrays of objects in Python can also be implemented using the NumPy library.
- NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions. It is commonly used for scientific computing and data analysis.

OBJECT ORIENTED PROGRAMMING (USING PYTHON)

INHERITANCE

Introduction | Polymorphism (Overriding)

OUTLINE

Introduction to Inheritance

Inheriting classes

Polymorphism and Method Overriding

Inheritance - Introduction

- Reusability is an important feature of Object Oriented Programming (OOP).
- Reusing an existing piece of code has many benefits.
- It not only saves effort and cost required to build a software product, but also enhances its reliability.

Inheritance - Introduction

- Instead of reinventing the same code that is already available, it makes sense in reusing existing code.
- Python permits two code reuse mechanisms:

1. Containership (also called composition or complex objects)
2. Inheritance

Inheritance - Introduction

- In both mechanisms we can reuse existing classes and create new enhanced classes based on them.
- We can reuse existing classes even if their source code is not available.

Inheritance - Introduction

Which to use and when?

- **Containership** should be used when the two classes have a '**has-a**' relationship.
- **For example**, a College has Professors. So College class's object can contain one or more Professor class's object(s).

Inheritance - Introduction

Which to use and when?

- **Inheritance** should be used when the two classes have a '**like-a** or **is-a**' relationship.
- **For example**, a **Button** is like a **Window**. So **Button class** can inherit features of an existing class called **Window**.

Inheritance - Introduction

Inheritance:

- The technique of creating a **new class** from an **existing class** is called ***inheritance***.
- The old or existing **class** is called the ***base class*** or ***super class*** or ***parent class*** and the new class is known as the ***derived class*** or ***sub-class*** or ***child class***.

Inheritance - Introduction

Inheritance:

- The ***derived classes*** are created by first inheriting the data and methods of the ***base class*** and then adding new specialized data and functions in it.
- In this process of **inheritance**, the ***base class*** remains unchanged.
- The concept of **inheritance** is used to implement the **is-a** relationship.

Inheritance - Introduction

Inheritance:

For example, teacher **IS-A** person, student **IS-A** person; while both teacher and student are a person in the first place, both also have some distinguishing features. So all the common traits of teacher and student are specified in the ***Person class*** and specialized features are incorporate in two separate classes- ***Teacher*** and ***Student***.

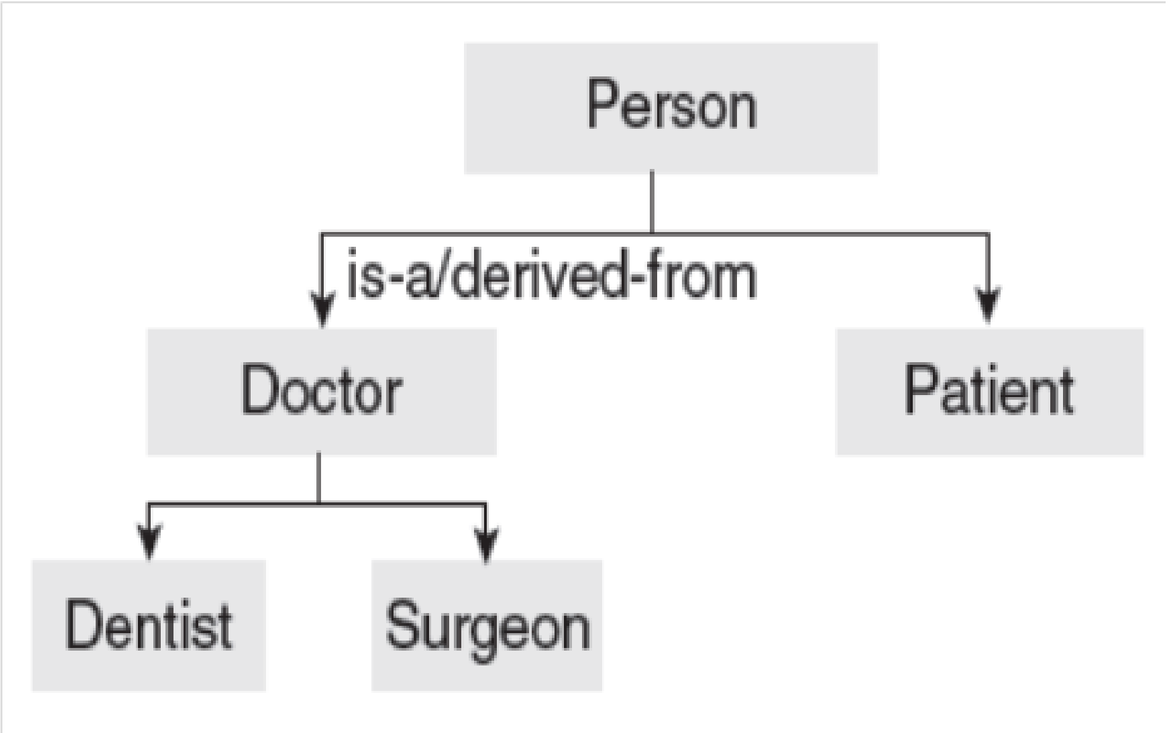
Inheritance - Introduction

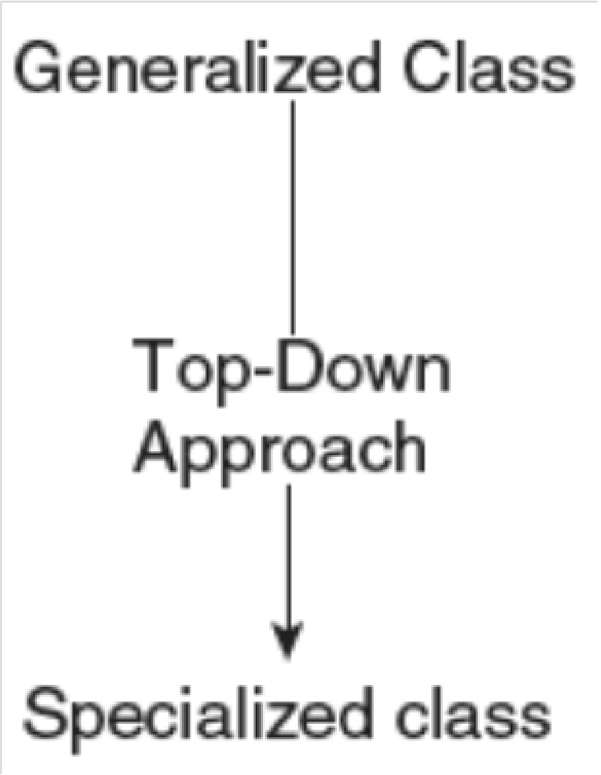
Inheritance:

- ***Inheritance*** which follows a top-down approach to problem solving.
- In top-down approach, generalized classes are designed first and then specialized classes are derived by inheriting/extending the generalized classes.

Inheritance - Introduction

Inheritance:





Inheriting Classes in Python

- In **python**, a derived class can ***inherit*** base class by just mentioning the base in the bracket after the derived class name.
- Syntax to ***inherit*** a base class into the derived class:

```
class derived-classname(base classname):
    list of variables
    list of methods
```

Inheriting Classes in Python

Example: Program to demonstrate the use of inheritance

```
class Person: # Here, Person is Base or Master or
Parent class
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
```

Inheriting Classes in Python

Example: Program to demonstrate the use of inheritance

```
class Teacher(Person): # Here, Teacher is derived class-1 and
Person is Base class
    def __init__(self, name, age, experience, research_area):
        Person.__init__(self, name, age)
        self.experience = experience
        self.research_area = research_area
    def displaydata(self):
        Person.display(self)
        print("Experience:", self.experience)
        print("Research Area:", self.research_area)
```

Inheriting Classes in Python

Example: Program to demonstrate the use of inheritance

```
class Student(Person): # Here, Student is derived class-2 and
Person is Base class
    def __init__(self, name, age, course, year_semester):
        Person.__init__(self, name, age)
        self.course = course
        self.year_semester = year_semester
    def displaydata(self):
        Person.display(self)
        print("Course:", self.course)
        print("Year & Semester:", self.year_semester)
```

Inheriting Classes in Python

Example: Program to demonstrate the use of inheritance

```
print("------TEACHER DETAILS------:")
T = Teacher("Dr. Kadiyala Ramana", 38, 17, "Artificial
Intelligence & Data Science")
T.displaydata()
print("------STUDENT DETAILS----:")
S = Student("RAM", 20, "B.E AI&DS-2", "I Year, II
Semester")
S.displaydata()
```

Inheriting Classes in Python

Example: Program to demonstrate the use of inheritance

OUTPUT:

```
------TEACHER DETAILS------:
Name: Dr. Kadiyala Ramana
Age: 38
Experience: 17
Research Area: Artificial Intelligence & Data Science
------STUDENT DETAILS----:
Name: RAM
Age: 20
Course: B.E AI&DS-2
Year & Semester: I Year, II Semester
```

Inheriting Classes in Python

- In the above program, classes **Teacher** and **Student** are both **inherited** from class **Person**.
- Therefore, the ***inherited classes*** have all the features (attributes and methods) of the ***base class***.
- A ***derived class*** is instantiated in the same way as any ***other class*** is.

Inheriting Classes in Python

- To create an object of the ***derived class***, just write the ***derived class*** name followed by an empty brackets as in **DerivedClassName()**.
- When we use **.__base__** attribute with class name, the **base** (or the ***parent***) **class** of the specified ***class*** is returned.
- Therefore, **print(Student.__base__)** will print (**<class '__main__.Person'>**)

Polymorphism & Method Overriding

- **Polymorphism** refers to having several different forms.
- It is one of the key features of **Object Oriented Programming (OOP)**.
- It enables the programmers to assign a different meaning or usage to a **variable**, **function**, or an **object** in different contexts.

Polymorphism & Method Overriding

- While **inheritance** is related to **classes** and their **hierarchy**, **polymorphism**, on the other hand, is related to methods.
- When **polymorphism** is applied to a **function** or **method** depending on the given parameters, a particular form of the **function** can be selected for execution.
- In **Python**, **method overriding** is one way of implementing **polymorphism**.

Polymorphism & Method Overriding

- In **Python**, method overriding is the process of defining a method in a subclass that has the same name and same parameters as a method in its superclass, but with different implementation.
- This allows the subclass to provide its own implementation of a method that is already defined in its superclass.

Polymorphism & Method Overriding

Example of how to override a method in a Python class:

```
class Animal:
    def make_sound(self):
        print("The animal makes a sound.")
class Cat(Animal):
    def make_sound(self):
        print("Meow")
class Dog(Animal):
    def make_sound(self):
        print("Woof")
```

Polymorphism & Method Overriding

Example of how to override a method in a Python class:

```
# create objects of the classes
animal = Animal()
cat = Cat()
dog = Dog()
# calling the methods
animal.make_sound() #output: The animal makes a sound.
cat.make_sound()      # output: Meow
dog.make_sound()      # output: Woof
```

Polymorphism & Method Overriding

Example of how to override a method in a Python class:

OUTPUT

```
The animal makes a sound.
Meow
Woof
```

Polymorphism & Method Overriding

Example of how to override a method in a Python class:

In this example, we have defined an **Animal** class with a **make_sound()** method that simply prints a generic message. We then define two subclasses, **Cat** and **Dog**, which both inherit from **Animal** class and override the **make_sound()** method with their own implementation. When we create objects of these classes and call the **make_sound()** method, the appropriate implementation is used based on the type of object.

Polymorphism & Method Overriding

Example: Program to demonstrate the use of inheritance

```
class Person: # Here, Person is Base or Master or
Parent class
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
```

Polymorphism & Method Overriding

Example: Program to demonstrate the use of inheritance

```
class Teacher(Person): # Here, Teacher is derived class-1 and
Person is Base class
    def __init__(self, name, age, experience, research_area):
        Person.__init__(self, name, age)
        self.experience = experience
        self.research_area = research_area
    def displaydata(self):
        Person.display(self)
        print("Experience:", self.experience)
        print("Research Area:", self.research_area)
```

Polymorphism & Method Overriding

Example: Program to demonstrate the use of inheritance

```
class Student(Person): # Here, Student is derived class-2 and
Person is Base class
    def __init__(self, name, age, course, year_semester):
        Person.__init__(self, name, age)
        self.course = course
        self.year_semester = year_semester
    def displaydata(self):
        Person.display(self)
        print("Course:", self.course)
        print("Year & Semester:", self.year_semester)
```

Polymorphism & Method Overriding

Example: Program to demonstrate the use of inheritance

```
print("------TEACHER DETAILS------:")
T = Teacher("Dr. Kadiyala Ramana", 38, 17, "Artificial
Intelligence & Data Science")
T.displaydata()
print("------STUDENT DETAILS----:")
S = Student("RAM", 20, "B.E AI&DS-2", "I Year, II
Semester")
S.displaydata()
```

Polymorphism & Method Overriding

- In the *above program*, notice that **__init__()** method was defined in all the three classes. When this happens, the method in the derived class **overrides** that in the base class.
- This means that **__init__()** in **Teacher** and **Student** gets preference over the **__init__()** method in the **Person class**.

Polymorphism & Method Overriding

- Thus, ***Method Overriding*** is the ability of a class to change the implementation of a method provided by one of its ancestors. It is an important concept of Object Oriented Programming (OOP) since it exploits the power of ***inheritance***.

Polymorphism & Method Overriding

- Observe another thing that when we **override** a base method, we extend the functionality of the base class method.
- This is done by calling the method in the base class method from the derived class method and also adding additional statements in the derived class method.

Polymorphism & Method Overriding

- Instead of writing **Person.__init__(self, name, age)**, we could have also written **super().__init__(self, name, age)**.
- Here, **super()** is a built-in function that denotes the **base class**. So when we invoke a method using the **super()** function, then parent version of the method is called.
- In **Python**, every class is **inherited** from the base class object.

Polymorphism & Method Overriding

- In case of **multiple inheritance** (*a class derived from more than one base class*), we need to invoke the **super()** function in **__init__()** method of every class.

Polymorphism & Method Overriding

Example: Program to demonstrate the issue of invoking __init__() case of multiple inheritance.

```
class Base1(object):
    def __init__(self):
        print("Base1 Class..")
class Base2(object):
    def __init__(self):
        print("Base2 Class..")
class Derived(Base1, Base2):
    pass
D = Derived()
```

Polymorphism & Method Overriding

Example: Program to demonstrate the issue of invoking __init__() case of multiple inheritance.

OUTPUT:

```
Base1 Class..
```

Polymorphism & Method Overriding

- In the ***above method***, an object of **derived class** is made.
- Since there is no **__init__()** method in the derived class, the **__init__()** method of the first class gets called.
- But since, there is no call to **super()** function in the **__init__()** method of **Base1** class, no further **__init__()** method is invoked.

Polymorphism & Method Overriding

Example: Program to demonstrate the call of super() from __init__() of a base class

```
class Base1(object):
    def __init__(self):
        print("Base1 Class..")
        super(Base1, self).__init__()
class Base2(object):
    def __init__(self):
        print("Base2 Class..")
class Derived(Base1, Base2):
    pass
D = Derived()
```

Polymorphism & Method Overriding

Example: Program to demonstrate the call of super() from __init__() of a base class

```
OUTPUT:
Base1 Class..
Base2 Class..
```

Polymorphism & Method Overriding

Example: Program to call the __init__() methods of all the classes.

```
class Base1(object):
    def __init__(self):
        print("Base1 Class..")
        super(Base1, self).__init__()
class Base2(object):
    def __init__(self):
        print("Base2 Class..")
class Derived(Base1, Base2):
    def __init__(self):
        super(Derived, self).__init__()
        print("Derived Class..")
D = Derived()
```

Polymorphism & Method Overriding

Example: Program to call the __init__() methods of all the classes.

```
OUTPUT:
Base1 Class..
Base2 Class..
Derived Class..
```

Polymorphism & Method Overriding

isinstance() and issubclass() methods:

- **isinatance()** and **issubclass()** are two built-in functions which are very useful in **Python** to check instances.
- The **isinstance()** function returns **True** if the object is an instance the class or other classes derived from it.
- Similarly, the **issubclass()** checks for class inheritance.

Polymorphism & Method Overriding

Example: Program to demonstrate isinstance() and issubclass() methods

```
class Person: # Here, Person is Base or Master or Parent
class
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
```

Polymorphism & Method Overriding

Example: Program to demonstrate isinstance() and issubclass() methods

```
class Teacher(Person): # Here, Teacher is derived class-1 and Person
is Base class
    def __init__(self, name, age, experience, research_area):
        Person.__init__(self, name, age)
        self.experience = experience
        self.research_area = research_area
    def displaydata(self):
        Person.display(self)
        print("Experience:", self.experience)
        print("Research Area:", self.research_area)
```

Polymorphism & Method Overriding

Example: Program to demonstrate isinstance() and issubclass() methods

```
class Student(Person): # Here, Student is derived class-2 and
Person is Base class
    def __init__(self, name, age, course, year_semester):
        Person.__init__(self, name, age)
        self.course = course
        self.year_semester = year_semester
    def displaydata(self):
        Person.display(self)
        print("Course:", self.course)
        print("Year & Semester:", self.year_semester)
```

Polymorphism & Method Overriding

Example: Program to demonstrate isinstance() and issubclass() methods

```
print("------TEACHER DETAILS------:")
T = Teacher("Dr. Kadiyala Ramana", 38, 17, "Artificial
Intelligence & Data Science")
T.displaydata()
print("------STUDENT DETAILS----:")
S = Student("RAM", 20, "B.E AI&DS-2", "I Year, II
Semester")
S.displaydata()
```

Polymorphism & Method Overriding

Example: Program to demonstrate isinstance() and issubclass() methods

```
print("----DEMO ON isinatnce() and issubclass() methods---:")
print("T is a Teacher?", isinstance(T, Teacher))
print("T is a Person?", isinstance(T, Person))
print("T is an Integer?", isinstance(T, int))
print("T is an Object?", isinstance(T, object))
print("Person is a subclass of Teacher?", issubclass(Person,
Teacher))
print("Boolean is a subclass of int?", issubclass(bool, int))
```

Polymorphism & Method Overriding

Example: Program to demonstrate isinstance() and issubclass() methods

OUTPUT:

```
------TEACHER DETAILS------:
Name: Dr. Kadiyala Ramana
Age: 38
Experience: 17
Research Area: Artificial Intelligence & Data Science
------STUDENT DETAILS----:
Name: RAM
Age: 20
Course: B.E AI&DS-2
Year & Semester: I Year, II Semester
```

Polymorphism & Method Overriding

Example: Program to demonstrate isinstance() and issubclass() methods

OUTPUT:

```
------DEMO ON isinatnce() and issubclass() methods------:
T is a Teacher? True
T is a Person? True
T is an Integer? False
T is an Object? True
Person is a subclass of Teacher? False
Boolean is a subclass of int? True
```

OBJECT ORIENTED PROGRAMMING (USING PYTHON)

INHERITANCE

Types of Inheritance

OUTLINE

Types of Inheritance

1. Single Level Inheritance
2. Multiple Inheritance
3. Multi-level Inheritance
4. Hierarchical Inheritance
5. Multi-path or Hybrid Inheritance

Types of Inheritance

- **Python** supports different types or variants of Inheritance.

There are 5 types of Inheritance:

1. Single Level Inheritance
2. Multiple Inheritance
3. Multi-level Inheritance
4. Hierarchical Inheritance
5. Multi-path or Hybrid Inheritance

Single Level Inheritance

- The concept of inheriting properties from only one class into another class is known as a **single inheritance**.

Syntax :

```
class BaseClassname:
    Body of base class
class DerivedClassname(BaseClassname):
    Body of derived class
```

Single Level Inheritance

Example: Program to implement single inheritance

```
class A:
    def m1(self):
        print("m1 is method of Main or Base Class A..")
class B(A): #Class B is derived from the Parent or Base Class
A
    def m2(self):
        print("m2 is method of Derived or Sub or Child Class
B..")
b1 = B()
b1.m1()
b1.m2()
```

Single Level Inheritance

Example: Program to implement single inheritance

OUTPUT:

```
m1 is method of Main or Base Class A..
m2 is method of Derived or Sub or Child Class B..
```

Multiple Inheritance

- When a **derived class** inherits features from more than one **base class**, it is called **multiple inheritance**.
- The **derived class** has all the features of both the **base classes** and in addition to them can have additional new features.

Multiple Inheritance

- The syntax for **multiple inheritance** is similar to that of **single inheritance** and can be given as:

```
class Base1:
    statement block
class Base2:
    statement block
class Derived (Base1, Base2):
    statement block
```

Multiple Inheritance

Example: Program to implement multiple inheritance

```
class Base1: #First Base Class
    def __init__(self):
        super(Base1,self).__init__()
        print("Base1 Class..")
class Base2:#Second Base Class
    def __init__(self):
        super(Base2,self).__init__()
        print("Base2 Class..")
```

Multiple Inheritance

Example: Program to implement multiple inheritance

```
class Derived(Base1, Base2): #Derived class
derived from 2 base classes Base1 & Base2
    def __init__(self):
        super(Derived,self).__init__()
        print("Derived Class..")
Derived()
```

Multiple Inheritance

Example: Program to implement multiple inheritance

OUTPUT:

```
Base2 Class..
Base1 Class..
Derived Class..
```

Multiple Inheritance

- In the above program output order is based on **Method Resolution Order (MRO)**, it works on depth-first traversal.
- The set of rules used to find the ***linearization order*** is called **Method Resolution Order (MRO)**.
- The **MRO** ensures that a class appears before its parent classes.

Multiple Inheritance

- However, in case of **multiple inheritance**, the **MRO** is same as a tuple of base classes.
- We can check the **MRO** of a class by either the **__mro__** attribute or **mro()** method.
- While the **__mro__** attribute returns a **tuple**, the **mro()** method returns a **list**.

Multiple Inheritance

- In the above program the order of class hierarchy can be given as:

  Derived -> Base1 and **Derived -> Base2.**

- When we create an instance of the derived class, the following things happen.

Step-1: The **__init__()** method of Derived class is called.

Step-2: The **__init__()** method of Base1 class is invoked (according to **MRO**) from the **__init__()** method of Derived class.

Multiple Inheritance

Step-3: The **__init__()** method of Base2 class is invoked (according to **MRO**) from the **__init__()** method of Base1 class.

Step-4: From the **__init__()** method of Base2 class the **__init__()** method of Object is invoked which does nothing. Finally, Base2 class gets printed on the screen and the control is returned to the **__init__()** method of Base1 class.

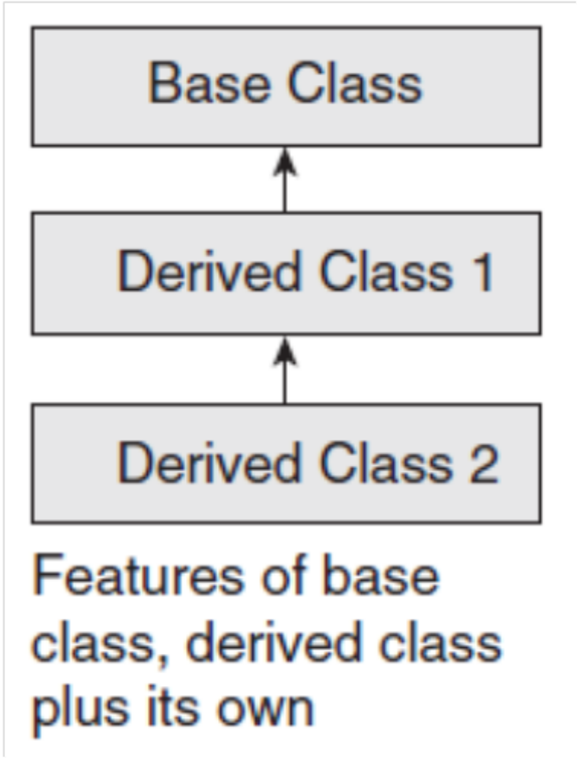
Step-5: Base1 class gets printed and the control is transferred back to the **__init__()** method of Derived class.

Step-6: Derived class gets printed on the screen and hence the result.

Multi-level Inheritance

- **Multi-Level inheritance** is possible in **python** like other object-oriented programming languages.
- The technique of deriving a class from an already derived class is called **multi - level inheritance**.
- In **multi - level inheritance**, number of levels can go up to any number based on the requirement.

Multi-level Inheritance



- In the above Fig., **Base Class** acts as the base for ***Derived Class1*** which in turn acts as a base for ***Derived Class2.***

Multi-level Inheritance

- The ***Derived Class1*** has the features of Base Class plus its own features.
- The ***Derived Class1*** is known as the intermediate class as this class provide a link for inheritance between the ***Base Class*** and the ***Derived Class2***.
- The chain of classes:

Base Class --> Derived Class1 --> Derived Class2 is known as the ***inheritance path***.

Multi-level Inheritance

The syntax of **multi-level inheritance** is given below:

```
class Base:
    body of the Base class
class Derived1(Base):
    body of the Derived1 class
class Derived2(Derived1):
    body of the Derived2 class
```

Multi-level Inheritance

- In **Multi-level inheritance** scenario, any specified attribute is first searched in the *current class* (***Derived Class2***). If it is not found there, then the ***Derived Class1*** is searched, if it is not found even there, then the ***Base Class*** is searched. If the attribute is still not found, then finally the object class is checked. This order is also called ***linearization*** of ***Derived Class2***.
- Correspondingly, the set of rules used to find this *linearization order* is called **Method Resolution Order(MRO).**

Multi-level Inheritance

Example: Program to implement multi-level inheritance

```
class Person: # Base class
    def name(self):
        faculty_name = input("Enter Eligible
Faculty Name for HOD position..")
        print("Name:", faculty_name)
```

Multi-level Inheritance

Example: Program to implement multi-level inheritance

```
class Teacher(Person):# Class Teacher derived
from the Base class Person
    def Qualification(self):
        print("Qualification of HOD is Ph.D
mandatory:")
```

Multi-level Inheritance

Example: Program to implement multi-level inheritance

```
class HOD(Teacher): # Class HOD is derived
from the Derived class Teacher
    def Experience(self):
        print("Experience of HOD is at least
15 years:")
```

Multi-level Inheritance

Example: Program to implement multi-level inheritance

```
print(Person.__mro__)
h = HOD()
h.name()
h.Qualification()
h.Experience()
```

Multi-level Inheritance

Example: Program to implement mult-level inheritance

OUTPUT:

```
Enter Eligible Faculty Name for HOD position..
Dr KADIYALA RAMANA
Name: Dr KADIYALA RAMANA
Qualification of HOD is Ph.D mandatory:
Experience of HOD is at least 15 years:
(<class '__main__.HOD'>, <class '__main__.Teacher'>, <class
'__main__.Person'>, <class 'object'>)<class 'object'>)
```

Multi-level Inheritance

Example: Program to implement mult-level inheritance

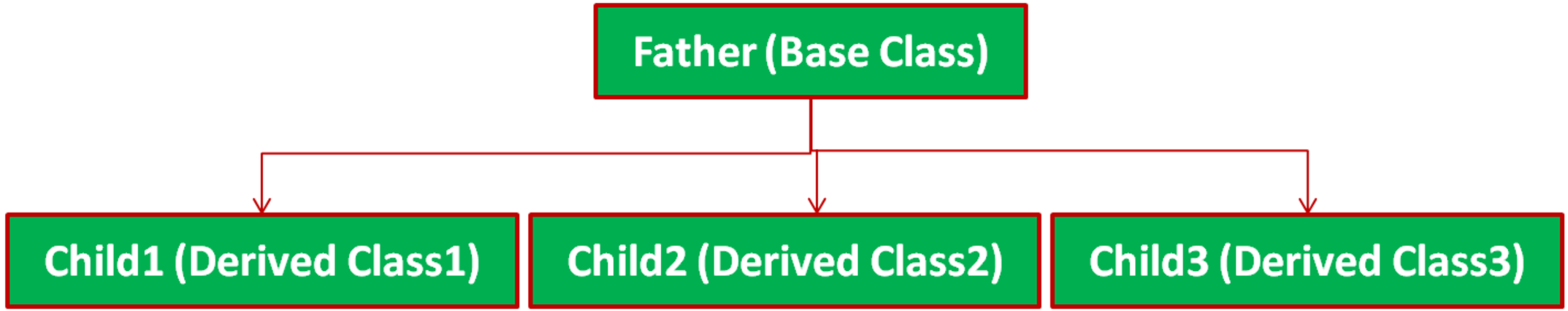
OUTPUT:

```
Enter Eligible Faculty Name for HOD position..
Dr KADIYALA RAMANA
Name: Dr KADIYALA RAMANA
Qualification of HOD is Ph.D mandatory:
Experience of HOD is at least 15 years:
(<class '__main__.HOD'>, <class '__main__.Teacher'>, <class
'__main__.Person'>, <class 'object'>)<class 'object'>)
```

Hierarchical Inheritance

- The concept of inheriting properties from one class into multiple classes is known as a **hierarchical inheritance**.
- When more than one derived class are created from a single base class, this type of inheritance is called **hierarchical inheritance.**
- It contains only one base class and multiple derived classes.

Hierarchical Inheritance



In the above Fig., **Father** is only one ***Base Class***. ***Child1***, ***Child2*** & ***Child3*** are three **sub** or ***derived classes*** derived from ***Base Class*** **Father**.

Hierarchical Inheritance

Syntax:

```
class A:
    body of base class
class B(A):
    body of derived-1 class
class C(A):
    body of derived-2 class
```

Hierarchical Inheritance

Ex: Program to implement hierarchical inheritance

```
class A: # Base class
    def m1(self):
        print("I am m1 of Base Class A!")
class B(A): # Derived Class1 from Base Class A
    def m2(self):
        print("I am m2 of Derived Class B!")
class C(A): # Derived Class2 from Base Class A
    def m3(self):
        print("I am m3 of Derived Class C!")
class D(A): # Derived Class3 from Base Class A
    def m4(self):
        print("I am m4 of Derived Class D!")
```

Hierarchical Inheritance

Ex: Program to implement hierarchical inheritance

```
b1 = B()
b1.m1() # derived from Base Class A
b1.m2() # Own method of Class B
c1 = C()
c1.m1() # derived from Base Class A
c1.m3() # Own method of Class C
d1 = D()
d1.m1() # derived from Base Class A
d1.m4() # Own method of Class D
```

Hierarchical Inheritance

Ex: Program to implement hierarchical inheritance

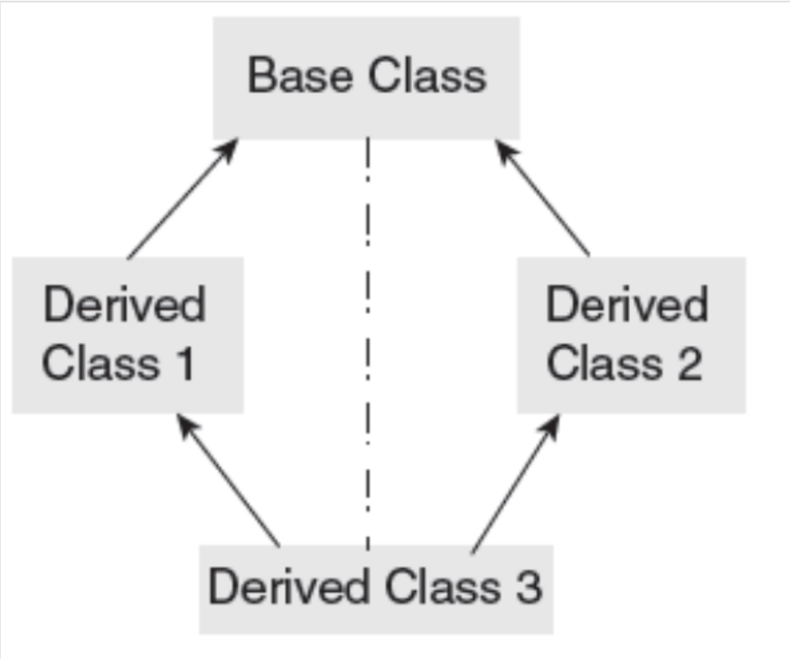
OUTPUT:

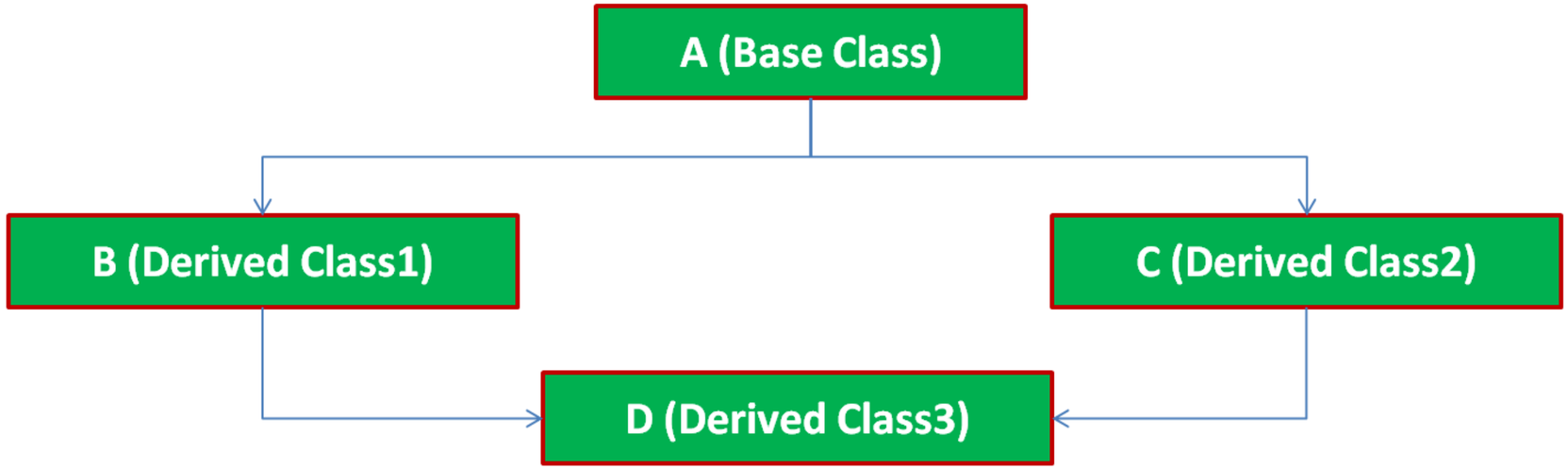
```
I am m1 of Base Class A!
I am m2 of Derived Class B!
I am m1 of Base Class A!
I am m3 of Derived Class C!
I am m1 of Base Class A!
I am m4 of Derived Class D!
```

Multi-path or Hybrid Inheritance

- Deriving a class from other derived classes that are in turn derived from the same base class is called **multi-path or Hybrid inheritance**.
- **Hybrid Inheritance** is that type in which we combine two or more types of inheritance.
- It is the combination of ***sinlge + Hierarchical + Multiple + Multilevel Inheritances*** are called **Hybrid or Multi-path Inheritance.**

Multi-path or Hybrid Inheritance

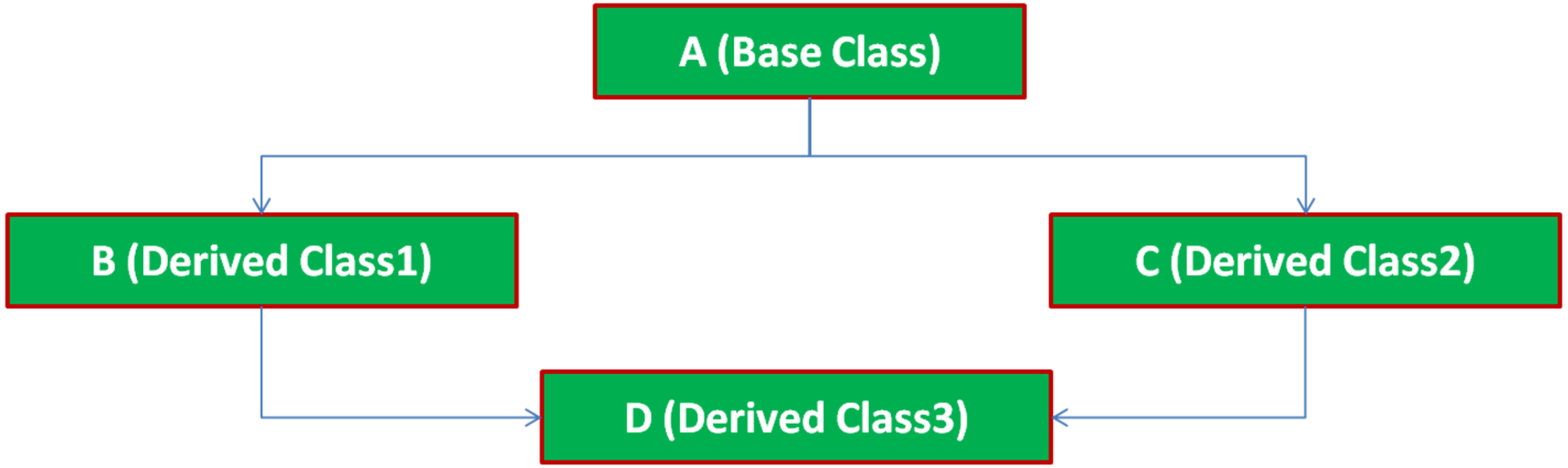




Multi-path or Hybrid Inheritance

- In the above Fig., the ***Derived Class3*** has two immediate Base classes **Derived Class1** and ***Derived Class2***. Both these base classes are themselves derived from the ***Base Class***, thereby forming a grand parent, parent, and child form of a relationship.
- The Derived Class inherits the features of of the B***ase Class*** (Grand Parents) via two separate paths. Therefore, the ***Base Class*** is also known as the ***Indirect Base Class***.

Multi-path or Hybrid Inheritance



- **A --> B, A --> C, B --> D & C --> D** are **Single Inheritances**
- **A --> B-->D & A --> C-->D** are **Multi-level Inheritances**
- **B, C --> D** is **Multiple Inheritance**
- **A-->B & A-->C** is **Hierarchical Inheritance**

Multi-path or Hybrid Inheritance

Syntax:

```
class A:
    body of class A
class B(A):
    body of class B
class C(A):
    body of class C:
class D(B, C):
    body of class D
```

Multi-path or Hybrid Inheritance

Ex. Program to demonstrate multi-path inheritance

```
class Student: # Base class
    def name(self):
        print("Name:")
class Academic_Performance(Student): # Derived
Class1 from Base Class Student
    def Academic_Score(self):
        print("Overall CGPA is 9.7 and above out
of 10")
```

Multi-path or Hybrid Inheritance

Ex. Program to demonstrate multi-path inheritance

```
class Written_Test(Student): # Derived Class2 from
Base Class Student
    def Written_test_Score(self):
        print("Written Test Score is 65% and
above")
```

Multi-path or Hybrid Inheritance

Ex. Program to demonstrate multi-path inheritance

```
class Result(Academic_Performance, Written_Test ): #
Derived Class3 from Derived Classes Academic_Performance
& Written_Test
    def Eligibility(self):
        print("---Minimum Eligibility to Apply for
Placements in Core Tier-1 Companies----")
        self.Academic_Score()
        self.Written_test_Score()
r1 = Result()
r1.Eligibility()
```

Multi-path or Hybrid Inheritance

Ex. Program to demonstrate multi-path inheritance

OUTPUT:

```
---Minimum Eligibility to Apply for
Placements in Core Tier-1 Companies----
Overall CGPA is 9.7 and above out of 10
Written Test Score is 65% and above
```

Object Oriented Programming (Using Python)

UNIT- II

Polymorphism and Inheritance:

- Type conversions
- Base classes and derived classes
- Various types of classes (including Metaclass/ abstract classes)
- Invocation of constructors and destructors in inheritance
- Aggregation and composition
- Classification hierarchies
- Unit testing and Exceptions

Type conversions in python classes

- In Python, you can convert one data type to another using type conversion functions.
- Here are some commonly used type conversion functions:

1. **int()** - converts a string or a float to an integer.
2. **float()** - converts a string or an integer to a float.
3. **str()** - converts an integer, float or any other data type to a string.
4. **bool()** - converts any data type to a boolean.

Type conversions in python classes

- When it comes to Python classes, you can define your own conversion functions by implementing the special methods **__int__()**, **__float__()**, **__str__()**, and **__bool__()** in your class.

Type conversions in python classes

Here is an example of a Python class that defines these conversion methods:

```
class Circle:
    def __init__(self, radius):
        self.radius = radius
    def __int__(self):
        return int(self.radius)
    def __float__(self):
        return float(self.radius)
    def __str__(self):
        return f"Circle with radius {self.radius}"
    def __bool__(self):
        return self.radius > 0
```

Type conversions in python classes

Here is an example of a Python class that defines these conversion methods:

```
# create an object of the Circle class
circle = Circle(5)
# calling the conversion methods
print(int(circle))    #output: 5
print(float(circle)) #output: 5.0
print(str(circle))    #output: Circle with radius 5
print(bool(circle))    # output: True
```

Type conversions in python classes

Here is an example of a Python class that defines these conversion methods:

OUTPUT:

5

5.0

Circle with radius 5

True

Type conversions in python classes

Here is an example of a Python class that defines these conversion methods:

In this example, we have defined a **`Circle`** class that has a **`radius`** attribute.
We have implemented the **`__int__()`**, **`__float__()`**, **`__str__()`**, and **`__bool__()`** methods to convert the circle object to an integer, float, string, and Boolean.

Base classes and derived classes

- Inheritance is one of the fundamental concepts of object-oriented programming in Python.
- In inheritance, a **derived class** is created by inheriting the attributes and methods of a **base class**.
- The **derived class** can then add or modify its own attributes and methods.
- The **base class** is also known as the **parent class**, while the **derived class** is also known as the **child class**.

Base classes and derived classes

Here's an example of how to create a derived class in Python:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def eat(self):
        print(f"{self.name} is eating.")
class Dog(Animal):
    def bark(self):
        print("Woof!")
```

Base classes and derived classes

Here's an example of how to create a derived class in Python:

```
# create objects of the classes
animal = Animal("Simba", 3)
dog = Dog("Buddy", 5)
# calling the methods
animal.eat()     # output: Simba is eating.
dog.eat()        # output: Buddy is eating.
dog.bark()       # output: Woof!
```

Base classes and derived classes

Here's an example of how to create a derived class in Python:

OUTPUT:

```
Simba is eating.
Buddy is eating.
Woof!
```

Base classes and derived classes

Here's an example of how to create a derived class in Python:

In this example, we have defined an **Animal** class with an **__init__()** method and an **eat()** method.

We then define a **Dog** class that inherits from the **Animal** class using the syntax

```
class Dog(Animal):
```

The **Dog** class also has a **bark()** method.

Base classes and derived classes

Here's an example of how to create a derived class in Python:

When we create objects of these classes, we can see that the **Dog** object inherits the **name** and **age** attributes as well as the **eat()** method from the **Animal** class. Additionally, the **Dog** object has its own **bark()** method.

By using inheritance, we can reuse code and avoid duplication of code. We can also create hierarchies of classes, where a derived class can be further inherited by another derived class.

Various types of classes

- In Python, there are several types of classes that you can create depending on your needs.
- Some of the common types of classes are:
    1. Regular Classes
    2. Abstract Classes
    3. Singleton Classes
    4. Metaclasses
    5. Inner Classes

Various types of classes

1. Regular Classes: Regular classes are the most common type of class in Python. They define a blueprint for creating objects with a set of attributes and methods.

Example:

```
class Dog:
    def __init__(self, name):
        self.name = name
    def bark(self):
        print(f"{self.name} says Woof!")
d1=Dog("DOG")
d1.bark()#Output: DOG says Woof!
```

Various types of classes

2. Abstract Classes: These are classes that cannot be instantiated, but are used as a base class for other classes. They contain abstract methods that need to be implemented in the derived class.

Example:

```
from abc import ABC, abstractmethod # abc - Abstract Base Classes module
class Shape(ABC): # ABC - Abstract Base Classes
    @abstractmethod
    def area(self):
        pass
    @abstractmethod
    def perimeter(self):
        pass
```

Various types of classes

2. Abstract Classes: Example: cont'd.

```
class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side * self.side
    def perimeter(self):
        return 4 * self.side
s=Square(5)
print(s.area())
print(s.perimeter())
```

OUTPUT:

```
25
20
```

Various types of classes

3. Singleton Classes: These are classes that allow only one instance to be created throughout the program. They are useful when you want to restrict the number of instances of a class.

Example:

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            return cls._instance
s=Singleton()
print(s)
s1=Singleton()
print(s1)
```

Various types of classes

3. Singleton Classes: Example cont'd.

'Output:

<__main__.Singleton object at 0x0000029CCD3868C0>
None

The __new__() static method creates a new instance of a class cls and takes in the class (of which the instance was requested) as the first argument.

Syntax

```
class ClassName:
  __new__(cls, *args, **kwargs):
    obj = super().__new__(cls)
    return obj
```

Various types of classes

3. Singleton Classes:

Object creation is typically done by invoking the superclass' __new__() in two ways. The first argument of __new__() is always the class which is passed in as the first parameter automatically. The name cls is not a keyword; it's used to reference the class (the first parameter) by convention. When __new__() returns an instance of cls it automatically invokes the __init__() method of that instance with the arguments passed to it. The first argument passed to __init__() will be the instance itself (this happens automatically) which is by convention referenced by self.

Various types of classes

4. Metaclasses: These are classes that are used to create other classes. They define the behavior and structure of a class.

Example:

```
class MyMeta(type):
    def __new__(cls, name, bases, attrs):
        attrs['x'] = 1
        return super().__new__(cls, name, bases, attrs)

class MyClass(metaclass=MyMeta):
    pass

print(MyClass.x) # Output: 1
```

Various types of classes

5. Inner Classes: These are classes that are defined inside another class. They can be used to group related classes together.

Example:

Various types of classes

5. Inner Classes: cont'd.

```
class OuterClass:
    def __init__(self):
        self.x = 10

    def outer_method(self):
        print("Outer method called")

        class InnerClass:
            def __init__(self):
                self.y = 20
                self.x = 10

            def inner_method(self):
                print("Inner method called")
                print("x = ", self.x)
                print("y = ", self.y)

        inner_obj = InnerClass()
        inner_obj.inner_method()

outer_obj = OuterClass()
outer_obj.outer_method()
```

Various types of classes

5. Inner Classes: cont'd.

```
OUTPUT:
Outer method called
Inner method called
x =  10
y =  20
```

Various types of classes

5. Inner Classes: cont'd.

In the above example, the **OuterClass** contains an inner class I**nnerClass**. When the **outer_method** is called, it creates an instance of the **InnerClass** and calls its **inner_method**. The **inner_method** accesses the attributes **x** and **y** of both the outer and inner classes.

- **Note** that the inner class can only be accessed within the scope of the outer class. It is not visible outside the outer class, and it can't be instantiated directly. The inner class can access all the attributes and methods of the outer class, including private ones.
- Inner classes can be used to organize and encapsulate code within a larger class, and they can access the attributes and methods of the outer class.

Object Oriented Programming (Using Python)

UNIT- II

Polymorphism and Inheritance: cont'd.

- Invocation of constructors and destructors in inheritance
- Aggregation and composition
- Classification hierarchies
- Unit testing
- Exceptions

Invocation of constructors and destructors

- In Python, constructors and destructors are special methods that are automatically called when an object is created and destroyed, respectively. The constructor method is called __init__() and the destructor method is called __del__().
- Here's how they are invoked in Python:

Constructor Invocation:

When an object is created, Python automatically calls the constructor method __init__() to initialize the object's attributes.

Example:

```
class MyClass:
    def __init__(self):
        print("Constructor called")
obj = MyClass()  # Output: Constructor called
```

In the above example, the constructor of the **MyClass** is invoked automatically when the object **obj** is created using the class name **MyClass()**.

Destructor Invocation:

When an object is no longer needed or when the program terminates, Python automatically calls the destructor method __del__() to perform any cleanup operations, such as releasing system resources or closing files.

Example:

```
class MyClass:
    def __del__(self):
        print("Destructor called")
obj = MyClass()  # Create object
del obj  # Delete object, Output: Destructor called
```

In the above example, the destructor of the **MyClass** is invoked automatically when the object **obj** is deleted using the **del** keyword.

Invocation of constructors and destructors

It's important to note that the destructor is not called immediately when an object is no longer needed. Instead, Python schedules the destructor to be called at a later time when it determines that the object is no longer being used.

The constructor and destructor methods in Python are automatically invoked when an object is created and destroyed, respectively.

Aggregation and composition

- Aggregation and composition are two forms of object composition in object-oriented programming that are closely related to inheritance.
- **Aggregation** is a form of object composition where one object contains references to one or more other objects, which are typically created independently of the parent object. In this way, the child objects exist independently of the parent object and can exist even after the parent object is destroyed.
- Aggregation is commonly used to represent a "has-a" relationship between objects.

Aggregation and composition

- **Composition**, on the other hand, is a stricter form of object composition where one object is composed of one or more other objects, and the child objects are created and destroyed along with the parent object. In this way, the child objects cannot exist independently of the parent object.
- **Composition** is commonly used to represent a "part-of" relationship between objects.

Aggregation and composition

Aggregation Example implemented in Python using inheritance:

```
class Person:
    def __init__(self, name):
        self.name = name

class Company:
    def __init__(self, name, employees):
        self.name = name
        self.employees = employees

person1 = Person("Alice")
person2 = Person("Bob")
company = Company("Acme Inc.", [person1, person2])
```

Aggregation and composition

Aggregation Example implemented in Python using inheritance:

In this example, the **Company** class contains a reference to a list of **Person** objects, which are created independently of the **Company** object. This is an example of aggregation.

Aggregation and composition

Composition Example implemented in Python using inheritance:

```
class Engine:
    def start(self):
        print("Engine started")

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        self.engine.start()

car = Car()
car.start()
```

Aggregation and composition

Composition Example implemented in Python using inheritance:

In this example, the **Car** class is composed of an **Engine** object, which is created along with the **Car** object. The **start** method of the **Car** class calls the **start** method of the **Engine** object. This is an example of composition.

Aggregation and composition

Note that while aggregation and composition are related to inheritance, they are not the same thing.

Inheritance is a mechanism for creating new classes by deriving them from existing classes, while aggregation and composition are mechanisms for composing objects by combining multiple objects into a single object.

Aggregation: Example full program

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Department:
    def __init__(self, name, employees):
        self.name = name
        self.employees = employees

    def add_employee(self, employee):
        self.employees.append(employee)

    def remove_employee(self, employee):
        self.employees.remove(employee)

class Manager(Person):
    def __init__(self, name, age, department):
        super().__init__(name, age)
        self.department = department

    def add_employee(self, employee):
        self.department.add_employee(employee)

    def remove_employee(self, employee):
        self.department.remove_employee(employee)

employee1 = Person("Ram", 24)
employee2 = Person("Robert", 28)
employee3 = Person("Turing", 32)

department = Department("Sales", [employee1, employee2])

manager = Manager("Alice", 40, department)
manager.add_employee(employee3)

for employee in department.employees:
    print(employee.name)
```

Output:

Ram
Robert
Turing

In this example, we have three classes: **Person**, **Department**, and **Manager**. **Person** is a base class that defines the **name** and **age** attributes of a person. **Department** is a class that defines a list of employees and provides methods to add and remove employees from the list. **Manager** is a subclass of **Person** that adds a **Department** attribute and methods to add and remove employees from the department's employee list.

We create three **Person** objects and a **Department** object with two employees. We then create a **Manager** object with the **Department** object as an argument. We use the add_employee() method of the **Manager** object to add a new employee to the **Department** object. Finally, we print out the names of all the employees in the **Department** object, including the one added by the **Manager**.

Composition: Example full program

```
class Engine:
    def __init__(self, fuel_type):
        self.fuel_type = fuel_type

class Car:
    def __init__(self, make, model, year, engine):
        self.make = make
        self.model = model
        self.year = year
        self.engine = engine

    def start(self):
        print(f"{self.make} {self.model} started")

    def stop(self):
        print(f"{self.make} {self.model} stopped")

engine = Engine("Gasoline")
car = Car("TATA", "Nexon", 2023, engine)

car.start()
car.stop()

# Output:
# TATA Nexon started
# TATA Nexon stopped
```

In this example, we have two classes: **Engine** and **Car**. **Engine** is a class that defines the fuel_type attribute of an engine. **Car** is a class that defines the make, model, year, and engine attributes of a car.

We create an **Engine** object with a fuel type of "Gasoline" and a **Car** object with the make "TATA", model “Nexon", year 2023, and the **Engine** object as an argument. We then use the start() and stop() methods of the **Car** object to simulate starting and stopping the car.

Classification Hierarchies

- Classification hierarchies in inheritance refer to a way of organizing classes into a tree-like structure based on their relationships.
- In this hierarchy, a class at a higher level is a more general concept, while a class at a lower level is a more specific concept.
- Each class in the hierarchy inherits properties and methods from its parent class, and it may also define new properties and methods that are specific to it.

Classification Hierarchies:Example-1. Animal Classification Hierarchy

```
class Animal:
    def __init__(self, name, habitat):
        self.name = name
        self.habitat = habitat

    def eat(self):
        print("The {} is eating.".format(self.name))

class Mammal(Animal):
    def __init__(self, name, habitat, num_legs):
        super().__init__(name, habitat)
        self.num_legs = num_legs

    def give_birth(self):
        print("The {} is giving birth to live young.".format(self.name))

class Reptile(Animal):
    def __init__(self, name, habitat, num_legs):
        super().__init__(name, habitat)
        self.num_legs = num_legs

    def lay_eggs(self):
        print("The {} is laying eggs.".format(self.name))

class Dog(Mammal):
    def __init__(self, name, habitat):
        super().__init__(name, habitat, num_legs=4)

    def bark(self):
        print("The {} is barking.".format(self.name))

class Lizard(Reptile):
    def __init__(self, name, habitat):
        super().__init__(name, habitat, num_legs=4)

    def change_color(self):
        print("The {} is changing color.".format(self.name))
```

```
dog = Dog("Rufus", "land")
dog.eat()
dog.give_birth()
dog.bark()

lizard = Lizard("Izzy", "desert")
lizard.eat()
lizard.lay_eggs()
lizard.change_color()
```

```
The Rufus is eating.
The Rufus is giving birth to live young.
The Rufus is barking.
The Izzy is eating.
The Izzy is laying eggs.
The Izzy is changing color.
```

Classification Hierarchies: Example-1. Animal Classification Hierarchy

In this example, we have a classification hierarchy for animals. The **Animal** class is the most general class, and it has two child classes: **Mammal** and **Reptile**. Each of these child classes has their own child classes, such as **Dog** and **Lizard**. The **Dog** and **Lizard** classes inherit properties and methods from their parent classes, but they also define new properties and methods specific to them.

Unit Testing

Unit testing is a process of testing individual units or components of a software application.

Inheritance plays a vital role in writing maintainable and testable code, and unit testing can help ensure that code is functioning as intended.

Unit Testing

Example: Unit Testing with Inheritance

unittesting.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/unittesting.py (3.10.0)

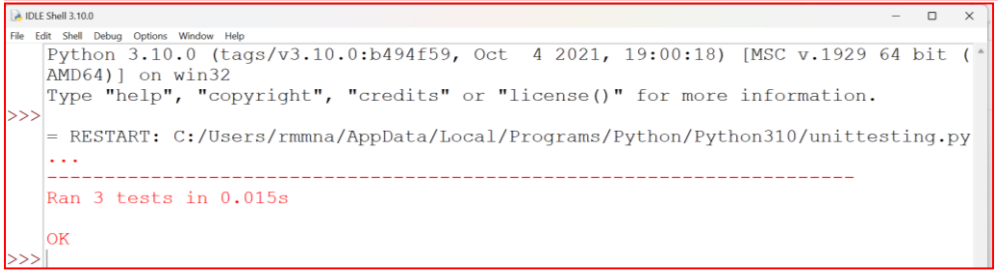
File Edit Format Run Options Window Help

```
import unittest

class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def speak(self):
        return f"{self.name} says something"

class Dog(Animal):
    def __init__(self, name):
        super().__init__(name, "Dog")

    def speak(self):
        return f"{self.name} barks"

class Cat(Animal):
    def __init__(self, name):
        super().__init__(name, "Cat")

    def speak(self):
        return f"{self.name} meows"
```

Unit Testing

Example: Unit Testing with Inheritance

```
class AnimalTests(unittest.TestCase):
    def test_animal_speak(self):
        animal = Animal("Bob", "Raccoon")
        self.assertEqual(animal.speak(), "Bob says something")

    def test_dog_speak(self):
        dog = Dog("Fido")
        self.assertEqual(dog.speak(), "Fido barks")

    def test_cat_speak(self):
        cat = Cat("Whiskers")
        self.assertEqual(cat.speak(), "Whiskers meows")

if __name__ == "__main__":
    unittest.main()
```



Unit Testing

assertEqual() in Python is a unittest library function that is used in unit testing to check the equality of two values.

This function will take three parameters as input and return a boolean value depending upon the assert condition.

If both input values are equal **assertEqual()** will return true else return false.

Unit Testing

Syntax: assertEqual(firstValue, secondValue, message)

Parameters: assertEqual() accept three parameter which are listed below with explanation:

firstValue variable of any type which is used in the comparison by function

secondValue: variable of any type which is used in the comparison by function

message: a string sentence as a message which got displayed when the test case got failed.

Unit Testing

In this example, we define a base class **Animal** and two subclasses **Dog** and **Cat**.

We also define a unit test class **AnimalTests** that tests the **speak()** method of each class.

The **test_animal_speak()** method tests the **Animal** class, while **test_dog_speak()** and **test_cat_speak()** test the **Dog** and **Cat** classes, respectively.

We use the **assertEqual()** method to check that the output of **speak()** is equal to the expected output.

OBJECT ORIENTED PROGRAMMING USING PYTHON

ERROR & EXCEPTION HANDLING

Introduction to Errors and Exceptions | Handling Exceptions

OUTLINE

- Introduction to Errors and Exceptions
- Handling Exceptions
- Multiple Except Blocks
- Multiple Exceptions in a Single Block
- Except Block Without Exception
- The else Clause

Introduction to Errors and Exceptions

- The programs that we write may behave abnormally or unexpectedly because of some errors and/or exceptions.
- The two common types of errors that we very often encounter are ***syntax errors*** and ***logic errors***.
- While ***logic errors*** occur due to poor understanding of problem and its solution, ***syntax errors***, on the other hand, arises due to poor understanding of the language.

Introduction to Errors and Exceptions

- However, *such errors* can be detected by exhaustive debugging and testing of procedures.
- But many times, we come across some peculiar problems which are often categorized as **exceptions**.
- **Exceptions** are run-time anomalies or unusual conditions (such as divide by zero, accessing arrays out of its bounds, running out of memory or disk space, overflow, and underflow) that a program may encounter during execution.

Introduction to Errors and Exceptions

- Like **errors**, **exceptions** can also be categorized as **synchronous** and **asynchronous exceptions**.
- While **synchronous exceptions** (like divide by zero, array index out of bound, etc.) can be controlled by the program, **asynchronous exceptions** (like an interrupt from the keyboard, hardware malfunction, or disk failure), on the other hand, are caused by events that are beyond the control of the program.

Introduction to Errors and Exceptions

Syntax Errors:

- **Syntax errors** occurs when we violate the rules of **Python** and they are the most common kind of error that we get while learning a new language.

For example, consider the lines of code given below.

```
>>>x=0
>>>if x == 0 print(x)
SyntaxError: invalid syntax
```

Introduction to Errors and Exceptions

Logic Error:

- **Logic error** specifies all those type of errors in which the program executes but gives incorrect results.
- **Logical error** may occur due to wrong algorithm or logic to solve a particular program.

Introduction to Errors and Exceptions

Logic Error:

- In some cases, **logic errors** may lead to divide by zero or accessing an item in a list where the index of the item is outside the bounds of the list. In this case, the **logic error** leads to a run-time error that causes the program to terminate suddenly. These types of run-time errors are known as **exceptions**.

Introduction to Errors and Exceptions

Exceptions:

- Even if a statement is syntactically correct, it may still cause an error when executed. Such errors that occur at run-time (or during execution) are known as ***exceptions***.
- An **exception** is an event, which occurs during the execution of a program and disrupts the normal flow of the program's instructions.

Introduction to Errors and Exceptions

Exceptions:

- When a program encounters a situation which it cannot deal with, it raises an exception.
- Therefore, we can say that an **exception** is a **Python object** that represents an error.
- When a program raises an exception, it must handle the **exception** or the program will be immediately terminated.

Introduction to Errors and Exceptions

Exceptions:

- We can handle exceptions in our programs to end it gracefully, otherwise, if **exceptions** are not handled by programs, then error messages are generated.

Introduction to Errors and Exceptions

Exceptions: *Examples*

```
>>>5/0
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    5/0
ZeroDivisionError: division by zero
```

Introduction to Errors and Exceptions

Exceptions: *Examples*

```
>>>var + 5
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    var + 5
NameError: name 'var' is not defined. Did you
mean: 'vars'?
```

Introduction to Errors and Exceptions

Exceptions: *Examples*

```
>>>'Python' + 3.10
Traceback (most recent call last):
  File "<pyshell#5>", line 1, in <module>
    'Python' + 3.10
TypeError: can only concatenate str (not
"float") to str
```

HANDLING EXCEPTIONS

- We can handle exceptions in our program by using **try** block and **except** block.
- A critical operation which can raise exception is placed inside the **try block** and the code that handles exception is written in **except block**.

HANDLING EXCEPTIONS

The syntax for **try-except** block can be given as,

```
try:
    statements
except ExceptionName:
    statements
```

HANDLING EXCEPTIONS

The **try** statement works as follows:

Step-1:

First, the ***try block*** (statements between the ***try*** and ***except*** keywords) is executed.

Step-2a:

If no exception occurs, the ***except block*** is skipped.

HANDLING EXCEPTIONS

Step-2b:

If an **exception** occurs, during execution of any statement in the ***try block***, then,

i) Rest of the statements in the ***try block*** are skipped.

ii) If the **exception** type matches the exception named after the ***except*** keyword, the ***except block*** is executed and then execution continues after ***try*** statement.

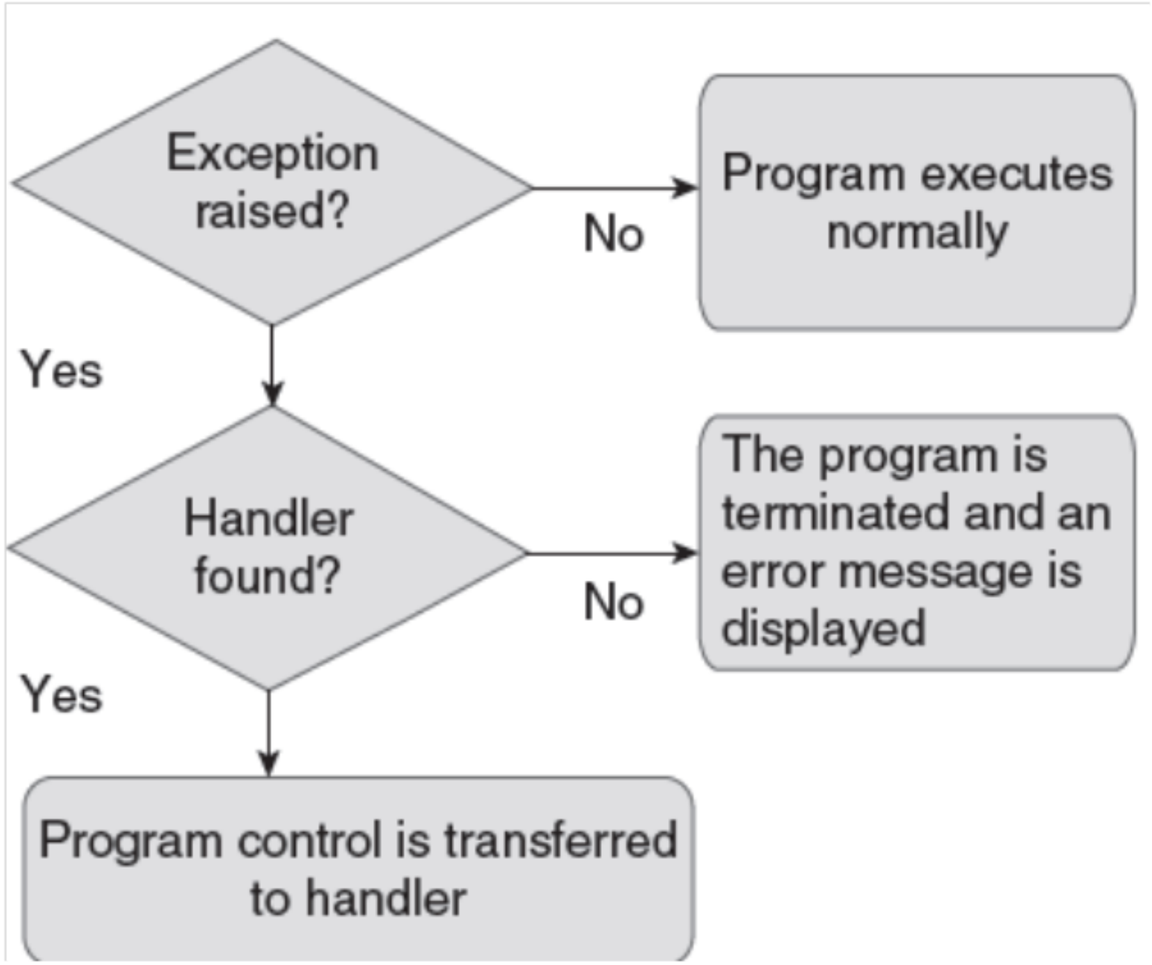
HANDLING EXCEPTIONS

Step-2b:

iii) If an **exception** occurs which does not match the **exception** named in the ***except block,*** then it is passed on to ***outer try block*** (in case of nested try blocks). If no exception handler is found in the program, then it is an unhandled exception and the program is terminated with an **error mesaage**.

HANDLING EXCEPTIONS

Step-2b:Case-iii



HANDLING EXCEPTIONS

Ex: Program to handle the divide by zero exception

```
num = int(input("Enter the numerator:"))
denom = int(input("Enter the denominator:"))
try:
    quo = num/denom
    print("Quotient:",quo)
except ZeroDivisionError:
    print("Denominator cannot be zero..")
```

HANDLING EXCEPTIONS

Ex: Program to handle the divide by zero exception

OUTPUT:

```
Enter the numerator:20
Enter the denominator:10
Quotient: 2.0
------------------------
Enter the numerator:30
Enter the denominator:0
Denominator cannot be zero..
```

MULTIPLE EXCEPT BLOCKS

- **Python** allows you to have **multiple except blocks** for a **single try block**.
- The block which matches with the **exception** generated will get executed.
- A **try block** can be associated with more than one **except block** to specify handlers for different exceptions. However, only one handler will be executed.

MULTIPLE EXCEPT BLOCKS

- **Exception handlers** only handle exceptions that occur in the corresponding **try block**.
- We can write our programs that handle selected exceptions.

MULTIPLE EXCEPT BLOCKS

The **syntax** for specifying **multiple except blocks** for a **single try block** can be given as:

```
try:
    operations are done in this block
    ..................
except Exception1:
    If there is Exception1, then execute this block.
except Exception2:
    If there is Exception2, then execute this block.
    ..................
else:
    If ther is no exception then execute this block.
    ..................
```

MULTIPLE EXCEPT BLOCKS

Ex. Program with multiple except block.

```
try:
    num = int(input("Enter the number:"))
    print(num**2)
except (KeyboardInterrupt):
        print("You shoud have entered a command from
keyboard....Program Terminating..")
except (ValueError):
        print("Please check before you enter...Program
Terminating..")
print("Bye..")
```

MULTIPLE EXCEPT BLOCKS

Ex. Program with multiple except block.

OUTPUT:

```
Enter the number:10
100
Bye..
Enter the number:"Hi"
Please check before you enter...Program Terminating..
Bye..
Enter the number:abc
Please check before you enter...Program Terminating..
Bye..
Enter the number: PRESS CTRL+C Command
You shoud have entered a command from keyboard....Program Terminating..
Bye..
```

MULTIPLE EXCEPTIONS IN A SINGLE BLOCK

- An **except clause** may name **multiple exceptions** as a parenthesized tuple as shown in the program given below.
- So whatever **exception** is raised, out of the three exceptions specified, the **same except block** will be executed.

MULTIPLE EXCEPTIONS IN A SINGLE BLOCK

Ex. Program having an except clause handling multiple exceptions simultaneously.

```
try:
    num = int(input("Enter the number:"))
    print(num**2)
except (KeyboardInterrupt, ValueError, TypeError):
     print("Please check before you
enter...Program Terminating..")
print("Bye..")
```

MULTIPLE EXCEPTIONS IN A SINGLE BLOCK

Ex. Program having an except clause handling multiple exceptions simultaneously.

OUTPUT:

```
Enter the number:10
100
Bye..
Enter the number:abc
Please check before you enter...Program Terminating..
Bye..
Enter the number:PRESS CTRL+C Command
Please check before you enter...Program Terminating..
Bye..
```

EXCEPT BLOCK WITHOUT EXCEPTION

- We can even specify an **except block** without mentioning any exception (i.e., **except:**).
- This type of **except block** if present should be the last one that can serve as a wildcard (when multiple except blocks are present). But use it with extreme caution, since it may mask a real programming error.

EXCEPT BLOCK WITHOUT EXCEPTION

- In large software programs, many times, it is difficult to anticipate all types of possible exceptional conditions.
- Therefore, the programmer may not be able to write a different handler (**except block**) for every individual type of exception.
- In such situations, a better idea is to write a **handler** that would catch all types of exceptions.

EXCEPT BLOCK WITHOUT EXCEPTION

The **syntax** to define a **handler** that would catch every possible **exception** from the **try block** is:

```
try:
    Write the operations here
    ..................
except:
    If there is any exception, then execute this block.
    ..................
else:
    If there is no exception then execute this block.
```

EXCEPT BLOCK WITHOUT EXCEPTION

Ex. Program to demonstrate the use of except: block

```
try:
    #f1 = open('File1.txt','r')
    str1 = f1.readlines()
    print(str1)
except IOError:
    print("Error occured during Input...Program
Terminated..")
except ValueError:
    print("Could not convert data to an integer..")
except:
    print("Unexpected Error..Program Terminating..")
```

EXCEPT BLOCK WITHOUT EXCEPTION

Ex. Program to demonstrate the use of except: block

OUTPUT:

```
By including f1 = open('File1.txt','r')
['Welcome to the course\n', 'Object Oriented
Porgramming using Python']
By excluding #f1 = open('File1.txt','r')
Unexpected Error..Program Terminating..
```

THE else CLAUSE

- The **try … except** block can optionally have an **else clause**, which, when present, must follow all except blocks.
- The statement(s) in the **else block** is executed only if the **try clause** does not **raise** an **exception**.

THE else CLAUSE

Ex. Program to demonstrate else block.

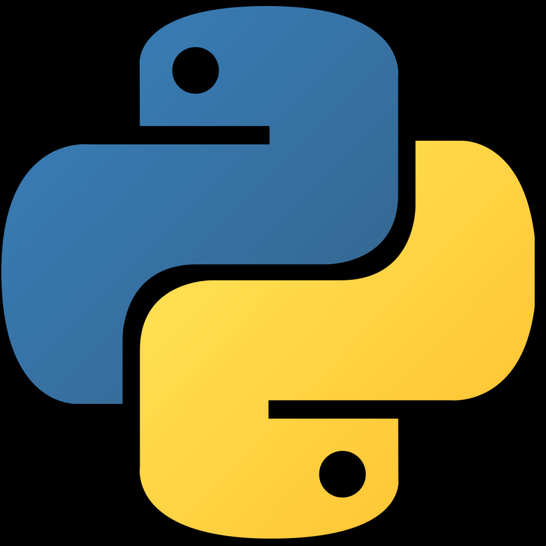
```
try:
    f1 = open('File1.txt')
    str1 = f1.readline()
    print(str1)
except IOError:
    print("Error occured during Input...Program Terminated..")
except:
    print("Error occured..Program Terminated..")
else:
    print("Program Terminated successfully..")
```

THE else CLAUSE

Ex. Program to demonstrate else block.

OUTPUT:

```
By including f1 = open('File1.txt')
Welcome to the course


Program Terminated successfully..
```



OBJECT ORIENTED PROGRAMMING USING PYTHON

ERROR & EXCEPTION HANDLING

Raising Exceptions | Built-in and User-defined Exceptions

OUTLINE

Raising Exceptions

Instantiating Exceptions

Handling Exceptions in Invoked Functions

Built-in and User-defined Exceptions

RAISING EXCEPTIONS

- We can deliberately **raise an exception** using the **raise** keyword.

The general syntax for the **raise** statement is:

```
raise [Exception [, args [, traceback]]]
```

RAISING EXCEPTIONS

```
raise [Exception [, args [, traceback]]]
```

- Here, **Exception** is the name of exception to be raised (example, **TypeError**).
- **args** is optional and specifies a value for the exception argument.
- If **args** is not specified, then the exception argument is None.
- The final argument, ***traceback***, is also optional and if present, is the ***traceback*** object used for the exception .

RAISING EXCEPTIONS

Ex1. Program to deliberately raise an exception

```
try:
    num = 10
    print(num)
    raise ValueError
except:
    print("Exception occurred...Program Terminating..")
```

OUTPUT:

```
10
Exception occurred...Program Terminating..
```

RAISING EXCEPTIONS

Ex2. Program to re-raise an exception

```
try:
    raise NameError
except:
    print("Re-Raising the Exception..")
    raise
```

OUTPUT:

```
Re-Raising the Exception..
Traceback (most recent call last):
  File "F:/RMM OOPP UNIT-5/Exceptions-Re-Raise.py", line 3, in
<module>
    raise NameError
NameError
```

INSTANTIATING EXCEPTIONS

- **Python** allows programmers to **instantiate an exception** first before raising it and add any attributes (or arguments) to it as desired.
- These attributes can be used to give additional information about the error.
- To i**nstantiate the exception**, the **except block** may specify a variable after the exception name.

INSTANTIATING EXCEPTIONS

- The variable then becomes an **exception instance** with the arguments stored in **instance.args**.
- The **exception instance** also has the **__str__()** method defined so that the arguments can be printed directly without using **instance.args**.
- The contents of the argument vary based on exception type.

INSTANTIATING EXCEPTIONS

Ex1. Program to understand the process of instantiating an exception.

```
try:
    raise Exception("Hello", "World")
except Exception as errorObj:
    print(type(errorObj)) # the exception instance
    print(errorObj.args) # arguments stored in .args
    print(errorObj) # __str__() allows args to be printed
directly
    arg1, arg2 = errorObj.args
    print("Argument1 = ", arg1)
    print("Argument2 = ", arg2)
```

INSTANTIATING EXCEPTIONS

Ex1. Program to understand the process of instantiating an exception.

OUTPUT:

```
<class 'Exception'>
('Hello', 'World')
('Hello', 'World')
Argument1 =  Hello
Argument2 =  World
```

Note:

If we raise an exception with arguments but do not handle it, then the name of the exception is printed along with its arguments.

INSTANTIATING EXCEPTIONS

Ex2. Program to raise an exception with arguments

```
try:
    raise Exception("Hello", "World")
except ValueError:
    print("Program Terminating..")
```

OUTPUT:

```
Traceback (most recent call last):
  File "F:/RMM OOPP UNIT-5/Exceptions-rasie with
arguments.py", line 3, in <module>
    raise Exception("Hello", "World")
Exception: ('Hello', 'World')
```

HANDLING EXCEPTIONS IN INVOKED FUNCTIONS

Exceptions can also be handled inside functions that are called in the **try block** as shown in the program given below:

Ex1.Program to handle exceptions from an invoked function

```
def Divide(num, denom):
    try:
        quo = num/denom
        print("Quotient = :",quo)
    except ZeroDivisionError:
        print("You cannot divide a number by zero..Program
terminating..")
Divide(20,0)
Divide(20,4)
```

HANDLING EXCEPTIONS IN INVOKED FUNCTIONS

Exceptions can also be handled inside functions that are called in the **try block** as shown in the program given below:

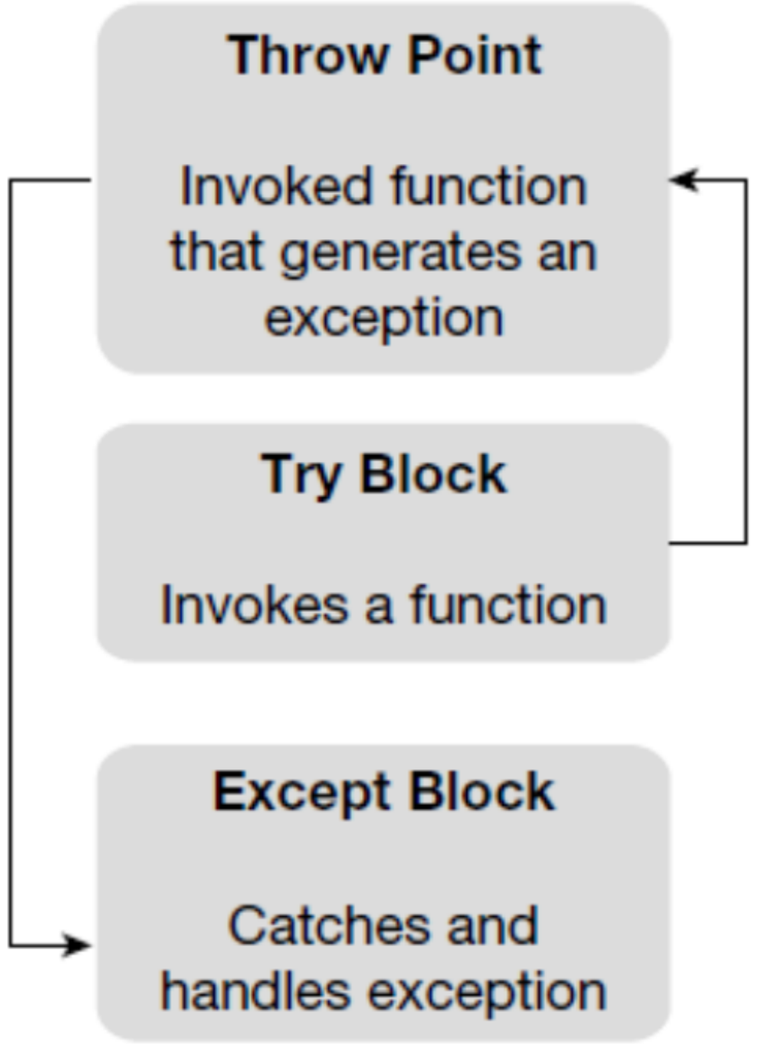
Ex1.Program to handle exceptions from an invoked function

OUTPUT:

```
You cannot divide a number by zero..Program terminating..
Quotient = : 5.0
```

HANDLING EXCEPTIONS IN INVOKED FUNCTIONS

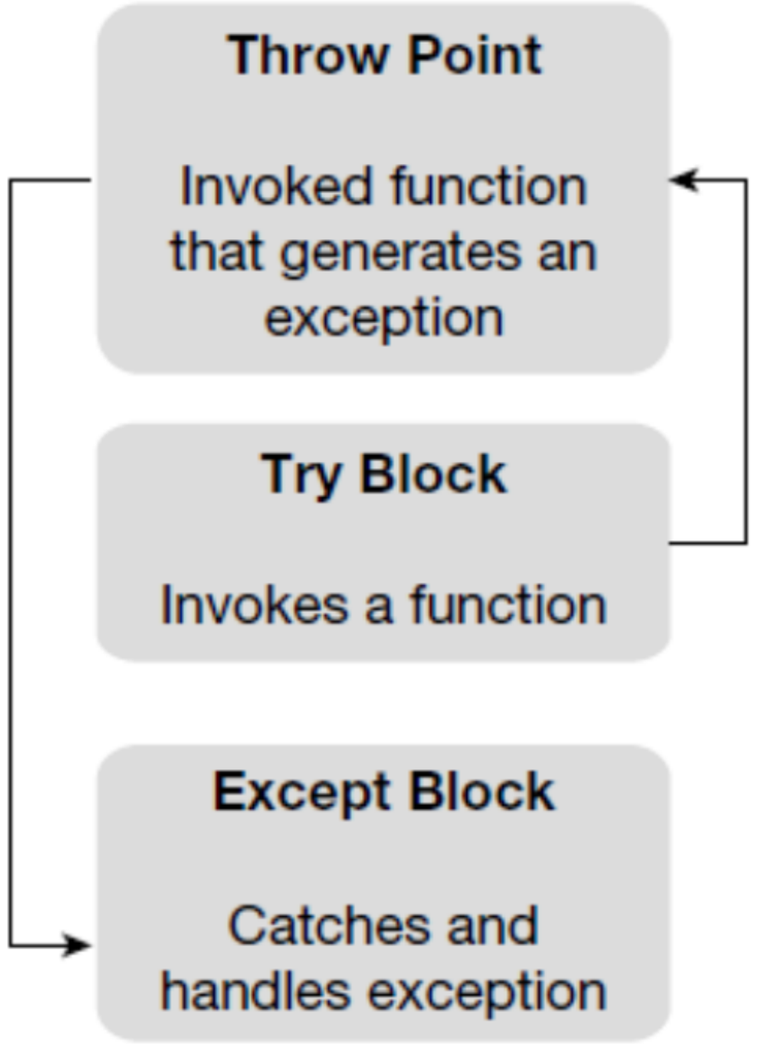
- Basically, a large program is usually divided into **n** number of functions.
- The possibility that the **invoked function** may generate an exceptional condition cannot be ignored.
- The figure, shows the scenario when the function invoked by the **try block** throws an exception which is handled by the **except block** in the calling function.



HANDLING EXCEPTIONS IN INVOKED FUNCTIONS

The syntax of such situation can be given as:

```
function_name(arg list):
    -------
    -------
try:
    ----
    function_name() //Function call
    ----
except ExceptionName:
    ------
    # Code to handle exception
    ------
```



HANDLING EXCEPTIONS IN INVOKED FUNCTIONS

Ex.Program to handle exceptions in the calling function

```
def Divide(num, denom):
    try:
        quo = num/denom
        print("Quotient = :",quo)
    except ZeroDivisionError:
        print("You cannot divide a number by zero..Program
terminating..")
Divide(20,0)
Divide(20,4)
```

OUTPUT:

```
Quotient = 2.857142857142857
You cannot divide a number by zero..Program Terminating..
```

BUILT-IN AND USER-DEFINED EXCEPTIONS

Exceptions are categorized into two types:

1. Built-in Exceptions
2. User-defined Exceptions

Built-in Exceptions:

The **following table** lists some **standard exceptions** that are already defined in **Python**. These **built-in exceptions** force our program to output an error when something in it goes wrong.

BUILT-IN AND USER-DEFINED EXCEPTIONS

Built-in Exceptions:

Exception	Description
`Exception`	Base class for all exceptions
`StopIteration`	Generated when the `next()` method of an iterator does not point to any object
`SystemExit`	Raised by `sys.exit()` function
`StandardError`	Base class for all built-in exceptions (excluding `StopIteration` and `SystemExit`)
`ArithmeticError`	Base class for errors that are generated due to mathematical calculations
`OverflowError`	Raised when the maximum limit of a numeric type is exceeded during a calculation
`FloatingPointError`	Raised when a floating point calculation could not be performed
`ZeroDivisionError`	Raised when a number is divided by zero
`AssertionError`	Raised when the assert statement fails
`AttributeError`	Raised when attribute reference or assignment fails
`EOFError`	Raised when end-of-file is reached or there is no input for `input()` function
`ImportError`	Raised when an import statement fails
`KeyboardInterrupt`	Raised when the user interrupts program execution (by pressing Ctrl+C)
`LookupError`	Base class for all lookup errors
`IndexError`	Raised when an index is not found in a sequence
`KeyError`	Raised when a key is not found in the dictionary
`NameError`	Raised when an identifier is not found in local or global namespace (referencing a non-existent variable)
`UnboundLocalError`, `EnvironmentError`	Raised when an attempt is made to access a local variable in a function or method when no value has been assigned to it.

BUILT-IN AND USER-DEFINED EXCEPTIONS

Built-in Exceptions:

IOError	Raised when input or output operation fails (for example, opening a file that does not exist)
SyntaxError	Raised when there is a syntax error in the program
IndentationError	Raised when there is an indentation problem in the program
SystemError	Raised when an internal system error occurs
ValueError	Raised when the arguments passed to a function are of invalid data type or searching a list for a non-existent value
RuntimeError	Raised when the generated error does not fall into any of the above category
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not implemented
TypeError	Raised when two or more data types are mixed without coercion

BUILT-IN AND USER-DEFINED EXCEPTIONS

User-defined Exceptions:

- **Python** allows programmers to create their **own exceptions** by creating a new exception class.
- The new exception class is derived from the base class Exception which is pre-defined in **Python**.

BUILT-IN AND USER-DEFINED EXCEPTIONS

User-defined Exceptions:

Ex. Program to define a user-defined exception

```
class myError(Exception):
    def __init__(self, val):
        self.val = val
    def __str__(self):
        return repr(self.val)
try:
    raise myError(30)
except myError as e:
    print("User-defined Exception Generated with value:
",e.val)
```

BUILT-IN AND USER-DEFINED EXCEPTIONS

User-defined Exceptions:

Ex. Program to define a user-defined exception

OUTPUT:

```
User-defined Exception Generated with value:  30
```

- In this program, the **__init__()** method of Exception class has been overridden by the new class.
- The customized exception class can be used to perform any task.
- However, these classes are usually kept simple and have only limited attributes to provide information about the error to be extracted by handlers for the exception.

BUILT-IN AND USER-DEFINED EXCEPTIONS

User-defined Exceptions:

- Creating our **own exception class** or defining a **user-defined exception** is known as ***custom exception***.

Note: An **exception** can be a string, a class, or an object. Most of the **exceptions** raised by **Python** are classes, with an argument that is an instance of the class.

- **as** is a keyword that allow programmers to name a variable within an **except statement**.

BUILT-IN AND USER-DEFINED EXCEPTIONS

User-defined Exceptions:

- When creating a **module** that can raise **different exceptions**, a better approach would be to create a **base class** for **exceptions** defined by that **module**, and **subclasses** to create specific exception classes for different error conditions.

BUILT-IN AND USER-DEFINED EXCEPTIONS

User-defined Exceptions:

Ex2. Program to create sub-classes for Exception class to handle exceptions in a better customized way

```
class Error(Exception):
    def message(self):
        raise NotImplementedError()
class InputError(Error):
    def __init__(self, expr, msg):
        self.expr = expr
        self.msg = msg
    def message(self):
        print("Error in input in Expression..")
        print(self.expr)
```

BUILT-IN AND USER-DEFINED EXCEPTIONS

User-defined Exceptions:

Ex2. Program to create sub-classes for Exception class to handle exceptions in a better customized way

```
try:
    a = input("Enter the value of a:")
    raise InputError("input(\"Enter the value of a:s\")","Input Error")
except InputError as ie:
    ie.message()
```

OUTPUT:

```
Enter the value of a:20
Error in input in Expression..
input("Enter the value of a:s")
```

BUILT-IN AND USER-DEFINED EXCEPTIONS

User-defined Exceptions:

- Although there is no **naming convention** for naming a **user-defined exception**, it is better to define **exceptions** with names that end in "**Error**", to make it consistent with the naming of the standard exceptions.

Note: Many standard modules define their **own exceptions** to report **errors** that may occur in functions they define.

UNIT- II

Polymorphism and Inheritance: cont'd.

- Exceptions – Example Program using Inheritance

Exceptions

Example:

exceptions with inheritance.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/exceptions with inheritance.py (3.10.0)

File Edit Format Run Options Window Help

```
class BankAccount:
    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount

class SavingsAccount(BankAccount):
    def __init__(self, account_number, balance, interest_rate):
        super().__init__(account_number, balance)
        self.interest_rate = interest_rate

    def add_interest(self):
        interest = self.balance * self.interest_rate
        self.balance += interest
```

Exceptions

Example: cont'd.

```
try:
    account = SavingsAccount("123456789", 1000, 0.05)
    account.withdraw(1500)
except ValueError as e:
    print(e)
```

```
>>>
    = RESTART: C:/Users/rmmna/AppData/Local/Programs/Python/Python310/exceptions wit
    h inheritance.py
    Insufficient funds
>>>
```

Exceptions

Example: cont'd.

In this example, we define a **BankAccount** class and a subclass **SavingsAccount**.

The **SavingsAccount** class has an additional method **add_interest()** that adds interest to the balance of the account.

We also define an exception to be raised if the user attempts to withdraw more money than is available in the account.

We use a **try-except** block to catch the exception and print out the error message.

This example demonstrates how exceptions can be used to handle errors in an inheritance hierarchy.

Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- Basics of Open Source Libraries for
  - Data pre-processing
  - Modeling and
  - Visualization

Python Libraries

- **Python** is one of the most popular languages used by data scientists and software developers alike for data science tasks.
- It can be used to predict outcomes, automate tasks, streamline processes, and offer business intelligence insights.
- There are quite a few open-source libraries that make Python data tasks much easier.

Python Libraries

The following are the most important libraries for data science tasks available in the Python ecosystem covering areas such as data processing, modeling, and visualization:

Data Mining

1. Scrapy
2. BeautifulSoup

Python Libraries

Data Processing and Modeling

3. NumPy (Numerical Python)
4. SciPy (Scientific Python)
5. Pandas (Python Data Analysis)
6. Keras
7. SciKit-Learn
8. PyTorch
9. TensorFlow
10. XGBoost

Python Libraries

Data Visualization

11. Matplotlib
12. Seaborn
13. Bokeh
14. Plotly
15. Pydot

Python Libraries

Scrapy

- One of the most popular Python data science libraries, Scrapy helps to build crawling programs (spider bots) that can retrieve structured data from the web – for example, URLs or contact info. It's a great tool for scraping data used in, for example, Python machine learning models.
- Developers use it for gathering data from APIs. This full-fledged framework follows the Don't Repeat Yourself principle in the design of its interface. As a result, the tool inspires users to write universal code that can be reused for building and scaling large crawlers.

Python Libraries

BeautifulSoup

- BeautifulSoup is another really popular library for web crawling and data scraping.
- If you want to collect data that's available on some website but not via a proper CSV or API, BeautifulSoup can help you scrape it and arrange it into the format you need.

Python Libraries

NumPy

- NumPy (Numerical Python) is a perfect tool for scientific computing and performing basic and advanced array operations.
- This library offers many handy features performing operations on n-arrays and matrices in Python.
- It helps to process arrays that store values of the same data type and makes performing math operations on arrays (and their vectorization) easier.
- In fact, the vectorization of mathematical operations on the NumPy array type increases performance and accelerates the execution time.

Python Libraries

SciPy

- This useful library includes modules for linear algebra, integration, optimization, and statistics.
- Its main functionality was built upon NumPy, so its arrays make use of this library.
- SciPy works great for all kinds of scientific programming projects (science, mathematics, and engineering).
- It offers efficient numerical routines such as numerical optimization, integration, and others in submodules.
- The extensive documentation makes working with this library really easy.

Python Libraries

Pandas

- Pandas is a library created to help developers work with "labeled" and "relational" data intuitively.
- It's based on two main data structures: "Series" (one-dimensional, like a list of items) and "Data Frames" (two-dimensional, like a table with multiple columns).
- Pandas allows converting data structures to DataFrame objects, handling missing data, and adding/deleting columns from DataFrame, assigning missing files, and plotting data with histogram or plot box. It’s a must-have for data wrangling, manipulation, and visualization.

Python Libraries

Keras

- Keras is a great library for building neural networks and modeling.
- It's very straightforward to use and provides developers with a good degree of extensibility.
- The library takes advantage of other packages, (Theano or TensorFlow) as its backends.
- Moreover, Microsoft integrated CNTK (Microsoft Cognitive Toolkit) to serve as another backend.
- It's a great pick if you want to experiment quickly using compact systems – the minimalist approach to design really pays off!

Python Libraries

SciKit-Learn

- This is an industry-standard for data science projects based in Python.
- Scikits is a group of packages in the SciPy Stack that were created for specific functionalities – for example, image processing.
- Scikit-learn uses the math operations of SciPy to expose a concise interface to the most common machine learning algorithms.
- Data scientists use it for handling standard machine learning and data mining tasks such as clustering, regression, model selection, dimensionality reduction, and classification.
- Another advantage: It comes with quality documentation and offers high performance.

Python Libraries

PyTorch

- PyTorch is a framework that is perfect for data scientists who want to perform deep learning tasks easily.
- The tool allows performing tensor computations with GPU (Graphics Processing Unit) acceleration.
- It's also used for other tasks – for example, for creating dynamic computational graphs and calculating gradients automatically.
- PyTorch is based on Torch, which is an open-source deep learning library implemented in C, with a wrapper in Lua (Lua is a lightweight, high-level, multi-paradigm programming language designed primarily for embedded use in applications).

Python Libraries

TensorFlow

- TensorFlow is a popular Python framework for machine learning and deep learning, which was developed at Google Brain.
- It's the best tool for tasks like object identification, speech recognition, and many others.
- It helps in working with artificial neural networks that need to handle multiple data sets.
- The library includes various layer-helpers (tflearn, tf-slim, skflow), which make it even more functional.
- TensorFlow is constantly expanded with its new releases – including fixes in potential security vulnerabilities or improvements in the integration of TensorFlow and GPU.

Python Libraries

XGBoost

- Use this library to implement machine learning algorithms under the Gradient Boosting framework.
- XGBoost is portable, flexible, and efficient.
- It offers parallel tree boosting that helps teams to resolve many data science problems.
- Another advantage is that developers can run the same code on major distributed environments such as Hadoop, SGE (Sun Grid Engine), and MPI (Message Passing Interface).

Python Libraries

Matplotlib

- This is a standard data science library that helps to generate data visualizations such as two-dimensional diagrams and graphs (histograms, scatterplots, non-Cartesian coordinates graphs).
- Matplotlib is one of those plotting libraries that are really useful in data science projects - it provides an object-oriented API for embedding plots into applications.
- Python can compete with scientific tools like MatLab or Mathematica.
- However, developers need to write more code than usual while using this library for generating advanced visualizations.
- Note that popular plotting libraries work seamlessly with Matplotlib.

Python Libraries

Seaborn

- Seaborn is based on Matplotlib and serves as a useful Python machine learning tool for visualizing statistical models – heatmaps and other types of visualizations that summarize data and depict the overall distributions.
- When using this library, you get to benefit from an extensive gallery of visualizations (including complex ones like time series, joint plots, and violin diagrams).

Python Libraries

Bokeh

- This library is a great tool for creating interactive and scalable visualizations inside browsers using JavaScript widgets.
- Bokeh is fully independent of Matplotlib.
- It focuses on interactivity and presents visualizations through modern browsers – similarly to Data-Driven Documents (d3.js).
- D3.js is a JavaScript library for producing dynamic, interactive data visualizations in web browsers. It makes use of Scalable Vector Graphics, HTML5, and Cascading Style Sheets standards.
- It offers a set of graphs, interaction abilities (like linking plots or adding JavaScript widgets), and styling.

Python Libraries

Plotly

- This web-based tool for data visualization that offers many useful out-of-box graphics – you can find them on the Plot.ly website (https://plotly.com/).
- The library works very well in interactive web applications.
- Its creators are busy expanding the library with new graphics and features for supporting multiple linked views, animation, and crosstalk integration.

Python Libraries

pydot

- This library helps to generate oriented and non-oriented graphs.
- It serves as an interface to Graphviz (written in pure Python).
- You can easily show the structure of graphs with the help of this library.
- That comes in handy when you're developing algorithms based on neural networks and decision trees.

Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- Numpy
  - Introduction
  - Installing Numpy
  - Arrays with Numpy
  - Create an array

Numpy

- **NumPy** (**Numerical Python**) is an open source Python library that's used in almost every field of science and engineering.
- It's the universal standard for working with numerical data in Python, and it's at the core of the scientific Python and PyData ecosystems (PyData - A community for developers and users of open source data tools).
- NumPy users include everyone from beginning coders to experienced researchers doing state-of-the-art scientific and industrial research and development.
- The NumPy API is used extensively in Pandas, SciPy, Matplotlib, scikit-learn, scikit-image and most other data science and scientific Python packages.

Numpy

- The **NumPy** library contains multidimensional array and matrix data structures.
- It provides ndarray, a homogeneous n-dimensional array object, with methods to efficiently operate on it.
- NumPy can be used to perform a wide variety of mathematical operations on arrays.
- It adds powerful data structures to Python that guarantee efficient calculations with arrays and matrices and it supplies an enormous library of high-level mathematical functions that operate on these arrays and matrices.

Numpy

- **NumPy** can be easily interfaced with other Python packages and provides tools for integrating with other programming languages like C, C++ etc.

Advantages of numpy

- Fundamental package for scientific computing with Python.
- N-dimensional array object.
- Linear algebra, Fourier transform, random number capabilities.
- Building block for other packages (e.g. Scipy)
- Open source

Installing NumPy

To install NumPy, we strongly recommend using a scientific Python distribution.

If you already have Python, you can install NumPy with:

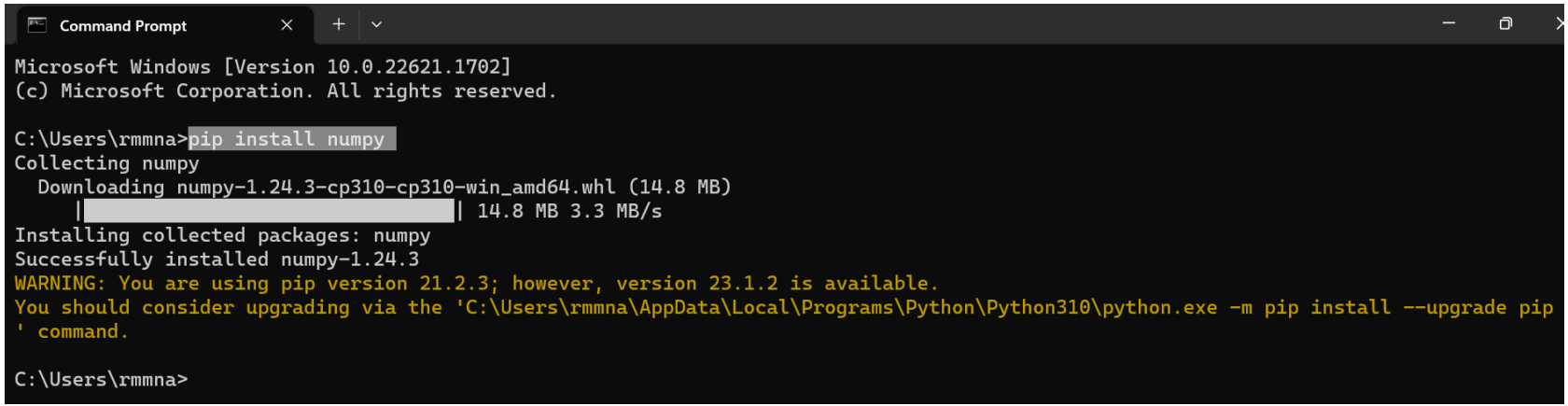
`pip install numpy # pip` is a package manager for Python packages, or modules

or

`conda install numpy # Conda` is a cross platform package and environment manager that installs and manages conda packages from the Anaconda repository as well as from the Anaconda Cloud.

How to install NumPy using official python IDLE?

- Press the Windows key on your keyboard.
- Type CMD and open Command Prompt. A black terminal should open up.
- Type `pip install numpy` and hit enter.
- It should start the installation. After you see the "Successfully Installed" message, go back to your IDLE and try importing numpy, it should work.



How to import NumPy

To access numpy and its functions import it in your Python code like this:

```
import numpy as np
```

We shorten the imported name to np for better readability of code using numpy. This is a widely adopted convention that you should follow so that anyone working with your code can easily understand it.

Arrays with numpy

- A NumPy array would require much less memory to store the same amount of data compared to a Python list, which helps in reading and writing from the array in a faster manner.

Attributes of an ndarray object

- NumPy's array class is called ndarray.
- It is also known by the alias array.
- Note that numpy.array is not the same as the Standard Python Library class array.array, which only handles one-dimensional arrays and offers less functionality.

Attributes of an ndarray object

The more important attributes of an ndarray object are:

`ndarray.ndim`

- the number of axes (dimensions) of the array.

`ndarray.shape`

- the dimensions of the array.
- This is a tuple of integers indicating the size of the array in each dimension.
- For a matrix with `n` rows and `m` columns, shape will be (`n,m`).
- The length of the shape tuple is therefore the number of axes, `ndim`.

Attributes of an ndarray object

`ndarray.size`

- the total number of elements of the array.
- This is equal to the product of the elements of shape.

`ndarray.dtype`

- an object describing the type of the elements in the array.
- One can create or specify dtype's using standard Python types.
- Additionally **`NumPy`** provides types of its own.
- **`numpy.int32`**, **`numpy.int16`**, and **`numpy.float64`** are some examples.

Attributes of an ndarray object

`ndarray.itemsize`

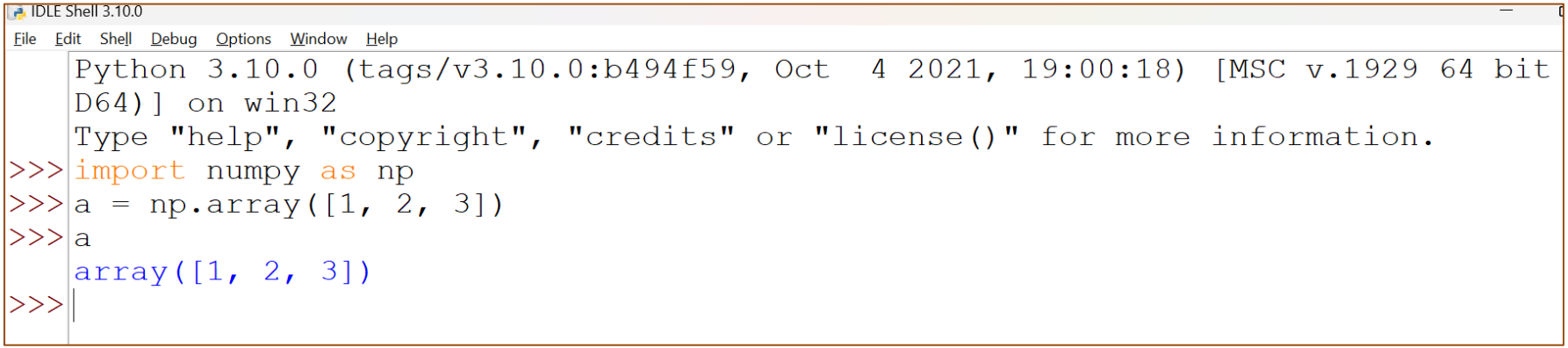
- the size in bytes of each element of the array.
- For example, an array of elements of type float64 has itemsize **`8`** (**`=64/8`**), while one of type complex32 has itemsize **`4`** (**`=32/8`**). It is equivalent to **`ndarray.dtype.itemsize`**.

`ndarray.data`

- the buffer containing the actual elements of the array.
- Normally, we won't need to use this attribute because we will access the elements in an array using indexing facilities.

Creating an array

- To create a NumPy array, you can use the function np.array().
- All you need to do to create a simple array is pass a list to it.
- If you choose to, you can also specify the type of data in your list.



Creating an array

Besides creating an array from a sequence of elements, you can easily create an array filled with 0's:

```
>>>np.zeros(2)
array([0., 0.])
>>>np.zeros((3, 4))
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

Creating an array

An array filled with 1's:

```
>>> np.ones(2)
array([1., 1.])
>>> np.ones((2, 3, 4), dtype=np.int16)
array([[[1, 1, 1, 1],
        [1, 1, 1, 1],
        [1, 1, 1, 1]],

       [[1, 1, 1, 1],
        [1, 1, 1, 1],
        [1, 1, 1, 1]]], dtype=int16)
```

Creating an array

Create an empty array :

- The function empty creates an array whose initial content is random and depends on the state of the memory.
- The reason to use empty over zeros (or something similar) is speed - just make sure to fill every element afterwards!

```
>>>import numpy as np
>>> np.empty(2)
array([-4.03016855e-113,  1.22438294e+210])
```

Creating an array

Create an empty array : Ex-2

```
>>>import numpy as np
>>> np.empty((2, 3))
array([[1.5, 2.5, 3. ],
       [4. , 5. , 6. ]])
```

Creating an array

Create an array with a range of elements:

To create sequences of numbers, NumPy provides the arange function which is analogous to the Python built-in range, but returns an array.

```
>>>import numpy as np
>>> np.arange(4)
array([0, 1, 2, 3])
>>> np.arange(10, 30, 5)
array([10, 15, 20, 25])
>>> np.arange(0, 2, 0.3)# it accepts float arguments
array([0. , 0.3, 0.6, 0.9, 1.2, 1.5, 1.8])
```

Creating an array

Create an array that contains a range of evenly spaced intervals:

To do this, you will specify the first number, last number, and the step size.

```
>>>import numpy as np
>>> np.arange(2, 9, 2)
array([2, 4, 6, 8])
```

Creating an array

Create an array that contains a range of evenly spaced intervals:

You can also use np.linspace() to create an array with values that are spaced linearly in a specified interval.

```
>>>import numpy as np
>>> np.linspace(0, 10, num=5)
array([ 0. ,  2.5,  5. ,  7.5, 10. ])
>>> np.linspace(0, 2, 9)
array([0.  , 0.25, 0.5 , 0.75, 1.  , 1.25, 1.5 ,
1.75, 2.  ])
```

Creating an array

Specifying your data type:

While the default data type is floating point (np.float64), you can explicitly specify which data type you want using the dtype keyword.

```
>>>import numpy as np
>>> x = np.ones(2, dtype=np.int64)
>>> x
array([1, 1], dtype=int64)
>>> y = np.zeros(2, dtype=np.int64)
>>> y
array([0, 0], dtype=int64)
```

Creating an array

Specifying your data type: Ex-1

While the default data type is floating point (np.float64), you can explicitly specify which data type you want using the dtype keyword.

```
>>>import numpy as np
>>> x = np.ones(2, dtype=np.int64)
>>> x
array([1, 1], dtype=int64)
>>> y = np.zeros(2, dtype=np.int64)
>>> y
array([0, 0], dtype=int64)
```

Creating an array

Specifying your data type: Ex-2

```
>>>import numpy as np
>>> a = np.array([[1, 2], [3, 4]], dtype=complex)
>>> a
array([[1.+0.j, 2.+0.j],
       [3.+0.j, 4.+0.j]])
```

Creating an array

Specifying your data type: cont'd.. Ex-3

```
>>>import numpy as np
>>> x = np.array([1, 2, 3])
>>> x
array([1, 2, 3])
>>> x.dtype
dtype('int32')
>>> y = np.array([1.0, 2.5, 3.5])
>>> y
array([1. , 2.5, 3.5])
>>> y.dtype
dtype('float64')
```

Creating an array

Transforming sequences of sequences into two-dimensional arrays, and three-dimensional arrays:

Array transforms sequences of sequences into two-dimensional arrays, sequences of sequences of sequences into three-dimensional arrays, and so on.

```
>>>import numpy as np
>>> z = np.array([(1.5, 2.5, 3), (4, 5, 6)])
>>> z
array([[1.5, 2.5, 3. ],
       [4. , 5. , 6. ]])
```

Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- Numpy
  - Printing arrays
  - Basic Operations

https://numpy.org/doc/stable/user/quickstart.html#quickstart-array-creation

Printing Arrays

- When you print an array, NumPy displays it in a similar way to nested lists, but with the following layout:
  - the last axis is printed from left to right,
  - the second-to-last is printed from top to bottom,
  - the rest are also printed from top to bottom, with each slice separated from the next by an empty line.
- One-dimensional arrays are then printed as rows, bidimensionals as matrices and tridimensionals as lists of matrices.

Printing Arrays

Example: 1d array

```
>>> import numpy as np
>>> a = np.arange(5) # 1d array
>>> print(a)
[0 1 2 3 4]
>>> a
array([0, 1, 2, 3, 4])
```

Printing Arrays

Example: 2d array

```
>>> import numpy as np
>>> b = np.arange(12).reshape(4, 3) # 2d array
>>> print(b)
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
>>> b
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

Printing Arrays

Example: 3d array

```
>>> import numpy as np
>>> c = np.arange(24).reshape(2, 3, 4)  # 3d array
>>> print(c)
[[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
```

Printing Arrays

numpy.reshape

Gives a new shape to an array without changing its data.

```
Syntax:
numpy.reshape(a, newshape, order='C')
Parameters:
a: array_like # Array to be reshaped.
newshape:int or tuple of ints
```

The newshape should be compatible with the original shape. If an integer, then the result will be a 1-D array of that length. One shape dimension can be -1. In this case, the value is inferred from the length of the array and remaining dimensions.

Printing Arrays

numpy.reshape

Syntax:

```
numpy.reshape(a, newshape, order='C')
```

Parameters:

order: *{'C', 'F', 'A'}, optional*

Read the elements of `a` using this index order, and place the elements into the reshaped array using this index order.

`'C'` means to read / write the elements using C-like index order, with the last axis index changing fastest, back to the first axis index changing slowest.

`'F'` means to read / write the elements using Fortran-like index order, with the first index changing fastest, and the last index changing slowest.

`'A'` means to read / write the elements in Fortran-like index order if a is Fortran contiguous in memory, C-like order otherwise.

Printing Arrays

If an array is too large to be printed, NumPy automatically skips the central part of the array and only prints the corners:

```
>>> import numpy as np
>>> print(np.arange(10000))
[   0    1    2 ... 9997 9998 9999]
>>> print(np.arange(10000).reshape(100, 100))
[[   0    1    2 ...   97   98   99]
 [ 100  101  102 ...  197  198  199]
 [ 200  201  202 ...  297  298  299]
 ...
 [9700 9701 9702 ... 9797 9798 9799]
 [9800 9801 9802 ... 9897 9898 9899]
 [9900 9901 9902 ... 9997 9998 9999]]
```

Printing Arrays

To disable this behaviour and force NumPy to print the entire array, you can change the printing options using set_printoptions.

```
>>>    np.set_printoptions(threshold=sys.maxsize)
# sys module should be imported
```

Basic Operations

- Arithmetic operators on arrays apply *elementwise*.
- A new array is created and filled with the result.

```
>>> import numpy as np
>>> a=np.array([10, 20, 30, 40])
>>> a
array([10, 20, 30, 40])
>>> b = np.arange(4)
>>> b
array([0, 1, 2, 3])
```

Basic Operations

```
>>> a=np.array([10, 20, 30, 40])
>>> a
array([10, 20, 30, 40])
>>> b = np.arange(4)
>>> b
array([0, 1, 2, 3])
>>> c=a-b
>>> c
array([10, 19, 28, 37])
>>> d=a+b
>>> d
array([10, 21, 32, 43])
```

Basic Operations

```
>>> a=np.array([10, 20, 30, 40])
>>> a
array([10, 20, 30, 40])
>>> b = np.arange(4)
>>> b
array([0, 1, 2, 3])
>>> b**2
array([0, 1, 4, 9])
>>> 10*np.sin(a)
array([-5.44021111,    9.12945251,  -9.88031624,
7.4511316 ])
```

Basic Operations

```
>>> a=np.array([10, 20, 30, 40])
>>> a
array([10, 20, 30, 40])
>>> b = np.arange(4)
>>> b
array([0, 1, 2, 3])
>>> a<30
array([ True,  True, False, False])
>>> a>30
array([False, False, False,  True])
```

Basic Operations

- The product operator * operates elementwise in NumPy arrays.
- The matrix product can be performed using the @ operator (in python >=3.5) or the dot function or method:

```
>>> import numpy as np
>>> A = np.array([[1, 1],
                  [0, 1]])
>>> A
array([[1, 1],
       [0, 1]])
>>> B = np.array([[2, 0],
                  [3, 4]])
>>> B
array([[2, 0],
       [3, 4]])
```

Basic Operations

```
>>> A * B # elementwise product
array([[2, 0],
       [0, 4]])
>>> A @ B # matrix product
array([[5, 4],
       [3, 4]])
>>> A.dot(B)# another matrix product
array([[5, 4],
       [3, 4]])
```

Basic Operations

- Some operations, such as += and *=, act in place to modify an existing array rather than create a new one:

```
>>> import numpy as np
>>> rg = np.random.default_rng(1)# create instance of
default random number generator
>>> a = np.ones((2, 3), dtype=int)
>>> a
array([[1, 1, 1],
       [1, 1, 1]])
>>> b = rg.random((2, 3))
>>> b
array([[0.51182162, 0.9504637 , 0.14415961],
       [0.94864945, 0.31183145, 0.42332645]])
```

Basic Operations

```
>>> a *= 3
>>> a
array([[3, 3, 3],
       [3, 3, 3]])
>>> b += a
>>> b
array([[3.51182162, 3.9504637 , 3.14415961],
       [3.94864945, 3.31183145, 3.42332645]])
>>> a += b # b is not automatically converted to integer type
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    a += b # b is not automatically converted to integer type
numpy.core._exceptions._UFuncOutputCastingError: Cannot cast ufunc 'add' output from dtype('float64') to dtype('int32') with casting rule 'same_kind'
```

Basic Operations

- When operating with arrays of different types, the type of the resulting array corresponds to the more general or precise one (a behavior known as upcasting):

```
>>> import numpy as np
>>> import math
>>> a = np.ones(3, dtype=np.int32)
>>> a
array([1, 1, 1])
>>> a
array([1, 1, 1])
>>> a.dtype.name
'int32'
```

Basic Operations

```
>>> b = np.linspace(0, math.pi, 3)
>>> b
array([0.        , 1.57079633, 3.14159265])
>>> b.dtype.name
'float64'
>>> c = a + b
>>> c
array([1.        , 2.57079633, 4.14159265])
>>> c.dtype.name
'float64'
```

Basic Operations

```
>>> d = np.exp(c * 1j)
>>> d
array([           0.54030231+0.84147098j,           -
0.84147098+0.54030231j,
       -0.54030231-0.84147098j])
>>> d.dtype.name
'complex128'
```

Basic Operations

- Many unary operations, such as computing the sum of all the elements in the array, are implemented as methods of the ndarray class:

```
>>> import numpy as np
>>> a = rg.random((2, 3))
>>> a.sum()
3.105710952998157
>>> a.min()
0.027559113243068367
>>> a.max()
0.8277025938204418
```

Basic Operations

- By default, these operations apply to the array as though it were a list of numbers, regardless of its shape.
- However, by specifying the axis parameter you can apply an operation along the specified axis of an array:

```
>>> import numpy as np
>>> b = np.arange(12).reshape(3, 4)
>>> b
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

Basic Operations

```
>>> import numpy as np
>>> b = np.arange(12).reshape(3, 4)
>>> b
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
>>> b.sum(axis=0)      # sum of each column
array([12, 15, 18, 21])
>>> b.min(axis=1)      # min of each row
array([0, 4, 8])
>>> b.cumsum(axis=1)  # cumulative sum along each row
array([[ 0,  1,  3,  6],
       [ 4,  9, 15, 22],
       [ 8, 17, 27, 38]])
```

Universal Functions

- NumPy provides familiar mathematical functions such as sin, cos, and exp.
- In NumPy, these are called “universal functions” (ufunc).
- Within NumPy, these functions operate elementwise on an array, producing an array as output.

```
>>> import numpy as np
```

Universal Functions

```
>>> import numpy as np
>>> B = np.arange(3)
>>> B
array([0, 1, 2])
>>> np.exp(B)
array([1.        , 2.71828183, 7.3890561 ])
>>> np.sqrt(B)
array([0.        , 1.        , 1.41421356])
>>> C = np.array([2., -1., 4.])
>>> C
array([ 2., -1.,  4.])
>>> np.add(B, C)
array([2., 0., 6.])
```

Universal Functions: cross product

cross product of two (arrays of) vectors:

- Return the cross product of two (arrays of) vectors.

Syntax:

```
numpy.cross(a, b, axisa=-1, axisb=-1, axisc=-1, axis=None)
```

Universal Functions: cross product

`numpy.cross(a, b, axisa=-1, axisb=-1, axisc=-1, axis=None)`

Parameters:

a: array_like # Components of the first vector(s)

b: array_like # Components of the second vector(s)

axisa: *int, optional*

Axis of a that defines the vector(s). By default, the last axis.

axisb: *int, optional*

Axis of b that defines the vector(s). By default, the last axis.

axisc: *int, optional*

Axis of c containing the cross product vector(s). Ignored if both input vectors have dimension 2, as the return is scalar. By default, the last axis.

axis: *int, optional*

If defined, the axis of a, b and c that defines the vector(s) and cross product(s). Overrides *axisa*, *axisb* and *axisc*.

Returns: *c: ndarray* # Vector cross product(s)

Raises: *ValueError*

When the dimension of the vector(s) in a and/or b does not equal 2 or 3.

Universal Functions: cross product

cross product of two (arrays of) vectors:

Assuming we have vector A with elements (A1, A2, A3) and vector B with elements (B1, B2, B3), we can calculate the cross product of these two vectors as:

```
Cross Product = [(A2*B3) – (A3*B2), (A3*B1) – (A1*B3), (A1*B2) – (A2*B1)]
```

Example:

Vector A: (1, 2, 3)

Vector B: (4, 5, 6)

Cross Product = [(A2*B3) – (A3*B2), (A3*B1) – (A1*B3), (A1*B2) – (A2*B1)]

= [(2*6) – (3*5), (3*4) – (1*6), (1*5) – (2*4)]

= (-3, 6, -3)

Universal Functions: cross product

cross product of two (arrays of) vectors:

Example program: *Vector cross-product*

```
>>> import numpy as np
>>> x = [1, 2, 3]
>>> x
[1, 2, 3]
>>> y = [4, 5, 6]
>>> y
[4, 5, 6]
>>> np.cross(x, y)
array([-3,  6, -3])
```

Universal Functions: cross product

cross product of two (arrays of) vectors:

Example program: *Vector cross-product*

One vector with dimension 2:

```
>>> import numpy as np
>>> x = [1, 2]
>>> x
[1, 2]
>>> y = [4, 5, 6]
>>> y
[4, 5, 6]
>>> np.cross(x, y)
array([12, -6, -3])
```

Universal Functions: cross product

cross product of two (arrays of) vectors:

Example program: *Vector cross-product*

Equivalently:

```
>>> import numpy as np
>>> x = [1, 2, 0]
>>> x
[1, 2,0]
>>> y = [4, 5, 6]
>>> y
[4, 5, 6]
>>> np.cross(x, y)
array([12, -6, -3])
```

Universal Functions: cross product

cross product of two (arrays of) vectors:

Example program: *Vector cross-product*

Both vectors with dimension 2:

```
>>> import numpy as np
>>> x = [1, 2]
>>> x
[1, 2]
>>> y = [4, 5]
>>> y
[4, 5]
>>> np.cross(x, y)
array(-3)
```

Universal Functions: cross product

cross product of two (arrays of) vectors:

Example program: *Vector cross-product*

Multiple vector cross-products.

Note that the direction of the cross product vector is defined by the right-hand rule:

```
>>> import numpy as np
>>> x = np.array([[1,2,3], [4,5,6]])
>>> y = np.array([[4,5,6], [1,2,3]])
>>> np.cross(x, y)
array([[-3,  6, -3],
       [ 3, -6,  3]])
```

Universal Functions: cross product

cross product of two (arrays of) vectors:

Example program: *Vector cross-product*

Multiple vector cross-products.

The orientation of c can be changed using the axisc keyword.

```
>>> import numpy as np
>>> x = np.array([[1,2,3], [4,5,6]])
>>> y = np.array([[4,5,6], [1,2,3]])
>>> np.cross(x, y, axisc=0)
array([[-3,  3],
       [ 6, -6],
       [-3,  3]])
```

Universal Functions: cross product

cross product of two (arrays of) vectors:

Example program: *Vector cross-product*

Multiple vector cross-products.

Change the vector definition of x and y using *axisa* and *axisb*.

```
>>> import numpy as np
>>> x = np.array([[1,2,3], [4,5,6], [7, 8, 9]])
>>> y = np.array([[7, 8, 9], [4,5,6], [1,2,3]])
>>> np.cross(x, y)
array([[ -6,  12,  -6],
       [  0,   0,   0],
       [  6, -12,   6]])
```

Universal Functions: cross product

cross product of two (arrays of) vectors:

Example program: *Vector cross-product*

Multiple vector cross-products.

Change the vector definition of x and y using *axisa* and *axisb*.

```
>>> import numpy as np
>>> x = np.array([[1,2,3], [4,5,6], [7, 8, 9]])
>>> y = np.array([[7, 8, 9], [4,5,6], [1,2,3]])
>>> np.cross(x, y, axisa=0, axisb=0)
array([[-24,  48, -24],
       [-30,  60, -30],
       [-36,  72, -36]])
```

Universal Functions: Others

all, any, apply_along_axis, argmax, argmin, argsort, average, bincount, ceil, clip, conj, corrcoef, cov, cross, cumprod, cumsum, diff, dot, floor, inner, invert, lexsort, max, maximum, mean, median, min, minimum, nonzero, outer, prod, re, round, sort, std, sum, trace, transpose, var, vdot, vectorize, where

Refer:

https://numpy.org/doc/stable/user/quickstart.html#quickstart-array-creation

Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- Numpy
  - Indexing, Slicing and Iterating
  - QR Decomposition

Indexing, Slicing and Iterating

- One-dimensional arrays can be indexed, sliced and iterated over, much like lists and other Python sequences.
- NumPy indexing is used for accessing an element from an array by giving it an index value that starts from 0.
- Slicing NumPy arrays means extracting elements from an array in a specific range. It obtains a substring, subtuple, or sublist from a string, tuple, or list.
- Negative Slicing index values start from the end of the array.
- Indexing starts from 0 and slicing is performed using indexing.

Indexing an Array

- Indexing is used to access individual elements.
- It is also possible to extract entire rows, columns, or planes from multi-dimensional arrays with numpy indexing.
- Indexing starts from 0.

Element of array	2	3	11	9	6	4	10	12
Index	0	1	2	3	4	5	6	7

Indexing using Index Arrays

- In NumPy arrays, when arrays are used as indexes to access groups of elements, this is called indexing using index arrays.
- NumPy arrays can be indexed with arrays or with any other sequence like a list, etc.

Indexing using Index Arrays

Example:

```
>>> import numpy as np
>>> a=np.arange(1,10,2)
>>> a
array([1, 3, 5, 7, 9])
>>> print("Elements of array: ",a)
Elements of array:  [1 3 5 7 9]
>>> a1=a[np.array([4,0,2,-1,-2])]
>>> print("Indexed Elements of array a: ",a1)
Indexed Elements of array a:  [9 1 5 9 7]
```

Indexing in one dimension Array

One-dimensional arrays can be indexed, sliced and iterated over, much like lists and other Python sequences.

```
>>> import numpy as np
>>> a=np.arange(1,10,2)
>>> a
array([1, 3, 5, 7, 9])
>>> print("Elements of array: ",a)
Elements of array:  [1 3 5 7 9]
>>> a[0]
1
```

Indexing in one dimension Array

```
>>> a
array([1, 3, 5, 7, 9])
>>> a[2]
5
>>> a[4]
9
>>> a[-1]
9
>>> a[-2]
7
```

Indexing in one dimension Array

```
>>> a
array([1, 3, 5, 7, 9])
>>> a[1:3]# slicing - excludes a[3]
array([3, 5])
>>> a[0:4] # slicing - excludes a[4]
array([1, 3, 5, 7])
>>>  a[:4:2]=200  #  from  start  to  position  4,
exclusive, set every 2nd element to 200
>>> a
array([200, 3, 200, 7, 9])
```

Indexing in one dimension Array

```
>>> a
array([1, 3, 5, 7, 9])
>>> a[::-1]  # reversed a
array([9, 7, 5, 3, 1])
>>> for i in a: # iteration
    print(i**2)
1
9
25
49
81
```

Indexing in Multidimensional Arrays

- Multidimensional arrays can have one index per axis.
- These indices are given in a tuple separated by commas.

```
>>> import numpy as np
>>> def f(x, y):
        return 10 * x + y
>>> b = np.fromfunction(f, (5, 4), dtype=int)
>>> b
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [30, 31, 32, 33],
       [40, 41, 42, 43]])
```

Indexing in Multidimensional Arrays

```
>>> b
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [30, 31, 32, 33],
       [40, 41, 42, 43]])
>>> b[2, 3]
23
>>> b[1,2]
12
>>> b[0,1]
1
```

Indexing in Multidimensional Arrays

```
>>> b
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [30, 31, 32, 33],
       [40, 41, 42, 43]])
>>> b[0:5, 1]  # each row in the second column of b
array([ 1, 11, 21, 31, 41])
>>> b[:, 1]    # equivalent to the previous one
array([ 1, 11, 21, 31, 41])
>>> b[1:3, :]  # each column in the second and third row of b
array([[10, 11, 12, 13],
       [20, 21, 22, 23]])
```

Indexing in Multidimensional Arrays

```
>>> b
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [30, 31, 32, 33],
       [40, 41, 42, 43]])
>>> b[-1]  # The last row. Equivalent to b[-1, :]
array([40, 41, 42, 43])
```

Indexing in Multidimensional Arrays

Iterating over multidimensional arrays is done with respect to the first axis:

```
>>> b
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [30, 31, 32, 33],
       [40, 41, 42, 43]])
>>> for row in b:
        print(row)
 [0 1 2 3]
[10 11 12 13]
[20 21 22 23]
[30 31 32 33]
[40 41 42 43]
```

Indexing in Multidimensional Arrays

However, if one wants to perform an operation on each element in the array, one can use the flat attribute which is an iterator over all the elements of the array:

```
>>> b
array([[ 0,  1,  2,  3],
       [10, 11, 12, 13],
       [20, 21, 22, 23],
       [30, 31, 32, 33],
       [40, 41, 42, 43]])
>>> for element in b.flat:
        print(element)
0
1
2
3
10
..
```

Indexing in Multidimensional Arrays

```
>>> c = np.array([[[  0,  1,  2],
                   [ 10, 12, 13]],
                   [[100, 101, 102],
                   [110, 112, 113]]])
# a 3D array (two stacked 2D arrays)
>>> c
array([[[  0,   1,   2],
        [ 10,  12,  13]],
       [[100, 101, 102],
        [110, 112, 113]]])
```

Indexing in Multidimensional Arrays

```
>>> c = np.array([[[  0,  1,  2],
                   [ 10, 12, 13]],
                   [[100, 101, 102],
                   [110, 112, 113]]])
>>> c.shape
(2, 2, 3)
>>> c[1, ...]  # same as c[1, :, :] or c[1]
array([[100, 101, 102],
       [110, 112, 113]])
>>> c[..., 2]  # same as c[:, :, 2]
array([[  2,  13],
       [102, 113]])
```

QR Decomposition (or Factorization) of a Matrix

- QR factorization of a matrix is the decomposition of a matrix say 'A' into 'A=QR' where Q is orthogonal and R is an upper-triangular matrix.
- We can calculate the QR decomposition of a given matrix with the help of `numpy.linalg.qr()`.

Syntax:

```
numpy.linalg.qr(a, mode='reduced')
```

Parameters:

a: *array_like, shape (..., M, N)*

An array-like object with the dimensionality of at least 2.

mode: *{'reduced', 'complete', 'r', 'raw'}, optional*

The mode is set by default to return the results of the reduced type.

QR Decomposition (or Factorization) of a Matrix

Syntax:

```
numpy.linalg.qr(a, mode='reduced')
```

Parameters:

mode: *{'reduced', 'complete', 'r', 'raw'}, optional*

The mode is set by default to return the results of the reduced type.

- For an input matrix of dimension MxN, the reduced mode results in Q and R matrices of dimensions MxK & KxN respectively.
- If the complete mode is being used for instance, then the Q and R matrices shall be returned with dimensions MxM & MxN respectively.
- Declaring an r mode shall return only the R matrix with KxN dimension.
- If one chooses the raw mode h & tau are returned with the dimensions NxM & K respectively. The array h contains the Householder reflectors that generate q along with r. The tau array contains scaling factors for the reflectors. In the deprecated 'economic' mode only h is returned.

QR Decomposition (or Factorization) of a Matrix

Example-1: Write NumPy program to calculate the QR decomposition of a given matrix.

```
>>> import numpy as np
# Define the input matrix
>>> A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
# Compute the QR decomposition of the matrix
>>> Q, R = np.linalg.qr(A)
# Print the Q and R matrices
>>> print("Q matrix:")
>>> print(Q)
>>> print("R matrix:")
>>> print(R)
```

QR Decomposition (or Factorization) of a Matrix

Example-1: Write NumPy program to calculate the QR decomposition of a given matrix.

Output:

```
>>> A
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
Q matrix:
[[-0.12309149  0.90453403  0.40824829]
 [-0.49236596  0.30151134 -0.81649658]
 [-0.86164044 -0.30151134  0.40824829]]
R matrix:
[[-8.12403840e+00 -9.60113630e+00 -1.10782342e+01]
 [ 0.00000000e+00  9.04534034e-01  1.80906807e+00]
 [ 0.00000000e+00  0.00000000e+00 -8.88178420e-16]]
```

QR Decomposition (or Factorization) of a Matrix

Example-2: Write NumPy program to calculate the QR decomposition of a given matrix.

```
import numpy as np
# Original matrix
matrix1 = np.array([[1, 2, 3], [3, 4, 5]])
print(matrix1)
# Decomposition of the said matrix
q, r = np.linalg.qr(matrix1)
print('\nQ:\n', q)
print('\nR:\n', r)
```

QR Decomposition (or Factorization) of a Matrix

Example-2: Write NumPy program to calculate the QR decomposition of a given matrix.

```
Output:
[[1 2 3]
 [3 4 5]]
Q:
 [[-0.31622777 -0.9486833 ]
 [-0.9486833   0.31622777]]
R:
 [[-3.16227766 -4.42718872 -5.69209979]
 [ 0.         -0.63245553 -1.26491106]]
```

Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- Pandas
  - Introduction
  - Pandas features
  - Installing Pandas
  - Object Creation
  - Viewing Data

Pandas (Python Data Analysis)

- Pandas is powerful Python data analysis toolkit
- Pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.
- Pandas is a Python package (library) designed to make working with "relational" or "labeled" data both easy and in-built.
- It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.
- Additionally, it has the broader goal of becoming the most powerful and flexible open source data analysis / manipulation tool available in any language.

Pandas - Main Features

- Easy handling of missing data (represented as `NaN`, `NA`, or `NaT`) in floating point as well as non-floating point data.
- Size mutability: columns can be inserted and deleted from DataFrame and higher dimensional objects.
- Automatic and explicit data alignment: objects can be explicitly aligned to a set of labels, or the user can simply ignore the labels and let `Series`, `DataFrame`, etc. automatically align the data for you in computations.
- Powerful, flexible group by functionality to perform split-apply-combine operations on data sets, for both aggregating and transforming data.

Pandas - Main Features

- Make it easy to convert ragged, differently-indexed data in other Python and NumPy data structures into DataFrame objects.
- Intelligent label-based slicing, fancy indexing, and subsetting of large data sets.
- Intuitive merging and joining data sets.
- Flexible reshaping and pivoting (rotating) of data sets.
- Hierarchical labeling of axes (possible to have multiple labels per tick).

Pandas - Main Features

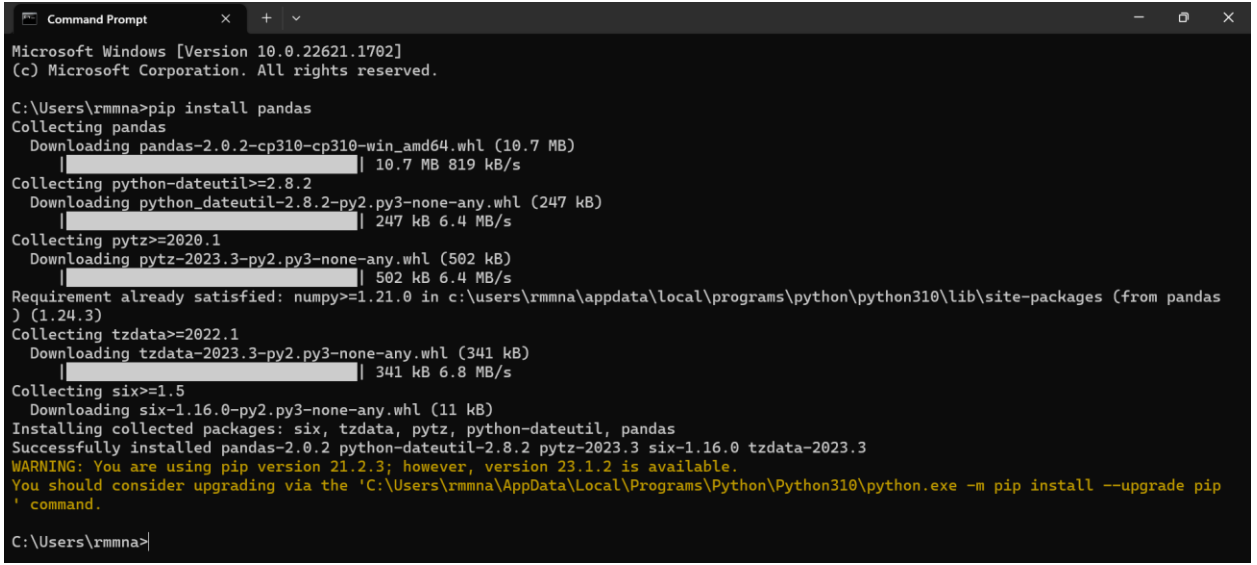
- Robust IO tools for loading data from flat files (CSV and delimited), Excel files, databases, and saving/loading data from the ultrafast HDF5 format (Hierarchical Data Format).
- Time series-specific functionality: date range generation and frequency conversion, moving window statistics, date shifting and lagging.

Installing Pandas

```
# conda
conda install pandas
# PyPI  (Python Package Index )
pip install pandas
```

How to install Pandas using official python IDLE?

- Press the Windows key on your keyboard.
- Type CMD and open Command Prompt. A black terminal should open up.
- Type pip install pandas and hit enter.
- It should start the installation. After you see the "Successfully Installed" message, go back to your IDLE and try importing pandas, it should work.



How to import Pandas

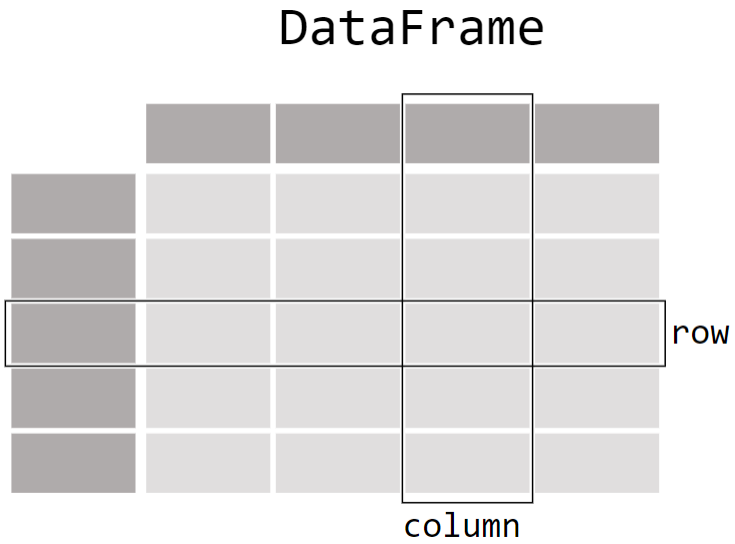
- To access pandas and its functions import it in your Python code like this:

```
import pandas as pd
```

- We shorten the imported name to pd for better readability of code using pandas. This is a widely adopted convention that you should follow so that anyone working with your code can easily understand it.
- https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html#min
- https://pandas.pydata.org/docs/getting_started/index.html

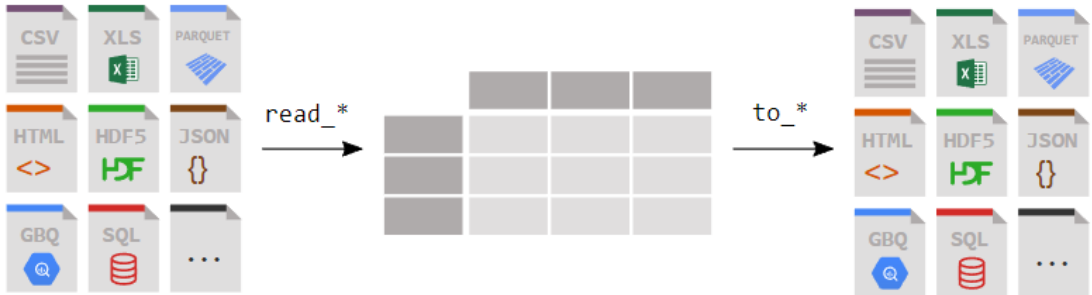
What kind of data does pandas handle?

- When working with tabular data, such as data stored in spreadsheets or databases, pandas is the right tool for us.
- Pandas will help us to explore, clean, and process your data.
- In pandas, a data table is called a DataFrame.



How do we read and write tabular data?

- Pandas supports the integration with many file formats or data sources out of the box (csv, excel, sql, json - JavaScript Object Notation , parquet-open source, column-oriented data file format,…).
- Importing data from each of these data sources is provided by function with the prefix `read_*`. Similarly, the `to_*` methods are used to store data.

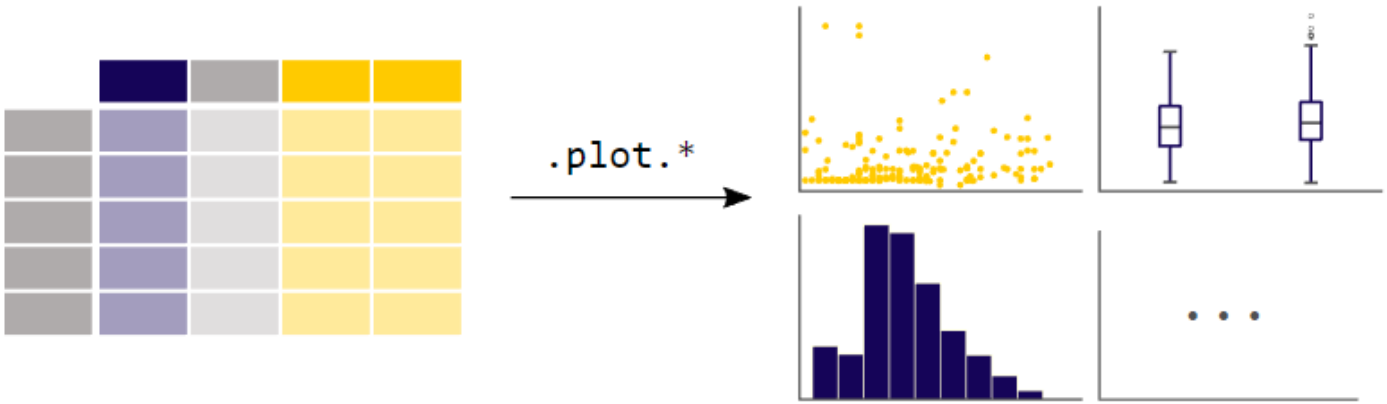


How do we select a subset of a table?

- Selecting or filtering specific rows and/or columns?
- Filtering the data on a condition?
- Methods for slicing, selecting, and extracting the data we need are available in pandas.

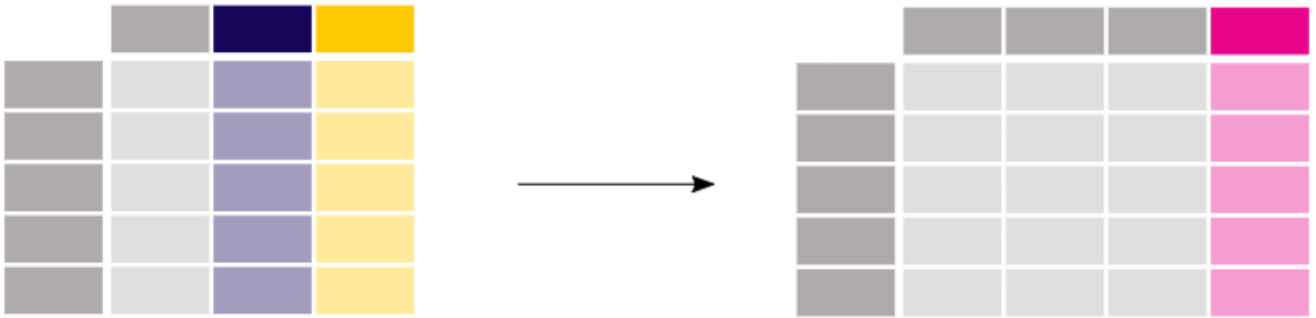
How to create plots in pandas?

- Pandas provides plotting our data out of the box, using the power of Matplotlib.
- We can pick the plot type (scatter, bar, boxplot,…) corresponding to our data.



How to create new columns derived from existing columns?

- There is no need to loop over all rows of our data table to do calculations.
- Data manipulations on a column work elementwise.
- Adding a column to a DataFrame based on existing data in other columns is straightforward.

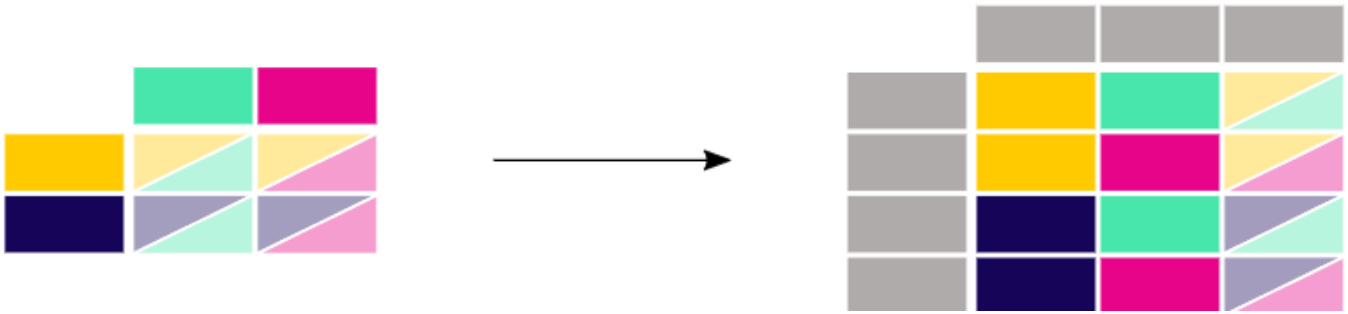


How to calculate summary statistics?

- Basic statistics (mean, median, min, max, counts…) are easily calculable.
- These or custom aggregations can be applied on the entire data set, a sliding window of the data, or grouped by categories.
- The latter is also known as the split-apply-combine approach.

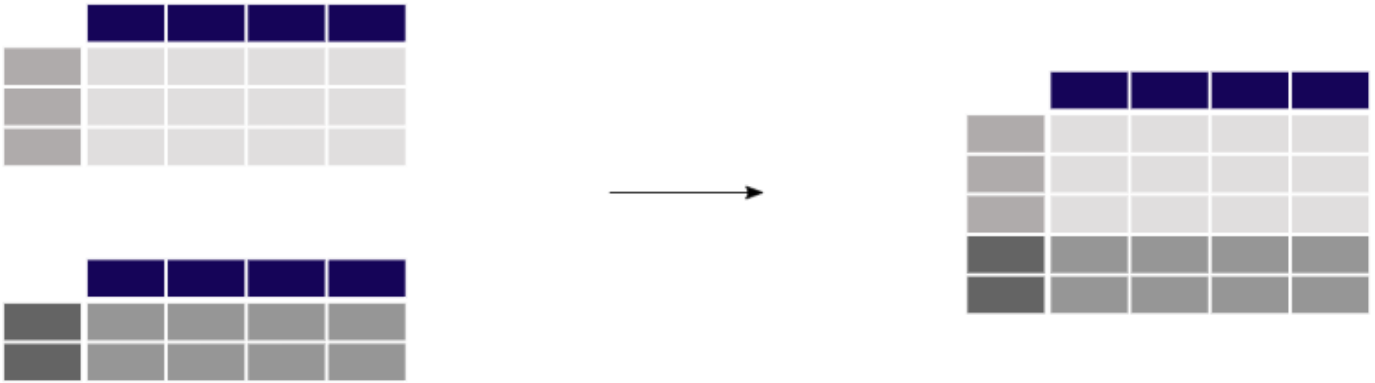
How to reshape the layout of tables?

- Change the structure of our data table in multiple ways.
- We can `melt()` our data table from wide to long/tidy form or `pivot()` from long to wide format.
- With aggregations built-in, a pivot table is created with a single command.



How to combine data from multiple tables?

- Multiple tables can be concatenated both column wise and row wise as database-like join/merge operations are provided to combine multiple tables of data.



How to handle time series data?

- Pandas has great support for time series and has an extensive set of tools for working with dates, times, and time-indexed data.

How to manipulate textual data?

- Data sets do not only contain numerical data.
- Pandas provides a wide range of functions to clean textual data and extract useful information from it.

Quick overview of pandas?

- https://pandas.pydata.org/docs/user_guide/10min.html#min

Object Creation

Creating a Series by passing a list of values, letting pandas create a default integer index:

```
>>> import numpy as np
>>> import pandas as pd
>>> s = pd.Series([1, 3, 5, np.nan, 6, 8])# nan - not a number
>>> s
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
dtype: float64
```

Object Creation

Creating a DataFrame by passing a NumPy array, with a datetime index using date_range() and labeled columns:

```
>>> import numpy as np
>>> import pandas as pd
>>> dates = pd.date_range("20130101", periods=6)
>>> dates
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-
03', '2013-01-04', '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')
```

Object Creation

Creating a DataFrame by passing a NumPy array, with a datetime index using date_range() and labeled columns: cont'd.

```
>>> df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
>>> df
```

	A	B	C	D
2013-01-01	-0.472821	1.541007	1.390524	-0.404981
2013-01-02	-0.229180	1.769937	-0.119053	-0.794888
2013-01-03	3.368224	0.708559	-0.251711	0.702893
2013-01-04	-0.397939	0.539684	1.556475	-0.326841
2013-01-05	1.853638	0.400270	1.168184	-0.363158
2013-01-06	-0.997194	-0.503115	0.190057	0.547042

Object Creation

Creating a DataFrame by passing a dictionary of objects that can be converted into a series-like structure:

```
>>> import numpy as np
>>> import pandas as pd
>>> df2 = pd.DataFrame(
    {
        "A": 1.0,
        "B": pd.Timestamp("20130102"),
        "C": pd.Series(1, index=list(range(4)), dtype="float32"),
        "D": np.array([3] * 4, dtype="int32"),
        "E": pd.Categorical(["test", "train", "test", "train"]),
        "F": "foo",
    }
)
```

Object Creation

Creating a DataFrame by passing a dictionary of objects that can be converted into a series-like structure:

```
>>> df2
     A           B   C    D      E    F
0  1.0 2013-01-02  1.0  3   test  foo
1  1.0 2013-01-02  1.0  3  train  foo
2  1.0 2013-01-02  1.0  3   test  foo
3  1.0 2013-01-02  1.0  3  train  foo
```

Object Creation

Creating a DataFrame by passing a dictionary of objects that can be converted into a series-like structure:

- The columns of the resulting DataFrame have different dtypes:

```
>>> df2.dtypes
A           float64
B    datetime64[ns]
C           float32
D             int32
E          category
F            object
dtype: object
```

Viewing Data

Use DataFrame.head() and DataFrame.tail() to view the top and bottom rows of the frame respectively:

```
>>> import numpy as np
>>> import pandas as pd
>>> dates = pd.date_range("20230615", periods=6)
>>> dates
DatetimeIndex(['2023-06-15', '2023-06-16', '2023-06-17',
'2023-06-18', '2023-06-19', '2023-06-20'],
dtype='datetime64[ns]', freq='D')
```

Viewing Data

Use DataFrame.head() and DataFrame.tail() to view the top and bottom rows of the frame respectively:

```
>>> df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
>>> df
                   A         B         C         D
2023-06-15  1.647283 -0.659818  0.058287  0.522661
2023-06-16 -0.699142 -0.078138 -1.802049 -0.753840
2023-06-17 -0.652668  0.755713 -1.039440 -0.678967
2023-06-18  0.625232  0.112527 -1.274081  0.724884
2023-06-19 -1.719933  1.858491  0.713219 -0.283876
2023-06-20  1.419357  0.952211 -0.146506 -1.337652
```

Viewing Data

Use DataFrame.head() and DataFrame.tail() to view the top and bottom rows of the frame respectively:

```
>>> df.head()
                   A         B         C         D
2023-06-15  1.647283 -0.659818  0.058287  0.522661
2023-06-16 -0.699142 -0.078138 -1.802049 -0.753840
2023-06-17 -0.652668  0.755713 -1.039440 -0.678967
2023-06-18  0.625232  0.112527 -1.274081  0.724884
2023-06-19 -1.719933  1.858491  0.713219 -0.283876
```

Viewing Data

Use DataFrame.head() and DataFrame.tail() to view the top and bottom rows of the frame respectively:

```
>>> df.tail(3)
                   A         B         C         D
2023-06-18  0.625232  0.112527 -1.274081  0.724884
2023-06-19 -1.719933  1.858491  0.713219 -0.283876
2023-06-20  1.419357  0.952211 -0.146506 -1.337652
```

Viewing Data

Display the DataFrame.index or DataFrame.columns:

```
>>> df.index
DatetimeIndex(['2023-06-15',  '2023-06-16',  '2023-06-17',
'2023-06-18', '2023-06-19', '2023-06-20'],
              dtype='datetime64[ns]', freq='D')
>>> df.columns
Index(['A', 'B', 'C', 'D'], dtype='object')
```

Viewing Data

DataFrame.to_numpy() gives a NumPy representation of the underlying data.

Note that this can be an expensive operation when your DataFrame has columns with different data types, which comes down to a fundamental difference between pandas and NumPy: **NumPy arrays have one dtype for the entire array, while pandas DataFrames have one dtype per column**. When you call DataFrame.to_numpy(), **pandas** will find the NumPy dtype that can hold *all* of the dtypes in the DataFrame. This may end up being object, which requires casting every value to a Python object.

Viewing Data

For df, our DataFrame of all floating-point values, and DataFrame.to_numpy() is fast and doesn't require copying data:

```
>>> df.to_numpy()
array([[ 1.64728338, -0.65981759,  0.05828652,  0.52266126],
       [-0.69914212, -0.07813804, -1.80204948, -0.7538402 ],
       [-0.65266814,  0.7557127 , -1.03944031, -0.67896746],
       [ 0.6252324 ,  0.11252683, -1.27408074,  0.7248838 ],
       [-1.71993319,  1.85849098,  0.7132188 , -0.28387634],
       [ 1.41935705,  0.95221056, -0.14650593, -1.33765214]])
```

Viewing Data

For df2, the DataFrame with multiple dtypes, DataFrame.to_numpy() is relatively expensive:

```
>>> df2
    A      B         C   D    E    F
0  1.0 2013-01-02  1.0  3   test  foo
1  1.0 2013-01-02  1.0  3  train  foo
2  1.0 2013-01-02  1.0  3   test  foo
3  1.0 2013-01-02  1.0  3  train  foo
>>> df2.to_numpy()
array([[1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'test', 'foo'],
       [1.0, Timestamp('2013-01-02 00:00:00'), 1.0, 3, 'train', 'foo']],
      dtype=object)
```

Viewing Data

Note

DataFrame.to_numpy() does *not* include the index or column labels in the output.

describe() shows a quick statistic summary of your data:

```
>>> df.describe()
              A         B         C         D
count  6.000000  6.000000  6.000000  6.000000
mean   0.073711 -0.431125 -0.687758 -0.233103
std    0.843157  0.922818  0.779887  0.973118
min   -0.861849 -2.104569 -1.509059 -1.135632
25%   -0.611510 -0.600794 -1.368714 -1.076610
50%    0.022070 -0.228039 -0.767252 -0.386188
75%    0.658444  0.041933 -0.034326  0.461706
max    1.212112  0.567020  0.276232  1.071804
```

Viewing Data

Transposing your data:

```
>>> df.T
   2023-06-15  2023-06-16  2023-06-17  2023-06-18  2023-06-19  2023-06-20
A    1.647283   -0.699142   -0.652668    0.625232   -1.719933    1.419357
B   -0.659818   -0.078138    0.755713    0.112527    1.858491    0.952211
C    0.058287   -1.802049   -1.039440   -1.274081    0.713219   -0.146506
D    0.522661   -0.753840   -0.678967    0.724884   -0.283876   -1.337652
```

Viewing Data

DataFrame.sort_index() sorts by an axis:

```
>>> df.sort_index(axis=1, ascending=False)
                   D         C         B         A
2023-06-15  0.522661  0.058287 -0.659818  1.647283
2023-06-16 -0.753840 -1.802049 -0.078138 -0.699142
2023-06-17 -0.678967 -1.039440  0.755713 -0.652668
2023-06-18  0.724884 -1.274081  0.112527  0.625232
2023-06-19 -0.283876  0.713219  1.858491 -1.719933
2023-06-20 -1.337652 -0.146506  0.952211  1.419357
```

Viewing Data

DataFrame.sort_values() sorts by values:

```
>>> df.sort_values(by="B")
                   A         B         C         D
2023-06-15  1.647283 -0.659818  0.058287  0.522661
2023-06-16 -0.699142 -0.078138 -1.802049 -0.753840
2023-06-18  0.625232  0.112527 -1.274081  0.724884
2023-06-17 -0.652668  0.755713 -1.039440 -0.678967
2023-06-20  1.419357  0.952211 -0.146506 -1.337652
2023-06-19 -1.719933  1.858491  0.713219 -0.283876
```

Write a Pandas program to convert a Panda Module Series to Python list and its type.

```
>>> import pandas as pd
# Define a Pandas Series
>>> data = pd.Series([1, 2, 3, 4, 5])
# Convert the Series to a Python list
>>> data_list = data.tolist()
# Print the list and its type
>>> print("List:", data_list)
List: [1, 2, 3, 4, 5]
>>> print("Type:", type(data_list))
Type: <class 'list'>
```

Write a Pandas program to convert a Panda Module Series to Python list and its type.

- In this PROGRAM, we first import the Pandas library.
- Then, we define a Pandas Series ‘data’ with some sample data.
- We then use the ‘tolist()’ function to convert the Series to a Python list, and store the result in the ‘data_list’ variable.
- Finally, we print the list and its type using the ‘print()’ function.

Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- Pandas
  - Selection
  - Missing Data
  - Operations

Quick overview of pandas?

- https://pandas.pydata.org/docs/user_guide/10min.html#min

Selection

Getting

- Selecting a single column, which yields a Series, equivalent to df.A:

```
>>> import numpy as np
>>> import pandas as pd
>>> dates = pd.date_range("20230616", periods=6)
>>> dates
DatetimeIndex(['2023-06-16', '2023-06-17', '2023-06-
18', '2023-06-19','2023-06-20', '2023-06-21'],
              dtype='datetime64[ns]', freq='D')
```

Selection

Getting

- Selecting a single column, which yields a Series, equivalent to df.A:

```
>>> df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
>>> df
                   A         B         C         D
2023-06-16  0.624990  0.280439 -1.135774 -1.313608
2023-06-17 -2.193261 -0.791604  0.190538 -1.416561
2023-06-18  0.490894  0.266283  0.420222  1.509806
2023-06-19  0.835607  0.468082 -0.636188  0.711356
2023-06-20  0.572577 -0.170767 -1.007022 -1.370098
2023-06-21 -0.829668  1.107512  0.742606 -1.473094
```

Selection

Getting

- Selecting a single column, which yields a Series, equivalent to df.A:

```
>>> df["A"]
2023-06-16    0.624990
2023-06-17   -2.193261
2023-06-18    0.490894
2023-06-19    0.835607
2023-06-20    0.572577
2023-06-21   -0.829668
Freq: D, Name: A, dtype: float64
>>> df.A
```

Selection

Getting

- Selecting via [] (__getitem__), which slices the rows:

```
>>> df[0:3]
                   A         B         C         D
2023-06-16  0.624990  0.280439 -1.135774 -1.313608
2023-06-17 -2.193261 -0.791604  0.190538 -1.416561
2023-06-18  0.490894  0.266283  0.420222  1.509806
>>> df["20230617":"20230620"]
                   A         B         C         D
2023-06-17 -2.193261 -0.791604  0.190538 -1.416561
2023-06-18  0.490894  0.266283  0.420222  1.509806
2023-06-19  0.835607  0.468082 -0.636188  0.711356
2023-06-20  0.572577 -0.170767 -1.007022 -1.370098
```

Selection

Selection by label

- Selection by Label using DataFrame.loc() or DataFrame.at()

DataFrame.loc()

- Access a group of rows and columns by label(s) or a boolean array.
- .loc[] is primarily label based, but may also be used with a boolean array.
- Allowed inputs are:
  - A single label, e.g. 5 or 'a', (note that 5 is interpreted as a label of the index, and never as an integer position along the index).
  - A list or array of labels, e.g. ['a', 'b', 'c'].
  - A slice object with labels, e.g. 'a':'f'.
  - A boolean array of the same length as the axis being sliced, e.g. [True, False, True].
  - An alignable boolean Series. The index of the key will be aligned before masking.
  - An alignable Index. The Index of the returned selection will be the input.
  - A callable function with one argument (the calling Series or DataFrame) and that returns valid output for indexing.

Selection

Selection by label

- Selection by Label using DataFrame.loc() or DataFrame.at()

DataFrame.at()

- Access a single value for a row/column label pair.
- Similar to loc, in that both provide label-based lookups.
- Use at if you only need to get or set a single value in a DataFrame or Series.

Selection

Selection by label

- For getting a cross section using a label:

```
>>> df.loc[dates[0]]
A    0.624990
B    0.280439
C   -1.135774
D   -1.313608
Name: 2023-06-16 00:00:00, dtype: float64
```

Selection

Selection by label

- Selecting on a multi-axis by label:

```
>>> df.loc[:, ["A", "B"]]
                   A         B
2023-06-16  0.624990  0.280439
2023-06-17 -2.193261 -0.791604
2023-06-18  0.490894  0.266283
2023-06-19  0.835607  0.468082
2023-06-20  0.572577 -0.170767
2023-06-21 -0.829668  1.107512
```

Selection

Selection by label

- Showing label slicing, both endpoints are included:

```
>>> df.loc["20230616":"20230619", ["A", "B"]]
                   A         B
2023-06-16  0.624990  0.280439
2023-06-17 -2.193261 -0.791604
2023-06-18  0.490894  0.266283
2023-06-19  0.835607  0.468082
```

Selection

Selection by label

- Reduction in the dimensions of the returned object:

```
>>> df.loc["20230616", ["A", "B"]]
A    0.624990
B    0.280439
Name: 2023-06-16 00:00:00, dtype: float64
```

Selection

Selection by label

- For getting a scalar value:

```
>>> df.loc[dates[0], "A"]
0.6249895880761244
```

Selection

Selection by label

- For getting fast access to a scalar (equivalent to the prior method):

```
>>> df.at[dates[0], "A"]
0.6249895880761244
```

Selection

Selection by position

- Selection by Position using DataFrame.iloc() or DataFrame.iat().

DataFrame.iloc()

- Purely integer-location based indexing for selection by position.
- .iloc[] is primarily integer position based (from 0 to length-1 of the axis), but may also be used with a boolean array.
- Allowed inputs are:
    - An integer, e.g. 5.
    - A list or array of integers, e.g. [4, 3, 0].
    - A slice object with integers, e.g. 1:7.
    - A boolean array.
    - A callable function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above).
    - A tuple of row and column indexes. The tuple elements consist of one of the above inputs, e.g. (0, 1).

Selection

Selection by position

- Selection by Position using DataFrame.iloc() or DataFrame.iat().

DataFrame.iat()

- Access a single value for a row/column pair by integer position.
- Similar to iloc, in that both provide integer-based lookups.
- Use iat if you only need to get or set a single value in a DataFrame or Series.

Selection

Selection by position

```
>>> import numpy as np
>>> import pandas as pd
>>> dates = pd.date_range("20230616", periods=6)
>>> dates
DatetimeIndex(['2023-06-16', '2023-06-17', '2023-06-
18', '2023-06-19','2023-06-20', '2023-06-21'],
              dtype='datetime64[ns]', freq='D')
```

Selection

Selection by position

```
>>> df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
>>> df
                   A         B         C         D
2023-06-16  0.624990  0.280439 -1.135774 -1.313608
2023-06-17 -2.193261 -0.791604  0.190538 -1.416561
2023-06-18  0.490894  0.266283  0.420222  1.509806
2023-06-19  0.835607  0.468082 -0.636188  0.711356
2023-06-20  0.572577 -0.170767 -1.007022 -1.370098
2023-06-21 -0.829668  1.107512  0.742606 -1.473094
```

Selection

Selection by position

Select via the position of the passed integers:

```
>>> df.iloc[3]
A    1.164038
B    0.358708
C   -1.164755
D    0.136925
Name: 2023-06-19 00:00:00, dtype: float64
```

Selection

Selection by position

By integer slices, acting similar to NumPy/Python:

```
>>> df.iloc[3:5, 0:2]
                   A         B
2023-06-19  1.164038  0.358708
2023-06-20  0.802864 -0.979016
```

Selection

Selection by position

By lists of integer position locations, similar to the NumPy/Python style:

```
>>> df.iloc[[1, 2, 4], [0, 2]]
                   A         C
2023-06-17  1.429741 -0.167137
2023-06-18 -0.308334 -0.111697
2023-06-20  0.802864  0.252669
```

Selection

Selection by position

For slicing rows explicitly:

```
>>> df.iloc[1:3, :]
                   A         B         C         D
2023-06-17  1.429741 -1.425115 -0.167137  0.344657
2023-06-18 -0.308334  1.187900 -0.111697  1.718216
```

Selection

Selection by position

For slicing columns explicitly:

```
>>> df.iloc[:, 1:3]
                   B         C
2023-06-16 -1.540570  1.397791
2023-06-17 -1.425115 -0.167137
2023-06-18  1.187900 -0.111697
2023-06-19  0.358708 -1.164755
2023-06-20 -0.979016  0.252669
2023-06-21 -1.504834  0.096893
```

Selection

Selection by position

For getting a value explicitly:

```
>>> df.iloc[1, 1]
-1.4251154212429558
```

Selection

Selection by position

For getting fast access to a scalar (equivalent to the prior method):

```
>>> df.iat[1, 1]
-1.4251154212429558
```

Selection

Boolean indexing

Using a single column's values to select data:

```
>>> df[df["A"] > 0]
                   A         B         C         D
2023-06-16  0.273229 -1.540570  1.397791 -2.013290
2023-06-17  1.429741 -1.425115 -0.167137  0.344657
2023-06-19  1.164038  0.358708 -1.164755  0.136925
2023-06-20  0.802864 -0.979016  0.252669 -0.110771
2023-06-21  1.002757 -1.504834  0.096893  0.086878
```

Selection

Boolean indexing

Selecting values from a DataFrame where a boolean condition is met:

```
>>> df[df > 0]
                   A         B         C         D
2023-06-16  0.273229       NaN  1.397791       NaN
2023-06-17  1.429741       NaN       NaN  0.344657
2023-06-18       NaN  1.187900       NaN  1.718216
2023-06-19  1.164038  0.358708       NaN  0.136925
2023-06-20  0.802864       NaN  0.252669       NaN
2023-06-21  1.002757       NaN  0.096893  0.086878
```

Selection

Boolean indexing

Using the isin() method for filtering:

```
>>> df2 = df.copy()
>>>  df2["E"]  =  ["one",  "one",  "two",  "three",
"four", "three"]
```

	A	B	C	D	E
2023-06-16	0.273229	-1.540570	1.397791	-2.013290	one
2023-06-17	1.429741	-1.425115	-0.167137	0.344657	one
2023-06-18	-0.308334	1.187900	-0.111697	1.718216	two
2023-06-19	1.164038	0.358708	-1.164755	0.136925	three
2023-06-20	0.802864	-0.979016	0.252669	-0.110771	four
2023-06-21	1.002757	-1.504834	0.096893	0.086878	three

Selection

Boolean indexing

Using the isin() method for filtering:

```
>>> df2[df2["E"].isin(["two", "four"])]
                   A         B         C         D      E
2023-06-18 -0.308334  1.187900 -0.111697  1.718216    two
2023-06-20  0.802864 -0.979016  0.252669 -0.110771   four
```

Selection

Setting

Setting a new column automatically aligns the data by the indexes:

```
>>> s1 = pd.Series([1, 2, 3, 4, 5, 6], index=pd.date_range("20230617", periods=6))
>>> s1
2023-06-17    1
2023-06-18    2
2023-06-19    3
2023-06-20    4
2023-06-21    5
2023-06-22    6
Freq: D, dtype: int64
```

Selection

Setting

Setting a new column automatically aligns the data by the indexes:

```
>>> df["F"] = s1
>>> df
                   A         B         C         D         F
2023-06-16  0.273229 -1.540570  1.397791 -2.013290       NaN
2023-06-17  1.429741 -1.425115 -0.167137  0.344657  1.0
2023-06-18 -0.308334  1.187900 -0.111697  1.718216  2.0
2023-06-19  1.164038  0.358708 -1.164755  0.136925  3.0
2023-06-20  0.802864 -0.979016  0.252669 -0.110771  4.0
2023-06-21  1.002757 -1.504834  0.096893  0.086878  5.0
```

Selection

Setting

Setting values by label:

```
>>> df.at[dates[0], "A"] = 0
>>> df
                   A         B         C         D         F
2023-06-16  0.000000 -1.540570  1.397791 -2.013290       NaN
2023-06-17  1.429741 -1.425115 -0.167137  0.344657  1.000000
2023-06-18 -0.308334  1.187900 -0.111697  1.718216  2.000000
2023-06-19  1.164038  0.358708 -1.164755  0.136925  3.000000
2023-06-20  0.802864 -0.979016  0.252669 -0.110771  4.000000
2023-06-21  1.002757 -1.504834  0.096893  0.086878  5.000000
```

Selection

Setting

Setting values by position:

```
>>> df.iat[0, 1] = 0
>>> df
                   A         B         C         D         F
2023-06-16  0.000000  0.000000  1.397791 -2.013290       NaN
2023-06-17  1.429741 -1.425115 -0.167137  0.344657  1.000000
2023-06-18 -0.308334  1.187900 -0.111697  1.718216  2.000000
2023-06-19  1.164038  0.358708 -1.164755  0.136925  3.000000
2023-06-20  0.802864 -0.979016  0.252669 -0.110771  4.000000
2023-06-21  1.002757 -1.504834  0.096893  0.086878  5.000000
```

Selection

Setting

Setting by assigning with a NumPy array:

```
>>> df.loc[:, "D"] = np.array([5] * len(df))
>>> df
                   A         B         C    D    F
2023-06-16  0.000000  0.000000  1.397791  5.0  NaN
2023-06-17  1.429741 -1.425115 -0.167137  5.0  1.0
2023-06-18 -0.308334  1.187900 -0.111697  5.0  2.0
2023-06-19  1.164038  0.358708 -1.164755  5.0  3.0
2023-06-20  0.802864 -0.979016  0.252669  5.0  4.0
2023-06-21  1.002757 -1.504834  0.096893  5.0  5.0
```

Selection

Setting

Setting by assigning with a NumPy array:

```
>>> df.loc[:, "D"] = np.array([6] * len(df))
>>> df
                   A         B         C    D    F
2023-06-16  0.000000  0.000000  1.397791  6.0  NaN
2023-06-17  1.429741 -1.425115 -0.167137  6.0  1.0
2023-06-18 -0.308334  1.187900 -0.111697  6.0  2.0
2023-06-19  1.164038  0.358708 -1.164755  6.0  3.0
2023-06-20  0.802864 -0.979016  0.252669  6.0  4.0
2023-06-21  1.002757 -1.504834  0.096893  6.0  5.0
```

Selection

Setting

A where operation with setting:

```
>>> df2 = df.copy()
>>> df2[df2 > 0] = -df2
>>> df2
```

	A	B	C	D	F
2023-06-16	0.000000	0.000000	-1.397791	-6.0	NaN
2023-06-17	-1.429741	-1.425115	-0.167137	-6.0	-1.0
2023-06-18	-0.308334	-1.187900	-0.111697	-6.0	-2.0
2023-06-19	-1.164038	-0.358708	-1.164755	-6.0	-3.0
2023-06-20	-0.802864	-0.979016	-0.252669	-6.0	-4.0
2023-06-21	-1.002757	-1.504834	-0.096893	-6.0	-5.0

Missing Data

- pandas primarily uses the value `np.nan` to represent missing data.
- It is by default not included in computations.
- Reindexing allows you to change/add/delete the index on a specified axis. This returns a copy of the data:

```
>>> df1 = df.reindex(index=dates[0:4], columns=list(df.columns) + ["E"])
>>> df1.loc[dates[0] : dates[1], "E"] = 1
>>> df1
                   A         B         C    D    F    E
2023-06-16  0.000000  0.000000  1.397791  6.0  NaN  1.0
2023-06-17  1.429741 -1.425115 -0.167137  6.0  1.0  1.0
2023-06-18 -0.308334  1.187900 -0.111697  6.0  2.0  NaN
2023-06-19  1.164038  0.358708 -1.164755  6.0  3.0  NaN
```

Missing Data

- DataFrame.dropna() drops any rows that have missing data:

```
>>> df1.dropna(how="any")
>>> df1
                   A         B         C    D    F    E
2023-06-17  1.429741 -1.425115 -0.167137  6.0  1.0  1.0
```

Missing Data

- DataFrame.fillna() fills missing data:

```
>>> df1.fillna(value=5)
>>> df1
                   A         B         C    D    F    E
2023-06-16  0.000000  0.000000  1.397791  6.0  5.0  1.0
2023-06-17  1.429741 -1.425115 -0.167137  6.0  1.0  1.0
2023-06-18 -0.308334  1.187900 -0.111697  6.0  2.0  5.0
2023-06-19  1.164038  0.358708 -1.164755  6.0  3.0  5.0
```

Missing Data

- isna() gets the boolean mask where values are nan:

```
>>> pd.isna(df1)
>>> df1
                A      B      C      D      F      E
2023-06-16  False  False  False  False   True  False
2023-06-17  False  False  False  False  False  False
2023-06-18  False  False  False  False  False   True
2023-06-19  False  False  False  False  False   True
```

Operations

Stats

- Operations in general *exclude* missing data.
- Performing a descriptive statistic:

```
>>> df
                   A         B         C    D    F
2023-06-16  0.000000  0.000000  1.397791  6.0  NaN
2023-06-17  1.429741 -1.425115 -0.167137  6.0  1.0
2023-06-18 -0.308334  1.187900 -0.111697  6.0  2.0
2023-06-19  1.164038  0.358708 -1.164755  6.0  3.0
2023-06-20  0.802864 -0.979016  0.252669  6.0  4.0
2023-06-21  1.002757 -1.504834  0.096893  6.0  5.0
```

Operations

Stats

- Performing a descriptive statistic:

```
>>> df.mean() #column wise mean
A    0.681844
B   -0.393726
C    0.050627
D    6.000000
F    3.000000
dtype: float64
```

Operations

Stats

- Same operation on the other axis:

```
>>> df.mean(1)#row wise mean
2023-06-16    1.849448
2023-06-17    1.367498
2023-06-18    1.753574
2023-06-19    1.871598
2023-06-20    2.015303
2023-06-21    2.118963
Freq: D, dtype: float64
```

Operations

Stats

- Operating with objects that have different dimensionality and need alignment.
- In addition, pandas automatically broadcasts along the specified dimension:

```
>>> s = pd.Series([1, 3, 5, np.nan, 6, 8], index=dates).shift(2)
>>> s
2023-06-16    NaN
2023-06-17    NaN
2023-06-18    1.0
2023-06-19    3.0
2023-06-20    5.0
2023-06-21    NaN
Freq: D, dtype: float64
```

Operations

Stats

- Operating with objects that have different dimensionality and need alignment.
- In addition, pandas automatically broadcasts along the specified dimension:

```
>>> df.sub(s, axis="index")
                   A         B         C    D    F
2023-06-16       NaN       NaN       NaN  NaN  NaN
2023-06-17       NaN       NaN       NaN  NaN  NaN
2023-06-18 -1.308334  0.187900 -1.111697  5.0  1.0
2023-06-19 -1.835962 -2.641292 -4.164755  3.0  0.0
2023-06-20 -4.197136 -5.979016 -4.747331  1.0 -1.0
2023-06-21       NaN       NaN       NaN  NaN  NaN
```

Operations

Apply

- DataFrame.apply() applies a user defined function to the data:

```
>>> df
                   A         B         C    D    F
2023-06-16  0.000000  0.000000  1.397791  6.0  NaN
2023-06-17  1.429741 -1.425115 -0.167137  6.0  1.0
2023-06-18 -0.308334  1.187900 -0.111697  6.0  2.0
2023-06-19  1.164038  0.358708 -1.164755  6.0  3.0
2023-06-20  0.802864 -0.979016  0.252669  6.0  4.0
2023-06-21  1.002757 -1.504834  0.096893  6.0  5.0
```

Operations

Apply

- DataFrame.apply() applies a user defined function to the data:

```
>>> df.apply(np.cumsum) # cumulative sum column wise
                   A         B         C     D     F
2023-06-16  0.000000  0.000000  1.397791   6.0   NaN
2023-06-17  1.429741 -1.425115  1.230654  12.0   1.0
2023-06-18  1.121407 -0.237215  1.118957  18.0   3.0
2023-06-19  2.285445  0.121493 -0.045798  24.0   6.0
2023-06-20  3.088308 -0.857523  0.206872  30.0  10.0
2023-06-21  4.091065 -2.362357  0.303764  36.0  15.0
```

Operations

Apply

- DataFrame.apply() applies a user defined function to the data:

```
>>> df.apply(lambda x: x.max() - x.min()) # column wise
A    1.738075
B    2.692734
C    2.562546
D    0.000000
F    4.000000
dtype: float64
```

Operations

String Methods

Series is equipped with a set of string processing methods in the str attribute that make it easy to operate on each element of the array, as in the code snippet below.

Note that pattern-matching in str generally uses regular expressions by default (and in some cases always uses them).

Operations

String Methods

```
>>> s = pd.Series(["A", "B", "C", "Aaba", "Baca", np.nan, "CABA", "dog", "cat"])
>>> s.str.lower()
0       a
1       b
2       c
3    aaba
4    baca
5     NaN
6    caba
7     dog
8     cat
dtype: object
```

Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- Pandas
  - Merge
  - Grouping
  - Time series
  - Categoricals
  - Importing and exporting data

Quick overview of pandas?

- https://pandas.pydata.org/docs/user_guide/10min.html#min

Merge

Concat

pandas provides various facilities for easily combining together Series and DataFrame objects with various kinds of set logic for the indexes and relational algebra functionality in the case of join / merge-type operations.

Merge

Concat

Concatenating pandas objects together along an axis with concat():

```
>>> df = pd.DataFrame(np.random.randn(10, 4))
>>> df
          0         1         2         3
0 -0.717196  0.473925 -0.395855 -1.621849
1 -0.219668 -1.228870 -0.914974 -0.241247
2  2.244973 -0.415200  0.056994  0.101080
3 -0.878046  0.416592  1.453482 -0.033171
4  1.114444 -1.857564 -0.410036 -0.823989
5 -1.025485  0.487437 -0.406924  1.596674
6 -0.001355  1.356347  0.042176  0.712419
7  1.518298 -0.282735  0.731253 -1.149422
8  0.862762 -0.563118  0.055865  0.801844
9 -0.608196 -1.381167  0.419710  0.268061
```

Merge

Concat

Concatenating pandas objects together along an axis with concat():

```
>>> pieces = [df[:3], df[3:7], df[7:]]
>>> pieces
[          0         1         2         3
0 -0.717196  0.473925 -0.395855 -1.621849
1 -0.219668 -1.228870 -0.914974 -0.241247
2  2.244973 -0.415200  0.056994  0.101080,
          0         1         2         3
3 -0.878046  0.416592  1.453482 -0.033171
4  1.114444 -1.857564 -0.410036 -0.823989
5 -1.025485  0.487437 -0.406924  1.596674
6 -0.001355  1.356347  0.042176  0.712419,
          0         1         2         3
7  1.518298 -0.282735  0.731253 -1.149422
8  0.862762 -0.563118  0.055865  0.801844
9 -0.608196 -1.381167  0.419710  0.268061]
```

Merge

Concat

Concatenating pandas objects together along an axis with concat():

```
>>> pd.concat(pieces)
          0         1         2         3
0 -0.717196  0.473925 -0.395855 -1.621849
1 -0.219668 -1.228870 -0.914974 -0.241247
2  2.244973 -0.415200  0.056994  0.101080
3 -0.878046  0.416592  1.453482 -0.033171
4  1.114444 -1.857564 -0.410036 -0.823989
5 -1.025485  0.487437 -0.406924  1.596674
6 -0.001355  1.356347  0.042176  0.712419
7  1.518298 -0.282735  0.731253 -1.149422
8  0.862762 -0.563118  0.055865  0.801844
9 -0.608196 -1.381167  0.419710  0.268061
```

Merge

Join

merge() enables SQL style join types along specific columns:

```
>>> import numpy as np
>>> import pandas as pd
>>> left = pd.DataFrame({"key": ["oop", "oop"], "lval": [1, 2]})
>>> left
   key  lval
0  oop     1
1  oop     2
```

Merge

Join

```
>>> right = pd.DataFrame({"key": ["oop", "oop"], "rval": [4, 5]})
>>> right
   key  rval
0  oop     4
1  oop     5
>>> pd.merge(left, right, on="key")
   key  lval  rval
0  oop     1     4
1  oop     1     5
2  oop     2     4
3  oop     2     5
```

Merge

Join – Example2

```
>>> left = pd.DataFrame({"key": ["oop", "python"], "lval": [1, 2]})
>>> left
      key  lval
0     oop     1
1  python     2
>>>   right   =   pd.DataFrame({"key":   ["oop",   "python"], "rval": [4, 5]})
>>> right
      key  rval
0     oop     4
1  python     5
```

Merge

Join – Example2

```
>>> pd.merge(left, right, on="key")
      key  lval  rval
0     oop     1     4
1  python     2     5
```

Grouping

By “group by” we are referring to a process involving one or more of the following steps:

- **Splitting** the data into groups based on some criteria
- **Applying** a function to each group independently
- **Combining** the results into a data structure

Grouping

```
>>> import numpy as np
>>> import pandas as pd
>>> df = pd.DataFrame(
    {
        "A": ["oop", "python", "oop", "python", "oop", "python", "oop", "oop"],
        "B": ["one", "one", "two", "three", "two", "two", "one", "three"],
        "C": np.random.randn(8),
        "D": np.random.randn(8),
    }
)
```

Grouping

```
>>> df
     A       B         C         D
0    oop     one  0.507365  1.299938
1  python    one -0.829337 -0.672491
2    oop     two -1.725394 -1.049454
3  python  three  0.739141  1.984817
4    oop     two  1.058922 -1.549976
5  python    two -0.285935 -0.435408
6    oop     one -1.351595  0.362258
7    oop   three -0.005763  1.304180
```

Grouping

Grouping and then applying the `sum()` function to the resulting groups:

```
>>> df
     A      B         C         D
0  oop    one  0.507365  1.299938
1  python    one -0.829337 -0.672491
2  oop    two -1.725394 -1.049454
3  python  three  0.739141  1.984817
4  oop    two  1.058922 -1.549976
5  python    two -0.285935 -0.435408
6  oop    one -1.351595  0.362258
7  oop  three -0.005763  1.304180
```

Grouping

Grouping and then applying the `sum()` function to the resulting groups:

```
>>> df.groupby("A")[["C", "D"]].sum()
                C         D
A
oop    -1.516466  0.366946
python -0.376131  0.876918
```

Grouping

Grouping by multiple columns forms a hierarchical index, and again we can apply the `sum()` function:

```
>>> df.groupby(["A", "B"]).sum()
                     C         D
A      B
oop    one    -0.844231  1.662196
       three  -0.005763  1.304180
       two    -0.666472 -2.599430
python one    -0.829337 -0.672491
       three   0.739141  1.984817
       two    -0.285935 -0.435408
```

Time series

- pandas has simple, powerful, and efficient functionality for performing resampling operations during frequency conversion (e.g., converting secondly data into 5-minutely data).
- This is extremely common in, but not limited to, financial applications.

```
>>> rng = pd.date_range("19/06/2023", periods=100, freq="S")
>>> rng
```

Time series

IDLE Shell 3.10.0

File Edit Shell Debug Options Window Help

```
>>> rng
DatetimeIndex(['2023-06-19 00:00:00', '2023-06-19 00:00:01',
               '2023-06-19 00:00:02', '2023-06-19 00:00:03',
               '2023-06-19 00:00:04', '2023-06-19 00:00:05',
               '2023-06-19 00:00:06', '2023-06-19 00:00:07',
               '2023-06-19 00:00:08', '2023-06-19 00:00:09',
               '2023-06-19 00:00:10', '2023-06-19 00:00:11',
               '2023-06-19 00:00:12', '2023-06-19 00:00:13',
               '2023-06-19 00:00:14', '2023-06-19 00:00:15',
               '2023-06-19 00:00:16', '2023-06-19 00:00:17',
               '2023-06-19 00:00:18', '2023-06-19 00:00:19',
               '2023-06-19 00:00:20', '2023-06-19 00:00:21',
               '2023-06-19 00:00:22', '2023-06-19 00:00:23',
               '2023-06-19 00:00:24', '2023-06-19 00:00:25',
               '2023-06-19 00:00:26', '2023-06-19 00:00:27',
               '2023-06-19 00:00:28', '2023-06-19 00:00:29',
               '2023-06-19 00:00:30', '2023-06-19 00:00:31',
               '2023-06-19 00:00:32', '2023-06-19 00:00:33',
               '2023-06-19 00:00:34', '2023-06-19 00:00:35',
               '2023-06-19 00:00:36', '2023-06-19 00:00:37',
               '2023-06-19 00:00:38', '2023-06-19 00:00:39',
               '2023-06-19 00:00:40', '2023-06-19 00:00:41',
               '2023-06-19 00:00:42', '2023-06-19 00:00:43',
               '2023-06-19 00:00:44', '2023-06-19 00:00:45',
               '2023-06-19 00:00:46', '2023-06-19 00:00:47',
```

IDLE Shell 3.10.0

File Edit Shell Debug Options Window Help

```
               '2023-06-19 00:00:52', '2023-06-19 00:00:53',
               '2023-06-19 00:00:54', '2023-06-19 00:00:55',
               '2023-06-19 00:00:56', '2023-06-19 00:00:57',
               '2023-06-19 00:00:58', '2023-06-19 00:00:59',
               '2023-06-19 00:01:00', '2023-06-19 00:01:01',
               '2023-06-19 00:01:02', '2023-06-19 00:01:03',
               '2023-06-19 00:01:04', '2023-06-19 00:01:05',
               '2023-06-19 00:01:06', '2023-06-19 00:01:07',
               '2023-06-19 00:01:08', '2023-06-19 00:01:09',
               '2023-06-19 00:01:10', '2023-06-19 00:01:11',
               '2023-06-19 00:01:12', '2023-06-19 00:01:13',
               '2023-06-19 00:01:14', '2023-06-19 00:01:15',
               '2023-06-19 00:01:16', '2023-06-19 00:01:17',
               '2023-06-19 00:01:18', '2023-06-19 00:01:19',
               '2023-06-19 00:01:20', '2023-06-19 00:01:21',
               '2023-06-19 00:01:22', '2023-06-19 00:01:23',
               '2023-06-19 00:01:24', '2023-06-19 00:01:25',
               '2023-06-19 00:01:26', '2023-06-19 00:01:27',
               '2023-06-19 00:01:28', '2023-06-19 00:01:29',
               '2023-06-19 00:01:30', '2023-06-19 00:01:31',
               '2023-06-19 00:01:32', '2023-06-19 00:01:33',
               '2023-06-19 00:01:34', '2023-06-19 00:01:35',
               '2023-06-19 00:01:36', '2023-06-19 00:01:37',
               '2023-06-19 00:01:38', '2023-06-19 00:01:39'],
              dtype='datetime64[ns]', freq='S')
```

Time series

```
>>> ts = pd.Series(np.random.randint(0, 500, len(rng)), index=rng)
>>> ts
2023-06-19 00:00:00    285
2023-06-19 00:00:01    414
2023-06-19 00:00:02    357
2023-06-19 00:00:03    328
2023-06-19 00:00:04     52
                      ...
2023-06-19 00:01:35    499
2023-06-19 00:01:36    482
2023-06-19 00:01:37    396
2023-06-19 00:01:38    259
2023-06-19 00:01:39     27
Freq: S, Length: 100, dtype: int32
```

Time series

```
>>> ts.resample("5Min").sum()
2023-06-19    24777
Freq: 5T, dtype: int32
```

Time series

- Series.tz_localize() localizes a time series to a time zone:

```
>>> rng = pd.date_range("19/06/2023 00:00", periods=5, freq="D")
>>> rng
DatetimeIndex(['2023-06-19', '2023-06-20', '2023-06-21', '2023-06-22', '2023-06-23'],
              dtype='datetime64[ns]', freq='D')
```

Time series

- Series.tz_localize() localizes a time series to a time zone:

```
>>> ts = pd.Series(np.random.randn(len(rng)), rng)
>>> ts
2023-06-19   -0.404256
2023-06-20    0.477535
2023-06-21    2.293098
2023-06-22   -0.408688
2023-06-23    0.972186
Freq: D, dtype: float64
```

Time series

- Series.tz_localize() localizes a time series to a time zone:

```
>>> ts_utc = ts.tz_localize("UTC")
>>> ts_utc
2023-06-19 00:00:00+00:00   -0.404256
2023-06-20 00:00:00+00:00    0.477535
2023-06-21 00:00:00+00:00    2.293098
2023-06-22 00:00:00+00:00   -0.408688
2023-06-23 00:00:00+00:00    0.972186
Freq: D, dtype: float64
```

Time series

- Series.tz_convert() converts a timezones aware time series to another time zone:
- aware object represents a specific moment in time that is not open to interpretation.

```
>>> ts_utc = ts.tz_localize("UTC")
>>> ts_utc
2023-06-19 00:00:00+00:00   -0.404256
2023-06-20 00:00:00+00:00    0.477535
2023-06-21 00:00:00+00:00    2.293098
2023-06-22 00:00:00+00:00   -0.408688
2023-06-23 00:00:00+00:00    0.972186
Freq: D, dtype: float64
```

Time series

- Series.tz_convert() converts a timezones aware time series to another time zone:

```
>>> ts_utc.tz_convert("US/Eastern")
2023-06-18 20:00:00-04:00   -0.404256
2023-06-19 20:00:00-04:00    0.477535
2023-06-20 20:00:00-04:00    2.293098
2023-06-21 20:00:00-04:00   -0.408688
2023-06-22 20:00:00-04:00    0.972186
Freq: D, dtype: float64
```

Time series

- Converting between time span representations:

```
>>> rng = pd.date_range("1/1/2023", periods=5, freq="M")
>>> rng
DatetimeIndex(['2023-01-31', '2023-02-28', '2023-03-31',
'2023-04-30',
               '2023-05-31'],
              dtype='datetime64[ns]', freq='M')
```

Time series

- Converting between time span representations:

```
>>> ts = pd.Series(np.random.randn(len(rng)), index=rng)
>>> ts
2023-01-31   -0.457527
2023-02-28   -0.320943
2023-03-31   -0.910579
2023-04-30   -1.829910
2023-05-31    1.248885
Freq: M, dtype: float64
```

Time series

- Converting between time span representations:

```
>>> ps = ts.to_period()
>>> ps
2023-01   -0.457527
2023-02   -0.320943
2023-03   -0.910579
2023-04   -1.829910
2023-05    1.248885
Freq: M, dtype: float64
```

Time series

- Converting between time span representations:

```
>>> ps.to_timestamp()
2023-01-01   -0.457527
2023-02-01   -0.320943
2023-03-01   -0.910579
2023-04-01   -1.829910
2023-05-01    1.248885
Freq: MS, dtype: float64
```

Categoricals

- pandas can include categorical data in a DataFrame.
- Categoricals are a pandas data type corresponding to categorical variables in statistics.
- A categorical variable takes on a limited, and usually fixed, number of possible values.
- Examples are gender, social class, blood type, country affiliation, observation time or rating via Likert scales.
- In contrast to statistical categorical variables, categorical data might have an order (e.g. ‘strongly agree’ vs ‘agree’ or ‘first observation’ vs. ‘second observation’), but numerical operations (additions, divisions, …) are not possible.
- All values of categorical data are either in `categories` or `np.nan`. Order is defined by the order of `categories`, not lexical order of the values. Internally, the data structure consists of a `categories` array and an integer array of `codes` which point to the real value in the `categories` array.

Categoricals

```
>>> df = pd.DataFrame(
    {"id": [1, 2, 3, 4, 5, 6], "raw_grade": ["a", "b", "b", "a",
"a", "e"]}
)
>>> df
   id raw_grade
0   1         a
1   2         b
2   3         b
3   4         a
4   5         a
5   6         e
```

Categoricals

Converting the raw grades to a categorical data type:

```
>>> df["grade"] = df["raw_grade"].astype("category")
>>> df["grade"]
0    a
1    b
2    b
3    a
4    a
5    e
Name: grade, dtype: category
Categories (3, object): ['a', 'b', 'e']
```

Categoricals

Rename the categories to more meaningful names:

```
>>> new_categories = ["very good", "good", "very bad"]
>>> df["grade"]= df["grade"].cat.rename_categories(new_categories)
>>> df["grade"]
0     very good
1          good
2          good
3     very good
4     very good
5      very bad
Name: grade, dtype: category
Categories (3, object): ['very good', 'good', 'very bad']
```

Categoricals

Reorder the categories and simultaneously add the missing categories (methods under `Series.cat()` return a new `Series` by default):

```
>>> df["grade"] = df["grade"].cat.set_categories(
    ["very bad", "bad", "medium", "good", "very good"]
)
>>> df["grade"]
0    very good
1         good
2         good
3    very good
4    very good
5     very bad
Name: grade, dtype: category
Categories (5, object): ['very bad', 'bad', 'medium', 'good', 'very good']
```

Categoricals

Sorting is per order in the categories, not lexical order:

```
>>> df.sort_values(by="grade")
   id  raw_grade      grade
5   6          e   very bad
1   2          b       good
2   3          b       good
0   1          a  very good
3   4          a  very good
4   5          a  very good
```

Categoricals

Grouping by a categorical column also shows empty categories:

```
>>> df.groupby("grade").size()
grade
very bad      1
bad           0
medium        0
good          2
very good     3
dtype: int64
```

Importing and exporting data

CSV

Reading from a csv file: using `read_csv()`:

```
"C:/Users/rmmna/Downloads/food.csv"
```

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Category	Description	Nutrient D	Data.Alph	Data.Beta	Data.Beta	Data.Carb	Data.Chol	Data.Chol	Data.Fiber	Data.Lutei	Data.Lyco	Data.Niaci	Data.Prote	Data.Retir
2	Milk	Milk, human	11000000	0	7	0	6.89	14	16	0	0	0	0.177	1.03	60
3	Milk	Milk, NFS	11100000	0	4	0	4.87	8	17.9	0	0	0	0.11	3.34	58
4	Milk	Milk, whole	11111000	0	7	0	4.67	12	17.8	0	0	0	0.105	3.28	31
5	Milk	Milk, low sodium, whole	11111100	0	7	0	4.46	14	16	0	0	0	0.043	3.1	28
6	Milk	Milk, calcium fortified, whole	11111150	0	7	0	4.67	12	17.8	0	0	0	0.105	3.28	31
7	Milk	Milk, calcium fortified, low fat (1%)	11111160	0	1	0	5.19	5	17.4	0	0	0	0.113	3.38	58
8	Milk	Milk, calcium fortified, fat free (skim)	11111170	0	0	0	4.85	2	16	0	0	0	0.088	3.4	137
9	Milk	Milk, reduced fat (2%)	11112110	0	3	0	4.91	8	18.2	0	0	0	0.112	3.35	83
10	Milk	Milk, acidophilus, low fat (1%)	11112120	0	1	0	5.19	5	17.4	0	0	0	0.113	3.38	58
11	Milk	Milk, acidophilus, reduced fat (2%)	11112130	0	3	0	4.91	8	18.2	0	0	0	0.112	3.35	83
12	Milk	Milk, low fat (1%)	11112210	0	1	0	5.19	5	17.4	0	0	0	0.113	3.38	58
13	Milk	Milk, fat free (skim)	11113000	0	2	0	4.89	3	18.2	0	0	0	0.118	3.43	64
14	Milk	Milk, lactose free, low fat (1%)	11114300	0	1	0	5.19	5	17.4	0	0	0	0.113	3.38	58
15	Milk	Milk, lactose free, fat free (skim)	11114320	0	2	0	4.89	3	18.2	0	0	0	0.118	3.43	64
16	Milk	Milk, lactose free, reduced fat (2%)	11114330	0	3	0	4.91	8	18.2	0	0	0	0.112	3.35	83
17	Milk	Milk, lactose free, whole	11114350	0	7	0	4.67	12	17.8	0	0	0	0.105	3.28	31
18	Buttermilk	Buttermilk, fat free (skim)	11115000	0	1	0	4.79	4	17.7	0	0	0	0.058	3.31	13
19	Buttermilk	Buttermilk, low fat (1%)	11115100	0	1	0	4.79	4	17.7	0	0	0	0.058	3.31	13
20	Buttermilk	Buttermilk, reduced fat (2%)	11115200	0	3	0	5.3	8	16	0	0	0	0.1	4.1	16
21	Buttermilk	Buttermilk, whole	11115300	0	7	0	4.88	11	14.6	0	0	0	0.09	3.21	46
22	Kefir	Kefir, NS as to fat content	11115400	0	2	0	7.48	5	15.2	0	0	0	0.128	3.59	174
23	Goat's mil	Goat's milk, whole	11116000	0	7	0	4.45	11	16	0	0	0	0.277	3.56	56

food

Importing and exporting data

CSV

Reading from a csv file: using read_csv():

```
>>> pd.read_csv("C:/Users/rmmna/Downloads/food.csv")
```

```
                                     Category  ... Data.Vitamins.Vitamin K
0                                        Milk  ...                    0.3
1                                        Milk  ...                    0.2
2                                        Milk  ...                    0.3
3                                        Milk  ...                    0.3
4                                        Milk  ...                    0.3
...                                       ...  ...                    ...
7078          Tomatoes as ingredient in omelet  ...                    8.8
7079  Other vegetables as ingredient in omelet  ...                    0.4
7080         Vegetables as ingredient in curry  ...                    8.9
7081         Sauce as ingredient in hamburgers  ...                   50.8
7082      Industrial oil as ingredient in food  ...                  155.8
[7083 rows x 38 columns]
```

Importing and exporting data

CSV

Writing to a csv file: using `DataFrame.to_csv()`:

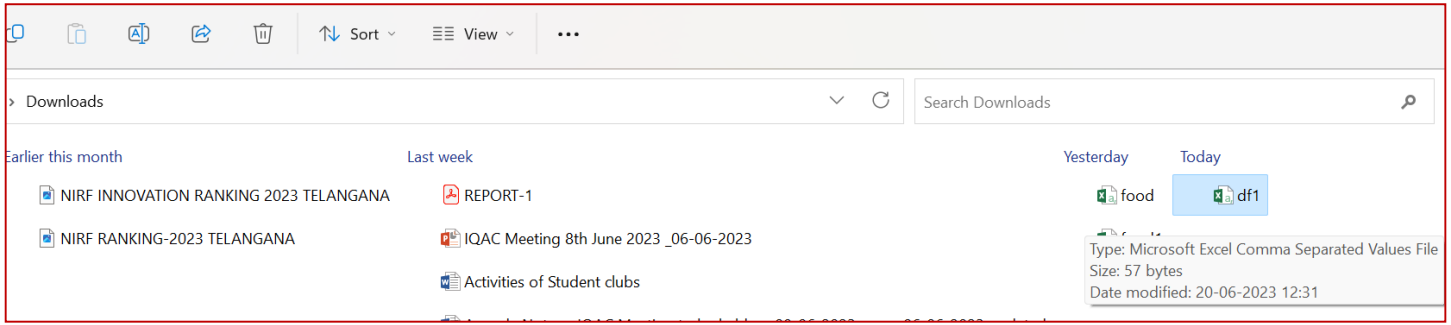
```
>>> import pandas as pd
>>> df = pd.DataFrame(
    {"id": [1, 2, 3, 4, 5, 6], "raw_grade": ["a", "b", "b", "a", "a", "e"]}
)
>>> df
   id raw_grade
0   1         a
1   2         b
2   3         b
3   4         a
4   5         a
5   6         e
```

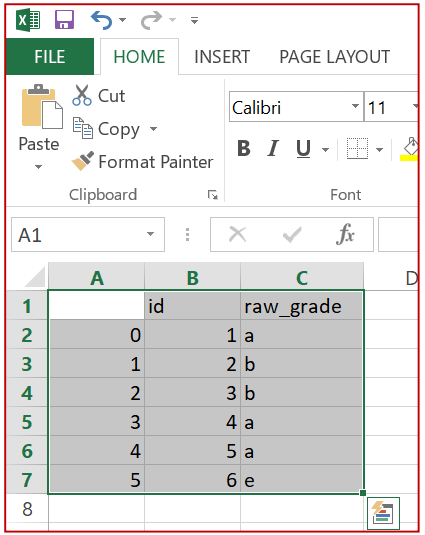
Importing and exporting data

CSV

Writing to a csv file: using `DataFrame.to_csv()`:

```
>>> df.to_csv("C:/Users/rmmna/Downloads/df1.csv")
```





Importing and exporting data

Excel

Writing to an excel file using `DataFrame.to_excel()`:

```
>>> import pandas as pd
>>> df1 = pd.DataFrame([['a', 'b'], ['c', 'd']],
                       index=['row 1', 'row 2'],
                       columns=['col 1', 'col 2'])
>>> df1
```

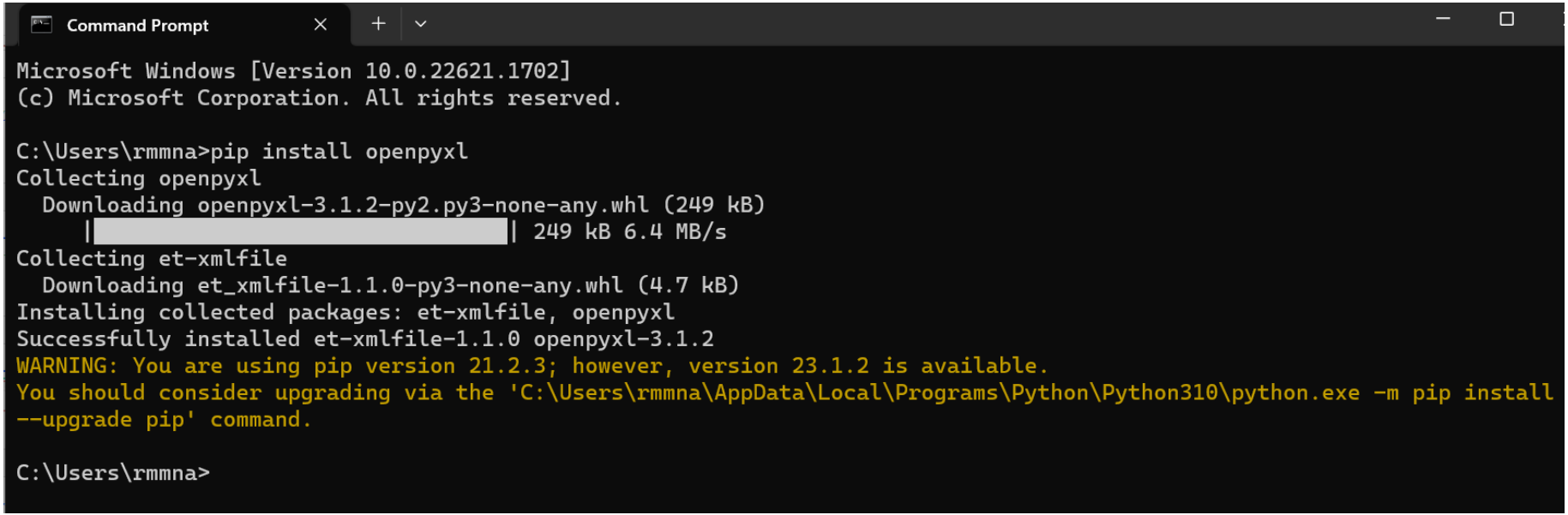
```
      col 1 col 2
row 1     a     b
row 2     c     d
```

Importing and exporting data

Excel

Writing to an excel file using `DataFrame.to_excel()`:

- install the module by running the `pip install openpyxl` command.

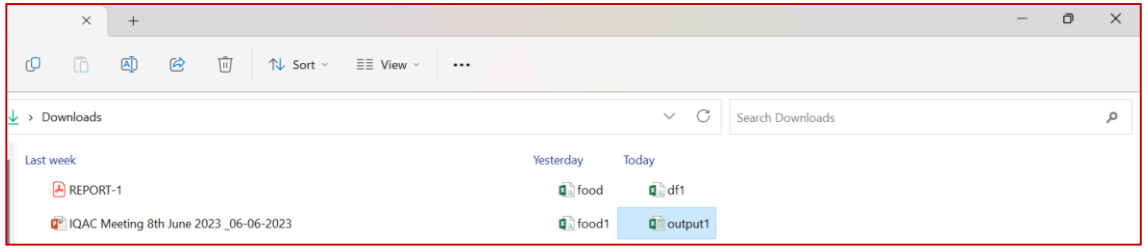


Importing and exporting data

Excel

Writing to an excel file using `DataFrame.to_excel()`:

```
>>> df1.to_excel("C:/Users/rmmna/Downloads/output1.xlsx")
```



	A	B	C
1		col 1	col 2
2	row 1	a	b
3	row 2	c	d
4			

Importing and exporting data

Excel

Reading from an excel file using `read_excel()`:

```
>>>pd.read_excel("C:/Users/rmmna/Downloads/output1.xlsx",
"Sheet1", index_col=None, na_values=["NA"])
```

```
 Unnamed: 0 col 1 col 2
0       row 1     a     b
1       row 2     c     d
```

Lab Experiment

Write a Pandas program to convert a NumPy array to a Pandas series

```
>>> import numpy as np
>>> import pandas as pd
# Define a NumPy array
>>> data = np.array([1, 2, 3, 4, 5])
>>> data
array([1, 2, 3, 4, 5])
>>> print(data)
[1 2 3 4 5]
```

Lab Experiment

Write a Pandas program to convert a NumPy array to a Pandas series cont'd.

```
# Convert the array to a Pandas Series
>>> data_series = pd.Series(data)
>>> data_series
0    1
1    2
2    3
3    4
4    5
dtype: int32
>>> print(data_series)
```

Lab Experiment

Write a Pandas program to convert a NumPy array to a Pandas series cont'd.

In this example,

- we first import the NumPy and Pandas libraries.
- Then, we define a NumPy array '`data`' with some sample data.
- We then use the '`pd.Series()`' function to convert the array to a Pandas Series, and store the result in the '`data_series`' variable.
- Finally, we print the Series using the '`print()`' function.

Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- matplotlib
  - Introduction
  - Installation
  - A simple example
  - Parts of a Figure
  - Types of inputs to plotting functions

matplotlib - Quick start guide

https://matplotlib.org/stable/tutorials/introductory/quick_start.html

Matplotlib

- This is a standard data science library that helps to generate data visualizations such as two-dimensional diagrams and graphs (histograms, scatterplots, non-Cartesian coordinates graphs).
- Matplotlib is one of those plotting libraries that are really useful in data science projects - it provides an object-oriented API for embedding plots into applications.
- Python can compete with scientific tools like MatLab or Mathematica.
- However, developers need to write more code than usual while using this library for generating advanced visualizations.
- Note that popular plotting libraries work seamlessly with Matplotlib.

Matplotlib

- Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.
- Matplotlib makes easy things easy and hard things possible.
  - Create publication quality plots.
  - Make interactive figures that can zoom, pan, update.
  - Customize visual style and layout.
  - Export to many file formats.
  - Embed in JupyterLab and Graphical User Interfaces.
  - Use a rich array of third-party packages built on Matplotlib.

Installing matplotlib

If you already have Python, you can install matplotlib using pip - PyPI (Python Package Index):

```
pip install matplotlib
```

`pip` is a package manager for Python packages, or modules

or

Install using conda:

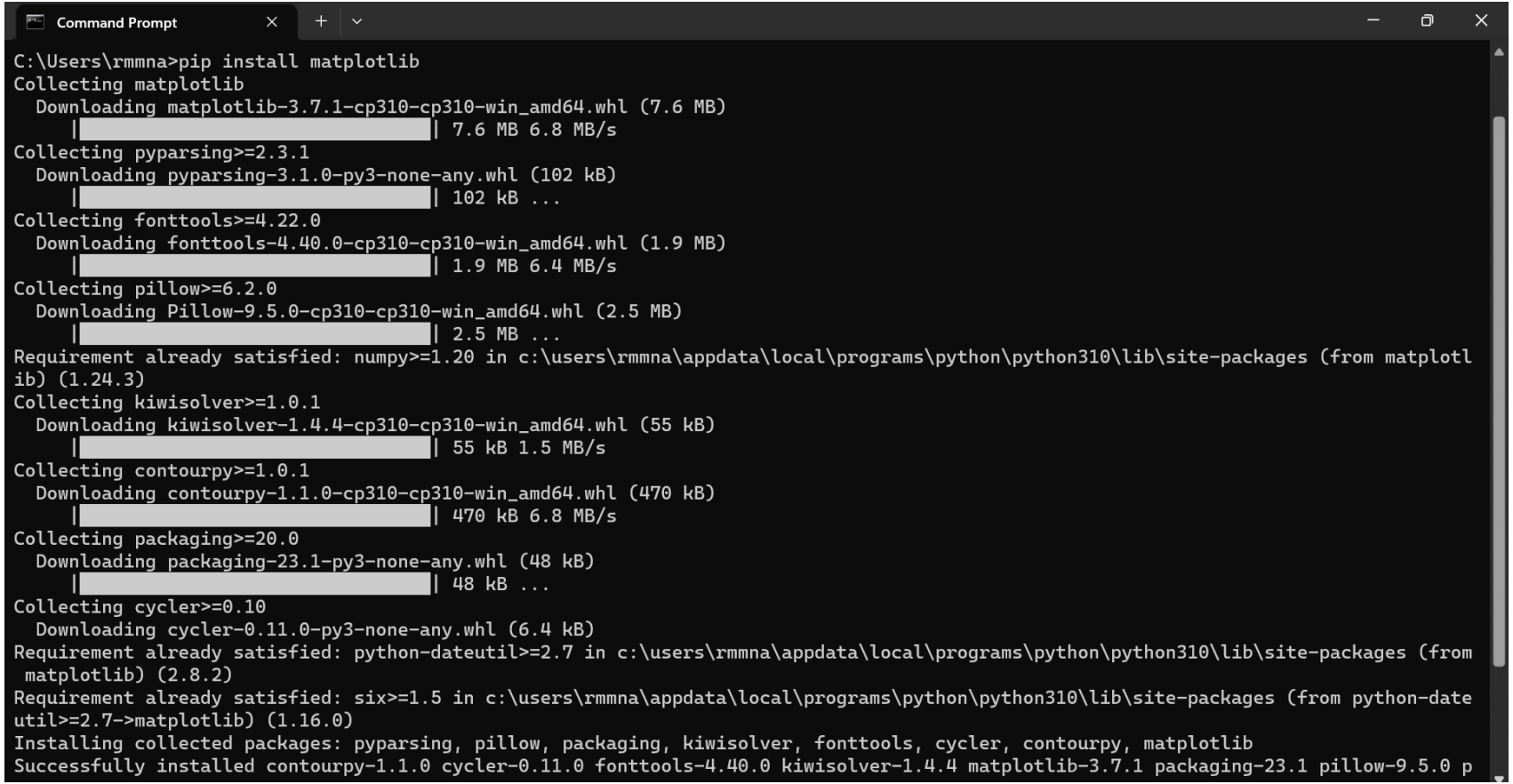
```
conda install -c conda-forge matplotlib
```

`Conda` is a cross platform package and environment manager that installs and manages conda packages from the Anaconda repository as well as from the Anaconda Cloud.

How to install matplotlib using official python IDLE?

- Press the Windows key on your keyboard.
- Type CMD and open Command Prompt. A black terminal should open up.
- Type pip install matplotlib and hit enter.
- It should start the installation. After you see the "Successfully Installed" message, go back to your IDLE and try importing matplotlib, it should work.

How to install matplotlib using official python IDLE?



How to import matplotlib

- To access matplotlib and its functions import it in your Python code like this:

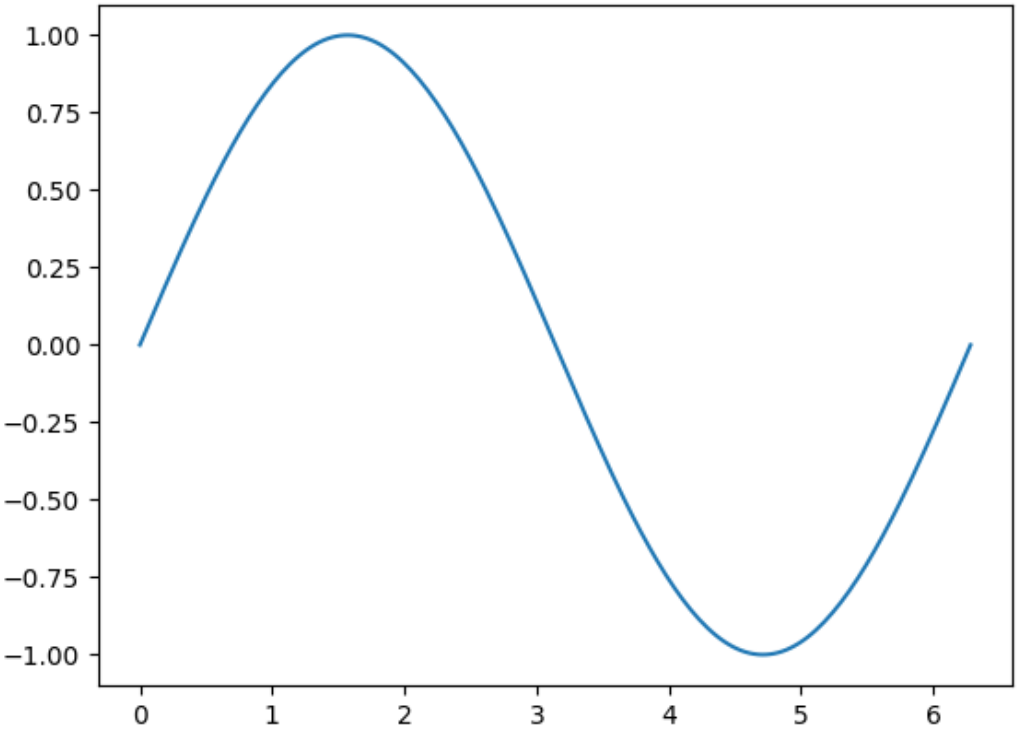
```
import matplotlib as mpl
```

- We shorten the imported name to mpl for better readability of code using matplotlib. This is a widely adopted convention that you should follow so that anyone working with your code can easily understand it.

Draw a first plot

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> x = np.linspace(0, 2 * np.pi, 200)
>>> y = np.sin(x)
>>> fig, ax = plt.subplots()
>>> ax.plot(x, y)
[<matplotlib.lines.Line2D object at 0x000001A0E5021780>]
>>> plt.show()
```

Draw a first plot



Quick start guide

```
>>> import matplotlib as mpl
>>> import matplotlib.pyplot as plt
>>> import numpy as np
```

Pyplot:

- `matplotlib.pyplot` is a collection of functions that make matplotlib work like MATLAB.
- Each pyplot function makes some change to a figure: e.g., creates a figure, creates a plotting area in a figure, plots some lines in a plotting area, decorates the plot with labels, etc.

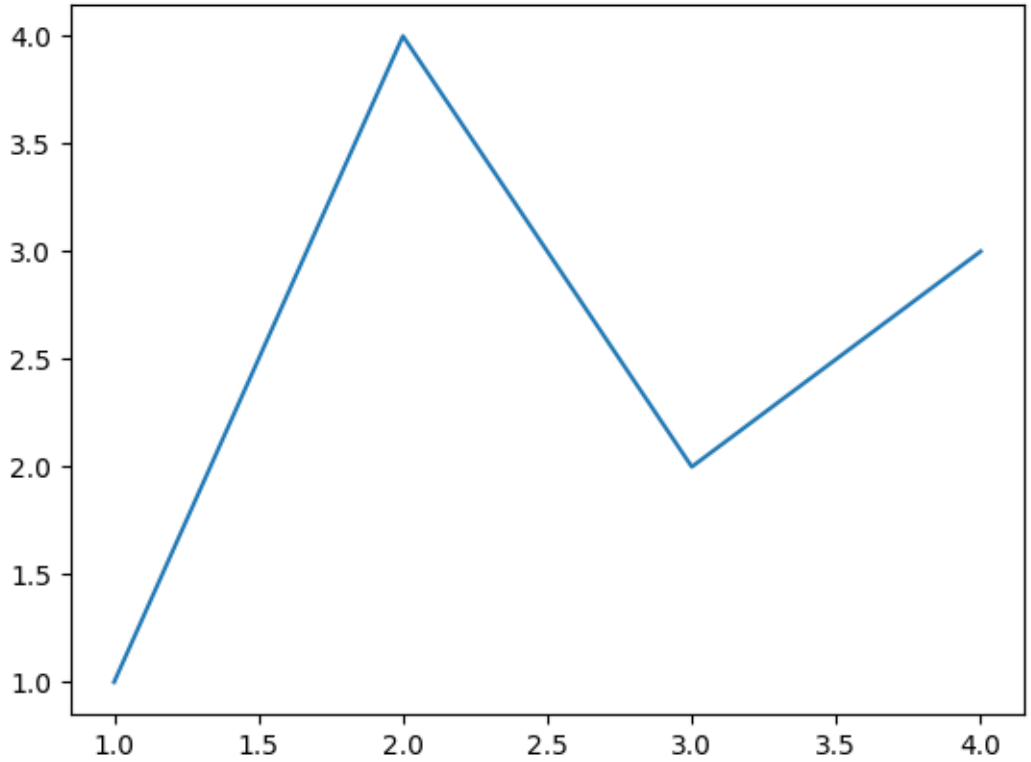
A simple example

- Matplotlib graphs your data on `Figure`s (e.g., windows, Jupyter widgets, etc.), each of which can contain one or more `Axes`, an area where points can be specified in terms of x-y coordinates (or theta-r in a polar plot, x-y-z in a 3D plot, etc.).
- The simplest way of creating a Figure with an Axes is using `pyplot.subplots`.
- We can then use `Axes.plot` to draw some data on the Axes:

A simple example

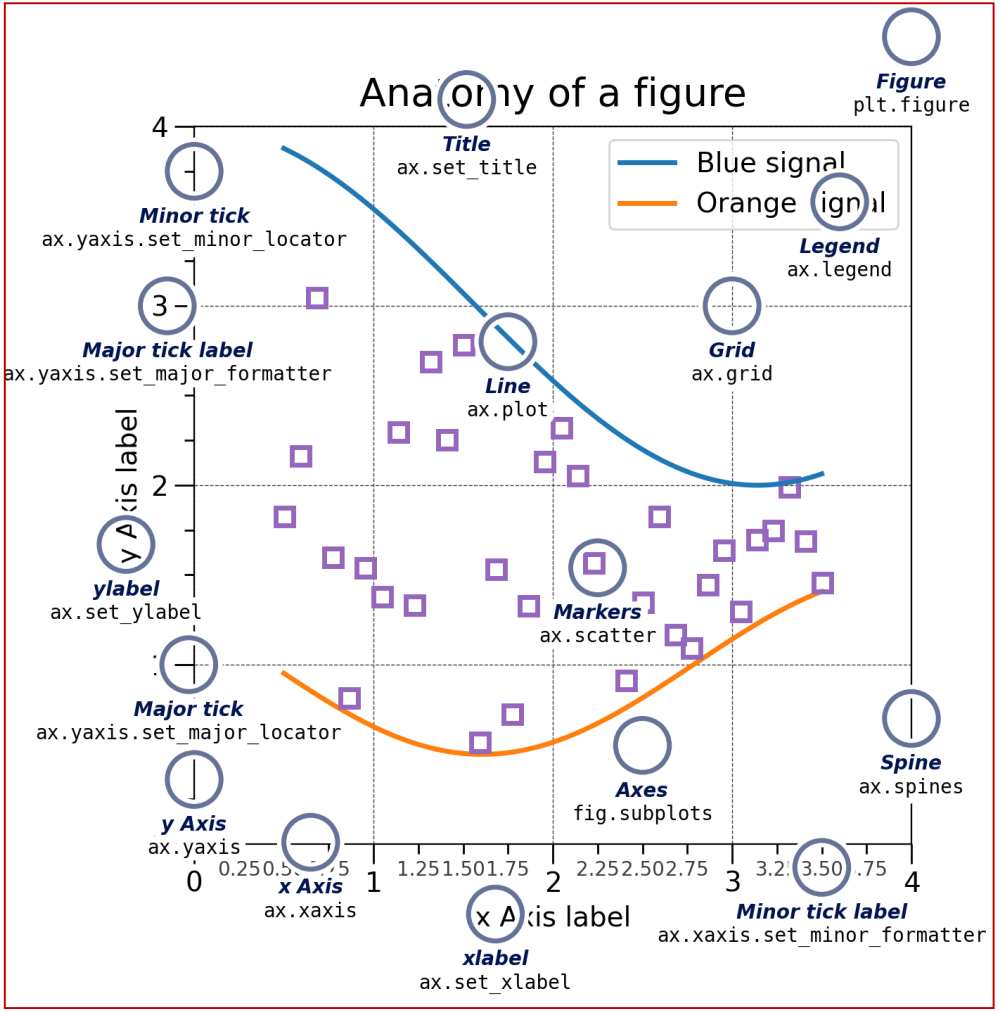
```
>>> fig, ax = plt.subplots() #Create a figure containing a single axes.
>>> ax.plot([1, 2, 3, 4], [1, 4, 2, 3]) # Plot some data on the axes.
[<matplotlib.lines.Line2D object at 0x000002191A694F10>]
>>> plt.show()
```

A simple example



Parts of a Figure

- Here are the components of a Matplotlib Figure.



Parts of a Figure

Figure

- The whole figure.
- The Figure keeps track of all the child `Axes`, a group of 'special' Artists (titles, figure legends, colorbars, etc), and even nested subfigures.
- The easiest way to create a new Figure is with pyplot:

Parts of a Figure

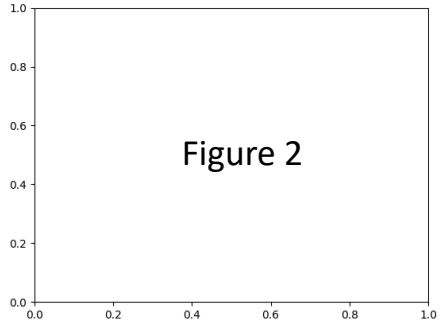
Figure

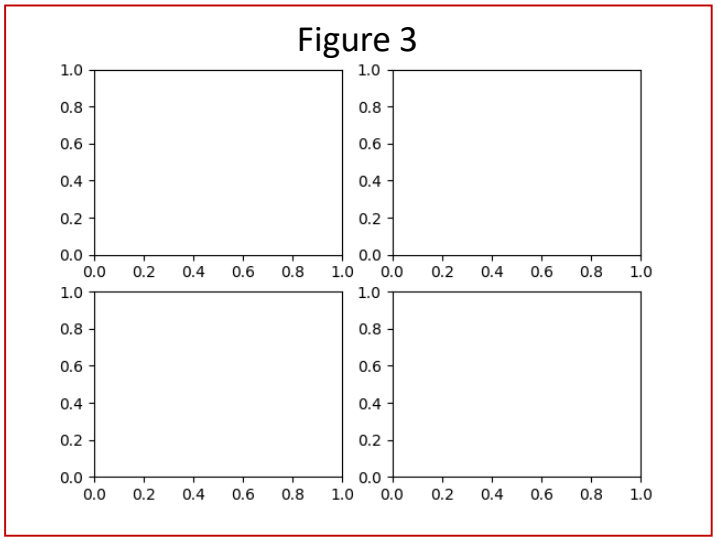
- The easiest way to create a new Figure is with pyplot:

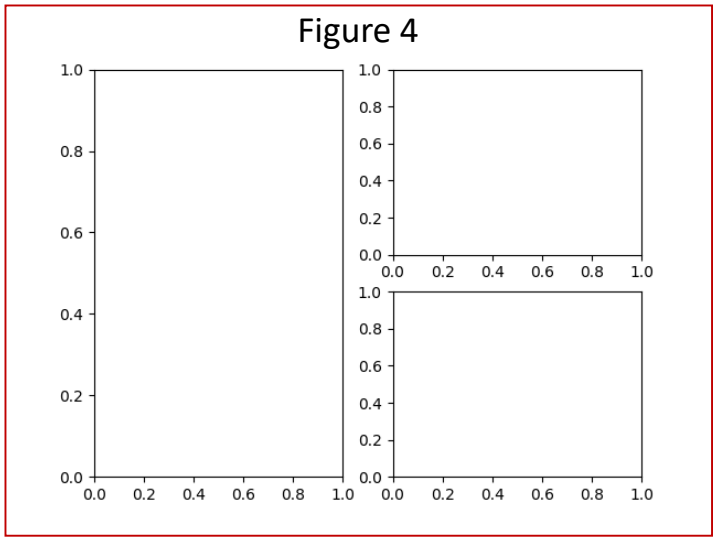
```
>>> fig = plt.figure()  # an empty figure with no Axes
>>> fig, ax = plt.subplots()  # a figure with a single Axes
>>> fig, axs = plt.subplots(2, 2)  # a figure with a 2x2 grid of Axes
# a figure with one axes on the left, and two on the right:
>>> fig, axs = plt.subplot_mosaic([['left', 'right-top'],
                                   ['left', 'right_bottom']])
>>> plt.show()
```

Parts of a Figure

Figure 1







Parts of a Figure

Axes

- An Axes is an Artist attached to a Figure that contains a region for plotting data, and usually includes two (or three in the case of 3D) `Axis` objects (be aware of the difference between Axes and Axis) that provide ticks and tick labels to provide scales for the data in the Axes.
- Each `Axes` also has a title (set via `set_title()`), an x-label (set via `set_xlabel()`), and a y-label set via `set_ylabel()`).
- The `Axes` class and its member functions are the primary entry point to working with the OOP interface, and have most of the plotting methods defined on them (e.g. `ax.plot()`, shown above, uses the `plot` method)

Parts of a Figure

Axis

- These objects set the scale and limits and generate ticks (the marks on the Axis) and ticklabels (strings labeling the ticks).
- The location of the ticks is determined by a `Locator` object and the ticklabel strings are formatted by a `Formatter`.
- The combination of the correct `Locator` and `Formatter` gives very fine control over the tick locations and labels.

Parts of a Figure

Artist

- Basically, everything visible on the Figure is an Artist (even `Figure`, `Axes`, and `Axis` objects).
- This includes `Text` objects, `Line2D` objects, `collections` objects, `Patch` objects, etc.
- When the Figure is rendered, all of the Artists are drawn to the canvas.
- Most Artists are tied to an Axes; such an Artist cannot be shared by multiple Axes, or moved from one to another.

Types of inputs to plotting functions

- Plotting functions expect `numpy.array` or `numpy.ma.masked_array` as input, or objects that can be passed to `numpy.asarray`.
- Classes that are similar to arrays ('array-like') such as `pandas` data objects and `numpy.matrix` may not work as intended.
- Common convention is to convert these to `numpy.array` objects prior to plotting.
- For example, to convert a `numpy.matrix`

Types of inputs to plotting functions

For example, to convert a `numpy.matrix`

```
>>> import numpy as np
>>> b = np.matrix([[1, 2], [3, 4]])
>>> b_asarray = np.asarray(b)
>>> b_asarray
array([[1, 2],
       [3, 4]])
>>> print(b_asarray)
[[1 2]
 [3 4]]
```

Types of inputs to plotting functions

- Most methods will also parse an addressable object like a *dict*, a `numpy.recarray`, or a `pandas.DataFrame`.
- Matplotlib allows you to provide the `data` keyword argument and generate plots passing the strings corresponding to the *x* and *y* variables.

Types of inputs to plotting functions

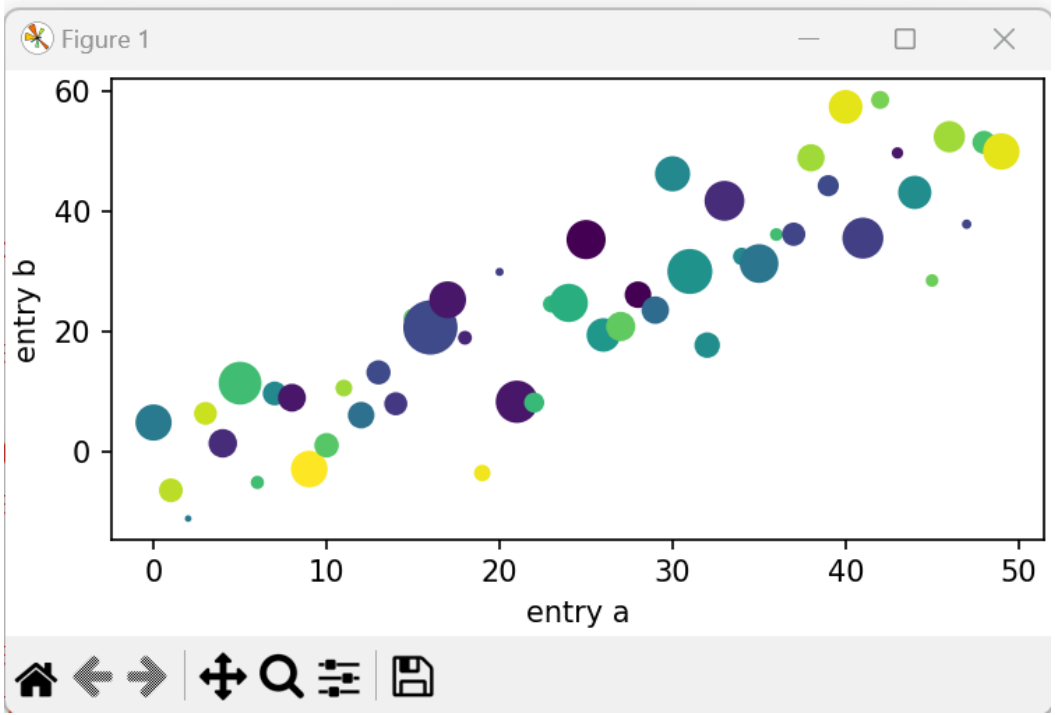
```
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> np.random.seed(19680801)
>>> data = {'a': np.arange(50),
        'c': np.random.randint(0, 50, 50),
        'd': np.random.randn(50)}
>>> data['b'] = data['a'] + 10 * np.random.randn(50)
>>> data['d'] = np.abs(data['d']) * 100
>>>    fig,    ax    =    plt.subplots(figsize=(5,    2.7),
layout='constrained')
```

Types of inputs to plotting functions

```
>>> ax.scatter('a', 'b', c='c', s='d', data=data)
<matplotlib.collections.PathCollection object at 0x000001C94970E380>
>>> ax.set_xlabel('entry a')
Text(0.5, 0, 'entry a')
>>> ax.set_ylabel('entry b')
Text(0, 0.5, 'entry b')
```

Types of inputs to plotting functions

```
>>> plt.show()
```



Object Oriented Programming (Using Python)

UNIT- III

Python Libraries:

- Matplotlib cont'd.
  - Coding Styles
  - Styling Artists
  - Labelling plots
  - Axis scales and ticks
  - Color mapped data

matplotlib - Quick start guide

https://matplotlib.org/stable/tutorials/introductory/quick_start.html

Coding Styles

The explicit and the implicit interfaces

There are essentially two ways to use Matplotlib:

- Explicitly create Figures and Axes, and call methods on them (the "object-oriented (OO) style").
- Rely on `pyplot` to implicitly create and manage the Figures and Axes, and use `pyplot` functions for plotting.

Coding Styles

The explicit and the implicit interfaces

Example of Object-oriented (OO) style:

```
>>> import matplotlib.pyplot as plt
# Note that even in the OO-style, we use `.pyplot.figure` to
create the Figure.
>>> fig, ax = plt.subplots(figsize=(5, 2.7),
layout='constrained')
>>> ax.plot(x, x, label='linear')  # Plot some data on the
axes.
[<matplotlib.lines.Line2D object at 0x000001417FE6F5B0>]
```

Coding Styles

The explicit and the implicit interfaces

Example of Object-oriented (OO) style: cont'd.

```
>>> ax.plot(x, x**2, label='quadratic')  # Plot more data on
the axes...
[<matplotlib.lines.Line2D object at 0x000001417FE6F880>]
>>> ax.plot(x, x**3, label='cubic')  # ... and some more.
[<matplotlib.lines.Line2D object at 0x000001417FE6FB20>]
>>> ax.set_xlabel('x label')  # Add an x-label to the axes.
Text(0.5, 0, 'x label')
>>> ax.set_ylabel('y label')  # Add a y-label to the axes.
Text(0, 0.5, 'y label')
```

Coding Styles

The explicit and the implicit interfaces

Example of Object-oriented (OO) style: cont'd.

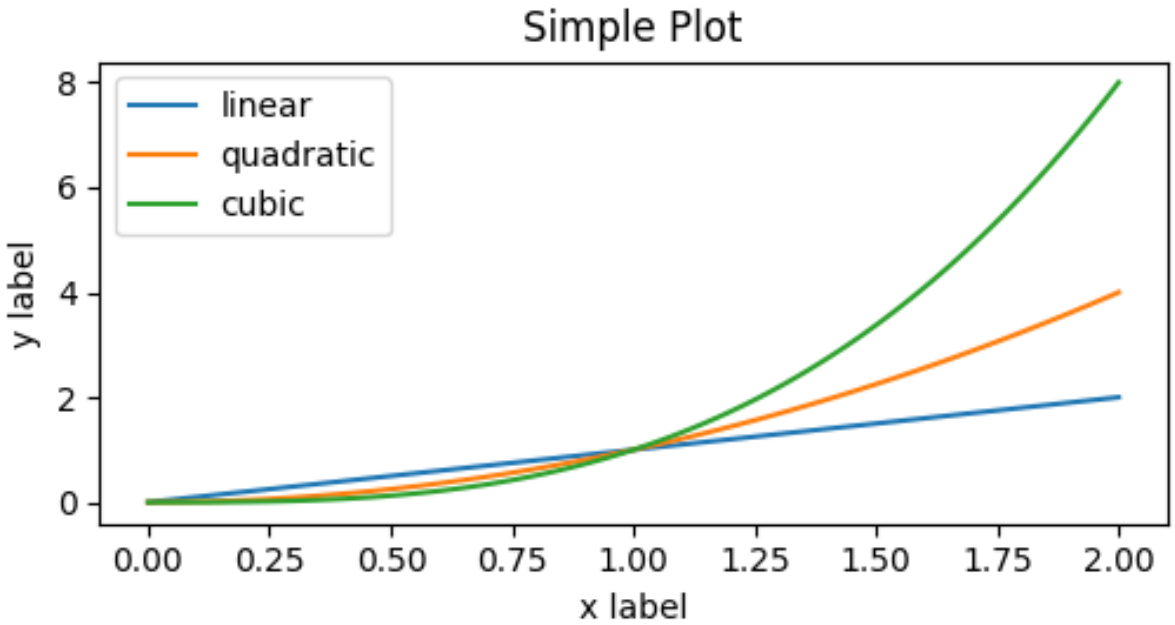
```
>>> ax.set_title("Simple Plot")  # Add a title to the axes.
Text(0.5, 1.0, 'Simple Plot')
>>> ax.legend()  # Add a legend.
<matplotlib.legend.Legend object at 0x000001417FE6FB50>
```

Coding Styles

The explicit and the implicit interfaces

Example of Object-oriented (OO) style: cont'd.

```
>>> plt.show()
```



Coding Styles

The explicit and the implicit interfaces

Example of pyplot-style:

```
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> x = np.linspace(0, 2, 100)  # Sample data.
>>> plt.figure(figsize=(5, 2.7), layout='constrained')
<Figure size 500x270 with 0 Axes>
>>> plt.plot(x, x, label='linear')  # Plot some data on
the (implicit) axes.
[<matplotlib.lines.Line2D object at 0x000001410000BA60>]
```

Coding Styles

The explicit and the implicit interfaces

Example of pyplot-style: cont'd.

```
>>> plt.plot(x, x**2, label='quadratic')  # etc.
[<matplotlib.lines.Line2D object at 0x000001410000BD30>]
>>> plt.plot(x, x**3, label='cubic')
[<matplotlib.lines.Line2D object at 0x000001410000BFD0>]
>>> plt.xlabel('x label')
Text(0.5, 0, 'x label')
>>> plt.ylabel('y label')
Text(0, 0.5, 'y label')
```

Coding Styles

The explicit and the implicit interfaces

Example of pyplot-style: cont'd.

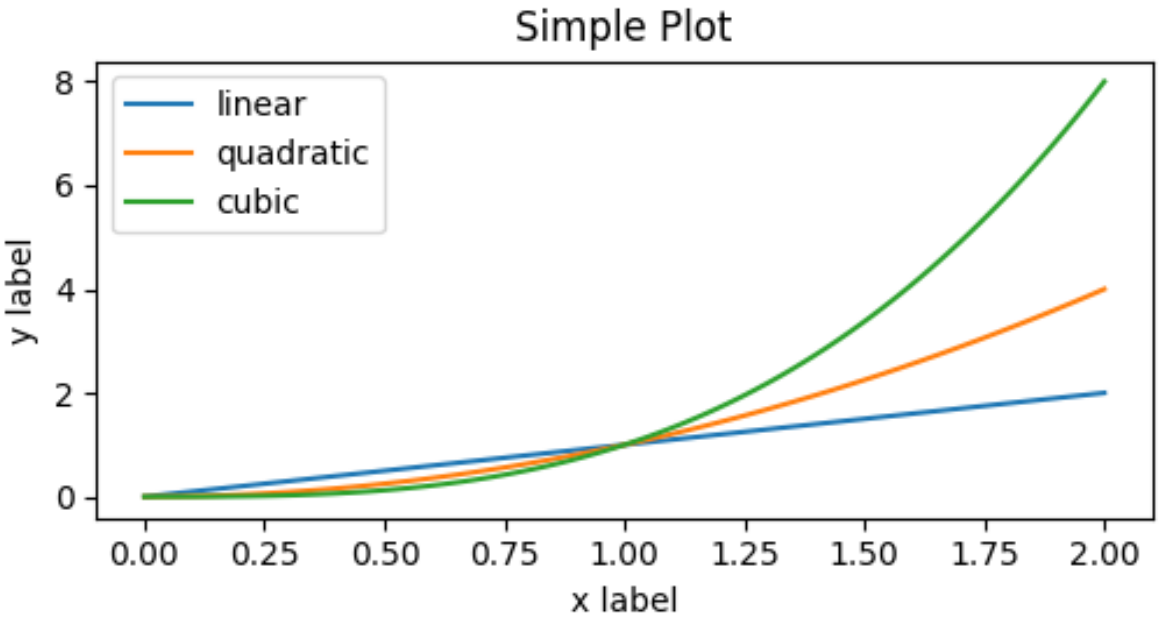
```
>>> plt.title("Simple Plot")
Text(0.5, 1.0, 'Simple Plot')
>>> plt.legend()
<matplotlib.legend.Legend object at 0x000001417FF4B190>
```

Coding Styles

The explicit and the implicit interfaces

Example of pyplot-style: cont'd.

```
>>> plt.show()
```



Coding Styles

Making a helper functions

If we need to make the same plots over and over again with different data sets, or want to easily wrap Matplotlib methods, use the recommended signature function below.

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> def my_plotter(ax, data1, data2, param_dict):
    """
    A helper function to make a graph.
    """
    out = ax.plot(data1, data2, **param_dict)
    return out
```

Coding Styles

Making a helper functions

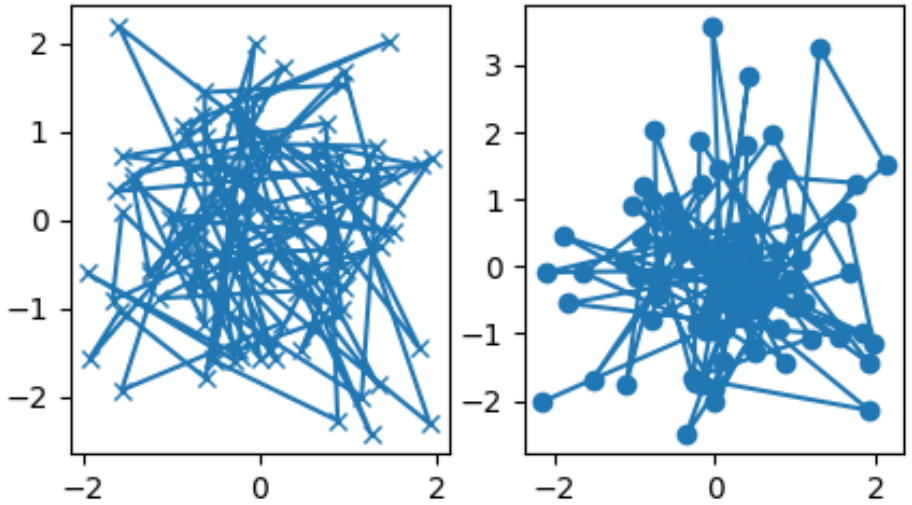
We would then use twice to populate two subplots:

```
>>> data1, data2, data3, data4 = np.random.randn(4, 100)
# make 4 random data sets
>>> fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(5,2.7))
>>> my_plotter(ax1, data1, data2, {'marker': 'x'})
[<matplotlib.lines.Line2D object at 0x000001F3D45F5B70>]
>>> my_plotter(ax2, data3, data4, {'marker': 'o'})
[<matplotlib.lines.Line2D object at 0x000001F3D45F5E70>]
```

Coding Styles

Making a helper functions

```
>>> plt.show()
```



Note that if you want to install these as a python package, or any other customizations you could use one of the many templates on the web; Matplotlib has one at `mpl-cookiecutter`

Styling Artists

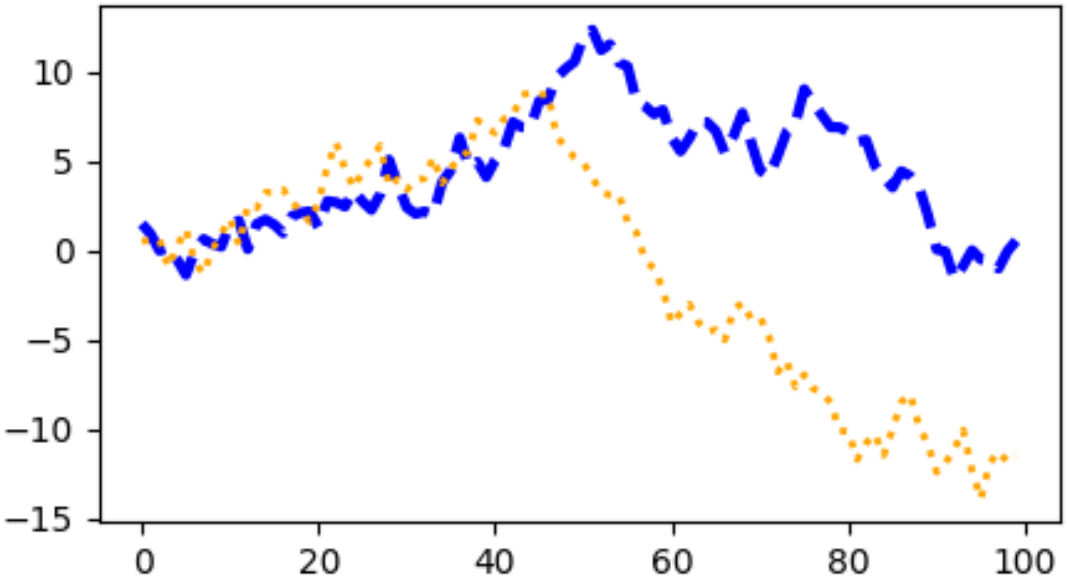
- Most plotting methods have styling options for the Artists, accessible either when a plotting method is called, or from a "setter" on the Artist.
- In the plot below we manually set the *color*, *linewidth*, and *linestyle* of the Artists created by `plot`, and we set the linestyle of the second line after the fact with `set_linestyle`.

Styling Artists

```
>>> import matplotlib.pyplot as plt
>>> data1, data2, data3, data4 = np.random.randn(4, 100)
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>> x = np.arange(len(data1))
>>>     ax.plot(x,     np.cumsum(data1),     color='blue',
linewidth=3, linestyle='--')
[<matplotlib.lines.Line2D object at 0x000001F3D2E2F580>]
>>>  l,  =  ax.plot(x,  np.cumsum(data2),  color='orange',
linewidth=2)
>>> l.set_linestyle(':')
```

Styling Artists

```
>>> plt.show()
```



Styling Artists

Colors

- Matplotlib has a very flexible array of colors that are accepted for most Artists.
- Some Artists will take multiple colors. i.e. for a scatter plot, the edge of the markers can be different colors from the interior:

Styling Artists

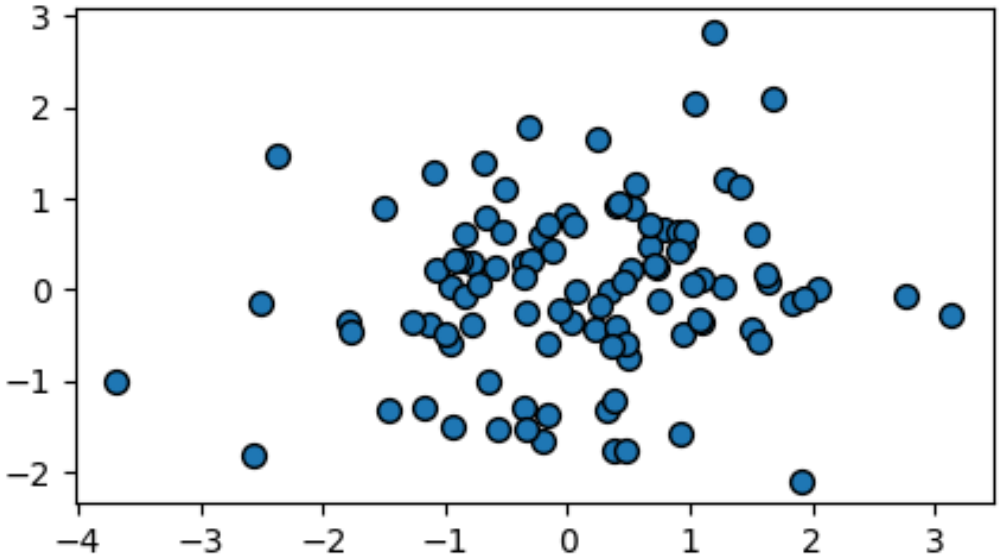
Colors

```
>>> import matplotlib.pyplot as plt
>>> data1, data2, data3, data4 = np.random.randn(4, 100)
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>>   ax.scatter(data1,   data2,   s=50,   facecolor='C0',
edgecolor='k')
<matplotlib.collections.PathCollection object at
0x000001ED8D6A4FD0>
```

Styling Artists

Colors

```
>>> plt.show()
```



Styling Artists

Linewidths, linestyles, and markersizes

- Line widths are typically in typographic points (1 pt = 1/72 inch) and available for Artists that have stroked lines.
- Similarly, stroked lines can have a linestyle.
- Marker size depends on the method being used.
- `plot` specifies markersize in points, and is generally the "diameter" or width of the marker.
- `scatter` specifies markersize as approximately proportional to the visual area of the marker.
- There is an array of markerstyles available as string codes, or users can define their own `MarkerStyle`.

Styling Artists

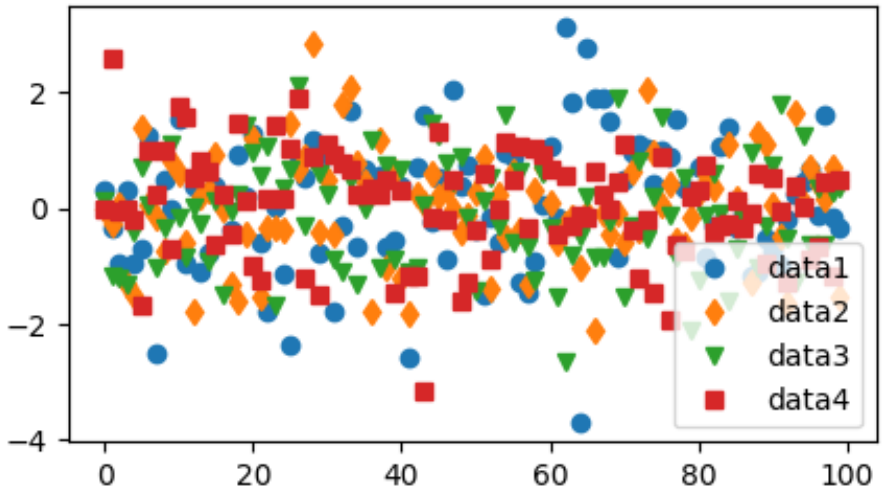
Linewidths, linestyles, and markersizes

```
>>> import matplotlib.pyplot as plt
>>> data1, data2, data3, data4 = np.random.randn(4, 100)
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>> ax.plot(data1, 'o', label='data1')
[<matplotlib.lines.Line2D object at 0x000001ED8BEF8970>]
>>> ax.plot(data2, 'd', label='data2')
[<matplotlib.lines.Line2D object at 0x000001ED8BEF8C40>]
>>> ax.plot(data3, 'v', label='data3')
[<matplotlib.lines.Line2D object at 0x000001ED8BEF8DC0>]
>>> ax.plot(data4, 's', label='data4')
[<matplotlib.lines.Line2D object at 0x000001ED8BEF9060>]
```

Styling Artists

Linewidths, linestyles, and markersizes

```
>>> ax.legend()
<matplotlib.legend.Legend object at 0x000001ED8BEF91E0>
>>> plt.show()
```



Labelling plots

Axes labels and text

- `set_xlabel`, `set_ylabel`, and `set_title` are used to add text in the indicated locations.
- Text can also be directly added to plots using `text`:

Labelling plots

Axes labels and text

```
>>> import matplotlib.pyplot as plt
>>> mu, sigma = 115, 15
>>> x = mu + sigma * np.random.randn(10000)
>>>    fig,    ax    =    plt.subplots(figsize=(5,    2.7),
layout='constrained')
>>>  n,  bins,  patches  =  ax.hist(x,  50,  density=True,
facecolor='C0', alpha=0.75)
>>> ax.set_xlabel('Length [cm]')
Text(0.5, 0, 'Length [cm]')
>>> ax.set_ylabel('Probability')
Text(0, 0.5, 'Probability')
```

Labelling plots

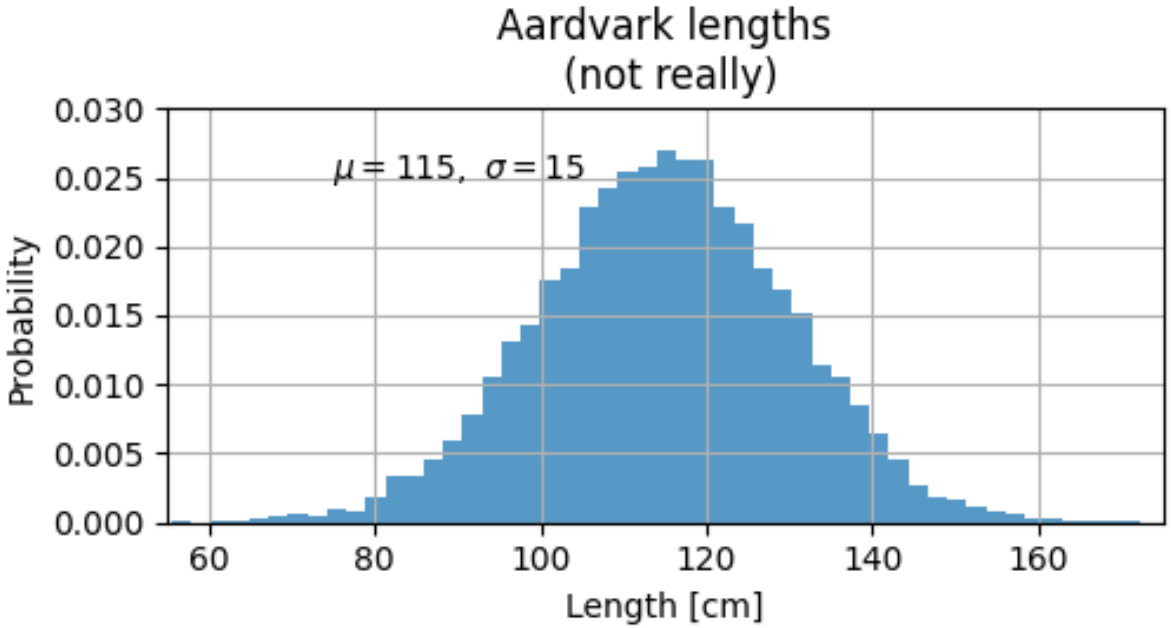
Axes labels and text

```
>>> ax.set_title('Aardvark lengths\n (not really)')
Text(0.5, 1.0, 'Aardvark lengths\n (not really)')
>>> ax.text(75, .025, r'$\mu=115,\ \sigma=15$')
Text(75, 0.025, '$\\mu=115,\\ \\sigma=15$')
>>> ax.axis([55, 175, 0, 0.03])
(55.0, 175.0, 0.0, 0.03)
>>> ax.grid(True)
```

Labelling plots

Axes labels and text

```
>>> plt.show()
```



Labelling plots

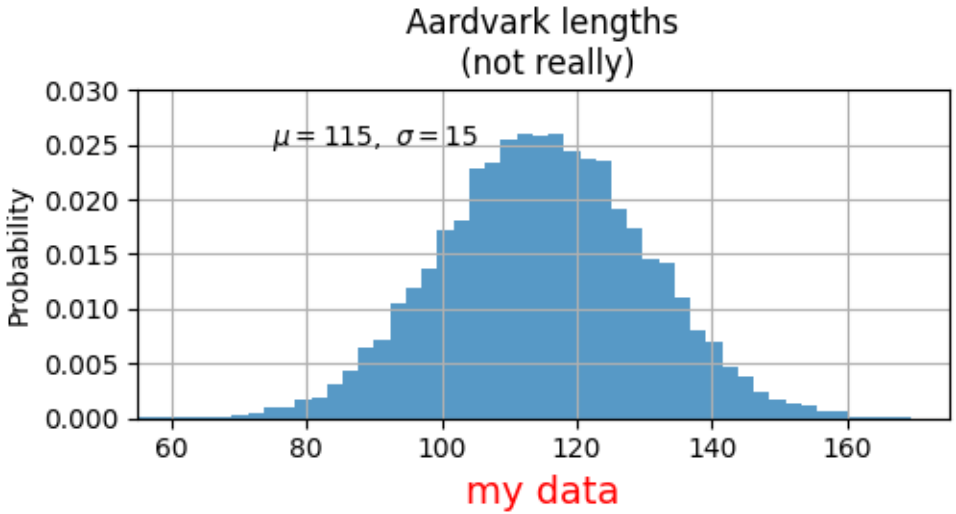
Axes labels and text

- All of the `text` functions return a `matplotlib.text.Text` instance.
- Just as with lines above, you can customize the properties by passing keyword arguments into the text functions:

Labelling plots

Axes labels and text

```
>>> ax.set_xlabel('my data', fontsize=14, color='red')
>>> plt.show()
```



Labelling plots

Annotations

- We can also annotate points on a plot, often by connecting an arrow pointing to `xy`, to a piece of text at *`xytext`*:

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>> t = np.arange(0.0, 5.0, 0.01)
>>> s = np.cos(2 * np.pi * t)
>>> line, = ax.plot(t, s, lw=2)
```

Labelling plots

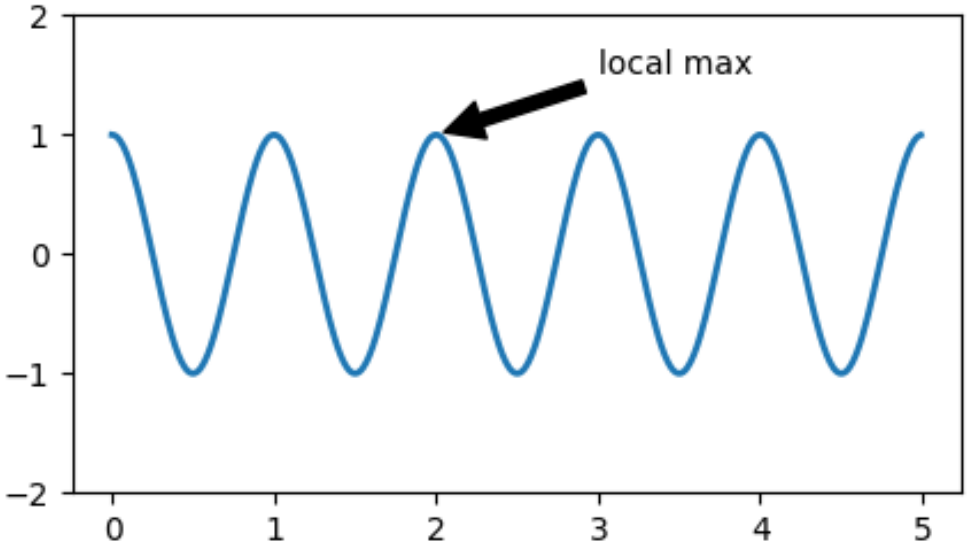
Annotations

```
>>> ax.annotate('local max', xy=(2, 1), xytext=(3, 1.5),
            arrowprops=dict(facecolor='black',
shrink=0.05))
Text(3, 1.5, 'local max')
>>> ax.set_ylim(-2, 2)
(-2.0, 2.0)
```

Labelling plots

Annotations

```
>>> plt.show()
```



Labelling plots

Legends

- Often we want to identify lines or markers with a `Axes.legend`:

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> fig, ax = plt.subplots(figsize=(5, 2.7))
>>> data1, data2, data3, data4 = np.random.randn(4, 100)
>>> ax.plot(np.arange(len(data1)), data1, label='data1')
[<matplotlib.lines.Line2D object at 0x000001ED9036CAC0>]
```

Labelling plots

Legends

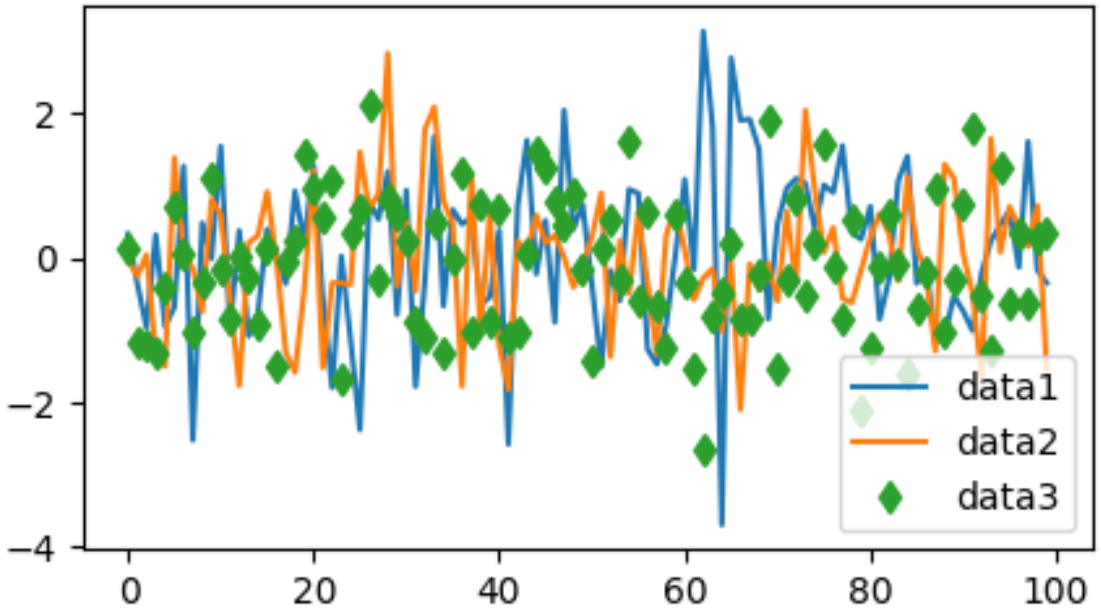
- Often we want to identify lines or markers with a `Axes.legend`:

```
>>> ax.plot(np.arange(len(data2)), data2, label='data2')
[<matplotlib.lines.Line2D object at 0x000001ED9036CD90>]
>>>     ax.plot(np.arange(len(data3)),     data3,     'd',
label='data3')
 [<matplotlib.lines.Line2D object at 0x000001ED9036D030>]
>>> ax.legend()
<matplotlib.legend.Legend object at 0x000001ED902EC6A0>
```

Labelling plots

Legends

```
>>> plt.show()
```



Axis scales and ticks

- Each Axes has two (or three) `Axis` objects representing the x- and y-axis.
- These control the *scale* of the Axis, the tick *locators* and the tick *formatters*.
- Additional Axes can be attached to display further Axis objects.

Axis scales and ticks

Scales

- In addition to the linear scale, Matplotlib supplies non-linear scales, such as a log-scale.
- Since log-scales are used so much there are also direct methods like `loglog`, `semilogx`, and `semilogy`.
- There are a number of scales.
- Here we set the scale manually:

Axis scales and ticks

Scales

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> fig, axs = plt.subplots(1, 2, figsize=(5, 2.7),
layout='constrained')
>>> data1, data2, data3, data4 = np.random.randn(4, 100)
>>> xdata = np.arange(len(data1))  # make an ordinal for
this
>>> data = 10**data1
```

Axis scales and ticks

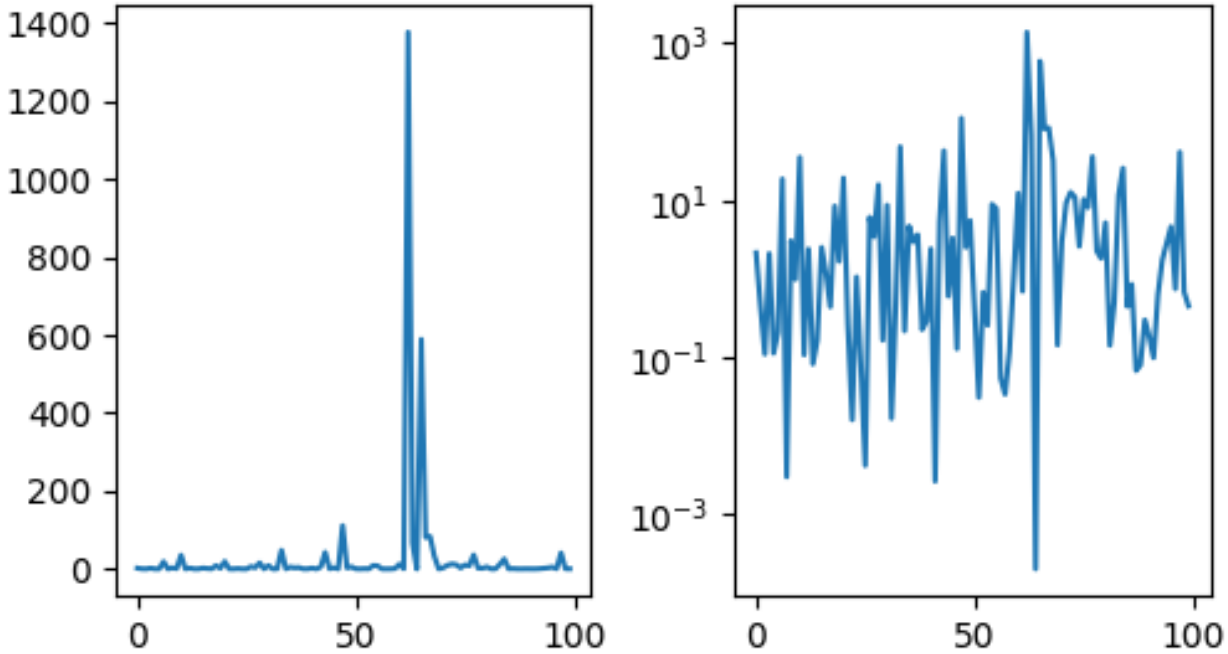
Scales

```
>>> axs[0].plot(xdata, data)
[<matplotlib.lines.Line2D object at 0x000001ED9041D1E0>]
>>> axs[1].set_yscale('log')
>>> axs[1].plot(xdata, data)
[<matplotlib.lines.Line2D object at 0x000001ED9041D4E0>]
```

Axis scales and ticks

Scales

```
>>> plt.show()
```



Axis scales and ticks

Scales

- The scale sets the mapping from data values to spacing along the Axis.
- This happens in both directions, and gets combined into a *transform*, which is the way that Matplotlib maps from data coordinates to Axes, Figure, or screen coordinates.

Axis scales and ticks

Tick locators and formatters

- Each Axis has a tick *locator* and *formatter* that choose where along the Axis objects to put tick marks.
- A simple interface to this is `set_xticks`:

Axis scales and ticks

Tick locators and formatters

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> fig, axs = plt.subplots(2, 1, layout='constrained')
>>> data1, data2, data3, data4 = np.random.randn(4, 100)
>>> xdata = np.arange(len(data1))  # make an ordinal for
this
>>> axs[0].plot(xdata, data1)
[<matplotlib.lines.Line2D object at 0x000001ED902C60E0>]
```

Axis scales and ticks

Tick locators and formatters

```
>>> axs[1].plot(xdata, data1)
[<matplotlib.lines.Line2D object at 0x000001ED8C038A00>]
>>> axs[1].set_xticks(np.arange(0, 100, 30), ['zero',
'30', 'sixty', '90'])
[<matplotlib.axis.XTick object at 0x000001ED8C039FF0>,
<matplotlib.axis.XTick object at 0x000001ED8C039E40>,
<matplotlib.axis.XTick object at 0x000001ED902C4520>,
<matplotlib.axis.XTick object at 0x000001ED8BF90700>]
```

Axis scales and ticks

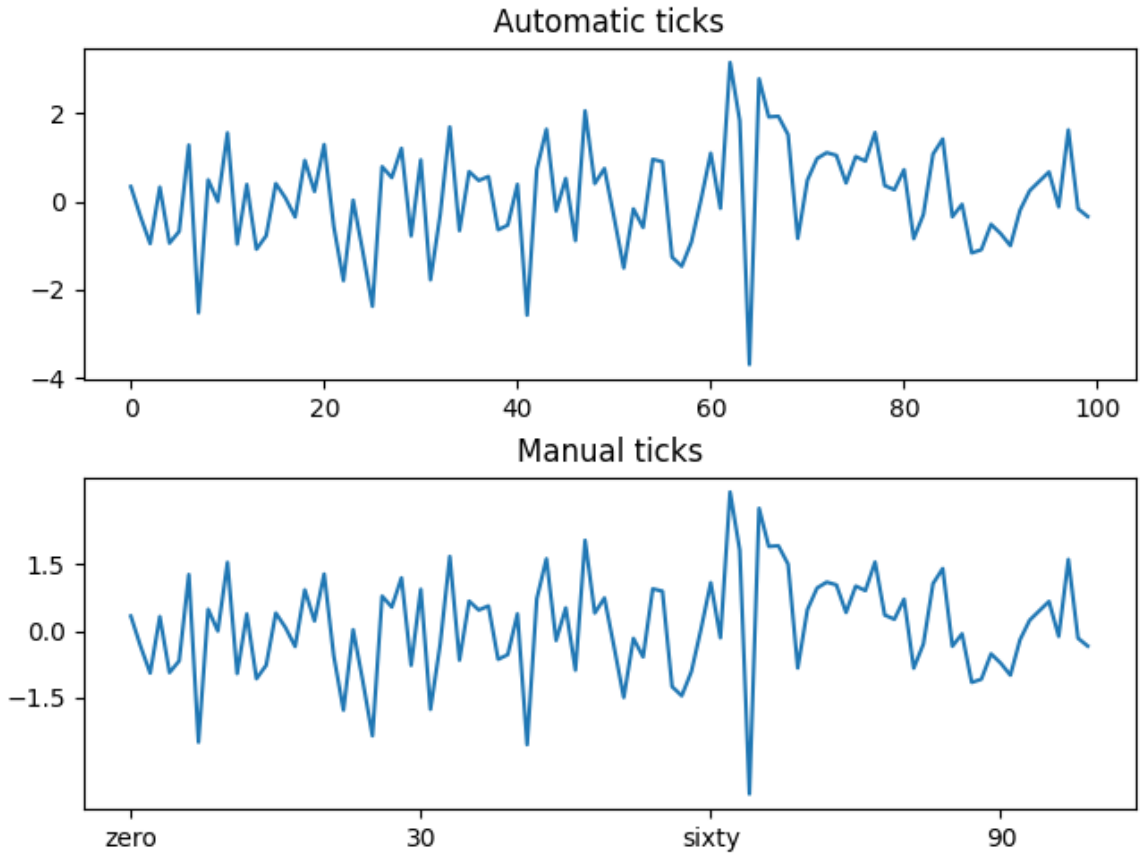
Tick locators and formatters

```
>>> axs[1].set_yticks([-1.5, 0, 1.5])  # note that we
don't need to specify labels
[<matplotlib.axis.YTick object at 0x000001ED9045C640>,
<matplotlib.axis.YTick object at 0x000001ED8C038190>,
<matplotlib.axis.YTick object at 0x000001ED902C6170>]
>>> axs[1].set_title('Manual ticks')
Text(0.5, 1.0, 'Manual ticks')
```

Axis scales and ticks

Tick locators and formatters

```
>>> plt.show()
```



Axis scales and ticks

Tick locators and formatters

- Different scales can have different locators and formatters; for instance the log-scale above uses `LogLocator` and `LogFormatter`.

Axis scales and ticks

Plotting dates and strings

- Matplotlib can handle plotting arrays of dates and arrays of strings, as well as floating point numbers.
- These get special locators and formatters as appropriate.
- For dates:

Axis scales and ticks

Plotting dates and strings

```
>>> import matplotlib as mpl
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> fig, ax = plt.subplots(figsize=(5, 2.7),
layout='constrained')
>>> dates = np.arange(np.datetime64('2023-06-28'),
np.datetime64('2023-07-15'),
                      np.timedelta64(1, 'h'))
```

Axis scales and ticks

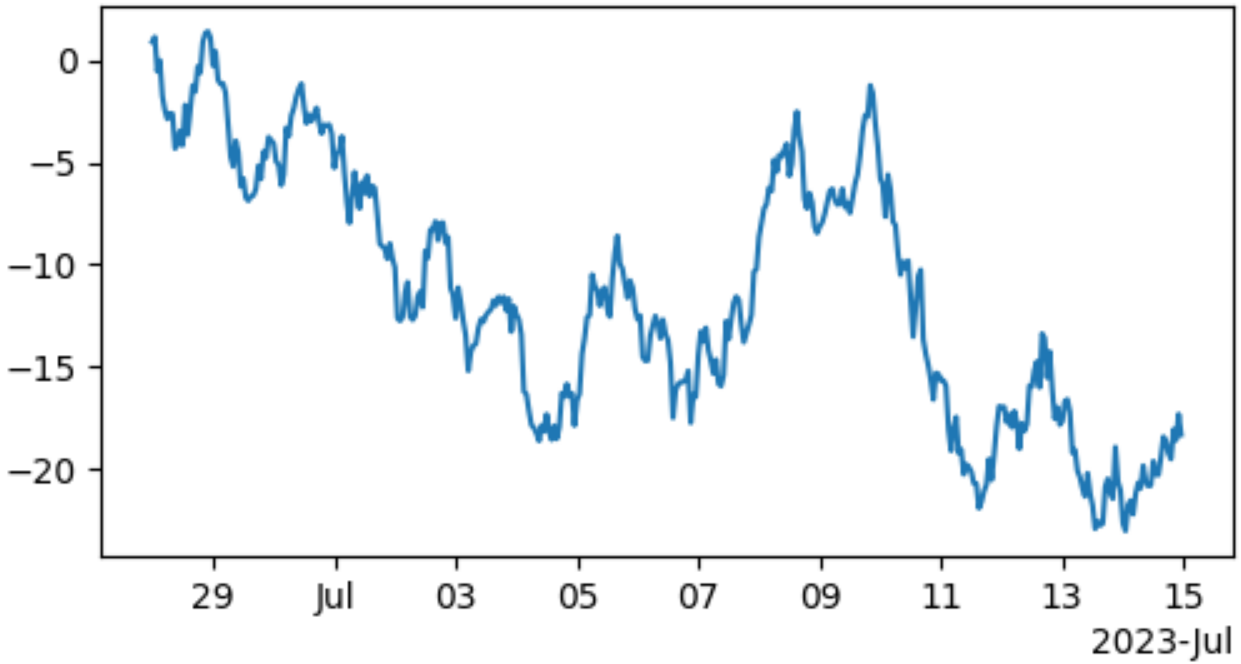
Plotting dates and strings

```
>>> data = np.cumsum(np.random.randn(len(dates)))
>>> ax.plot(dates, data)
[<matplotlib.lines.Line2D object at 0x000001ED8BF51060>]
>>> cdf =
mpl.dates.ConciseDateFormatter(ax.xaxis.get_major_locator
())
>>> ax.xaxis.set_major_formatter(cdf)
```

Axis scales and ticks

Plotting dates and strings

```
>>> plt.show()
```



Axis scales and ticks

Plotting dates and strings

- For strings: we get categorical plotting

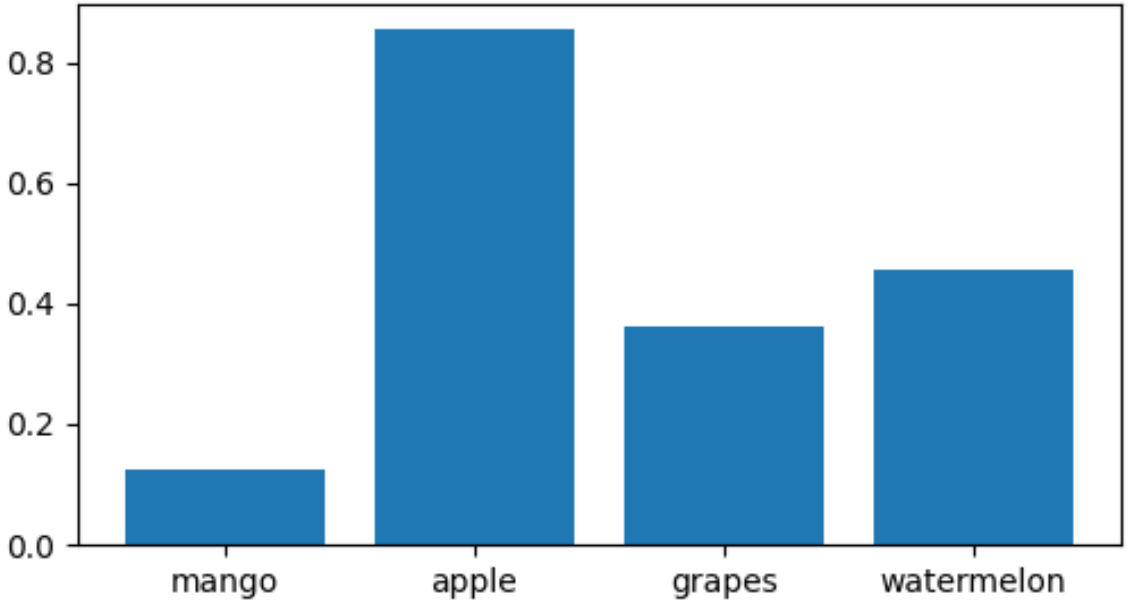
```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> fig, ax = plt.subplots(figsize=(5, 2.7),
layout='constrained')
>>> categories = ['mango', 'apple', 'grapes',
'watermelon']
>>> ax.bar(categories, np.random.rand(len(categories)))
<BarContainer object of 4 artists>
```

Axis scales and ticks

Plotting dates and strings

- For strings: we get categorical plotting

```
>>> plt.show()
```



Axis scales and ticks

Plotting dates and strings

- One limitation about categorical plotting is that some methods of parsing text files return a list of strings, even if the strings all represent numbers or dates.
- If you pass 1000 strings, Matplotlib will think you meant 1000 categories and will add 1000 ticks to your plot.

Axis scales and ticks

Additional Axis objects

- Plotting data of different magnitude in one chart may require an additional y-axis.
- Such an Axis can be created by using `twinx` to add a new Axes with an invisible x-axis and a y-axis positioned at the right (analogously for `twiny`).
- Similarly, you can add a `secondary_xaxis` or `secondary_yaxis` having a different scale than the main Axis to represent the data in different scales or units.

Axis scales and ticks

Additional Axis objects

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> fig, (ax1, ax3) = plt.subplots(1, 2, figsize=(7, 2.7), layout='constrained')
>>> l1, = ax1.plot(t, s)
>>> ax2 = ax1.twinx()
>>> l2, = ax2.plot(t, range(len(t)), 'C1')
>>> ax2.legend([l1, l2], ['Sine (left)', 'Straight (right)'])
<matplotlib.legend.Legend object at 0x000001ED905501C0>
```

Axis scales and ticks

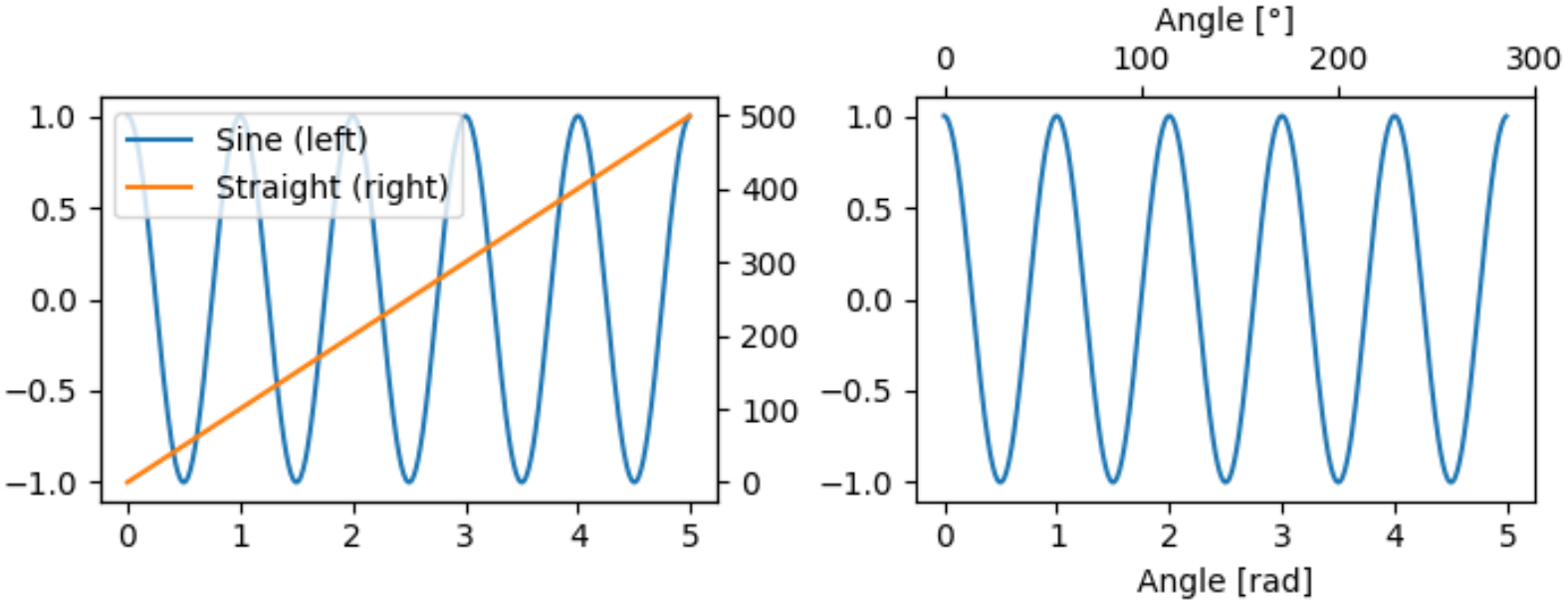
Additional Axis objects

```
>>> ax3.plot(t, s)
[<matplotlib.lines.Line2D object at 0x000001ED90581C90>]
>>> ax3.set_xlabel('Angle [rad]')
Text(0.5, 0, 'Angle [rad]')
>>> ax4 = ax3.secondary_xaxis('top',
functions=(np.rad2deg, np.deg2rad))
>>> ax4.set_xlabel('Angle [°]')
Text(0.5, 0, 'Angle [°]')
```

Axis scales and ticks

Additional Axis objects

```
>>> plt.show()
```



Color mapped data

- Often we want to have a third dimension in a plot represented by a colors in a colormap.
- Matplotlib has a number of plot types that do this:

Color mapped data

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> X, Y = np.meshgrid(np.linspace(-3, 3, 128),
np.linspace(-3, 3, 128))
>>> Z = (1 - X/2 + X**5 + Y**3) * np.exp(-X**2 - Y**2)
>>> fig, axs = plt.subplots(2, 2, layout='constrained')
>>> pc = axs[0, 0].pcolormesh(X, Y, Z, vmin=-1, vmax=1,
cmap='RdBu_r')
>>> fig.colorbar(pc, ax=axs[0, 0])
<matplotlib.colorbar.Colorbar object at
0x000001ED9372DDB0>
```

Color mapped data

```
>>> axs[0, 0].set_title('pcolormesh()')
Text(0.5, 1.0, 'pcolormesh()')
>>> co = axs[0, 1].contourf(X, Y, Z, levels=np.linspace(-
1.25, 1.25, 11))
>>> fig.colorbar(co, ax=axs[0, 1])
<matplotlib.colorbar.Colorbar object at
0x000001ED9001D3C0>
>>> axs[0, 1].set_title('contourf()')
Text(0.5, 1.0, 'contourf()')
```

Color mapped data

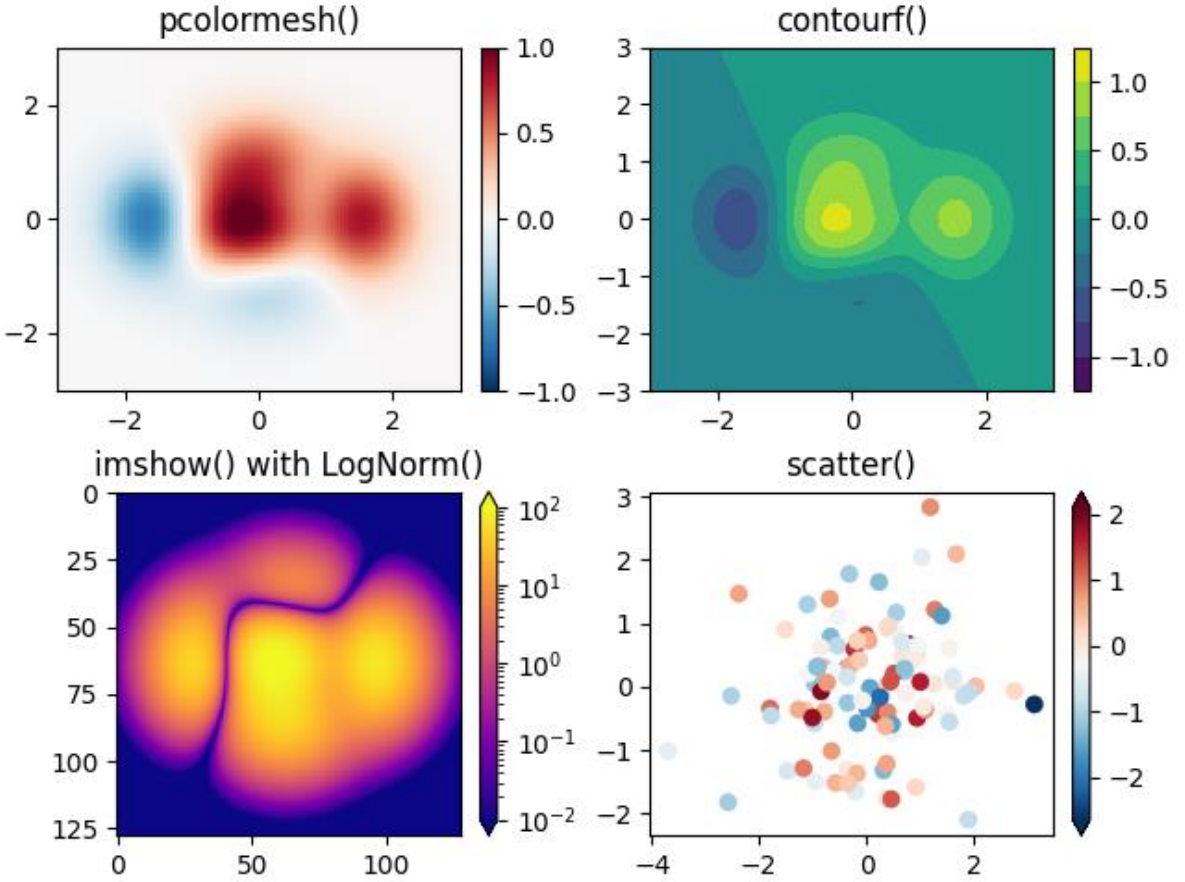
```
>>> pc = axs[1, 0].imshow(Z**2 * 100, cmap='plasma',

norm=mpl.colors.LogNorm(vmin=0.01, vmax=100))
>>> fig.colorbar(pc, ax=axs[1, 0], extend='both')
<matplotlib.colorbar.Colorbar object at
0x000001ED93696500>
>>> axs[1, 0].set_title('imshow() with LogNorm()')
Text(0.5, 1.0, 'imshow() with LogNorm()')
```

Color mapped data

```
>>> pc = axs[1, 1].scatter(data1, data2, c=data3,
cmap='RdBu_r')
>>> fig.colorbar(pc, ax=axs[1, 1], extend='both')
<matplotlib.colorbar.Colorbar object at
0x000001ED936C2530>
>>> axs[1, 1].set_title('scatter()')
Text(0.5, 1.0, 'scatter()')
```

Color mapped data

```
>>> plt.show()
```



Color mapped data

Colormaps

- These are all examples of Artists that derive from `ScalarMappable` objects.
- They all can set a linear mapping between *vmin* and *vmax* into the colormap specified by *cmap*.
- Matplotlib has many colormaps to choose from the existing, you can make your own or download as third-party packages.

Color mapped data

Normalizations

- Sometimes we want a non-linear mapping of the data to the colormap, as in the `LogNorm` example above.
- We do this by supplying the ScalarMappable with the *norm* argument instead of *vmin* and *vmax*.

Color mapped data

Colorbars

- Adding a `colorbar` gives a key to relate the color back to the underlying data.
- Colorbars are figure-level Artists, and are attached to a ScalarMappable (where they get their information about the norm and colormap) and usually steal space from a parent Axes.
- Placement of colorbars can be complex.
- You can also change the appearance of colorbars with the extend keyword to add arrows to the ends, and shrink and aspect to control the size.
- Finally, the colorbar will have default locators and formatters appropriate to the norm.
- These can be changed as for other Axis objects.

Object Oriented Programming (Using Python)

UNIT- IV

Python to access Web Data :

- Regular Expressions
- Extracting data
- Sockets
- Using the Developer Console to Explore HTTP
- Retrieving Web Page, and Passing Web Pages

Reference

https://www.coursera.org/learn/python-network-data

https://github.com/I-am-Harsh/Using-Python-to-Access-Web-Data

https://docs.python.org/3/howto/regex.html

https://www.py4e.com/code3/

https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_Languages/Book%3A_Python_for_Everybody_(Severance)

https://www.w3schools.com/python/python_regex.asp

Regular Expressions

- In computing, a regular expression, also referred as “regex” or “regexp”, provides a concise and flexible means for matching of strings of text, such as particular characters, words, or patterns of characters.
- A regular expression is written in a formal language that can be interpreted by a regular expression processor.
- Really clever “wild card” expressions for matching and parsing strings.
- Really smart “Find” or “Search”
- Regular expressions are widely used in various programming languages, including python for tasks such as pattern matching, string searching, data validation and text manipulation.

Understanding Regular Expressions

- Very powerful and quite cryptic (confusing)
- Fun once we understand them
- Regular expressions are a language unto (something continued until a particular time) themselves
- A language of “marker characters” – programming with characters
- It is kind of an “old school” language - compact

Python Regular Expression Quick Guide

Regular Expression	Description
`^`	Matches the beginning of a line
`$`	Matches the end of the line
`.`	Matches any character
`\s`	Matches whitespace
`\S`	Matches any non-whitespace character
`*`	Repeats a character zero or more times
`*?`	Repeats a character zero or more times (non-greedy)
`+`	Repeats a character one or more times
`+?`	Repeats a character one or more times (non-greedy)
`[aeiou]`	Matches a single character in the listed set
`[^XYZ]`	Matches a single character not in the listed set

Python Regular Expression Quick Guide

Regular Expression	Description
`[a-z0-9]`	The set of characters can include a range
`(`	Indicates where string extraction is to start
`)`	Indicates where string extraction is to end
`d`	Matches any digit character
`\D`	Matches any non-digit character
`\w`	Matches any alphanumeric character (word character)
`\W`	Matches any non-alphanumeric character
`?`	Matches zero or one occurrence of the preceding pattern
`{n}`	Matches exactly n occurrences of the preceding pattern
`{n,}`	Matches n or more occurrences of the preceding pattern
`{n,m}`	Matches between n and m occurrences of the preceding pattern

Python Regular Expression Quick Guide

Regular Expression	Description
`[]`	Matches any single character within the brackets
`[^]`	Matches any single character not within the brackets
`(pattern)`	Groups multiple patterns together
`(?:pattern)`	Non-capturing group. Matches the pattern but does not capture it
`\b`	Matches a word boundary
`\B`	Matches a non-word boundary
`[0-9]+`	One or more digits
`^F.+:`	It will look for lines or strings that start with 'F' followed by one or more characters and ending with a ':'. (Greedy Matching)
`^F.+?:`	It will look for lines or strings that start with 'F' followed by one or more characters (non-greedily) and ending with a ':'.

The Regular Expression Module

- Before we can use regular expression in our program, we must import the library using "**`import re`**"

```
>>> import re
```

Metacharacters

Metacharacters are characters with a special meaning.

Regular Expression	Description
[ ]	Matches any single character within the brackets
\	Signals a special sequence (can also be used to escape special characters)
.	Matches any character
^	Matches the beginning of a line
$	Matches the end of the line
*	Repeats a character zero or more times
+	Repeats a character one or more times
?	Matches zero or one occurrence of the preceding pattern
{ }	Matches exactly the specified number of occurrences of the preceding pattern
|	Either or
( )	Capture and group

Special Sequences

A special sequence is a \ followed by one of the characters in the list below, and has a special meaning:

Regular Expression	Description
`\A`	Returns a match if the specified characters are at the beginning of the string
`\b`	Returns a match where the specified characters are at the beginning or at the end of a word
`\B`	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word
`\d`	Returns a match where the string contains digits (numbers from 0-9)
`\D`	Returns a match where the string DOES NOT contain digits
`\s`	Returns a match where the string contains a white space character
`\S`	Returns a match where the string DOES NOT contain a white space character
`\w`	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)

Special Sequences

A special sequence is a \ followed by one of the characters in the list below, and has a special meaning: *cont'd.*

Regular Expression	Description
`\W`	Returns a match where the string DOES NOT contain any word characters
`\Z`	Returns a match if the specified characters are at the end of the string

Sets

A set is a set of characters inside a pair of square brackets [ ] with a special meaning:

Regular Expression	Description
`[arn]`	Returns a match where one of the specified characters (a, r, or n) is present
`[a-n]`	Returns a match for any lower case character, alphabetically between a and n
`[^arn]`	Returns a match for any character EXCEPT a, r, and n
`[0123]`	Returns a match where any of the specified digits (0, 1, 2, or 3) are present
`[0-9]`	Returns a match for any digit between 0 and 9
`[0-5][0-9]`	Returns a match for any two-digit numbers from 00 and 59
`[a-zA-Z]`	Returns a match for any character alphabetically between a and z, lower case OR upper case
`[+]`	In sets, +, *, ., |, (), $,{} has no special meaning, so [+] means: return a match for any + character in the string

Metacharacters - Examples

[] - A set of characters

Example: "[a-m]"

Program:

```
>>>import re
>>> txt = "The rain in Hyderabad"
#Find all lower case characters alphabetically between "a" and "m"
>>> x = re.findall("[a-m]", txt)
>>> print(x)
['h', 'e', 'a', 'i', 'i', 'd', 'e', 'a', 'b', 'a', 'd']
```

Metacharacters - Examples

\: Signals a special sequence (can also be used to escape special characters)

Example: "\d"

Program:

```
>>>import re
>>> txt = "That will be 83 dollars"
#Find all digit characters
>>> x = re.findall("\d", txt)
>>> print(x)
['8', '3']
```

Metacharacters - Examples

`.` : Any character (except newline character)

Example: `"he..o"`

Program:

```
>>>import re
>>> txt = "hello planet"
#Search for a sequence that starts with "he", followed by two (any) characters, and an "o":
>>> x = re.findall("he..o", txt)
>>> print(x)
['hello']
```

Metacharacters - Examples

For other Metacharacters, Special Sequences and Sets refer the following link:

https://www.w3schools.com/python/python_regex.asp

UNIT- IV

Python to access Web Data : cont'd.

- Regular Expressions
- Extracting data
- Sockets
- Using the Developer Console to Explore HTTP
- Retrieving Web Page, and Passing Web Pages

Reference

https://www.coursera.org/learn/python-network-data

https://github.com/I-am-Harsh/Using-Python-to-Access-Web-Data

https://docs.python.org/3/howto/regex.html

https://www.py4e.com/code3/

https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_Languages/Book%3A_Python_for_Everybody_(Severance)

https://www.w3schools.com/python/python_regex.asp

Extracting data using Regular Expression operations

- We can use `re.search()` to see if a string matches a regular expression, similar to using the `find()` method for strings
- We can use `re.findall()` to extract portions of a string that match our regular expression, similar to a combination of `find()` and slicing: `var[4:12]`

Regular Expression Functions

Function	Description	Syntax
`match`	Attempts to match the pattern at the beginning of the string	`re.match(pattern, string)`
`findall`	Returns a list containing all matches	`re.findall(pattern, string)`
`search`	Returns a Match object if there is a match anywhere in the string	`re.search(pattern, string)`
`split`	Returns a list where the string has been split at each match	`re.split(pattern, string, maxsplit=0)` If maxsplit is nonzero, at most maxsplit splits occur, and the remainder of the string is returned as the final element of the list.
`sub`	*Replacement and Substitution* Replaces one or many matches with a replacement string	`re.sub(pattern, repl, string)`

The findall() Function

- The findall() function returns a list containing all matches
- The list contains the matches in the order they are found
- If no matches are found, an empty list is returned

The findall() Function

Example-1:

Print a list of all matches

```
>>> import re
#Return a list containing every occurrence of “in”:
>>> txt = “The rain in Hyderabad“
>>> x = re.findall("in", txt)
>>> print(x)
['in', 'in']
```

The findall() Function

Example-2:

Return an empty list if no match was found:

```
>>> import re
>>> txt = "The rain in Hyderabad"
#Check if "Telangana" is in the string::
>>> x = re.findall("Telangana", txt)
>>> print(x)
[]
>>> if (x):
        print("Yes, there is at least one match!")
    else:
        print("No match")
No match
```

The search() Function

- The search() function searches the string for a match, and returns a Match object if there is a match
- If there is more than one match, only the first occurrence of the match will be returned

The search() Function

Example-1:

Search for the first white-space character in the string

```
>>> import re
>>> txt = " The rain in Hyderabad"
>>> x = re.search("\s", txt)
>>> print("The first white-space character is located in position:", x.start())
The first white-space character is located in position: 0
```

The search() Function

- If no matches are found, the value None is returned

Example-2:

Make a search that returns no match:

```
>>> import re
>>> txt = "The rain in Hyderabad"
>>> x = re.search("Telangana", txt)
>>> print(x)
None
```

The search() Function

Example-3:

Search the string to see if it starts with "The" and ends with “Hyderabad":

```
>>> import re
#Check if the string starts with "The" and ends with “Hyderabad"
>>> txt = "The rain in Hyderabad“
>>> x=re.search("^The.*Hyderabad$",txt)
>>> if x:
        print("YES! We have a match!")
    else:
        print("No match")
```

Output:

```
YES! We have a match!
```

The split() Function

- The split() function returns a list where the string has been split at each match

The split() Function

Example-1:

Split at each white-space character:

```
>>> import re
#Split the string at every white-space character
>>> txt = "The rain in Hyderabad"
>>> x = re.split("\s", txt)
>>> print(x)

Output:
['The', 'rain', 'in', 'Hyderabad']
```

The split() Function

- You can control the number of occurrences by specifying the `maxsplit` parameter

The split() Function

Example-2:

Split the string only at the first occurrence:

```
>>> import re
#Split the string at the first white-space character
>>> txt = "The rain in Hyderabad"
>>> x = re.split("\s", txt, 1)
>>> print(x)
```

Output:

```
['The', 'rain in Hyderabad']
```

The sub() Function

- The `sub()` function replaces the matches with the text of your choice

The sub() Function

Example-1:

Replace every white-space character with the symbol '-':

```
>>> import re
#Replace all white-space characters with the symbol "-":
>>> txt = "The rain in Hyderabad"
>>> x = re.sub("\s", "-", txt)
>>> print(x)

Output:
The-rain-in-Hyderabad
```

The sub() Function

- We can control the number of replacements by specifying the count parameter

The sub() Function

Example-2:

Replace the first 2 occurrences:

```
>>> import re
# Replace the first two occurrences of a white-space character
with the symbol"-":
>>> txt = "The rain in Hyderabad"
>>> x = re.sub("\s", "-", txt, 2)
>>> print(x)

Output:
The-rain-in Hyderabad
```

Match Object

- A Match Object is an object containing information about the search and the result.
- If there is no match, the value `None` will be returned, instead of the Match Object.

Match Object

Example-1:

Do a search that will return a Match Object:

```
>>> import re
#The search() function returns a Match object
>>> txt = "The rain in Hyderabad"
>>> x = re.search("ai", txt)
>>> print(x)
```

Output:

```
<re.Match object; span=(5, 7), match='ai'>
```

Match Object

The Match object has properties and methods used to retrieve information about the search, and the result:

- span() returns a tuple containing the start-, and end positions of the match.
- string returns the string passed into the function
- group() returns the part of the string where there was a match

Match Object

Example-2:

Print the position (start- and end-position) of the first match occurrence:

```
>>> import re
#The search() function returns a Match object
>>> txt = "The rain in Hyderabad"
>>> x = re.search(r"\bH\w+", txt)
>>> print(x.span())
(12, 21)
>>> y = re.search(r"\bT\w+", txt)
>>> print(y.span())
(0, 3)
>>> z = re.search("T", txt)
>>> print(z.span())
(0, 1)
```

Match Object

Example-3:

Print the string passed into the function:

```
>>> import re
#The string property returns the search string
>>> txt = "The rain in Hyderabad"
>>> x = re.search(r"\bH\w+", txt)
>>> print(x.string)
The rain in Hyderabad
```

Match Object

Example-4:

Print the part of the string where there was a match.

```
>>> import re
#Search for an upper case "H" character in the beginning of a word, and print the word:
>>> txt = "The rain in Hyderabad"
>>> x = re.search(r"\bH\w+", txt)
>>> print(x.group())
Hyderabad
```

Note: If there is no match, the value `None` will be returned, instead of the Match Object.

Extracting Data using Regex Operations

Matching Patterns

- re.match(pattern, string): Attempts to match the pattern at the beginning of the string.
- re.search(pattern, string): Searches for a match anywhere in the string.
- re.findall(pattern, string): Returns all non-overlapping matches of the pattern in the string.

Extracting Data using Regex Operations

Matching Patterns: Example

```
>>> import re
#Example text
>>> text = "My email address is abcd@cbit.ac.in and my phone number is (91) 1234567890"
# Pattern for matching email address
>>> email_pattern = r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"
# Pattern for matching phone number
>>> phone_pattern = r"\(\d{2}\) \d{10}"
```

Extracting Data using Regex Operations

Matching Patterns: Example cont'd.

```
# Find email address in the text
>>> email_matches = re.findall(email_pattern, text)
>>> print("Email address:", email_matches)
# Find phone number in the text
>>> phone_matches = re.findall(phone_pattern, text)
>>> print("Phone number:", phone_matches)
```

Output:

```
Email address: ['abcd@cbit.ac.in']
Phone number: ['(91) 1234567890']
```

Extracting Data using Regex Operations

Pattern Compilation

re.compile(pattern):

- Compiles a regular expression pattern into a pattern object.
- This allows us to reuse the compiled pattern for matching multiple strings efficiently.

Extracting Data using Regex Operations

Pattern Compilation: Example

```
>>> import re
#Example text
>>> text = "My email address is abcd@cbit.ac.in and my phone number is (91) 1234567890“
# Patterns
>>> email_pattern = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
>>> phone_pattern = re.compile(r"\(\d{2}\) \d{10}")
```

Extracting Data using Regex Operations

Pattern Compilation: Example cont'd

```
#Find email addresses using compiled pattern
>>> email_matches = email_pattern.findall(text)
>>> print("Email address:", email_matches)
#Find phone numbers using compiled pattern
>>> phone_matches = phone_pattern.findall(text)
>>> print("Phone number:", phone_matches)
Output:
Email address: ['abcd@cbit.ac.in']
Phone number: ['(91) 1234567890']
```

Extracting Data using Regex Operations

Pattern Compilation: Example cont'd

- In this example, we use the `re.compile()` function to compile regular expression patterns email_pattern and phone_pattern.
- The `re.compile()` function returns a regular expression object that can be used to perform matching operations.
- We then use the compiled patterns with the `findall()` method of the regular expression objects to find all matches in the given text.
- By compiling the patterns beforehand, you can reuse the compiled objects for matching multiple texts, which can improve performance when dealing with a large number of matching operations.

Extracting Data using Regex Operations

Grouping and Capturing

- Parentheses (and): Used to group parts of the pattern.
- (?:pattern): Non-capturing group. Matches the pattern but does not capture the matched substring.
- re.group(n): Returns the n-th captured group from the match.

Extracting Data using Regex Operations

Grouping and Capturing: Example

```
>>> import re
#Example text
>>> text = "My email addresses are
madanamohana_aids@cbit.ac.in,
madanamohana_aids@cbit.ac.in  and
director_iqac@cbit.ac.in"
# Patterns
>>> pattern = re.compile(r"(\b[A-Za-z0-9._%+-]+@[A-Za-z0-
9.-]+\.[A-Za-z]{2,}\b)")
```

Extracting Data using Regex Operations

Grouping and Capturing: Example

```
>>> matches = pattern.findall(text)
>>> for i, match in enumerate(matches, start=1):
    print(f"Email {i}: {match}")
```

Output:

```
Email 1: madanamohana_aids@cbit.ac.in
Email 2: madanamohana_aids@cbit.ac.in
Email 3: director_iqac@cbit.ac.in
```

Extracting Data using Regex Operations

Character Classes and Quantifiers

- [abc]: Matches any single character in the set.
- [^abc]: Matches any single character not in the set.
- *, +, ?: Quantifiers for matching zero or more, one or more, or zero or one occurrences of the preceding pattern.
- {n}, {n,m}: Matches exactly n occurrences or between n and m occurrences of the preceding pattern.

Extracting Data using Regex Operations

Character Classes and Quantifiers - Example

```
>>> import re
#Example text
>>> text = "AI&DS II Semester Subject codes are 22MTC04, 22CYC01, 22EEC01 and 22CSC03“
# Patterns
>>> pattern = re.compile(r'\b[0-9A-Z]{7}\b')
>>> subject_codes = pattern.findall(text)
```

Extracting Data using Regex Operations

Character Classes and Quantifiers – Example cont'd

```
>>> for code in subject_codes:
        print("Subject Code:", code)
```

Output:

```
Subject Code: 22MTC04
Subject Code: 22CYC01
Subject Code: 22EEC01
Subject Code: 22CSC03
```

Extracting Data using Regex Operations

Character Classes and Quantifiers – Example cont'd

In this program, the regular expression pattern `r'\b[0-9A-Z]{7}\b'` is used. Let's break down this pattern:

- `\b` denotes a word boundary, ensuring that we match whole words.
- `[0-9A-Z]` specifies that we want to match any digit (0-9) or uppercase letter (A-Z).
- `{7}` indicates that we want to match exactly 7 occurrences of the preceding character class.
- `\b` is another boundary to ensure we match the complete word.

Extracting Data using Regex Operations

Anchors and Boundaries

- `^`: Matches the beginning of the string or line.
- `$`: Matches the end of the string or line.
- `\b`: Matches a word boundary.

Extracting Data using Regex Operations

Anchors and Boundaries - Example

```
>>> import re
#Example text
>>> urls = [
"https://www.cbit.ac.in/",
"https://www.aicte-india.org/",
"https://www.osmania.ac.in/",
"https://www.python.org/",
"https://docs.python.org/3/howto/regex.html"
]
```

Extracting Data using Regex Operations

Anchors and Boundaries – Example cont'd

```
# Patterns
>>> pattern = re.compile(r"^https://\b.*\.html$")
>>> matched_urls = [url for url in urls if
pattern.match(url)]
>>> for url in matched_urls:
        print("Matched URL:", url)
Output:
Matched URL: https://docs.python.org/3/howto/regex.html
```

Extracting Data using Regex Operations

Anchors and Boundaries – Example cont'd

- In this example, we want to match URLs that start with `"https://"` and end with `".html"`, while considering word boundaries.
- We define a regular expression pattern using re.compile().
- The pattern `^https://\b.*\.html$` includes the following elements:
  - `^`: Represents the start of the line.
  - `https://`: Matches the literal string "https://".
  - `\b`: Represents a word boundary.
  - `.*`: Matches any character (except newline) zero or more times.
  - `\.html$`: Matches the literal string ".html" at the end of the line.

Greedy and Non-greedy Regex in Python

- In regular expressions, "greedy" and "non-greedy" refer to the behavior of quantifiers, such as *, +, ?, and {}.
- These quantifiers control the number of times a preceding pattern should match in a regular expression.
- By *default*, quantifiers are greedy, which means they try to match as much as possible while still allowing the overall pattern to match.
- On the other hand, non-greedy quantifiers match as little as possible while still allowing the pattern to match.

Greedy and Non-greedy Regex in Python

- In Python, the `re` module supports both greedy and non-greedy matching using the following quantifiers:
- Greedy quantifiers:
  - `*` Matches zero or more occurrences (greedy).
  - `+` Matches one or more occurrences (greedy).
  - `?` Matches zero or one occurrence (greedy).
  - `{m,n}` Matches at least m and at most n occurrences (greedy).
- Non-greedy quantifiers:
  - `*?` Matches zero or more occurrences (non-greedy).
  - `+?` Matches one or more occurrences (non-greedy).
  - `??` Matches zero or one occurrence (non-greedy).
  - `{m,n}?` Matches at least m and at most n occurrences (non-greedy).

Greedy and Non-greedy Regex in Python

Greedy and Non-greedy matching – Example1

```
>>> import re
#Example text
>>> str1 = "fabcdaxyzapq"
# Greedy matching
>>> greedy_result = re.search("a.*a", str1)
>>> print(greedy_result.group())
Output:
abcdaxyza
```

Greedy and Non-greedy Regex in Python

Greedy and Non-greedy matching - Example

```
>>> import re
#Example text
>>> str2 = "fabcdaxyzapq"
# Matching made non-greedy by adding the "?" character
after the "*" quantifier.
>>> non_greedy_result = re.search("a.*?a", str2)
>>> print(non_greedy_result.group())
Output:
abcda
```

Greedy and Non-greedy Regex in Python

Greedy and Non-greedy matching – Example2

```
>>> import re
#Example text
>>> text = "This is a <b>bold</b> statement."
# Greedy matching
>>> greedy_pattern = r'<.*>'
>>> greedy_match = re.search(greedy_pattern, text)
>>> if greedy_match:
        print("Greedy match:", greedy_match.group())
Output:
Greedy match: <b>bold</b>
```

Greedy and Non-greedy Regex in Python

Greedy and Non-greedy matching – Example2

```
>>> import re
#Example text
>>> text = "This is a <b>bold</b> statement."
# Non-Greedy matching
>>> non_greedy_pattern = r'<.*?>'
>>> non_greedy_match = re.search(non_greedy_pattern, text)
>>> if non_greedy_match:
        print("Non-greedy match:", non_greedy_match.group())
Output:
Non-greedy match: <b>
```

Greedy and Non-greedy Regex in Python

Greedy and Non-greedy matching – Example2

- In this example, we're trying to match HTML tags within the text.
- The greedy pattern `<.*>` starts matching from the first `<` and continues until the last `>`, including all the characters in between. It matches the entire `<b>bold</b>` substring.
- On the other hand, the non-greedy pattern `<.*?>` matches as little as possible. It starts matching from the first `<` and stops as soon as it encounters the first `>`, resulting in just `<b>`.
- By adding `?` after the quantifier, we instruct the regular expression to prefer the shortest possible match, leading to non-greedy behavior.
- The greediness of quantifiers can affect the overall matching behavior of regular expressions, so it's essential to understand and choose the appropriate greediness based on your specific use case.

UNIT- IV

Python to access Web Data : cont'd.

- Regular Expressions
- Extracting data
- Sockets
- Using the Developer Console to Explore HTTP
- Retrieving Web Page, and Passing Web Pages

Reference

https://www.coursera.org/learn/python-network-data

https://github.com/I-am-Harsh/Using-Python-to-Access-Web-Data

https://www.py4e.com/code3/

https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_Languages/Book%3A_Python_for_Everybody_(Severance)

http://docs.python.org/library/socket.html

https://youtu.be/T0rYSFPAR0A

Object Oriented Programming (Using Python)

Networked Programs:

- Sockets

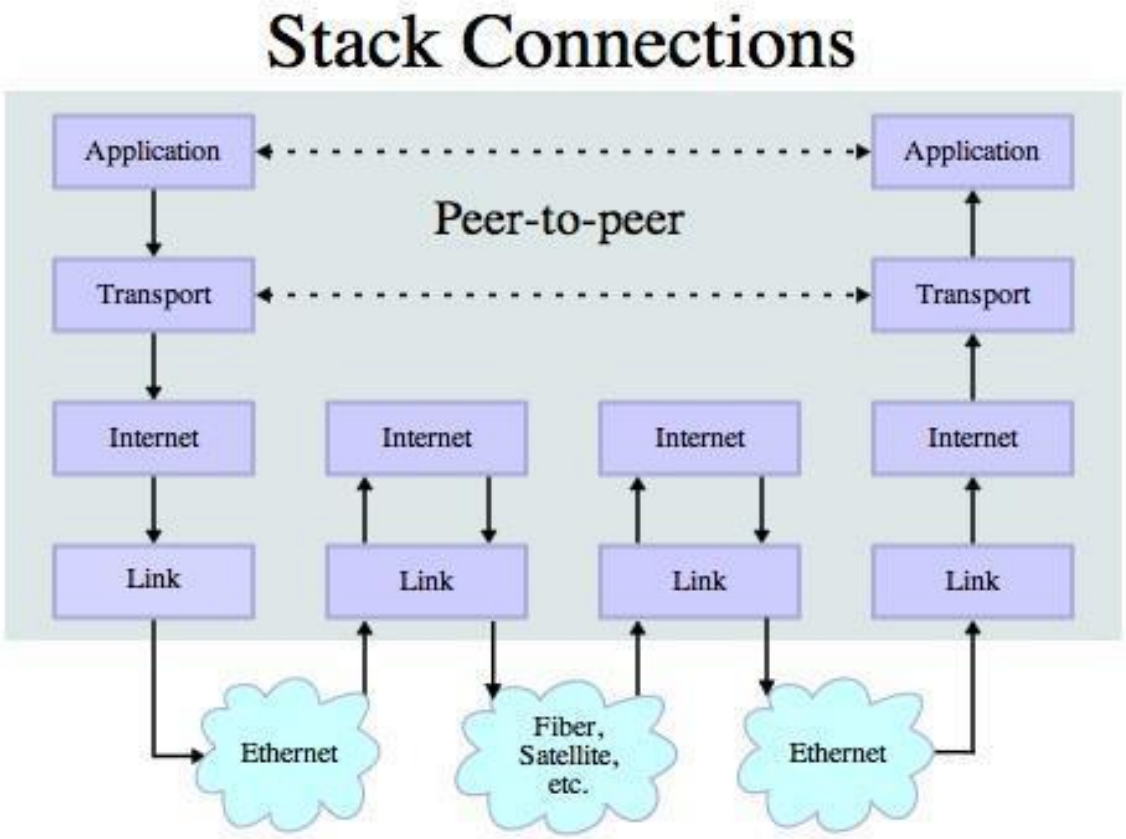
Transport Control Protocol (TCP)

- Built on top of IP (Internet Protocol)
- Assumes IP might lose some data - stores and retransmits data if it seems to be lost
- Handles “flow control” using a transmit window
- Provides a nice reliable channel

IP Address

- An IP (Internet Protocol) address is a numerical label assigned to each device (such as a computer or a smartphone) connected to a network
- It serves *two main purposes*:
  - ✓ Identifying the host or network interface and
  - ✓ Providing the location of the device in the network
- IPv4 (Internet Protocol version 4) addresses are the most common and consist of four sets of numbers separated by periods (e.g., 192.168.0.1)
- IPv6 (Internet Protocol version 6) addresses are the *newer version* and are represented as eight groups of hexadecimal digits separated by colons (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334)

Transport Control Protocol (TCP)



Socket

- A socket is an endpoint for communication between two processes over a computer network.
- Socket serves as Application Programming Interface (API) that allows processes to establish connections, send, and receive data.
- The concept of sockets is based on the client-server model, where one process acts as a server and listens for incoming connections, while other processes act as clients and establish connections to the server.
- Once a connection is established, processes can exchange data through the socket.

TCP Connections / Sockets

- “In computer networking, an Internet socket or network socket is an endpoint of a bidirectional inter-process communication flow across an Internet Protocol-based computer network, such as the Internet.”
- A single network will have two sockets.
- Sockets are a combination of an IP address and a Port.

Sockets Terminology

Domain

- The family of protocols that is used as the transport mechanism.
- These values are constants such as AF_INET (Address Family Internet), PF_INET (Protocol Family Internet), PF_UNIX, PF_X25 (X.25 is an ITU-T (developed by Telecommunication Standardization Sector of International Telecommunication Union) standard protocol suite for packet-switched data communication in wide area networks), and so on.

Type

- The type of communications between the two endpoints, typically SOCK_STREAM (Stream (connection) Socket) for connection-oriented protocols and SOCK_DGRAM (Datagram (conn.less) Socket) for connectionless protocols.

Sockets Terminology

Protocol

- Typically zero, this may be used to identify a variant of a protocol within a domain and type.

hostname

- The identifier of a network interface:
  - ✓ A string, which can be a host name, a dotted-quad address, or an IPV6 address in colon (and possibly dot) notation
  - ✓ A string "<broadcast>", which specifies an INADDR_BROADCAST address (IPv4 broadcast address).
  - ✓ A zero-length string, which specifies INADDR_ANY (IPv4 local host address), or
  - ✓ An Integer, interpreted as a binary address in host byte order

Sockets Terminology

port

- Each server listens for clients calling on one or more ports.
- A port may be a Fixnum port number, a string containing a port number, or the name of a service.

Ports

- Ports are virtual endpoints for communication in a network
- A port is associated with an IP address and allows multiple applications to run simultaneously on a device
- Ports are identified by numbers ranging from 0 to 65535
- Well-known ports (0-1023) are assigned to specific services (e.g., HTTP uses port 80, HTTPS uses port 443)
- Registered ports (1024-49151) are assigned by the Internet Assigned Numbers Authority (IANA) for specific purposes
- Dynamic or private ports (49152-65535) can be used by applications dynamically or for private purposes

TCP Port Numbers

- A port is an application-specific or process-specific software communications endpoint
- It allows multiple networked applications to coexist on the same server
- There is a list of well-known TCP port numbers

Common TCP Port Numbers

Protocol	Port Number	Python Library	Function
HTTP (Hyper Text Transfer Protocol)	80	`httplib, urllib, xmlrpclib`	Web pages
FTP (File Transfer Protocol)	20	`ftplib, urllib`	File transfers
NNTP (Network News Transfer Protocol)	119	`nntplib`	Unsent news
SMTP (Simple Mail Transfer Protocol)	25	`smtplib`	Sending mail
Telenet	23	`Telnetlib`	Command lines
POP3 (Post Office Protocol)	110	`Poplib`	Fetching email
Gopher (It is an application-layer protocol that provides the ability to extract and view Web documents stored on remote Web servers)	70	`Gopherlib`	Document transfer

Python *socket* Module

Python has built-in support for TCP Sockets

First import the Python socket module for this

```
>>> import socket
```

To create a socket, we must use the `socket.socket()` function available in socket module, which has the following general syntax:

```
s = socket.socket (socket_family, socket_type, protocol=0)
```

Here is the description of the parameters:

`socket_family` – This is either AF_UNIX or AF_INET

`socket_type` – This is either SOCK_STREAM or SOCK_DGRAM

`protocol` – This is usually left out, defaulting to `0`

Important Methods of the Socket module

Methods	Description
`socket.socket()`	Used to create sockets (required on both server as well as client ends to create sockets)
`socket.accept()`	Used to accept a connection, It returns a pair if values (conn, address)
`socket.bind()`	Used to bind to the address that us specified a parameter
`socket.close()`	Used to bind the address that is specified as a parameter
`socket.connect()`	Used to mark the socket as closed
`socket.listen()`	Used to connect to a remote address specified as the parameter

Server and Clients

Server

- Either a program, a computer, or a device
- Dedicated to managing network resources
- Can be on the same device or computer or local or even remote
- There are various *types of servers* such as database server, network server, front-end server etc.
- Servers commonly make use of *methods* like socket methods, bind and listen methods etc. to establish a connection and bind to the client

Server and Clients

Client

- Computer or software that receives information or services
- Clients requests for services from servers
- Example: Web browsers such as Google Chrome, Firefox etc.
- These web browsers request web servers to send required web pages and services as directed by the user
- Other examples of clients include online games, online chats etc.

Python Program to demo Client-Server Communication

Server-side program:

File Name: sockets_server.py

```
import socket
myserver=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
# AF_INET referes the address from the Internet and
SOCK_STREAM is used to create TCP protocol
myserver.bind((socket.gethostname(), 1024))
#Here 1024 is the Port Number. Provide any Port number is
>1023 which is non previliged.
myserver.listen(5)
#This asks for permission on Windows
```

Python Program to demo Client-Server Communication

Server-side program: cont'd.

File Name: sockets_server.py

```
while True:
    myclient,addr=myserver.accept()
    print(f"Connection to {addr} established")
    # f string is literal string
    myclient.send(bytes("Welcome to Socket Programming in Python!", "utf-8"))
    # To pass the message from server to client. The kind of byte is utf-8 (Unicode Transformation Format – 8-bit)
```

Python Program to establish a connection between Server & Client

Client-side program:

File Name: sockets_client.py

```
import socket
myserver=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
myserver.connect((socket.gethostname(), 1024))
# 1024 is same port number given in server
msg=myserver.recv(1024)
# 1024 bytes is is just sample byte number. Received
message from Server is stored in "msg"
print(msg.decode("utf-8"))
# "utf-8" is bytes code given in server. Message received
from the Server can be decoded.
```

Python Program to establish a connection between Server & Client

Running Procedure:

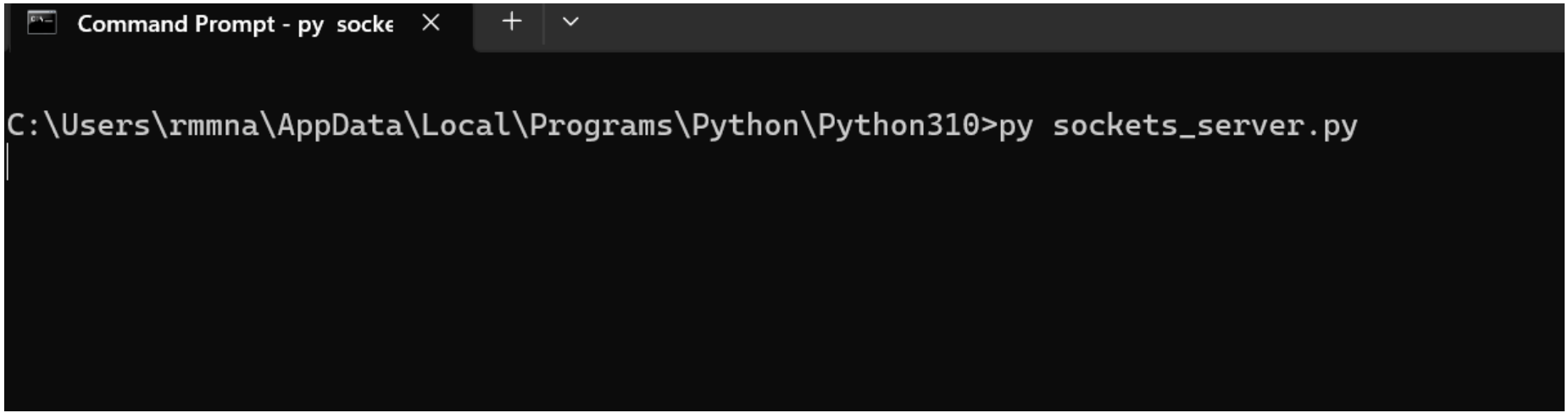
Step-1: Goto Command prompt and then change directory to where `sockets_server.py` and `sockets_client.py` programs are located

```
Microsoft Windows [Version 10.0.22621.1848]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rmmna>cd AppData

C:\Users\rmmna\AppData>cd Local

C:\Users\rmmna\AppData\Local>cd Programs

C:\Users\rmmna\AppData\Local\Programs>cd Python

C:\Users\rmmna\AppData\Local\Programs\Python>cd Python310

C:\Users\rmmna\AppData\Local\Programs\Python\Python310>dir *.py
 Volume in drive C is Windows
 Volume Serial Number is 4C6B-281F

 Directory of C:\Users\rmmna\AppData\Local\Programs\Python\Python310

25-04-2023  10:25               487 abstractclass.py
09-05-2023  21:07                97 arithmetic operators.py
20-04-2023  21:48                 2 classtype.py
20-04-2023  20:36               465 derivedclass.py
17-03-2023  22:36                60 ex.py
30-04-2023  21:56               771 exceptions with inheritance.py
06-07-2023  19:43               441 GREEDYEX.py
20-04-2023  19:17               467 overriding.py
09-05-2023  21:51                88 relational operators.py
25-04-2023  10:36               308 singletonclass.py
11-07-2023  15:02               175 sockets_client.py
11-07-2023  15:10               360 sockets_server.py
20-04-2023  20:07               599 typeconversions.py
```

Python Program to establish a connection between Server & Client

Running Procedure:

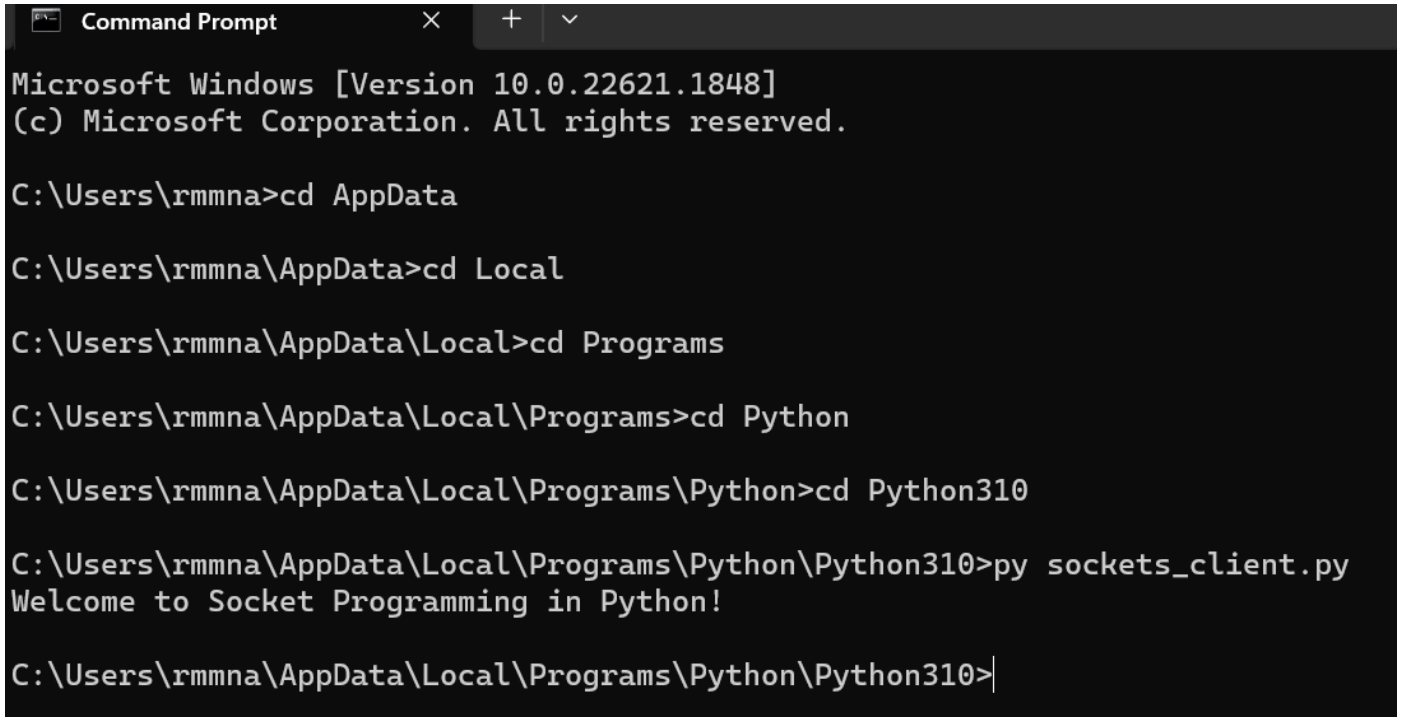
Step-2: Run the Server Program "`sockets_server.py`" using the following python command in command prompt and minimize this window.



Python Program to establish a connection between Server & Client

Running Procedure:

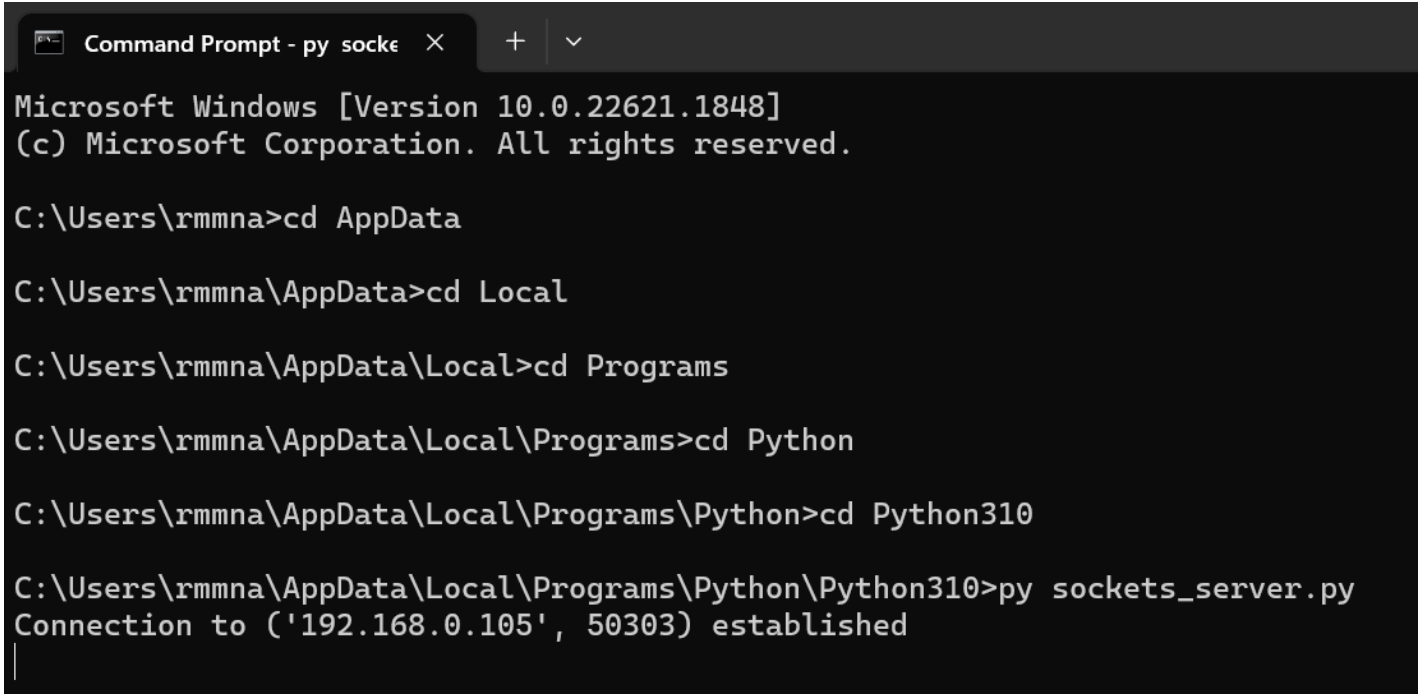
Step-3: Run the Client Program "`sockets_clients.py`" by opening the new command prompt window using the following python command.



Python Program to establish a connection between Server & Client

Running Procedure:

Step-4: Server shows connection establishment



UNIT- IV

Python to access Web Data : cont'd.

- Regular Expressions
- Extracting data
- Sockets
- Using the Developer Console to Explore HTTP and Retrieving Web Page
- Parsing Web Pages

Reference

https://www.coursera.org/learn/python-network-data

https://github.com/I-am-Harsh/Using-Python-to-Access-Web-Data

https://www.py4e.com/code3/

https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_Languages/Book%3A_Python_for_Everybody_(Severance)

http://docs.python.org/library/socket.html

https://youtu.be/T0rYSFPAR0A

Using the Developer Console to Explore HTTP

HTTP - Hypertext Transfer Protocol

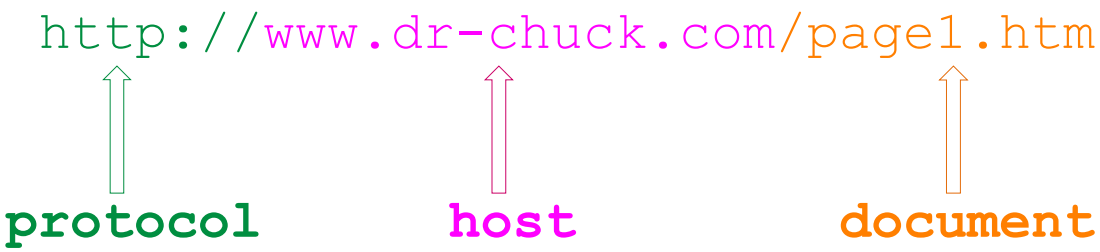
- The dominant Application Layer Protocol on the Internet
- Invented for the Web - to Retrieve HTML, Images, Documents, etc.
- Extended to retrieve data in addition to documents – RSS (Really Simple Syndication, a web feed that allows users and applications to access updates to websites in a standardized, computer-readable format), Web Services, etc.
- Basic Concept:
  - ✓ Make a Connection
  - ✓ Request a document
  - ✓ Retrieve the Document
  - ✓ Close the Connection

HTTP - Hypertext Transfer Protocol

- The HyperText Transfer Protocol is the set of rules to allow browsers to retrieve web documents from servers over the Internet
- In Python, we can interact with the HTTP protocol using various libraries, but one commonly used library is ‘requests’

HTTP - Hypertext Transfer Protocol

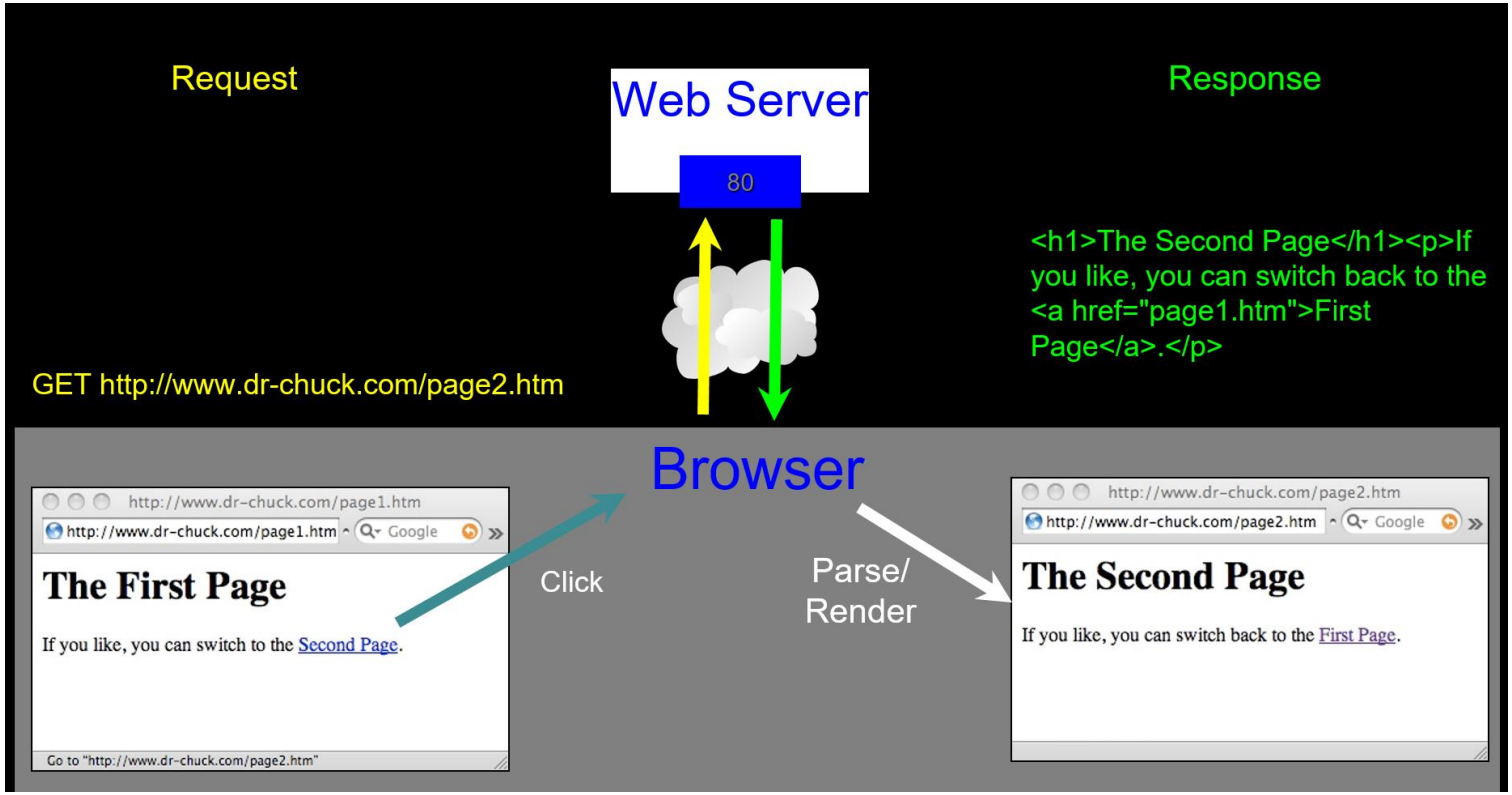
- The HyperText Transfer Protocol is the set of rules to allow browsers to retrieve web documents from servers over the Internet
- In Python, we can interact with the HTTP protocol using various libraries, but one commonly used library is 'requests'
- Example:



Getting Data from the Server

- Each time the user clicks on an anchor tag with an href= value (href-hypertext reference) to switch to a new page, the browser makes a connection to the web server and issues a "**`GET`**" request - to GET the content of the page at the specified URL
- The server returns the HTML document to the browser, which formats and displays the document to the user

Getting Data from the Server - Example

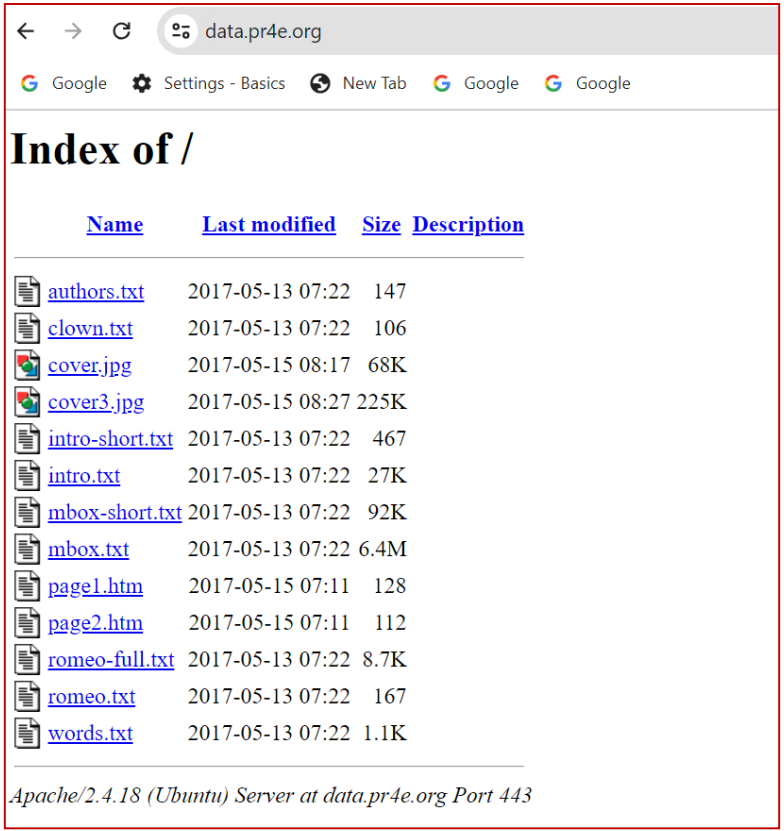


Making an HTTP request

- To request a response from the server, there are mainly two methods:
  - ✓ **`GET:`** To request data from the server
  - ✓ **`POST:`** To submit data to be processed to the server
- To make HTTP requests in Python, we can use several HTTP libraries like:
  - ✓ `httplib`
  - ✓ `urllib`
  - ✓ `requests`

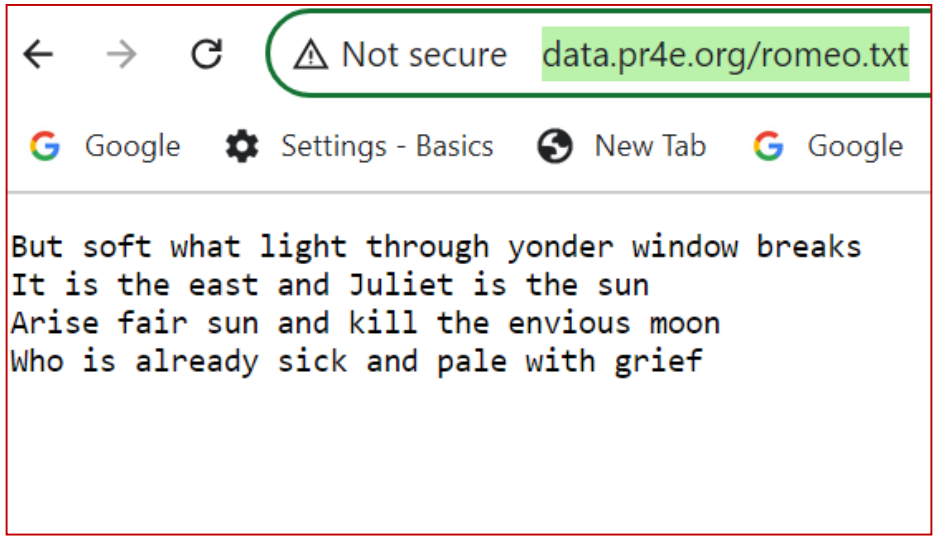
An HTTP request in Python using Sockets

Example URL: `data.pr4e.org`



An HTTP request in Python using Sockets

Example data: `http://data.pr4e.org/romeo.txt`



An HTTP request in Python using Sockets

Python Program: `htmlsocket.py`

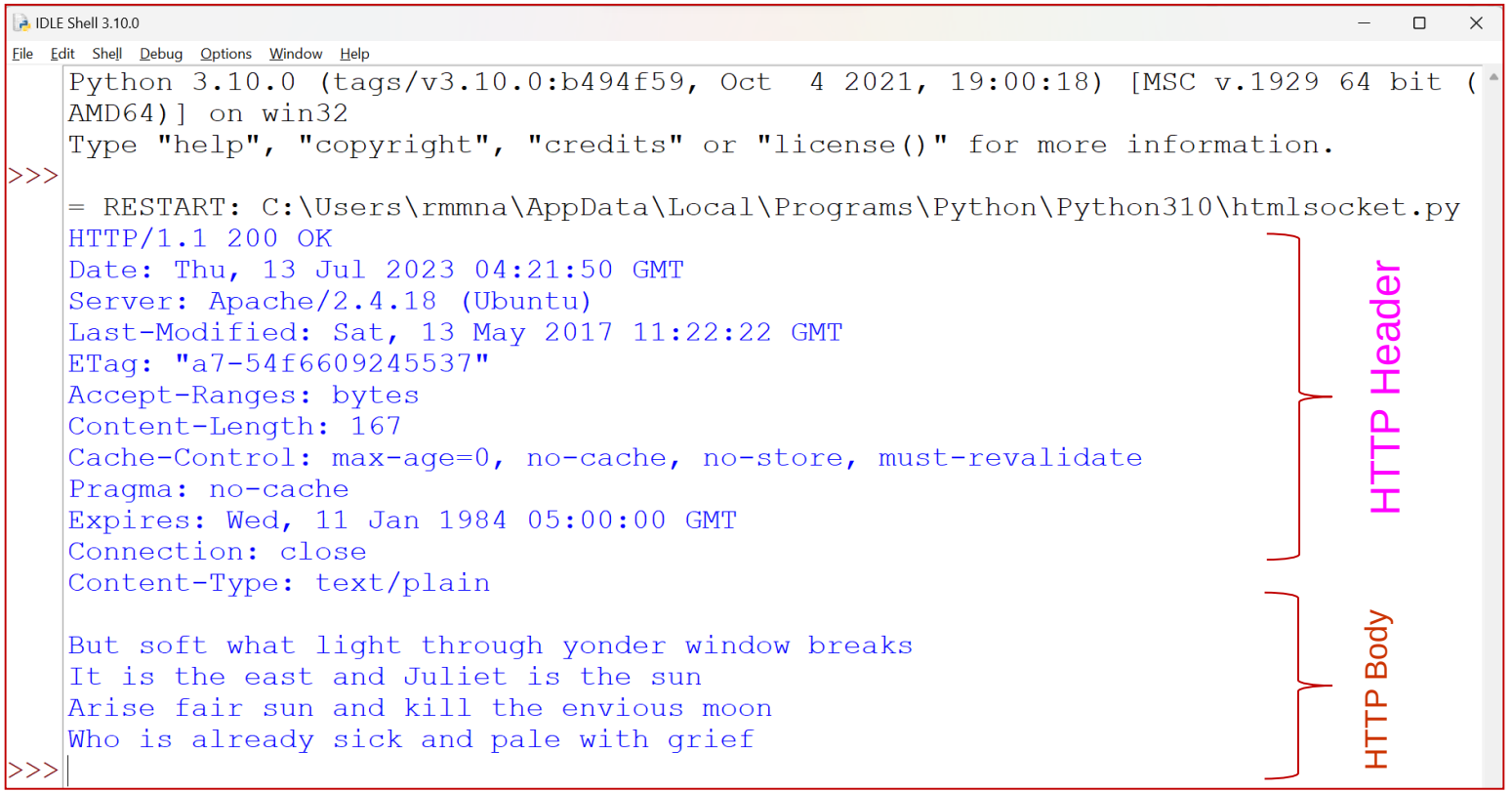
htmlsocket.py - C:\Users\rmmna\AppData\Local\Programs\Python\Python310\htmlsocket.py (3.10.0)

File Edit Format Run Options Window Help

```
import socket

mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()

mysock.send(cmd)

while True:
    data = mysock.recv(512)
    if (len(data) < 1):
        break
    print(data.decode(),end='')
mysock.close()
```

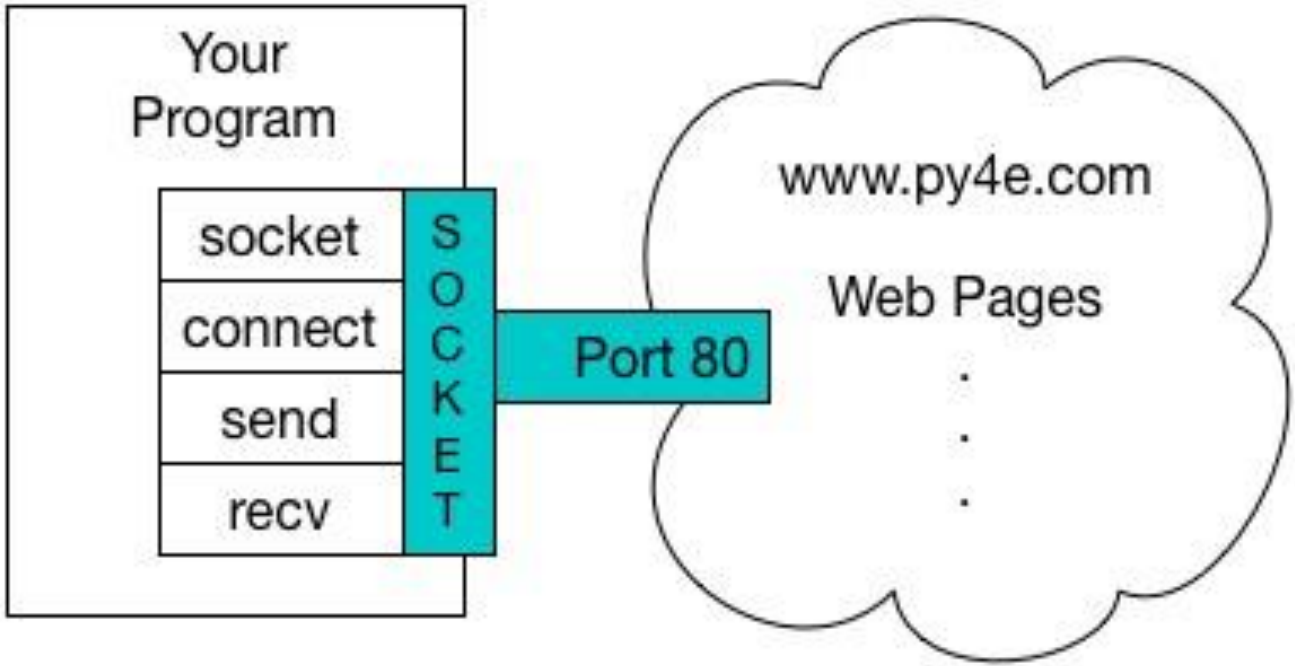
An HTTP request in Python using Sockets

Python Program: `htmlsocket.py`



An HTTP request in Python using Sockets

Python Program: `htmlsocket.py`



An HTTP request in Python using Sockets

Python Program: `htmlsocket.py`

`bytes.`**`decode`**`(`*`encoding="utf-8", errors="strict"`*`)`

`bytearray.`**`decode`**`(`*`encoding="utf-8", errors="strict"`*`)`

Return a string decoded from the given bytes. Default encoding is `'utf-8'`. *errors* may be given to set a different error handling scheme. The default for *errors* is `'strict'`, meaning that encoding errors raise a `UnicodeError`. Other possible values are `'ignore'`, `'replace'` and any other name registered via `codecs.register_error()`, see section Error Handlers. For a list of possible encodings, see section Standard Encodings.

`str.`**`encode`**`(`*`encoding="utf-8", errors="strict"`*`)`

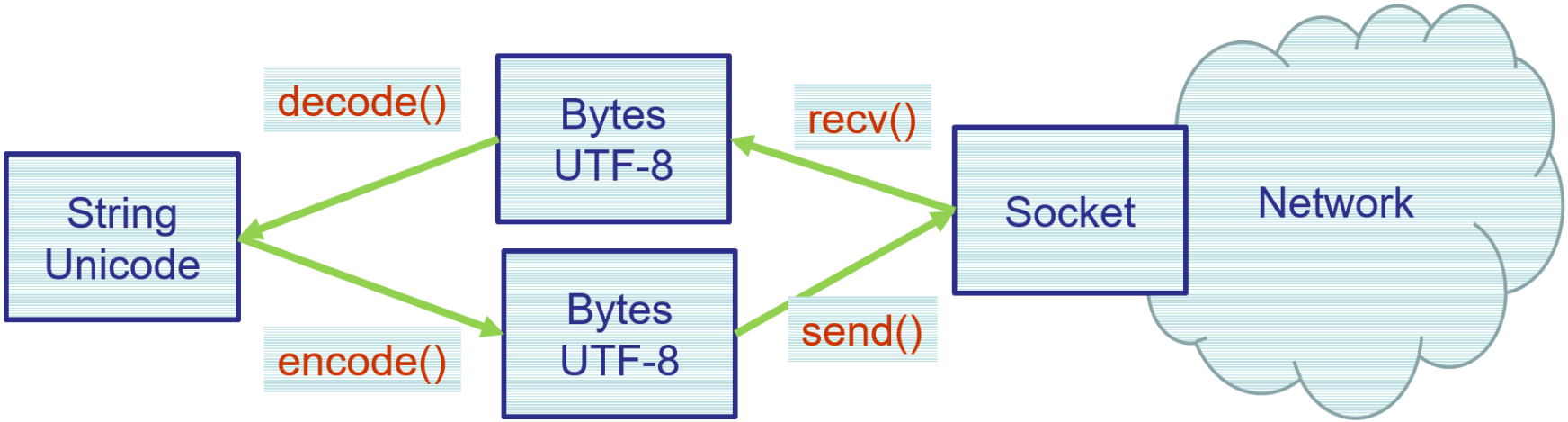
Return an encoded version of the string as a bytes object. Default encoding is `'utf-8'`. *errors* may be given to set a different error handling scheme. The default for *errors* is `'strict'`, meaning that encoding errors raise a `UnicodeError`. Other possible values are `'ignore'`, `'replace'`, `'xmlcharrefreplace'`, `'backslashreplace'` and any other name registered via `codecs.register_error()`, see section Error Handlers. For a list of possible encodings, see section Standard Encodings.

UTF-8

UTF-8 is a variable-length character encoding standard used for electronic communication. Defined by the Unicode Standard, the name is derived from Unicode Transformation Format – 8-bit. UTF-8 is capable of encoding all 1,112,064 valid character code points in Unicode using one to four one-byte code units. Wikipedia

An HTTP request in Python using Sockets

Python Program: `htmlsocket.py`



Retrieving Web Pages

Making HTTP Easier With urllib

- To make HTTP requests in Python, we can use several HTTP libraries like:
  - ✓ `httplib`
  - ✓ `urllib`
  - ✓ `requests`

httplib - HTTP protocol client

- The `httplib` module has been renamed to **`http.client`** in Python 3. The `2to3` tool will automatically adapt imports when converting our sources to Python 3.
- This module defines classes which implement the client side of the HTTP and HTTPS protocols.
- It is normally not used directly - the module `urllib` uses it to handle URLs that use HTTP and HTTPS.

httplib - HTTP protocol client

- Here's an example program demonstrating how to use `http.client` to make a simple HTTP GET request:

httplib - HTTP protocol client

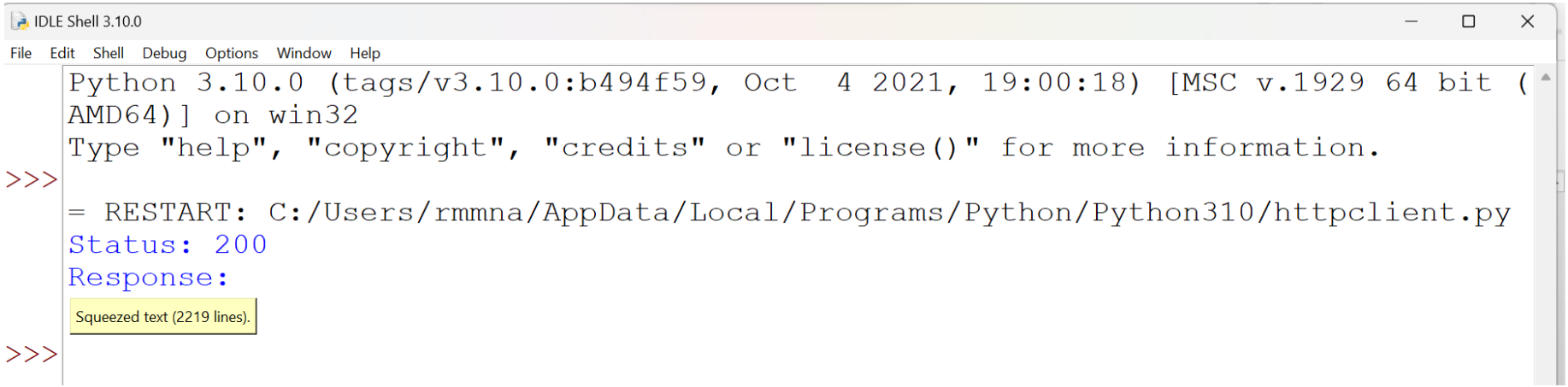
httpclient.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/httpclient.py (3.10.0)

File Edit Format Run Options Window Help

```
import http.client

# Create an HTTP connection
conn = http.client.HTTPSConnection("www.cbit.ac.in")

# Send an HTTP request
conn.request("GET", "/")

# Get the response from the server
response = conn.getresponse()

# Print the response status code
print("Status:", response.status)

# Read the response data
data = response.read()

# Print the response data
print("Response:")
print(data.decode())

# Close the connection
conn.close()
```

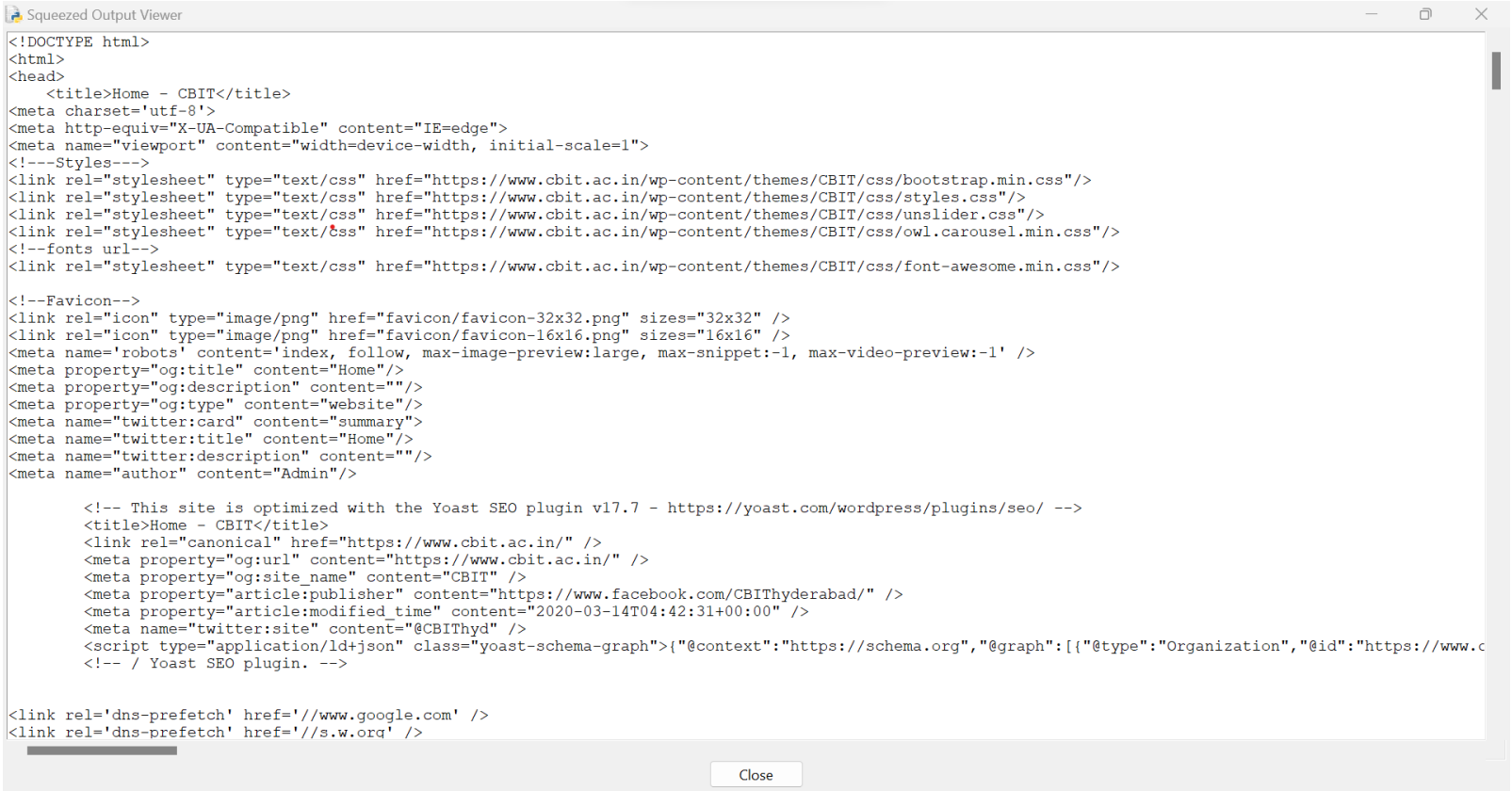
httplib - HTTP protocol client

Output:



httplib - HTTP protocol client

Output:



httplib - HTTP protocol client

- In this example, we create an `HTTPSConnection` object using the server address ("www.cbit.ac.in" in this case).
- We then send an `HTTP GET` request using the `request()` method, specifying the path ("/" in this case).
- The server responds with an HTTP response, which we retrieve using the `getresponse()` method.
- We can access various properties of the response, such as the status code (`response.status`) and the response data (`response.read()`).
- In this example, we print the status code and decode the response data as a string using the `decode()` method before printing it.
- Finally, we close the connection using the `close()` method.

Using urllib in Python

- In Python, the `urllib` module provides a collection of modules for working with URLs.
- One of the commonly used submodules is `urllib.request`, which allows you to make HTTP requests and handle responses.

Using urllib in Python

- Here's an example program demonstrating how to use `urllib.request` to make a simple HTTP GET request:

Using urllib in Python

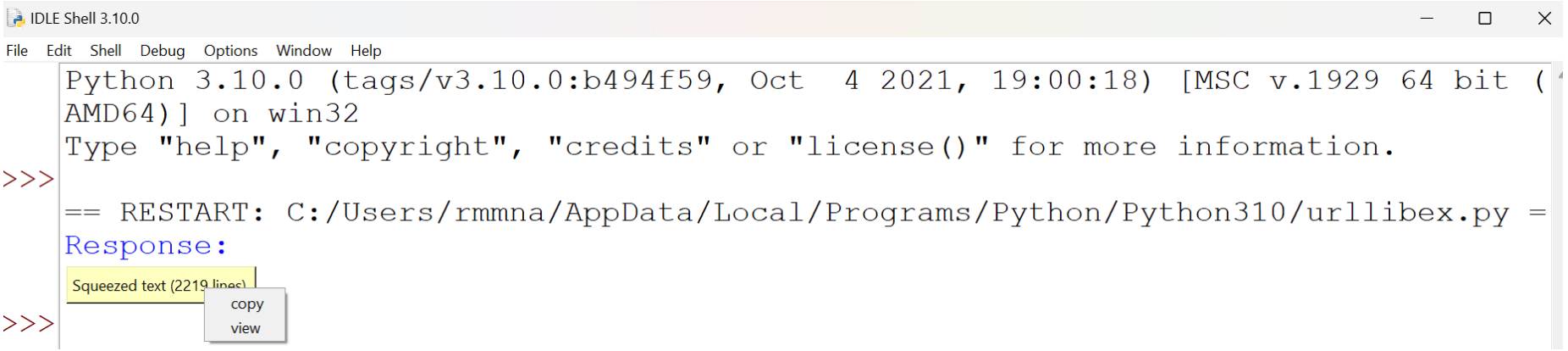
urllibex.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/urllibex.py (3.10.0)

File Edit Format Run Options Window Help

```
import urllib.request

# Make an HTTP GET request
response = urllib.request.urlopen("https://www.cbit.ac.in/")

# Read the response data
data = response.read()

# Print the response data
print("Response:")
print(data.decode())

# Close the response
response.close()
```

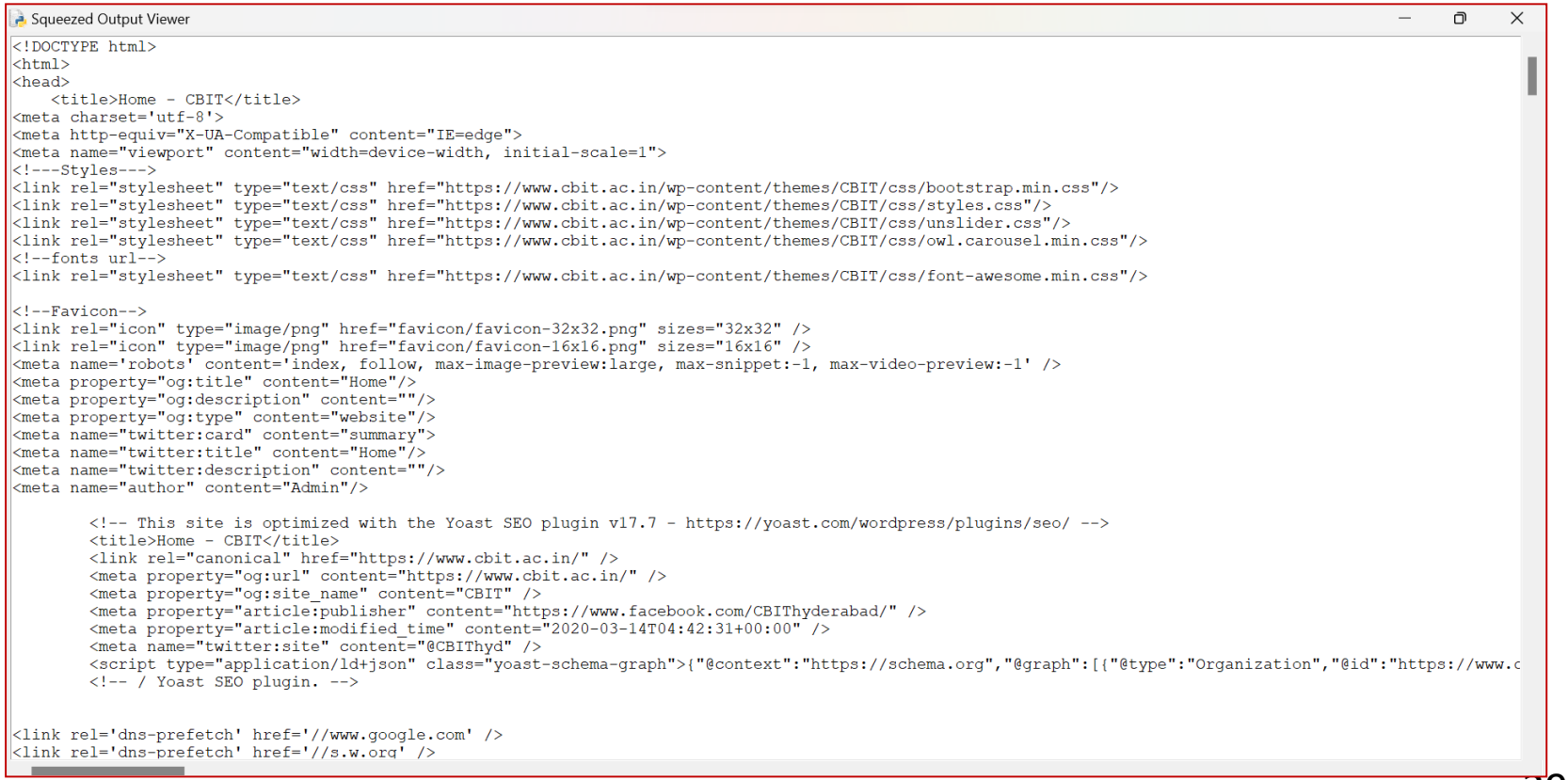
Using urllib in Python

- Output:



Using urllib in Python

- Output:



Using urllib in Python

- In this example, we use the `urlopen()` function from `urllib.request` to open a connection to the specified URL ("https://www.cbit.ac.in/" in this case) and make an HTTP GET request.
- The `urlopen()` function returns a file-like object that represents the response.
- We can then use the `read()` method on the response object to read the response data as bytes.
- In this example, we decode the response data as a string using the `decode()` method before printing it.
- Finally, we close the response using the `close()` method to free up system resources.
- Note that the urllib module provides various other functionalities for working with URLs, such as URL encoding, parsing, and handling errors.

Using urllib in Python – Example2

urllibsocket.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/urllibsocket.py (3.10.0)

File Edit Format Run Options Window Help

```
import urllib.request, urllib.parse, urllib.error

fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')
for line in fhand:
    print(line.decode().strip())
```

IDLE Shell 3.10.0

File Edit Shell Debug Options Window Help

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rmmna/AppData/Local/Programs/Python/Python310/urllibsocket.py
But soft what light through yonder window breaks
It is the east and Juliet is the sun
Arise fair sun and kill the envious moon
Who is already sick and pale with grief
>>>
```

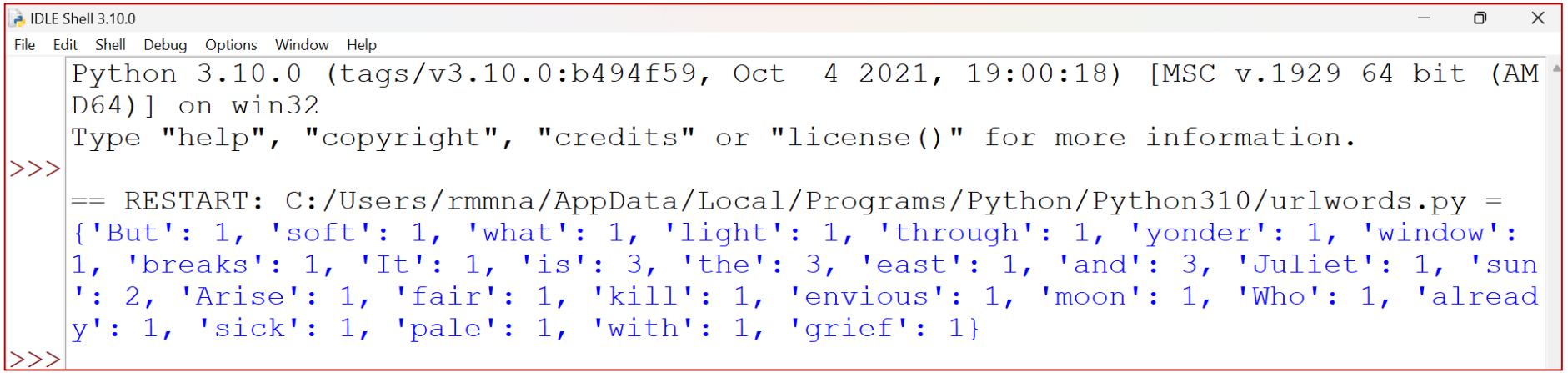
Using urllib in Python – Example3

urlwords.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/urlwords.py (3.10.0)

File Edit Format Run Options Window Help

```
import urllib.request, urllib.parse, urllib.error

fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')

counts = dict()
for line in fhand:
    words = line.decode().split()
    for word in words:
        counts[word] = counts.get(word, 0) + 1
print(counts)
```

Using urllib in Python – Example3



http requests

- To make HTTP requests in Python, we can use the popular '**`requests`**' library.
- The requests library provides a convenient and user-friendly API for making HTTP requests and handling responses.
- To install the requests library before running the program, we can install it using pip:

```
pip install requests
```

- By using the requests library, we can easily make HTTP requests and process the responses in our Python programs.

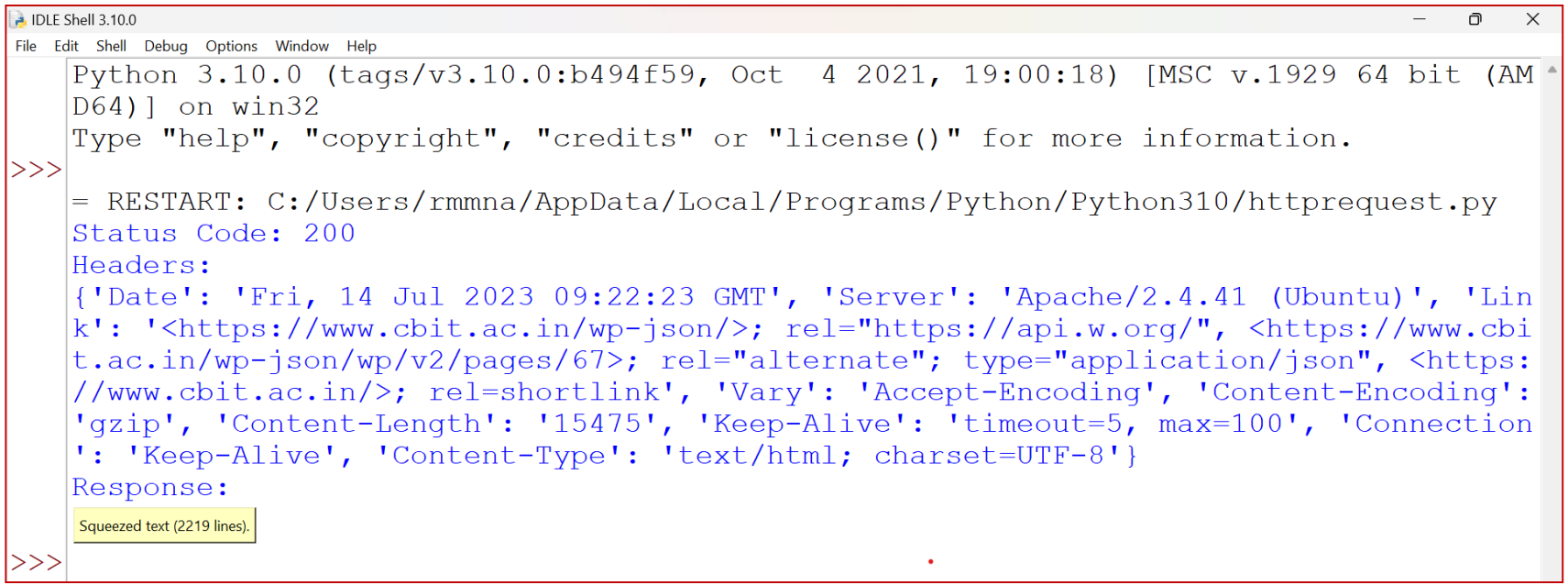
http requests - Example

httprequest.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/httprequest.py (3.10.0)

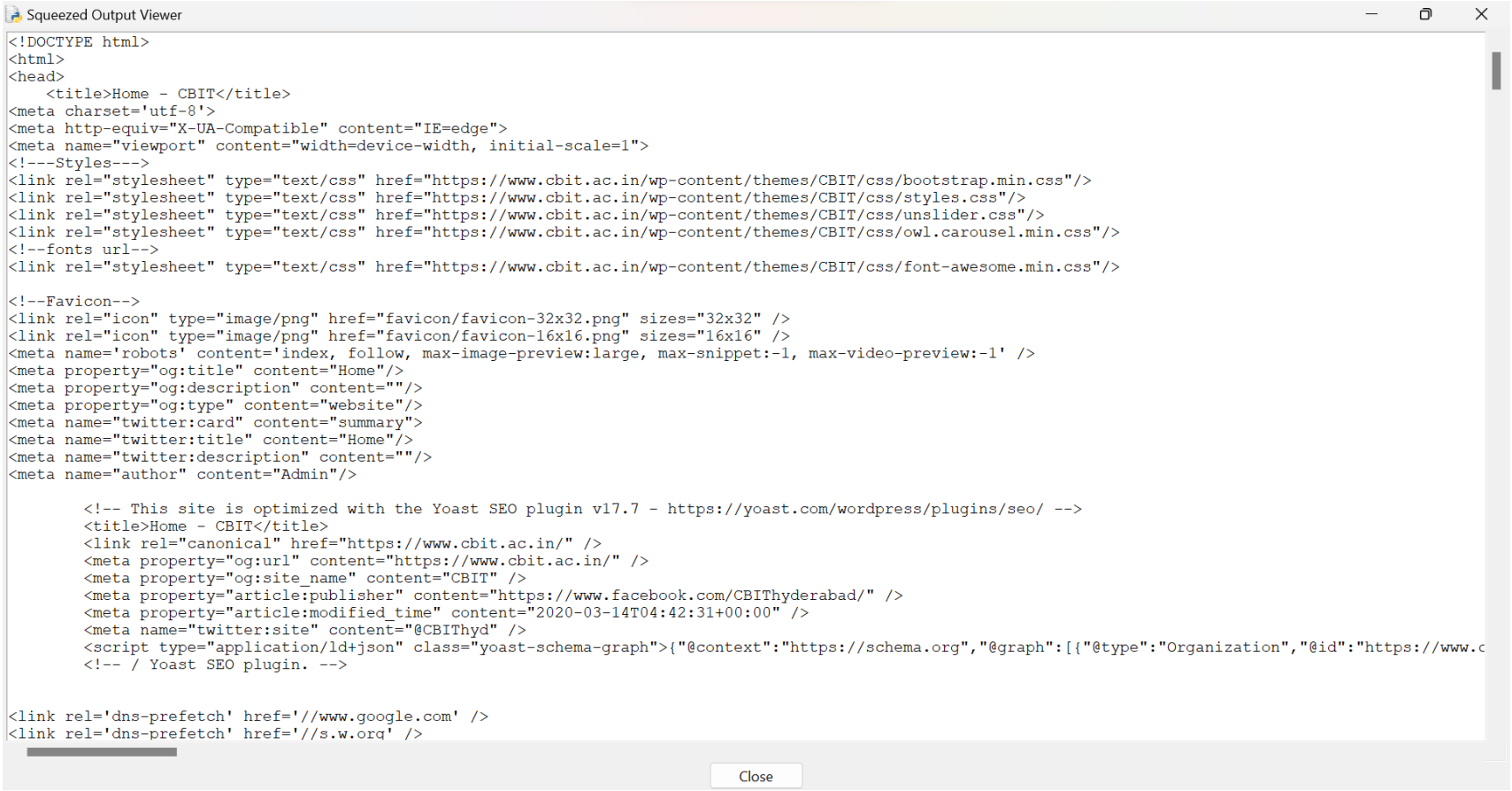
File Edit Format Run Options Window Help

```
import requests

# Make an HTTP GET request
response = requests.get("https://www.cbit.ac.in/")

# Print the response status code
print("Status Code:", response.status_code)

# Print the response headers
print("Headers:")
print(response.headers)

# Print the response body
print("Response:")
print(response.text)
```

http requests - Example



http requests - Example



Reading Web Pages

Reading web pages - Example

readingwebpages.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/readingwebpages.py (3.10.0)

File Edit Format Run Options Window Help

```
import urllib.request, urllib.parse, urllib.error

fhand = urllib.request.urlopen('http://www.dr-chuck.com/page1.htm')
for line in fhand:
    print(line.decode().strip())
```

IDLE Shell 3.10.0

File Edit Shell Debug Options Window Help

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rmmna/AppData/Local/Programs/Python/Python310/readingwebpages.
py
<h1>The First Page</h1>
<p>
If you like, you can switch to the
<a href="http://www.dr-chuck.com/page2.htm">
Second Page</a>.
</p>
>>>
```

UNIT- IV

Python to access Web Data : cont'd.

- Regular Expressions
- Extracting data
- Sockets
- Using the Developer Console to Explore HTTP and Retrieving Web Page
- Parsing Web Pages

Reference

https://www.coursera.org/learn/python-network-data

https://github.com/I-am-Harsh/Using-Python-to-Access-Web-Data

https://www.py4e.com/code3/

https://eng.libretexts.org/Bookshelves/Computer_Science/Programming_Languages/Book%3A_Python_for_Everybody_(Severance)

http://docs.python.org/library/socket.html

https://youtu.be/T0rYSFPAR0A

Parsing Web Pages / Parsing HTML (also known as Web Scraping)

What is Web Scraping?

- When a program or script pretends to be a browser and retrieves web pages, looks at those web pages, extracts information, and then looks at more web pages.
- Search engines scrape web pages - we call this “spidering the web” or “web crawling” or “web indexing”.
- web crawling is the process of automatically browsing and discovering web pages on the internet. It involves systematically following hyperlinks from one web page to another, recursively traversing through websites to collect data and build an index of the web.

What is Web Scraping?

- Web scraping refers to the automated extraction of data from websites.
- It involves writing code in a programming language, such as Python, to retrieve and parse HTML or XML content from web pages.
- Web scraping enables us to extract specific information, such as text, images, tables, or links, from websites and save it for further analysis, processing, or storage.

Why Scrape?

- Pull data - particularly social data - who links to who?
- Get our own data back out of some system that has no "export capability"
- Monitor a site for new information
- Spider the web to make a database for a search engine

Scraping Web Pages

- There is some controversy about web page scraping and some sites are a bit snippy about it.
- Republishing copyrighted information is not allowed
- Violating terms of service is not allowed
- Python is a popular language for web scraping due to its simplicity, rich ecosystem of libraries, and excellent tools for parsing HTML and XML

Scraping Web Pages

Key steps involved in a typical web scraping process:

- Sending HTTP requests: Use a library like `requests` or `urllib` to send an HTTP GET or POST request to the target website's URL.
- Retrieving the HTML content: Once the request is sent, we receive the HTML content of the web page as the response. We can access this content using the `response.text` property or similar methods.
- Parsing the HTML content: To extract data from the HTML, we need to parse it using a library like `BeautifulSoup` or `lxml`. These libraries provide functions and methods to traverse and search the HTML structure, extract specific elements, and retrieve their attributes or contents.
- `lxml` is one of the fastest and feature-rich libraries for processing XML and HTML in Python. This library is essentially a wrapper over C libraries libxml2 and libxslt (XSLT - Extensible Stylesheet Language Transformations). This combines the speed of the native C library and the simplicity of Python.

Scraping Web Pages

Key steps involved in a typical web scraping process: *cont'd.*

- Extracting data: Using the parsed HTML, we can apply various techniques to locate the desired data. This may involve searching for specific HTML tags, CSS (Cascading Style Sheets is a style sheet language used for describing the presentation of a document written in a markup language such as HTML or XML) classes, or attributes. We can use methods like `find_all()` or `Xpath` (XML path language) expressions to extract the required data.
- Storing or processing the data: Once we have extracted the data, we can save it to a file, a database, or process it further according to our needs.

Scraping Web Pages

- Web scraping can be a powerful tool for gathering data from the web, automating repetitive tasks, conducting research, and building data-driven applications.
- However, it's crucial to use it responsibly and ethically, respecting the legal and ethical boundaries of web scraping.

Scraping Web Pages using BeautifulSoup

Free software library called BeautifulSoup from https://www.crummy.com/software/BeautifulSoup/

[Download | Documentation | Hall of Fame | For enterprise | Source | Changelog | Discussion group | Zine]

Beautiful Soup

You didn't write that awful page. You're just trying to get some data out of it. Beautiful Soup is here to help. Since 2004, it's been saving programmers hours or days of work on quick-turnaround screen scraping projects.

Beautiful Soup is a Python library designed for quick turnaround projects like screen-scraping. Three features make it powerful:

1. Beautiful Soup provides a few simple methods and Pythonic idioms for navigating, searching, and modifying a parse tree: a toolkit for dissecting a document and extracting what you need. It doesn't take much code to write an application
2. Beautiful Soup automatically converts incoming documents to Unicode and outgoing documents to UTF-8. You don't have to think about encodings, unless the document doesn't specify an encoding and Beautiful Soup can't detect one. Then you just have to specify the original encoding.
3. Beautiful Soup sits on top of popular Python parsers like lxml and html5lib, allowing you to try out different parsing strategies or trade speed for flexibility.

Beautiful Soup parses anything you give it, and does the tree traversal stuff for you. You can tell it "Find all the links", or "Find all the links of class `externalLink`", or "Find all the links whose urls match "foo.com", or "Find the table heading that's got bold text, then give me that text."

Valuable data that was once locked up in poorly-designed websites is now within your reach. Projects that would have taken hours take only minutes with Beautiful Soup.

BeautifulSoup Installation

- Beautiful Soup is a Python library for pulling data out of HTML and XML files.
- It works with our favorite parser to provide native ways of navigating, searching, and modifying the parse tree.
- It commonly saves programmers hours or days of work.
- The latest Version of Beautifulsoup is v4.9.3 as of now.

BeautifulSoup Installation

- To run this, we can install BeautifulSoup

```
pip install beautifulsoup4
```

- Import bs4 from BeautifulSoup

```
from bs4 import BeautifulSoup
```

BeautifulSoup - Example

webscrap.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/webscrap.py (3.10.0)

File Edit Format Run Options Window Help

```
import urllib.request, urllib.parse, urllib.error
from bs4 import BeautifulSoup

url = input('Enter - ')
html = urllib.request.urlopen(url).read()
soup = BeautifulSoup(html, 'html.parser')

# Retrieve all of the anchor tags
tags = soup('a')
for tag in tags:
    print(tag.get('href', None))
```

BeautifulSoup - Example

Lab Experiment

Create a Python project to get the citation from Google scholar using title and year of publication and volume and pages of journal

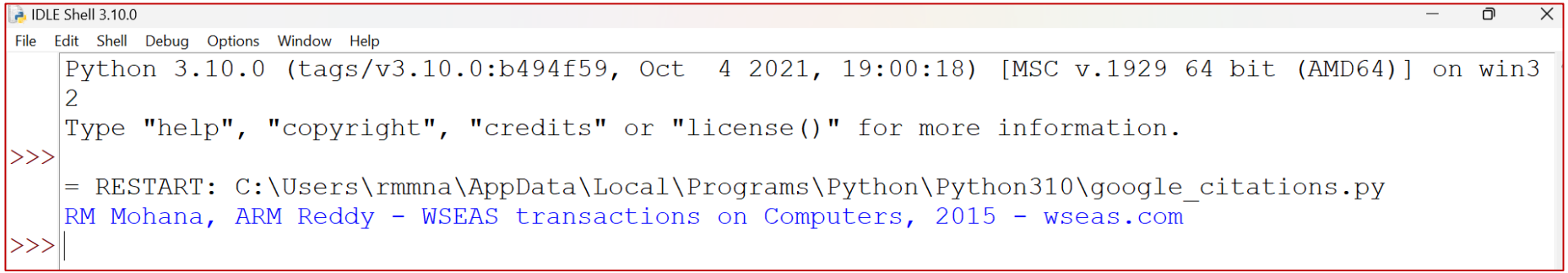
google_citations.py - C:\Users\rmmna\AppData\Local\Programs\Python\Python310\google_citations.py (3.10.0)

File Edit Format Run Options Window Help

```
import requests
from bs4 import BeautifulSoup
def get_citation(title, year, volume, pages):
    # Format the search query
    query = f"{title} {year} {volume}:{pages}"
    query = query.replace(' ', '+')
    url = f"https://scholar.google.com/scholar?q={query}"
    # Send a GET request to Google Scholar
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')
    # Find the first search result
    result = soup.find('div', {'class': 'gs_ri'})
    if result:
        # Extract the citation text
        citation = result.find('div', {'class': 'gs_a'}).text
        return citation
    else:
        return "Citation not found"
# Example usage
article_title = "CCBIR: A cloud based implementation of content based image retrieval"
article_year = "2015"
article_volume = "14"
article_pages = "333-346"
citation = get_citation(article_title, article_year, article_volume, article_pages)
print(citation)
```

Lab Experiment

Create a Python project to get the citation from Google scholar using title and year of publication and volume and pages of journal



Object Oriented Programming (Using Python)

UNIT- V

Using Databases with Python:

- Using Databases
- Single Table CRUD (Create, Read, Update, and Delete)
- Designing and representing a data model
- Reconstructing data with JOIN
- Many-to-many relationships

References

https://www.coursera.org/learn/python-databases/home/info

https://www.py4e.com/book

http://do1.dr-chuck.com/pythonlearn/EN_us/pythonlearn.pdf

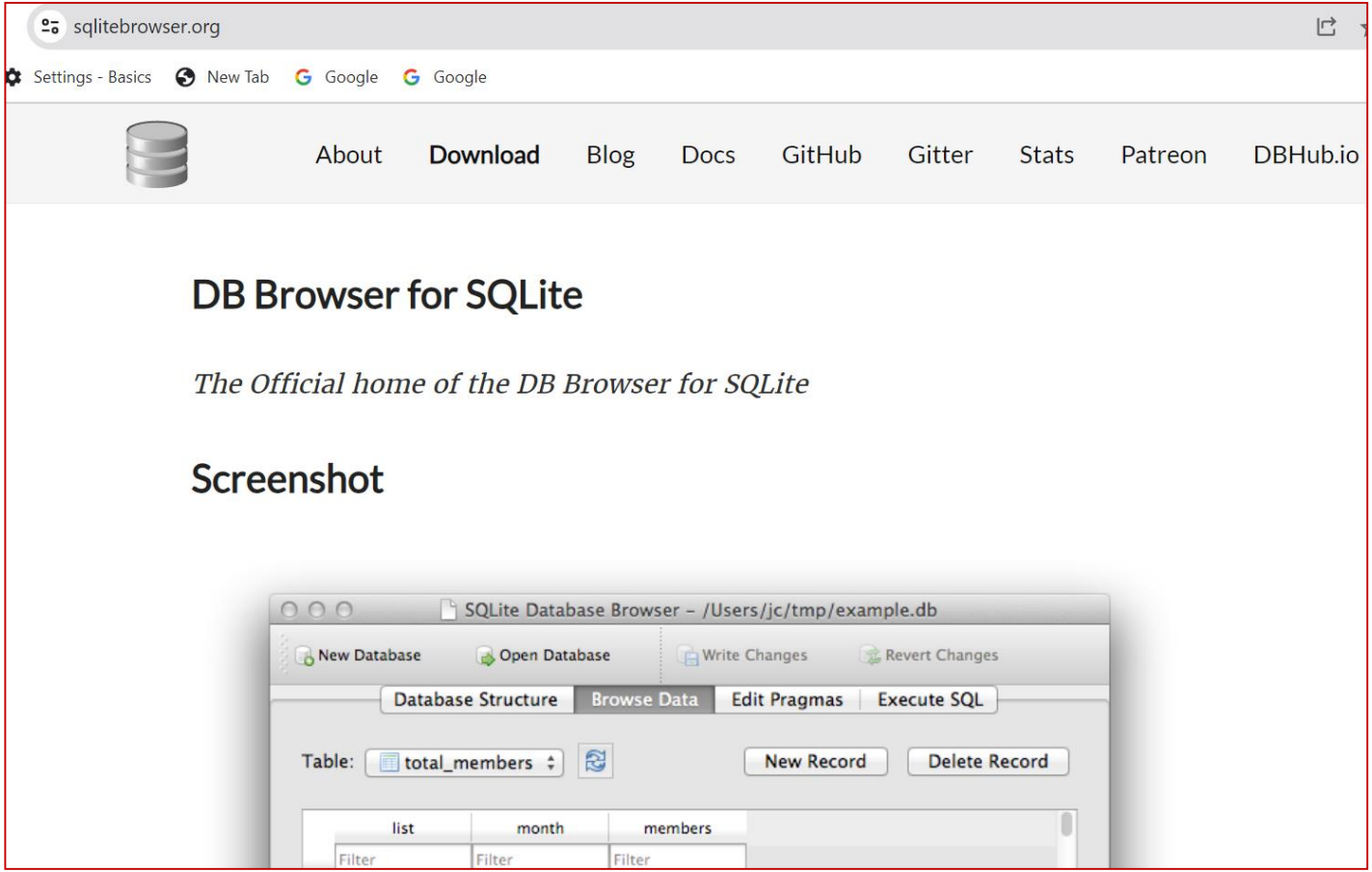
https://www.py4e.com/code3/

https://www.py4e.com/lectures3/

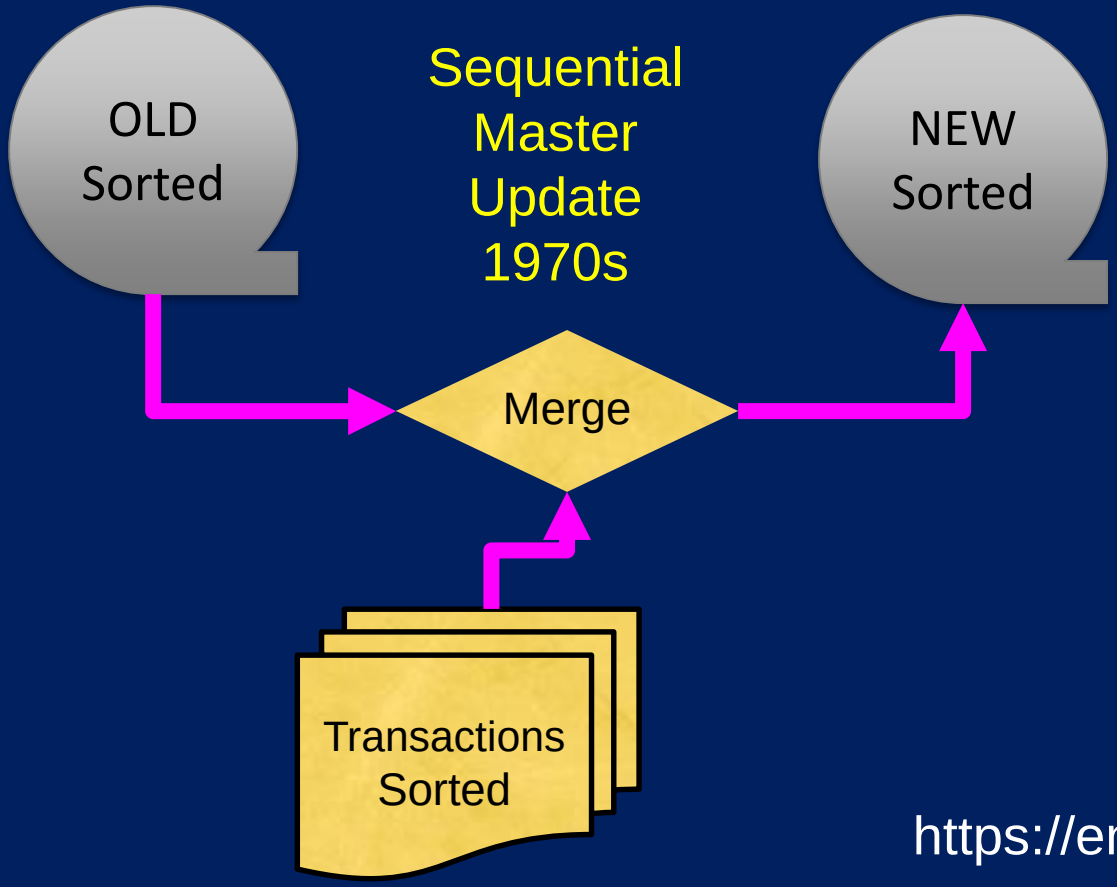
https://www.youtube.com/playlist?list=PLlRFEj9H3Oj7Bp8-DfGpfAfDBiblRfl5p

http://sqlitebrowser.org/

Relational Databases and SQLite



http://sqlitebrowser.org/



https://en.wikipedia.org/wiki/IBM_729

Random Access

- When you can randomly access data...
- How can you layout data to be most efficient?
- Sorting might not be the best idea

https://en.wikipedia.org/wiki/Hard_disk_drive_platter

Relational Databases

Relational databases model data by storing rows and columns in tables. The power of the relational database lies in its ability to efficiently retrieve data from those tables and in particular where there are multiple tables and the relationships between those tables involved in the query.

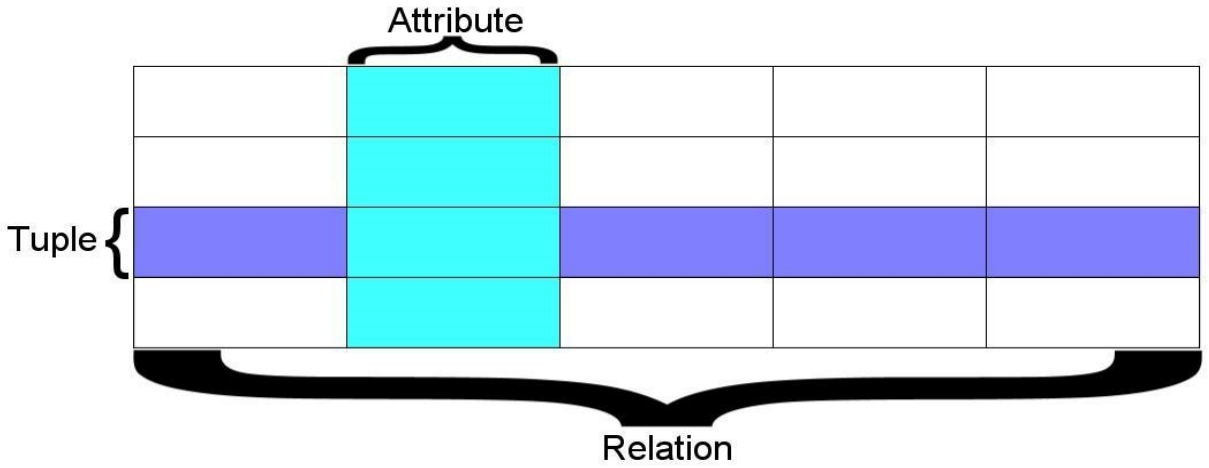
http://en.wikipedia.org/wiki/Relational_database

Terminology

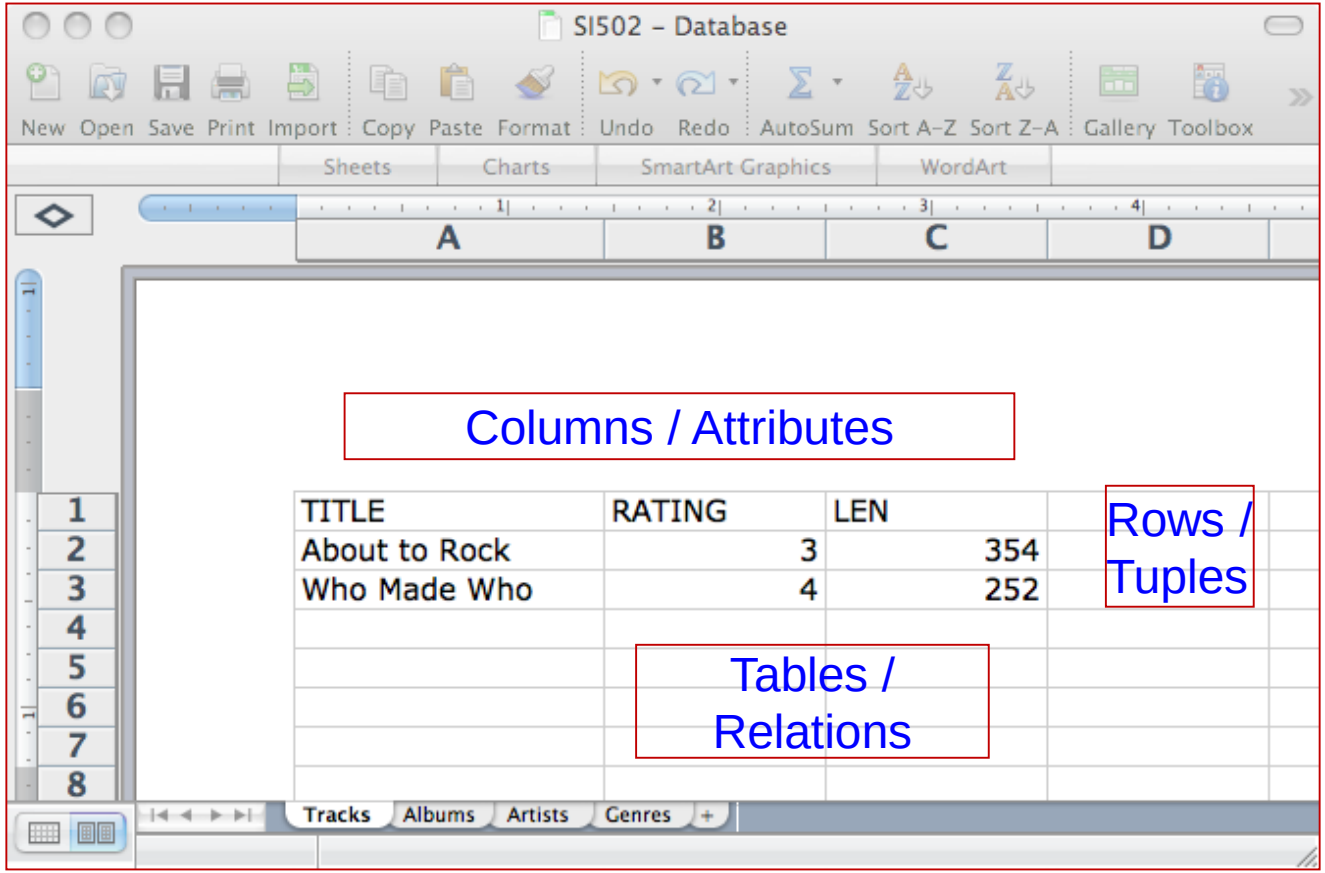
- Database - contains many tables
- Relation (or table) - contains tuples and attributes
- Tuple (or row) - a set of fields that generally represents an "object" like a person or a music track
- Attribute (also column or field) - one of possibly many elements of data corresponding to the object represented by the row

Relation

A relation is defined as a set of tuples that have the same attributes. A tuple usually represents an object and information about that object. Objects are typically physical objects or concepts. A relation is usually described as a table, which is organized into rows and columns. All the data referenced by an attribute are in the same domain and conform to the same constraints. (Wikipedia)



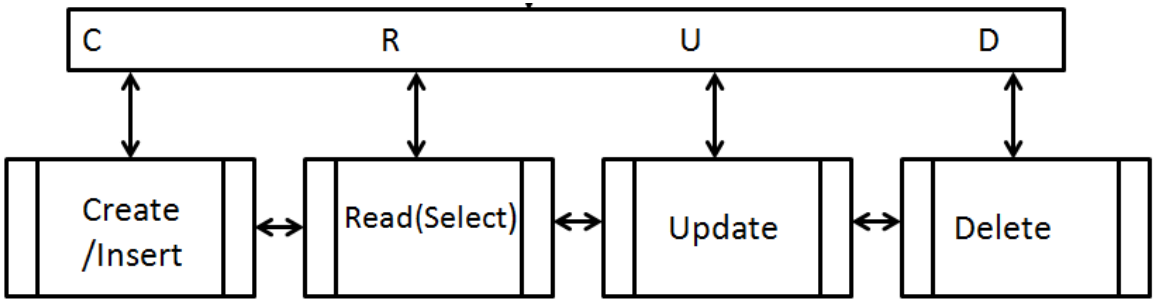
Relation - Example



SQL

SQL (Structured Query Language) is the language we use to issue commands to the database

- Create data (also known as Add/Insert)
- Retrieve data
- Update data
- Delete data



http://en.wikipedia.org/wiki/SQL

Using Databases

Two Roles in Large Projects

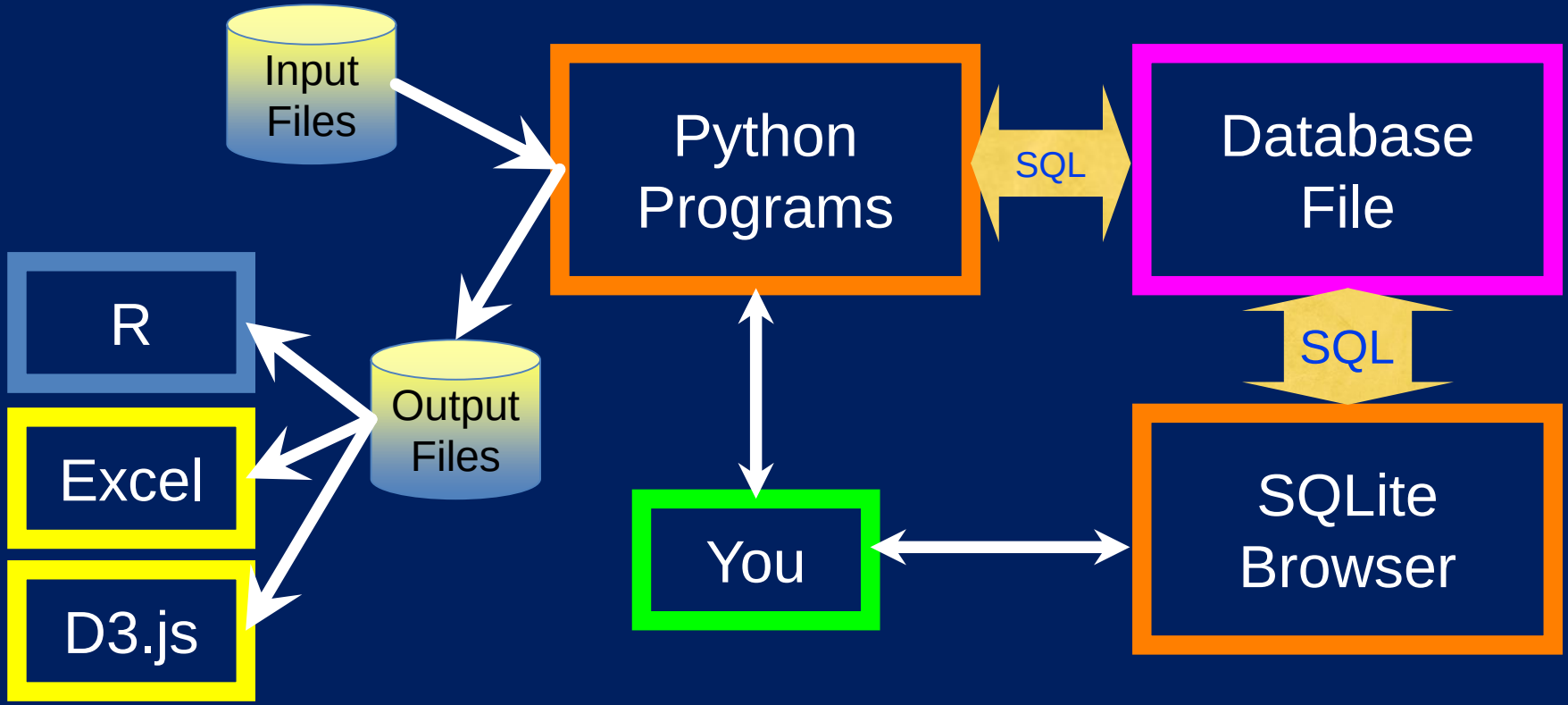
- Application Developer - Builds the logic for the application, the look and feel of the application - monitors the application for problems
- Database Administrator - Monitors and adjusts the database as the program runs in production
- Often both people participate in the building of the “Data model”

Database Administrator (DBA)

- A Database Administrator (DBA) is a person responsible for the design, implementation, maintenance, and repair of an organization's database.
- The role includes the development and design of database strategies, monitoring and improving database performance and capacity, and planning for future expansion requirements.
- They may also plan, coordinate, and implement security measures to safeguard the database.

http://en.wikipedia.org/wiki/Database_administrator

Data Analysis Structure



Database Model

- A database model or database schema is the structure or format of a database, described in a formal language supported by the database management system.
- In other words, a "database model" is the application of a data model when used in conjunction with a database management system.

http://en.wikipedia.org/wiki/Database_model

Common Database Systems

- Three major Database Management Systems in wide use
  - ✓ Oracle - Large, commercial, enterprise-scale, very very tweakable
  - ✓ MySql - Simpler but very fast and scalable - commercial open source
  - ✓ SqlServer - Very nice - from Microsoft (also Access)
- Many other smaller projects, free and open source
  - ✓ HSQL (Hyper SQL), **SQLite**, PostgreSQL (or Postgres)- is a powerful, open source object-relational database system etc.

SQLite is in Lots of Software...

http://www.sqlite.org/famous.html

SQLite Browser

- SQLite is a very popular database - it is free and fast and small
- SQLite Browser allows us to directly manipulate SQLite files

http://sqlitebrowser.org/

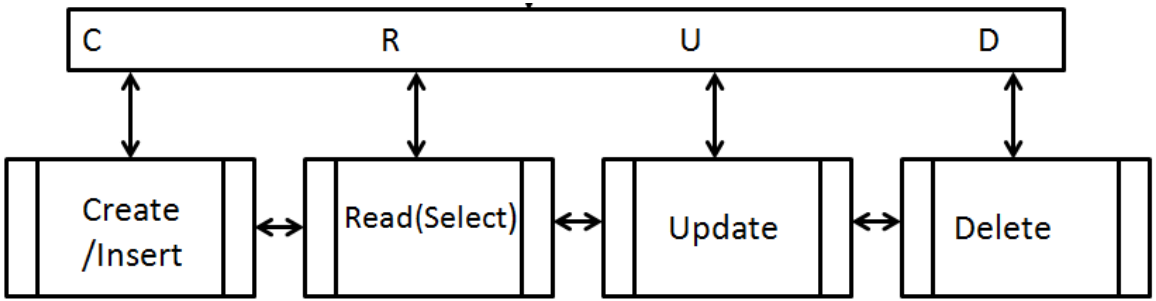
- SQLite is embedded in Python and a number of other languages

Single Table CRUD
(Create, Read, Update, and Delete)

SQL

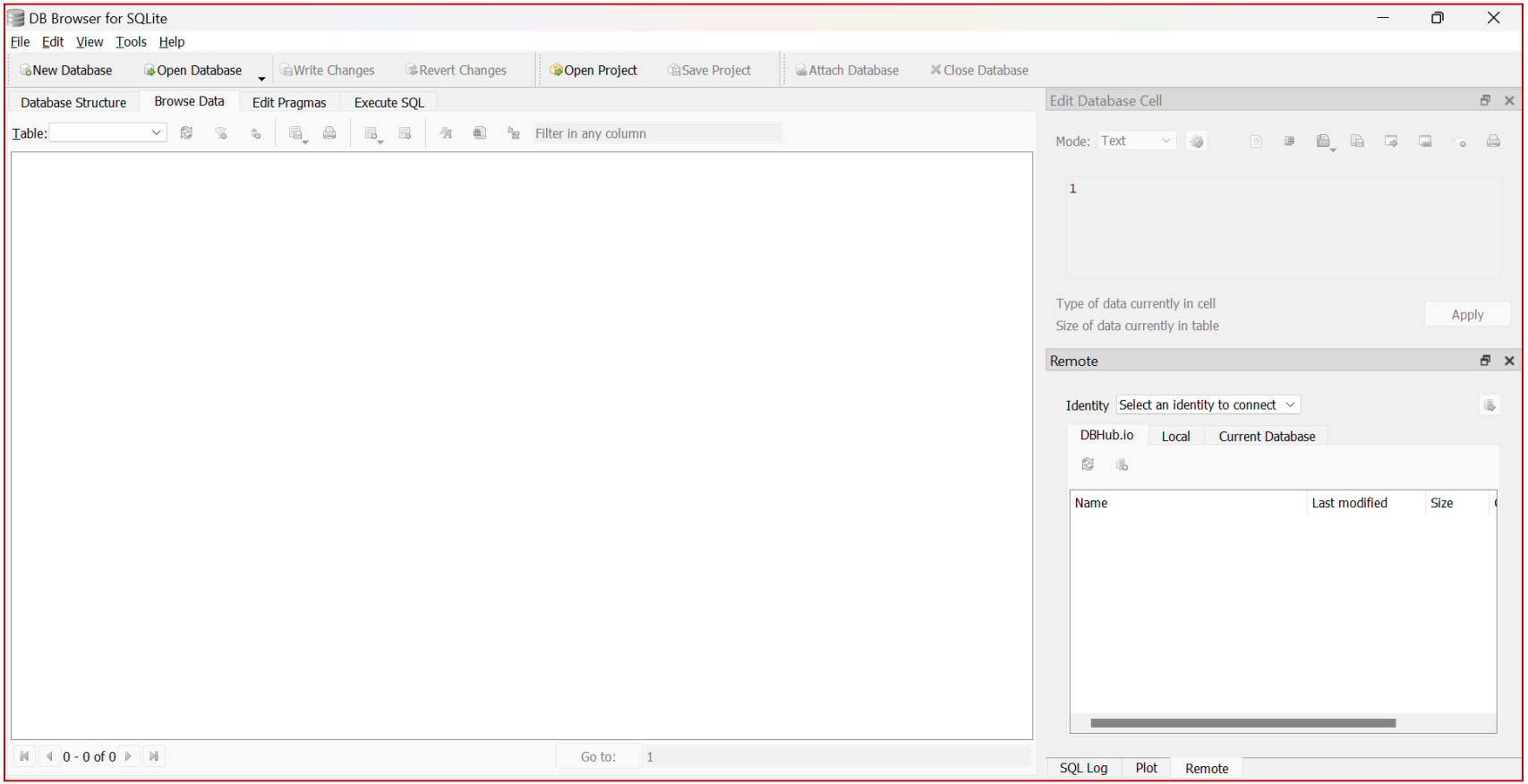
SQL (Structured Query Language) is the language we use to issue commands to the database

- Create data (also known as Add/Insert)
- Retrieve data (SELECT)
- Update data
- Delete data

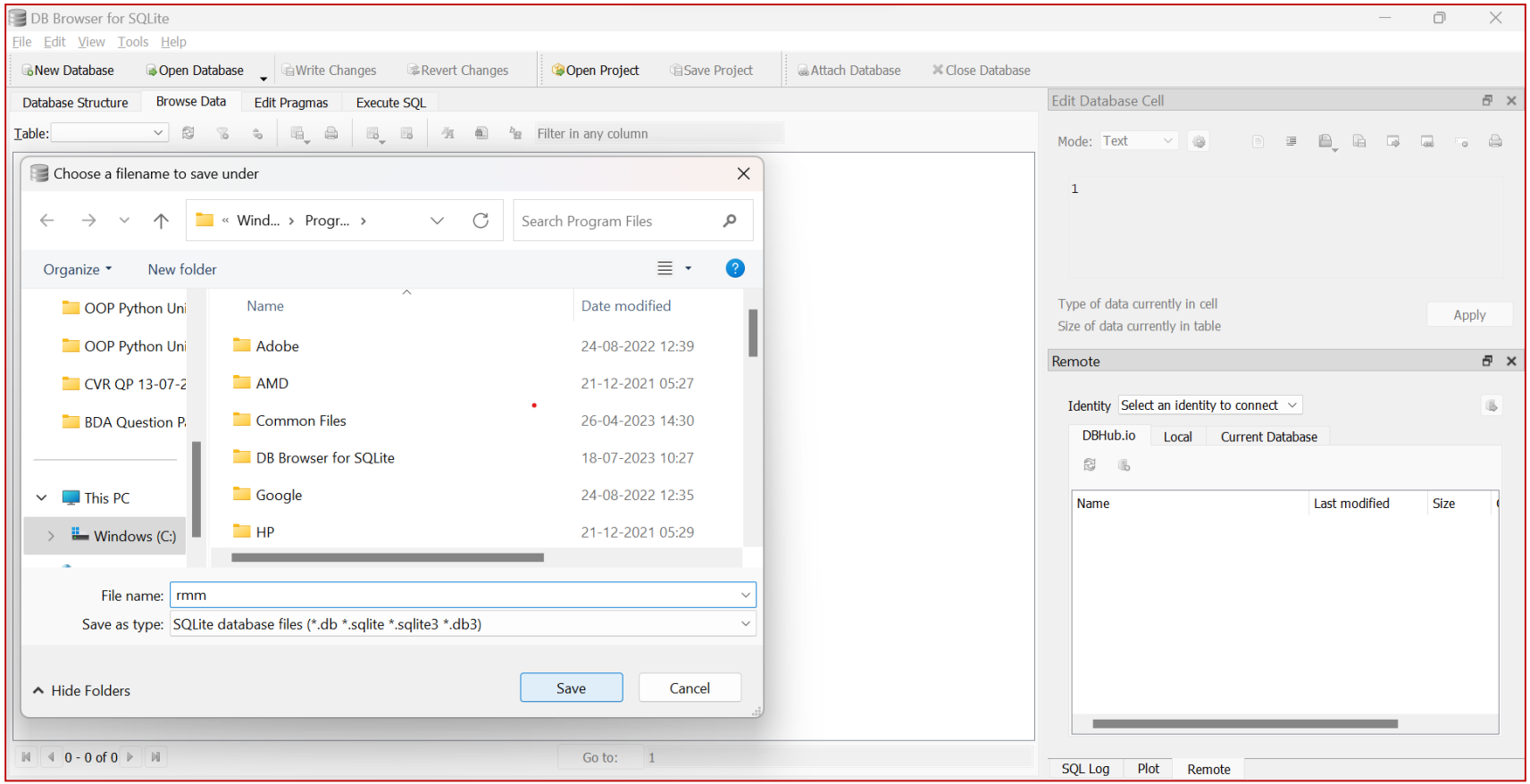


http://en.wikipedia.org/wiki/SQL

SQLite Browser



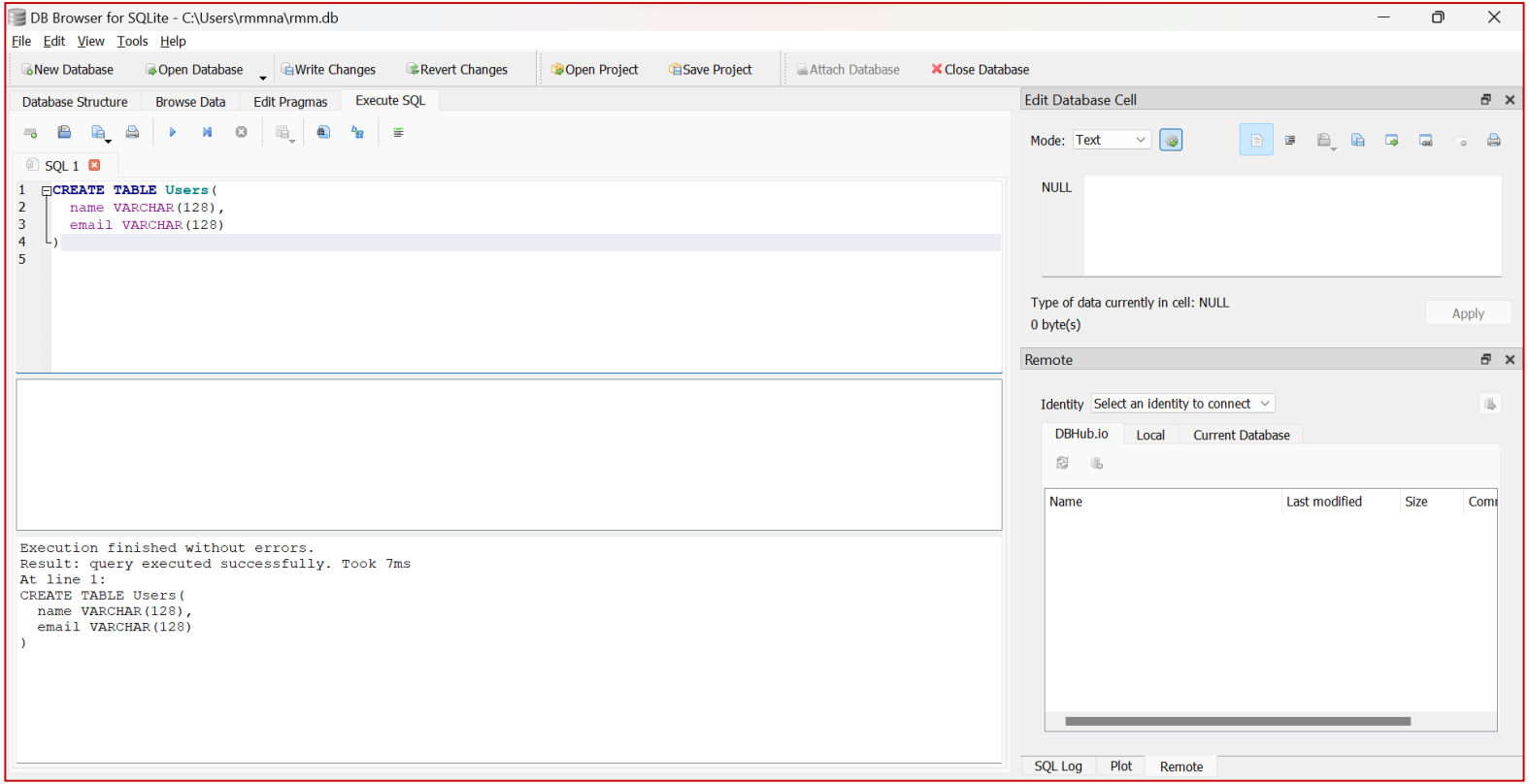
SQLite Browser



SQL - Create

The CREATE statement creates a table

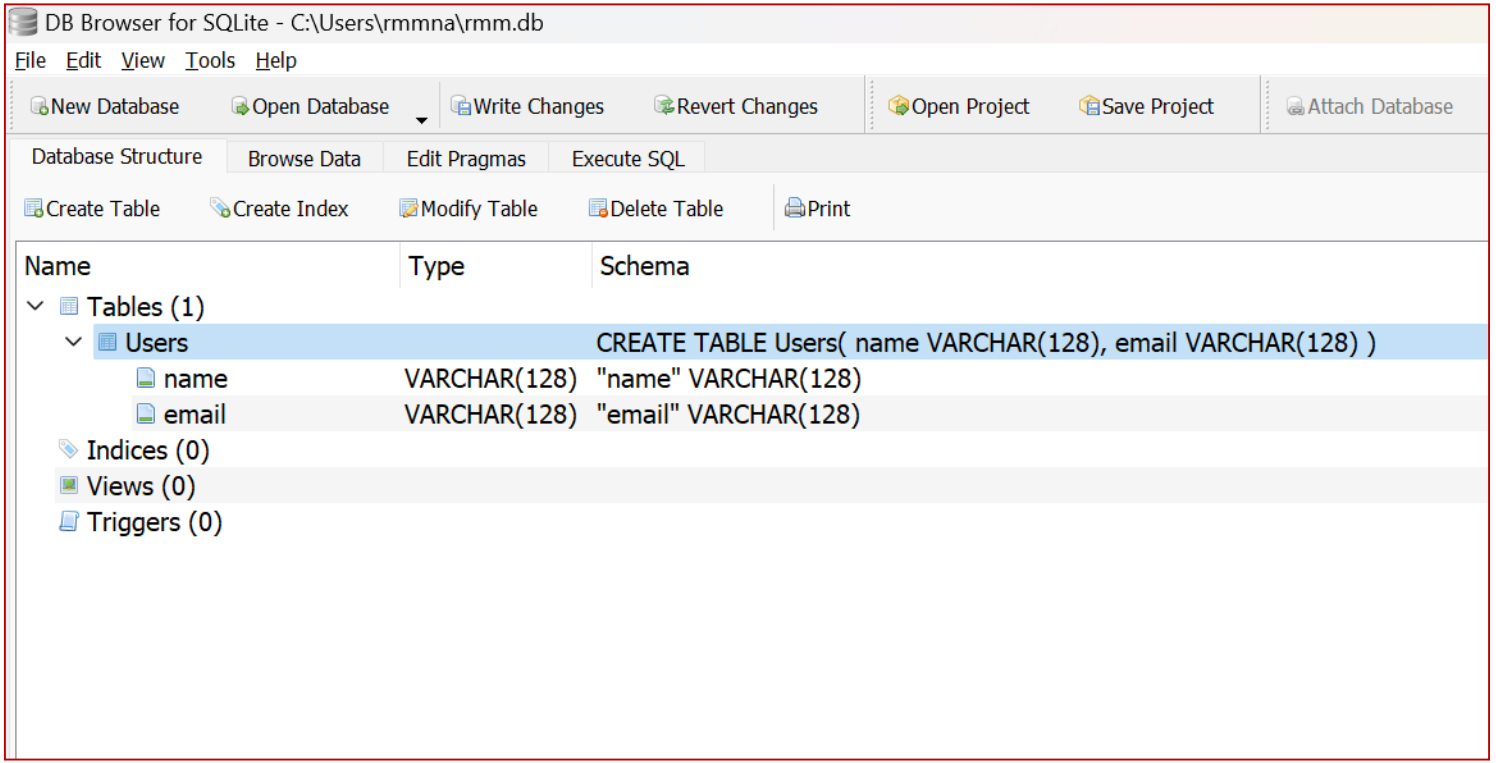
```
CREATE TABLE Users(name VARCHAR(128), email VARCHAR(128))
```



SQL - Create

The CREATE statement creates a table

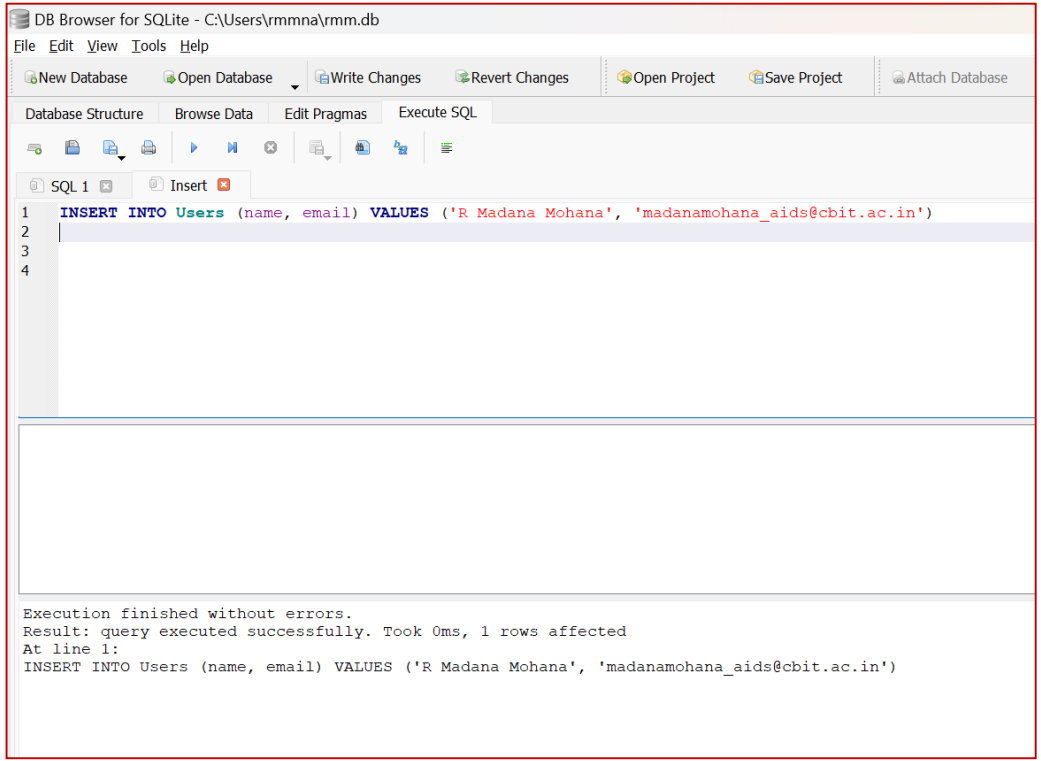
```
CREATE TABLE Users(name VARCHAR(128), email VARCHAR(128))
```



SQL - Insert

The Insert statement inserts a row into a table

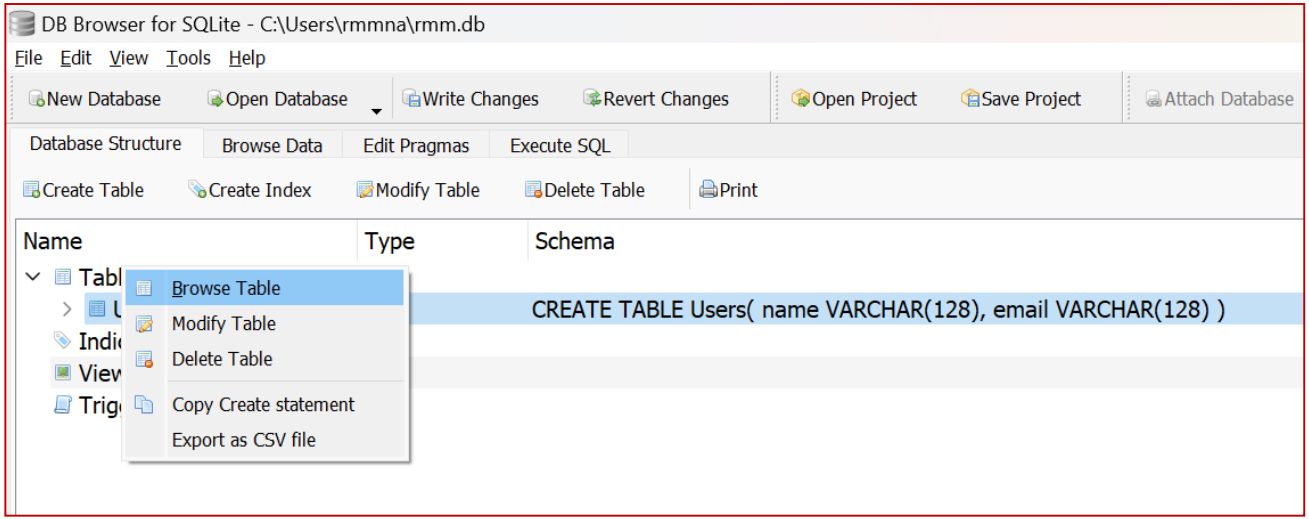
```
INSERT INTO Users (name, email) VALUES ('R Madana Mohana',
'madanamohana_aids@cbit.ac.in')
```



SQL - Insert

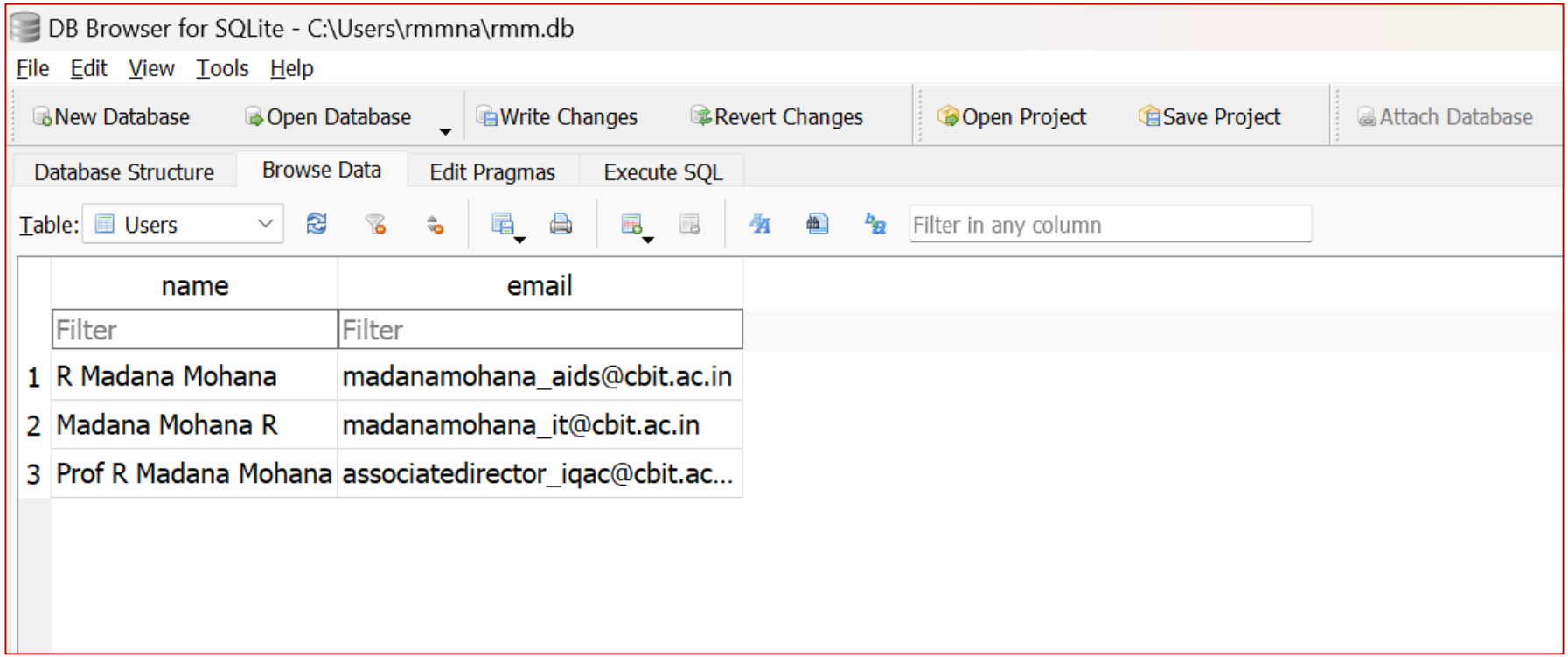
The Insert statement inserts a row into a table

```
INSERT INTO Users (name, email) VALUES ('R Madana Mohana',
'madanamohana_aids@cbit.ac.in')
INSERT INTO Users (name, email) VALUES ('Madana Mohana R',
'madanamohana_it@cbit.ac.in')
INSERT INTO Users (name, email) VALUES ('Prof R Madana Mohana',
'associatedirector_iqac@cbit.ac.in')
```



SQL - Insert

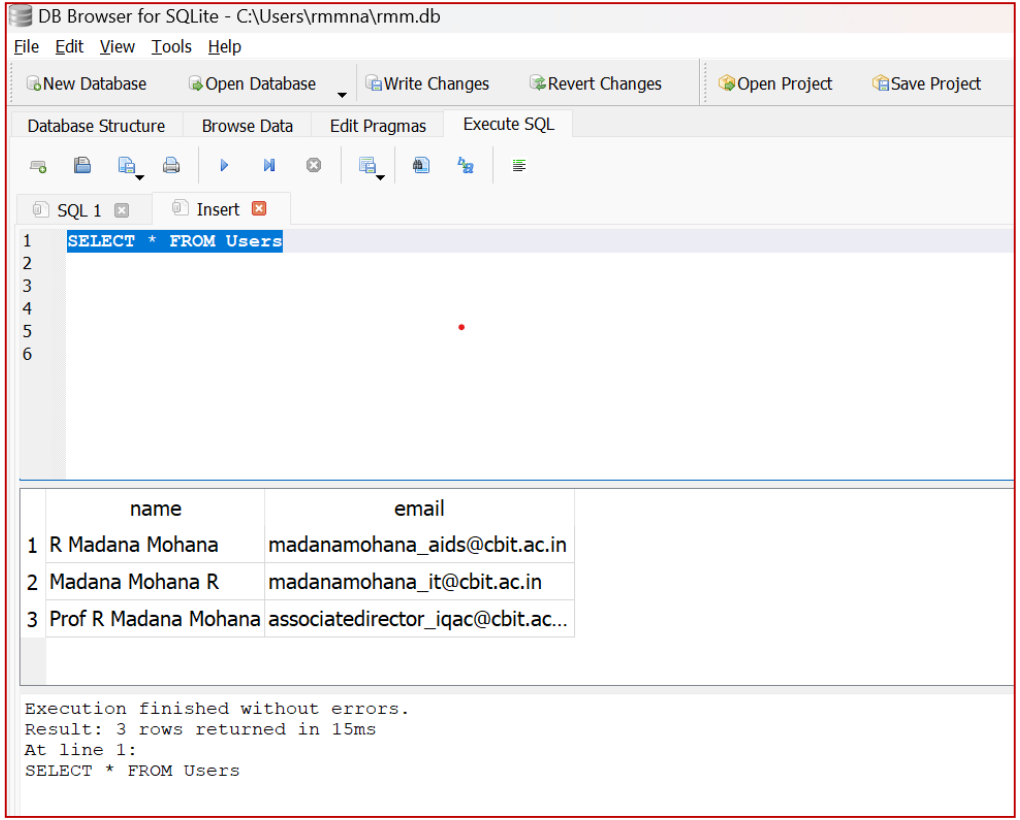
The Insert statement inserts a row into a table



SQL – Retrieving Records: Select

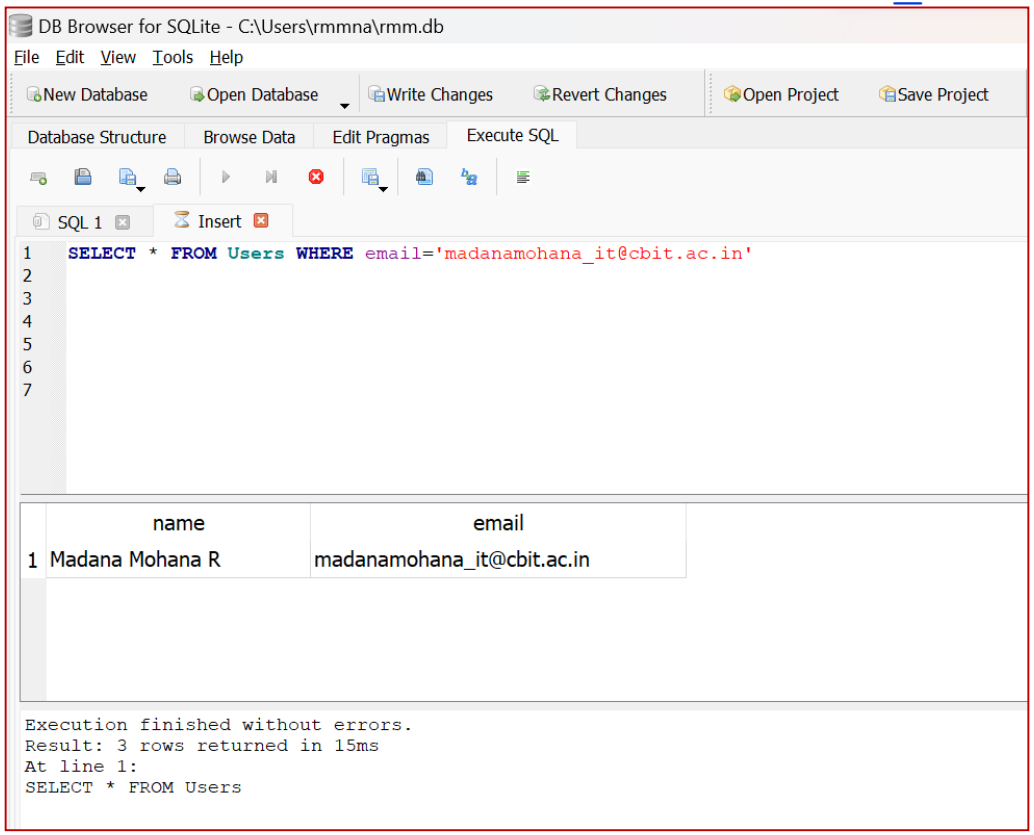
The SELECT statement retrieves a group of records - we can either retrieve all the records or a subset of the records with a WHERE clause.

```
SELECT * FROM Users
```



SQL – Retrieving Records: Select

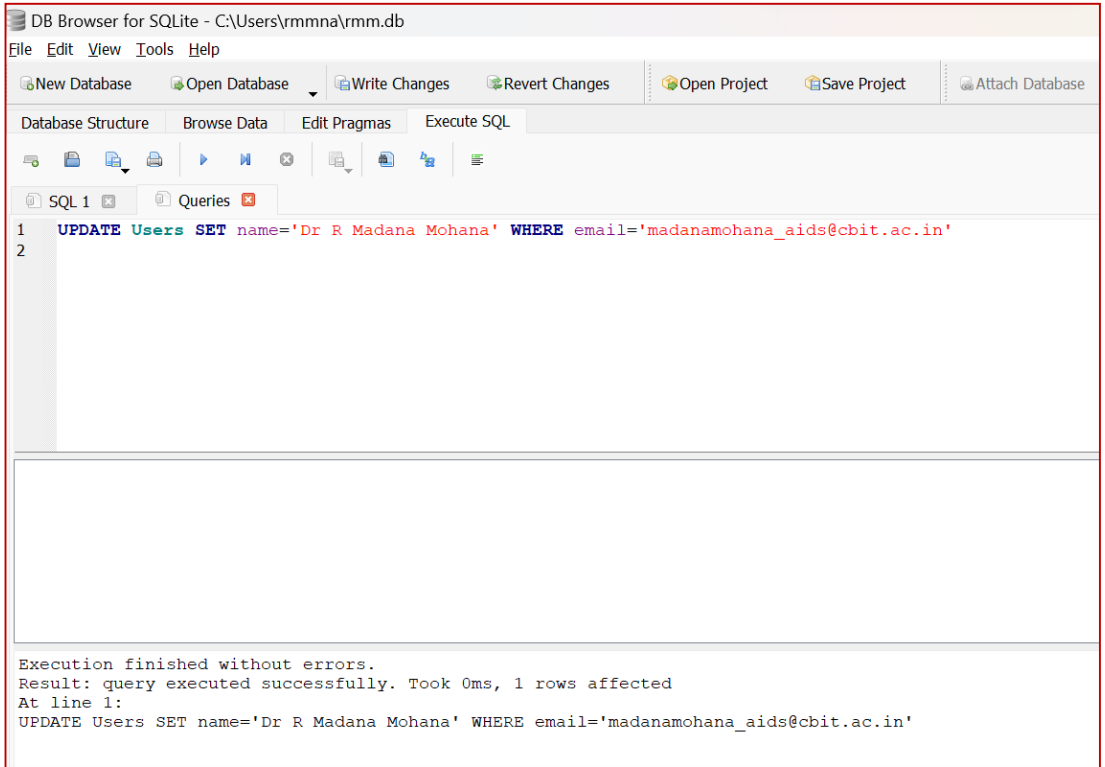
```
SELECT * FROM Users WHERE email='madanamohana_it@cbit.ac.in'
```



SQL – Update

Allows the updating of a field with a WHERE clause

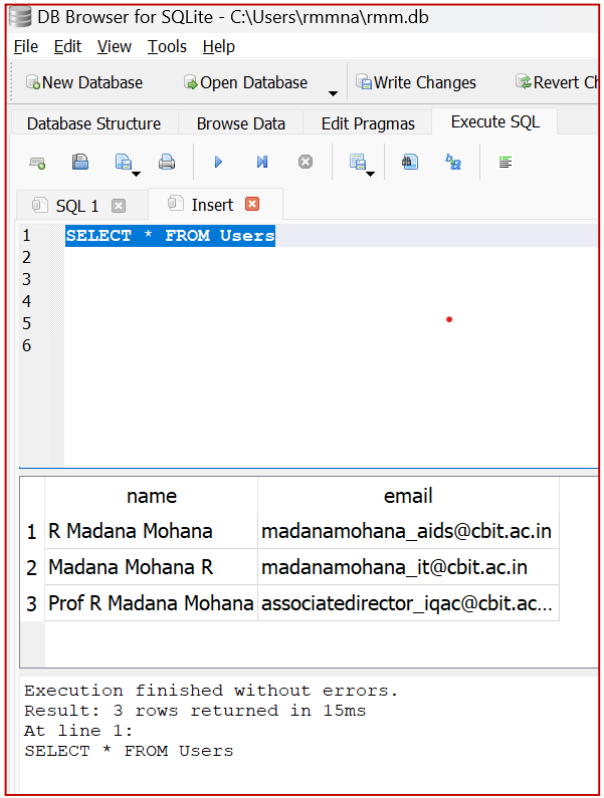
```
UPDATE Users SET name='Dr R Madana Mohana' WHERE
email='madanamohana_aids@cbit.ac.in'
```

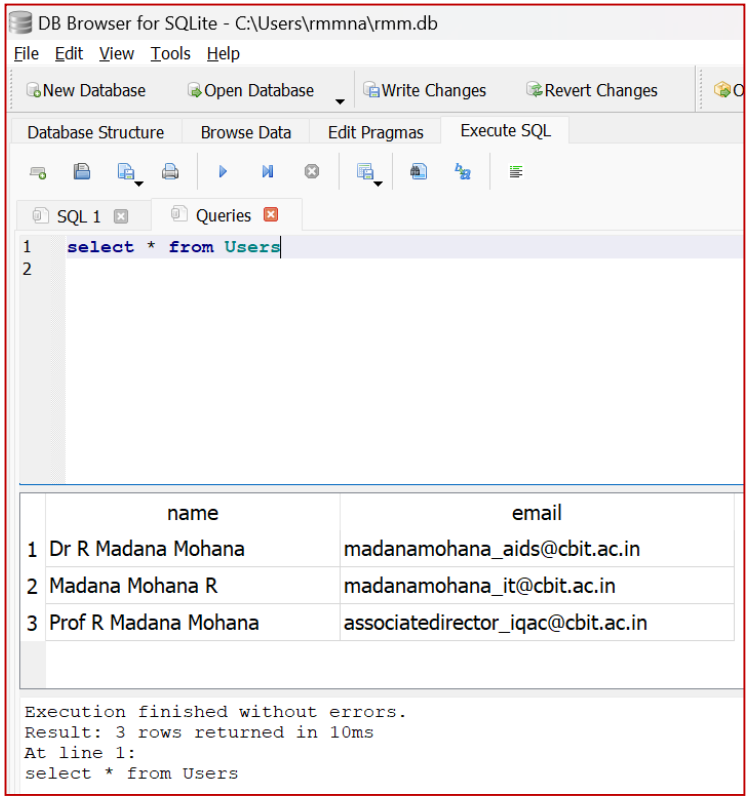


SQL – Update

Output before and after Update:

```
select * from Users
```

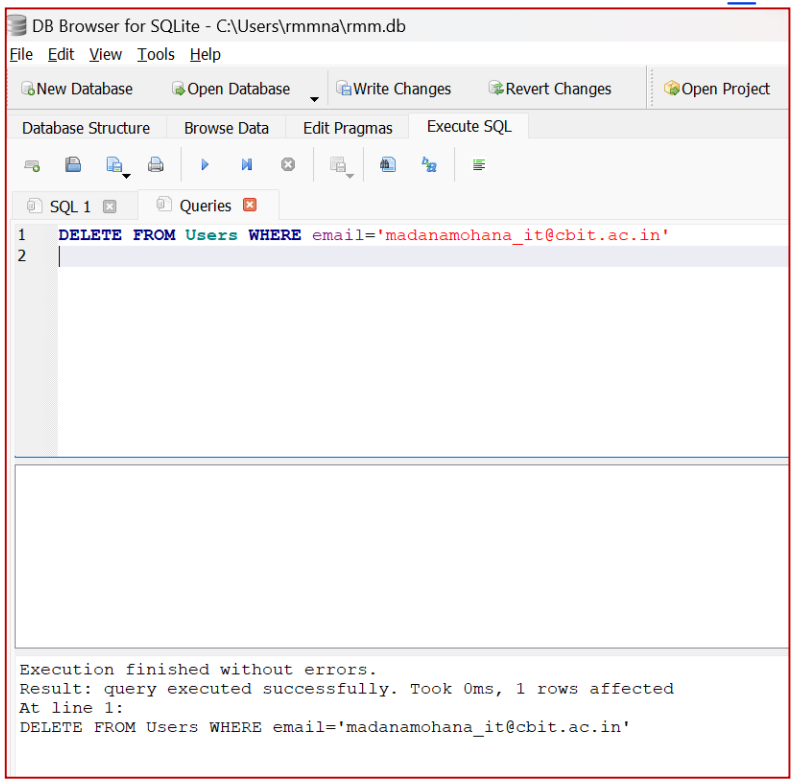




SQL – Delete

Deletes a row in a table based on selection criteria

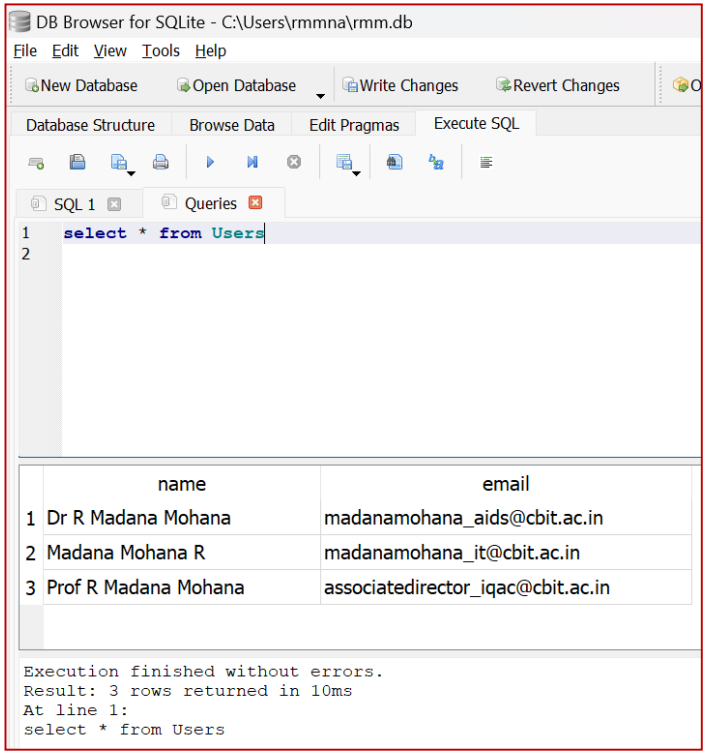
```
DELETE FROM Users WHERE email='madanamohana_it@cbit.ac.in'
```

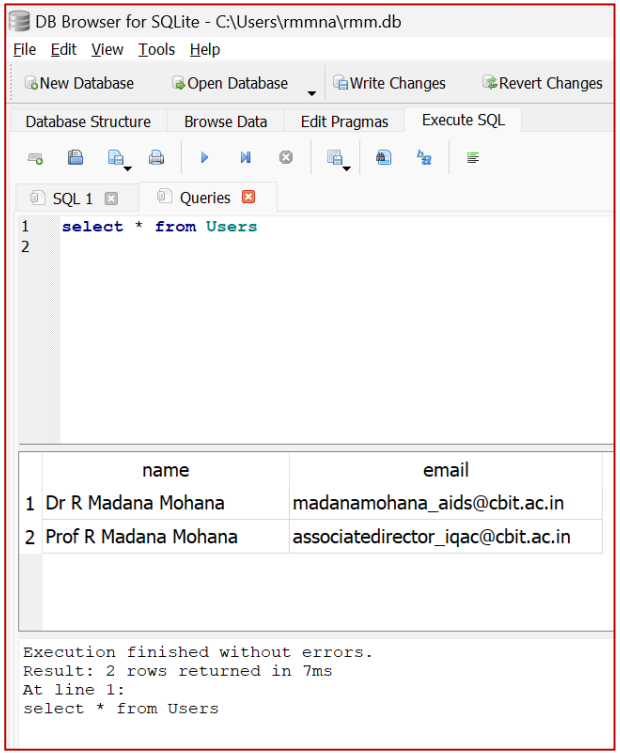


SQL – Delete

Output Before and after Delete

```
select * from Users
```

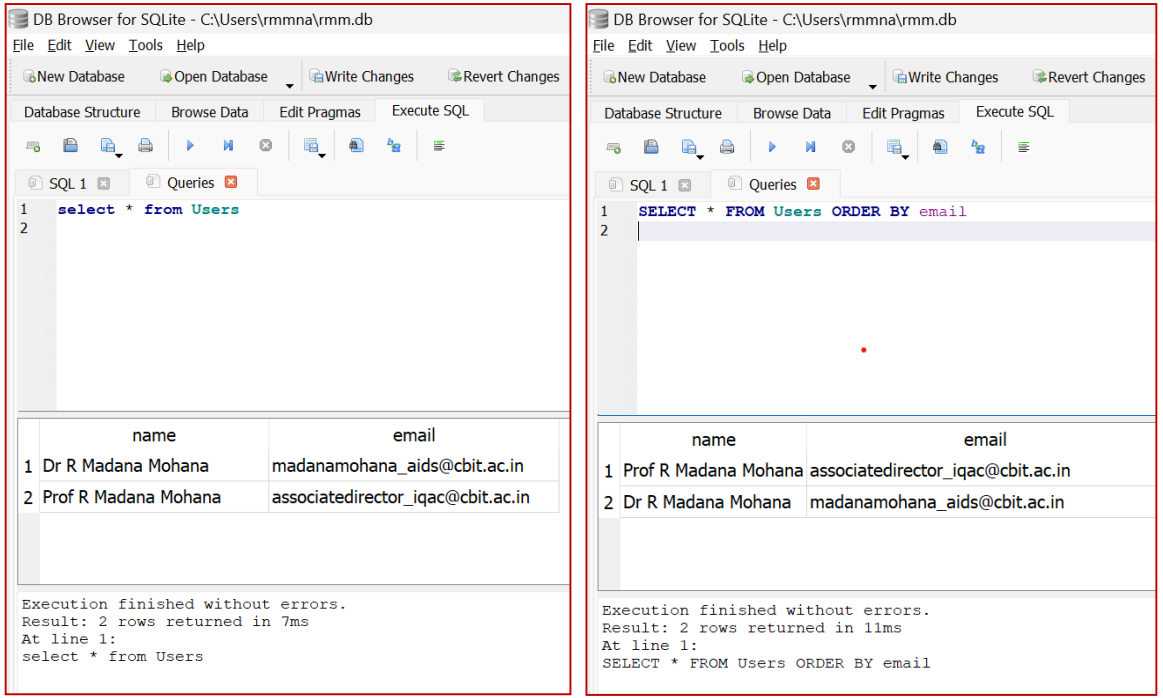




Sorting with ORDER BY

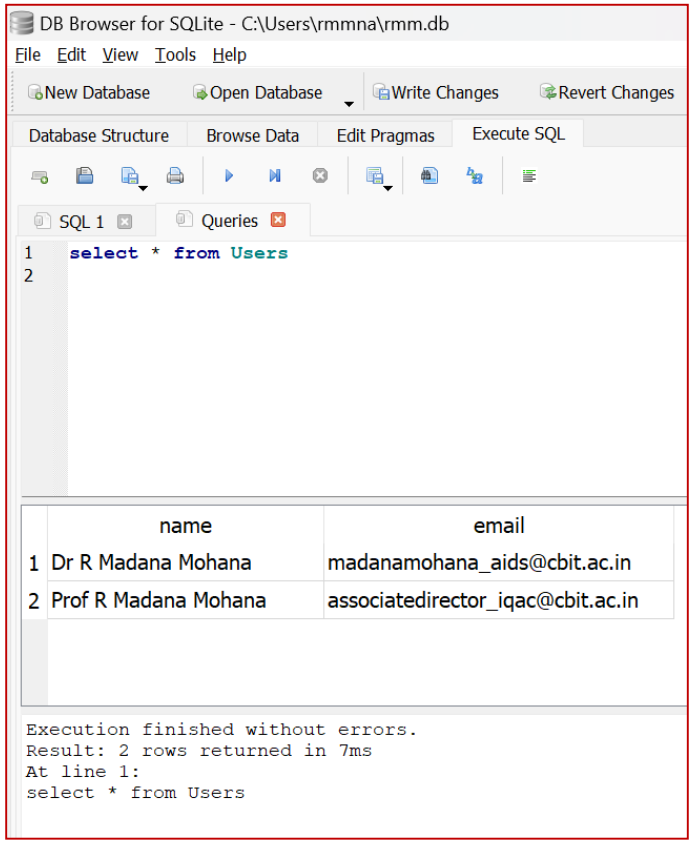
We can add an ORDER BY clause to SELECT statements to get the results sorted in ascending or descending order.

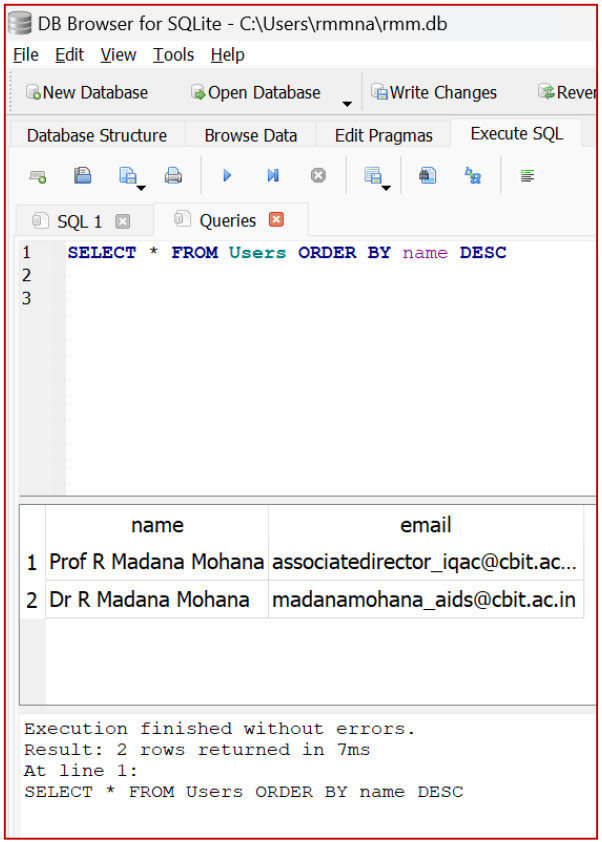
```
SELECT * FROM Users ORDER BY email
```



Sorting with ORDER BY

```
SELECT * FROM Users ORDER BY name DESC
```





SQL Summary

Output Before and after Delete

```
CREATE TABLE Users(name VARCHAR(128), email VARCHAR(128))

INSERT INTO Users (name, email) VALUES ('R Madana Mohana',
'madanamohana_aids@cbit.ac.in')

SELECT * FROM Users

SELECT * FROM Users WHERE email='madanamohana_it@cbit.ac.in'

UPDATE Users SET name='Dr R Madana Mohana' WHERE
email='madanamohana_aids@cbit.ac.in'

DELETE FROM Users WHERE email='madanamohana_it@cbit.ac.in'

SELECT * FROM Users ORDER BY email

SELECT * FROM Users ORDER BY name DESC
```

Single Table CRUD
(Create, Read, Update, and Delete)
Python Demo-1

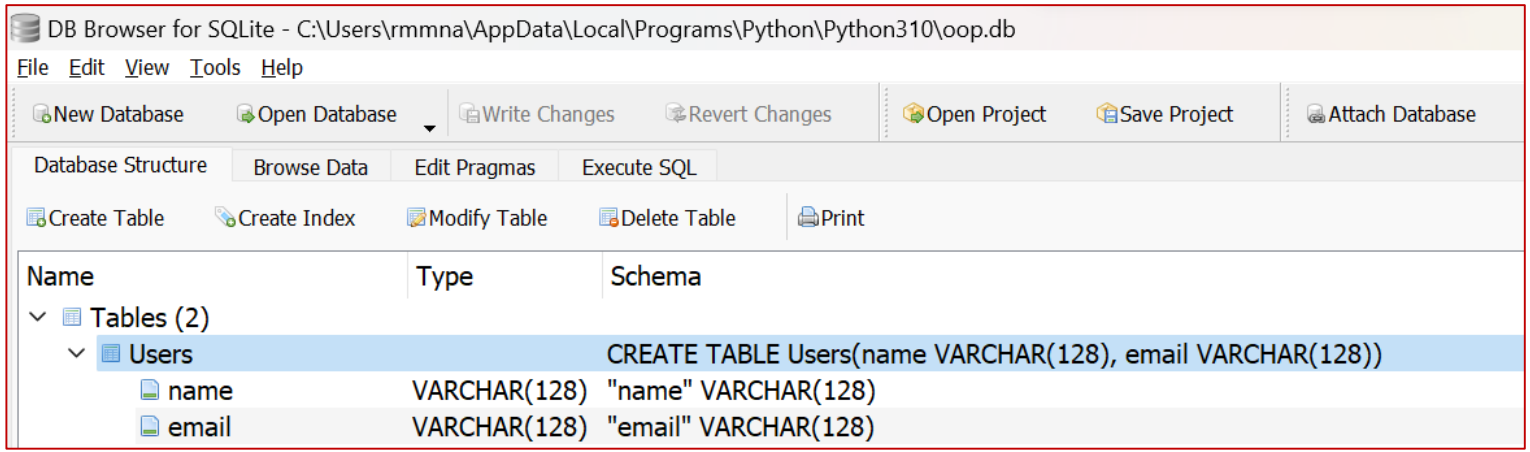
Single Table CRUD – Python Demo

CREATE: sqlite_create.py

sqlite_create.py - C:\Users\rmmna\AppData\Local\Programs\Python\Python310\sqlite_create.py (3.10.0)

File Edit Format Run Options Window Help

```
import sqlite3
conn = sqlite3.connect('oop.db')
print('Opened database successfully');
conn.execute('''CREATE TABLE Users(name VARCHAR(128), email VARCHAR(128))
''')
print('Table created successfully')
conn.close()
```



Single Table CRUD – Python Demo

INSERT: sqlite_insert.py

sqlite_insert.py - C:\Users\rmmna\AppData\Local\Programs\Python\Python310\sqlite_insert.py (3.10.0)

File Edit Format Run Options Window Help

```
import sqlite3
conn = sqlite3.connect('oop.db')
print('Opened database successfully');
conn.execute("INSERT INTO Users (name, email) VALUES ('R Madana Mohana', 'madanamohana_aids@cbit.ac.in')")
conn.execute("INSERT INTO Users (name, email) VALUES ('Prof R Madana Mohana', 'madanamohana_it@cbit.ac.in')")
conn.execute("INSERT INTO Users (name, email) VALUES ('Dr R Madana Mohana', 'associatedirector_iqac@cbit.ac.in')")
conn.commit()
print('Records Inserted successfully')
conn.close()
```

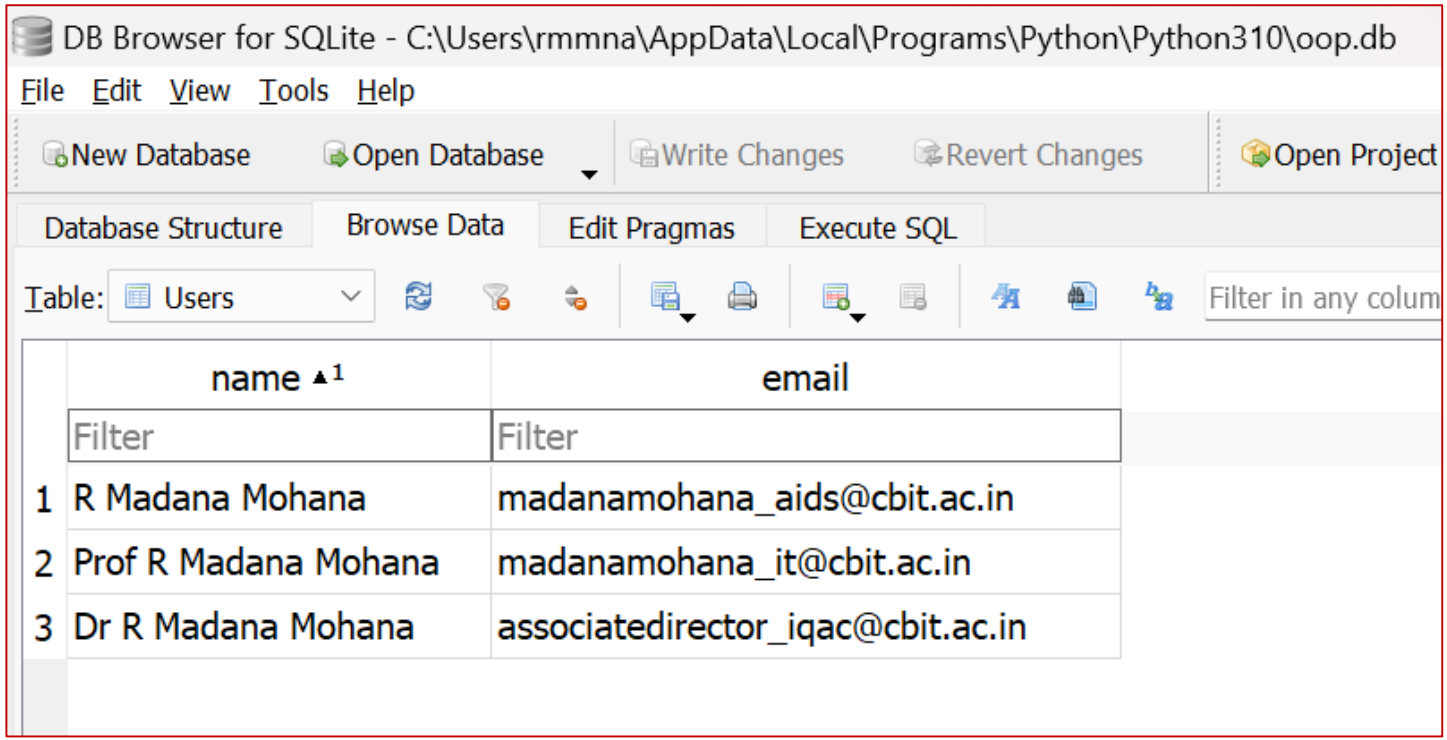
IDLE Shell 3.10.0

File Edit Shell Debug Options Window Help

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win3
2
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rmmna/AppData/Local/Programs/Python/Python310/sqlite_insert.py
Opened database successfully
Records Inserted successfully
>>>
```

Single Table CRUD – Python Demo

INSERT: `sqlite_insert.py`



Single Table CRUD – Python Demo

SELECT (Retrieve): `sqlite_select.py`

sqlite_select.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/sqlite_select.py (3.10.0)

File Edit Format Run Options Window Help

```
import sqlite3
conn = sqlite3.connect('oop.db')
print('Opened database successfully');
cursor=conn.execute("SELECT * from Users")
for row in cursor:
    print("Name = ", row[0])
    print("E-mail = ", row[1], "\n")
print('Records Retrieved successfully')
conn.close()
```

Single Table CRUD – Python Demo

SELECT (Retrieve): sqlite_select.py

```
>>>
= RESTART: C:/Users/rmmna/AppData/Local/Programs/Python/Python310/sqlite_select.py
Opened database successfully
Name =  R Madana Mohana
E-mail =  madanamohana_aids@cbit.ac.in

Name =  Prof R Madana Mohana
E-mail =  madanamohana_it@cbit.ac.in

Name =  Dr R Madana Mohana
E-mail =  associatedirector_iqac@cbit.ac.in

Records Retrieved successfully
>>>
```

Single Table CRUD – Python Demo

UPDATE: sqlite_update.py

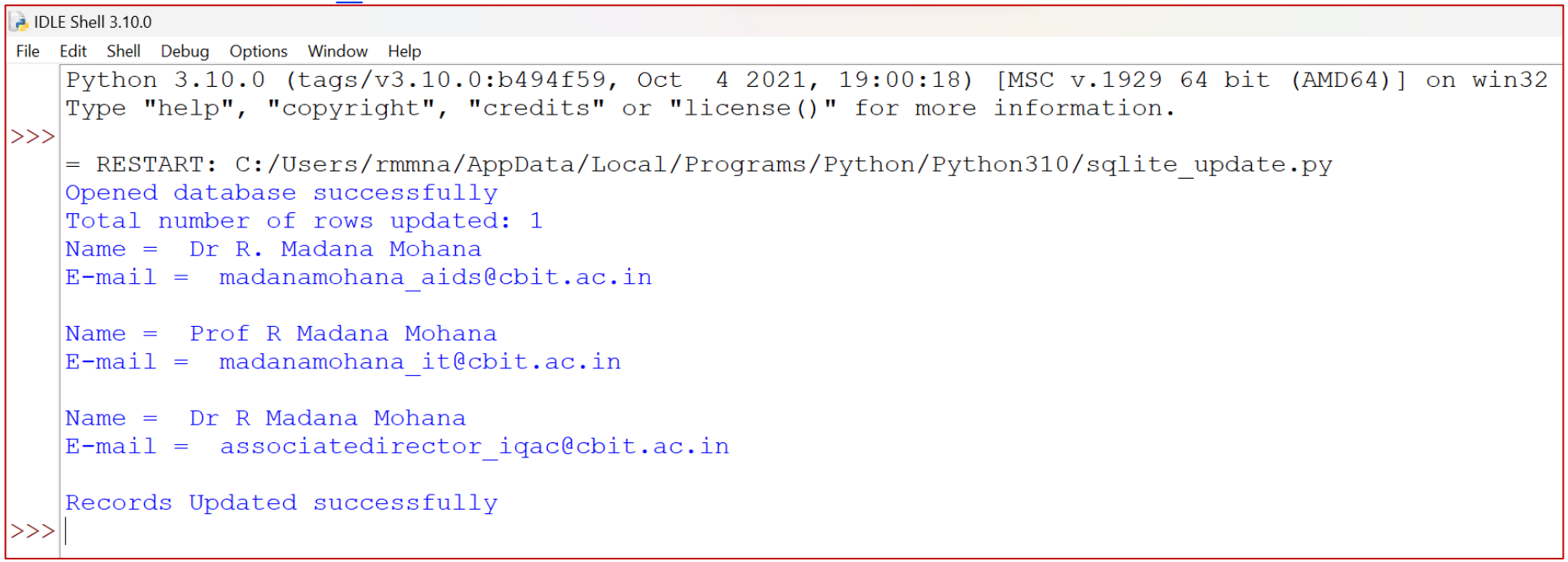
sqlite_update.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/sqlite_update.py (3.10.0)

File Edit Format Run Options Window Help

```
import sqlite3
conn = sqlite3.connect('oop.db')
print('Opened database successfully');
cursor=conn.execute("UPDATE Users SET name='Dr R. Madana Mohana' WHERE email='madanamohana_aids@cbit.ac.in'")
conn.commit()
print("Total number of rows updated:", conn.total_changes)
cursor=conn.execute("SELECT * from Users")
for row in cursor:
    print("Name = ", row[0])
    print("E-mail = ", row[1], "\n")
print('Records Updated successfully')
conn.close()
```

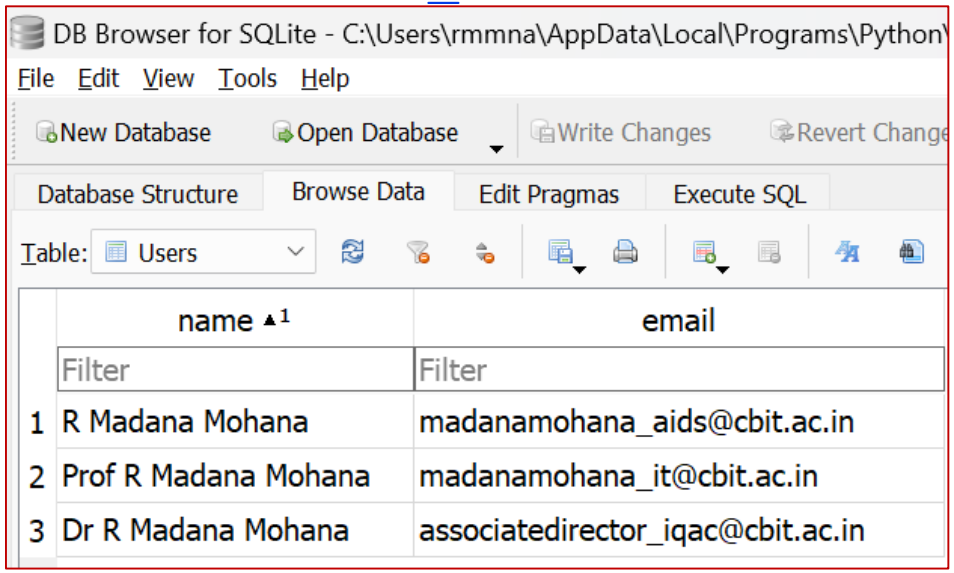
Single Table CRUD – Python Demo

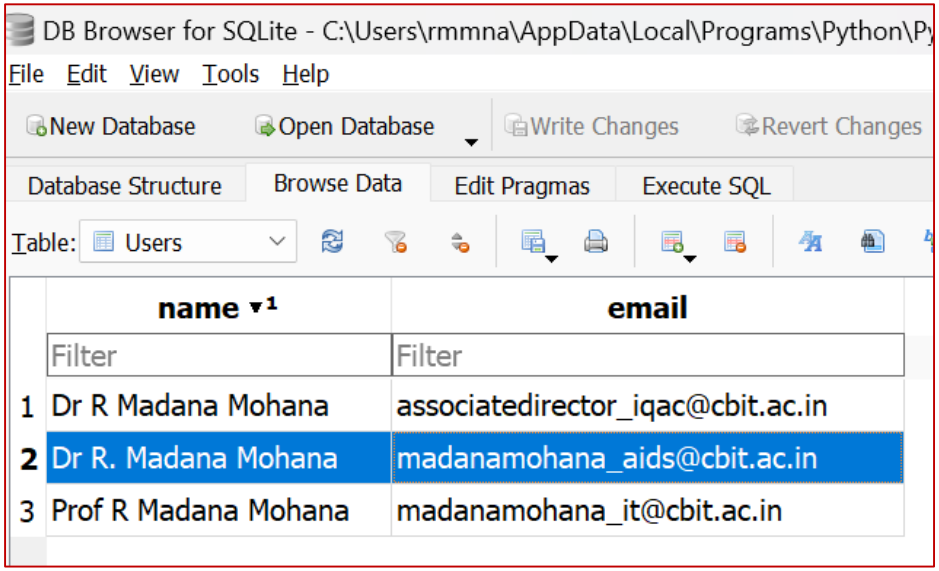
UPDATE: sqlite_update.py



Single Table CRUD – Python Demo

UPDATE: `sqlite_update.py`





Single Table CRUD – Python Demo

DELETE: sqlite_delete.py

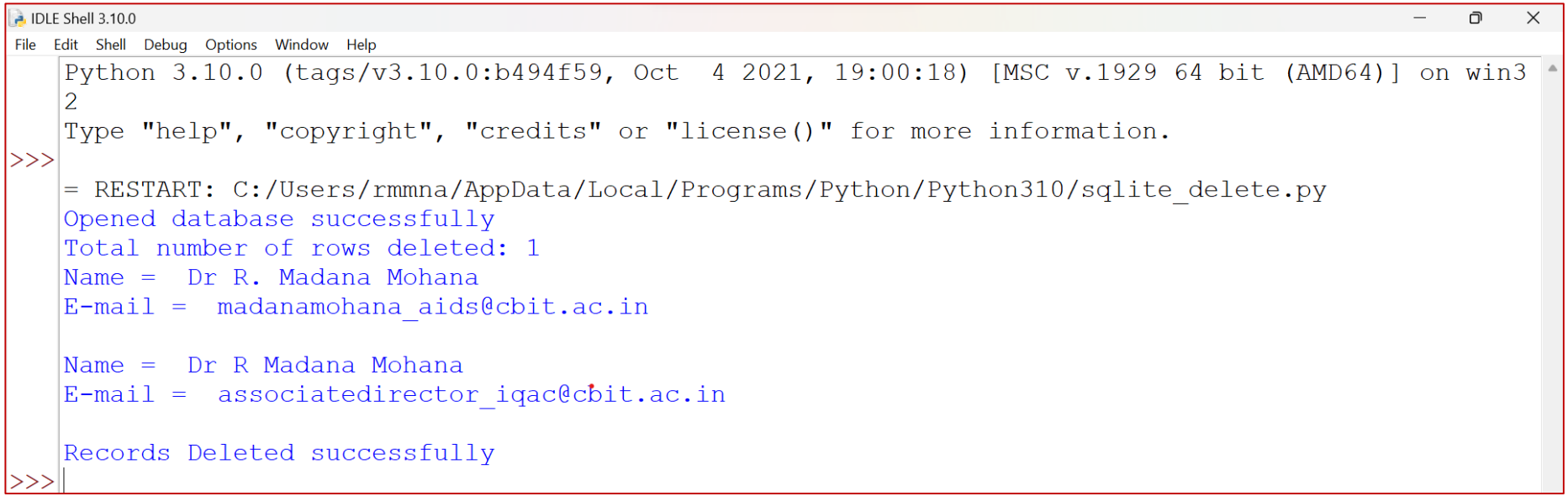
sqlite_delete.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/sqlite_delete.py (3.10.0)

File Edit Format Run Options Window Help

```
import sqlite3
conn = sqlite3.connect('oop.db')
print('Opened database successfully');
cursor=conn.execute("DELETE FROM Users WHERE email='madanamohana_it@cbit.ac.in'")
conn.commit()
print("Total number of rows deleted:", conn.total_changes)
cursor=conn.execute("SELECT * from Users")
for row in cursor:
    print("Name = ", row[0])
    print("E-mail = ", row[1], "\n")
print('Records Deleted successfully')
conn.close()
```

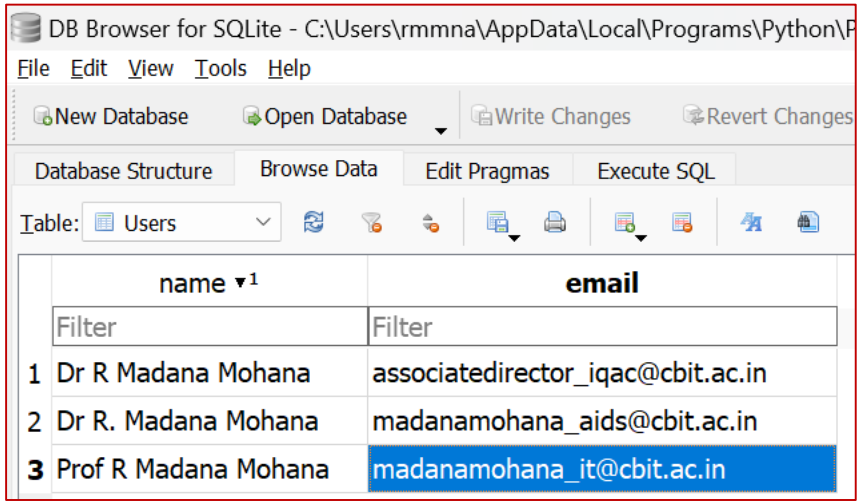
Single Table CRUD – Python Demo

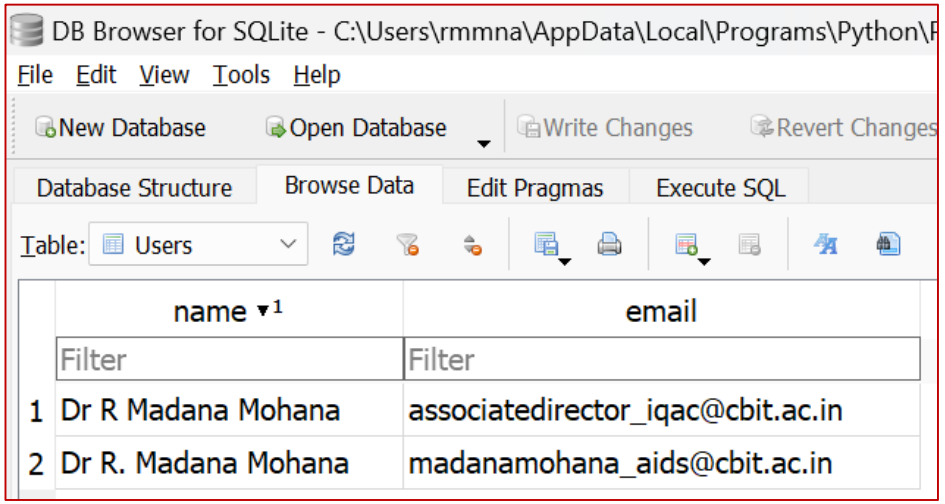
DELETE: sqlite_delete.py



Single Table CRUD – Python Demo

DELETE: `sqlite_delete.py`





Single Table CRUD
(Create, Read, Update, and Delete)
Python Demo-2

Single Table CRUD – Python Demo

File Name: emaildb.py (1/3)

emaildb.py - C:\Users\rmmna\Downloads\OOPS-PYTHON II SEM R22\OOP Python Unit-5\emaildb.py (3.10.0)

File Edit Format Run Options Window Help

```
import sqlite3

conn = sqlite3.connect('emaildb.sqlite')
cur = conn.cursor()

cur.execute('DROP TABLE IF EXISTS Counts')

cur.execute('''
CREATE TABLE Counts (email TEXT, count INTEGER)''')
```

Single Table CRUD – Python Demo

File Name: emaildb.py (2/3)

emaildb.py - C:\Users\rmmna\Downloads\OOPS-PYTHON II SEM R22\OOP Python Unit-5\emaildb.py (3.10.0)

File Edit Format Run Options Window Help

```
fname = input('Enter file name: ')
if (len(fname) < 1): fname = 'mbox-short.txt'
fh = open(fname)
for line in fh:
    if not line.startswith('From: '): continue
    pieces = line.split()
    email = pieces[1]
    cur.execute('SELECT count FROM Counts WHERE email = ? ', (email,))
    row = cur.fetchone()
    if row is None:
        cur.execute('''INSERT INTO Counts (email, count)
                VALUES (?, 1)''', (email,))
    else:
        cur.execute('UPDATE Counts SET count = count + 1 WHERE email = ?'
                    (email,))
    conn.commit()
```

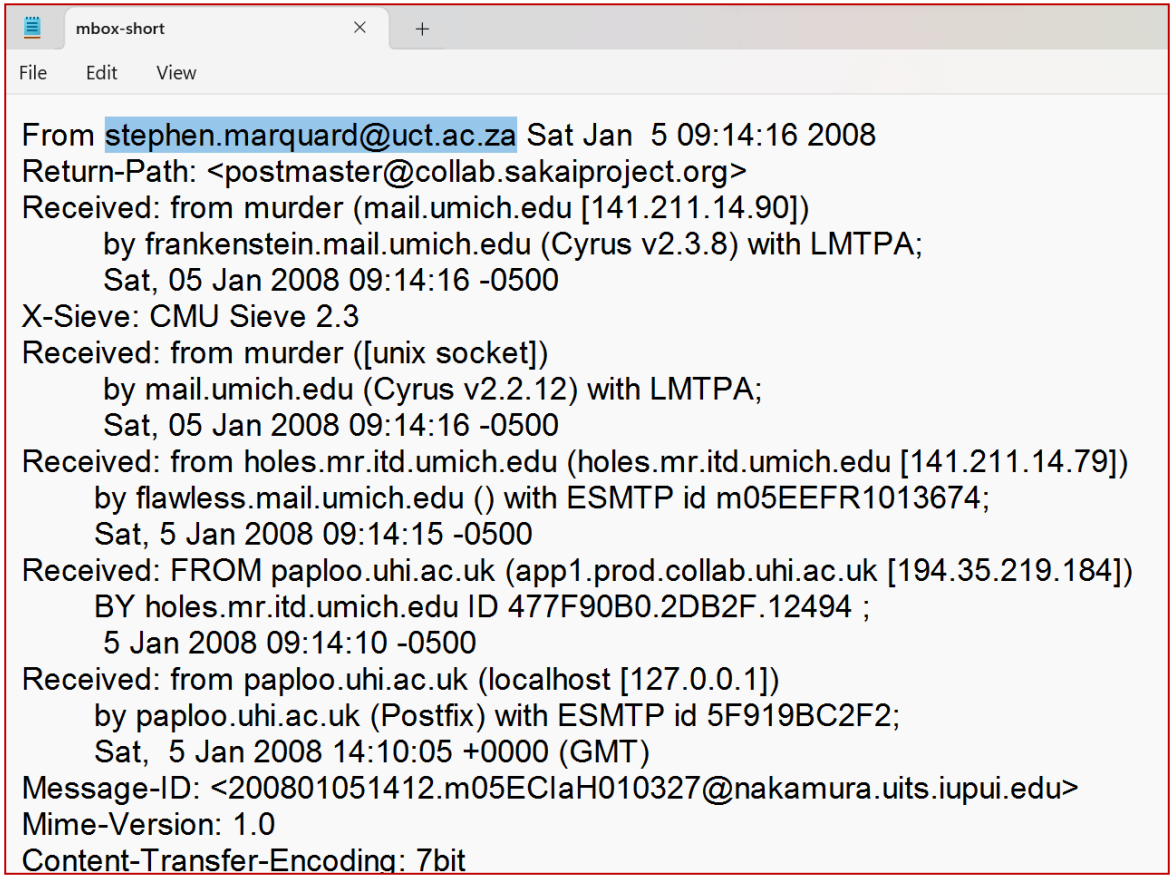
Single Table CRUD – Python Demo

File Name: emaildb.py (3/3)

```
# https://www.sqlite.org/lang_select.html
sqlstr = 'SELECT email, count FROM Counts ORDER BY count DESC LIMIT 10'

for row in cur.execute(sqlstr):
    print(str(row[0]), row[1])

cur.close()
```

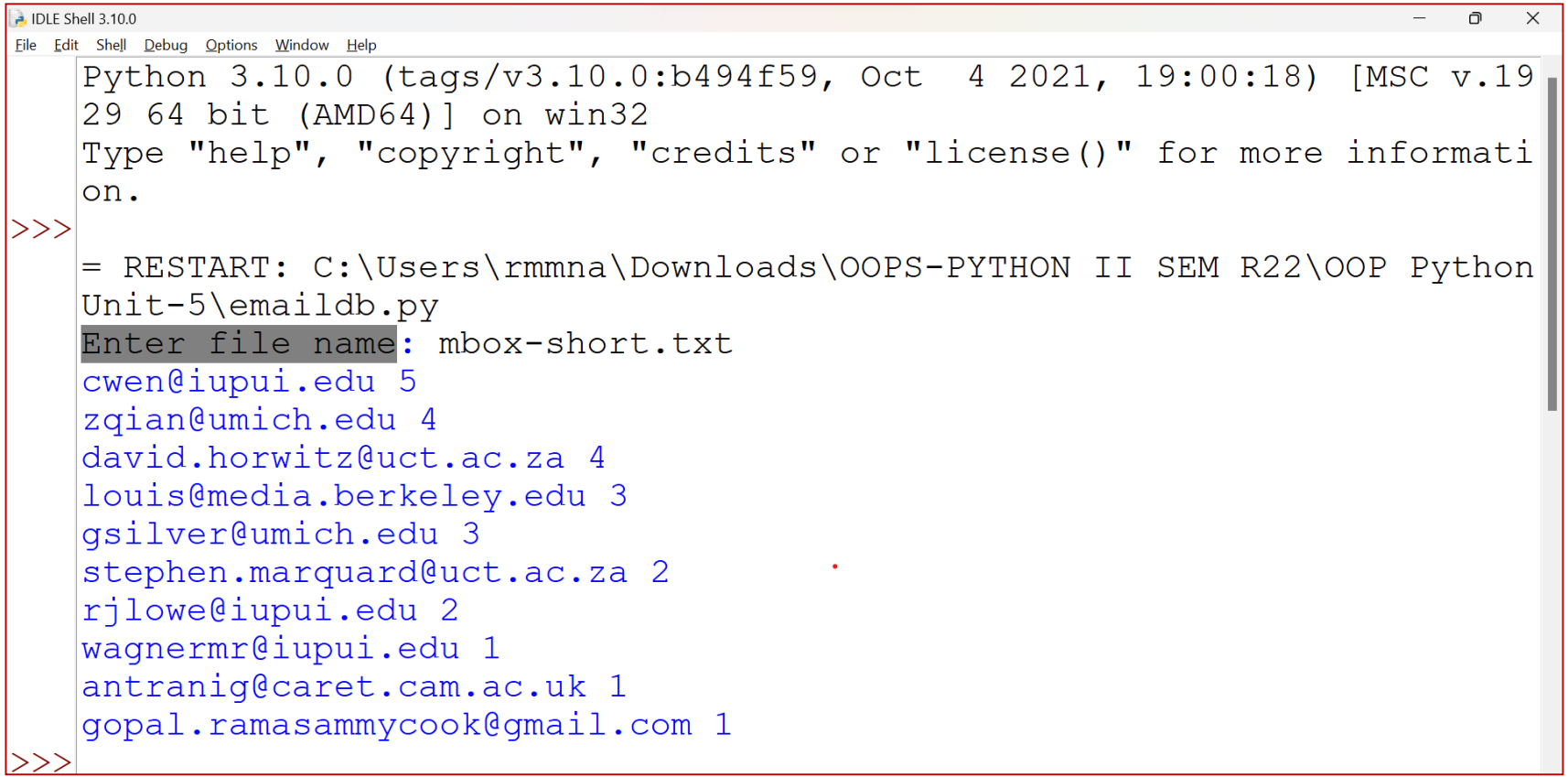
Single Table CRUD – Python Demo

Input File Name: `mbox-short.txt`



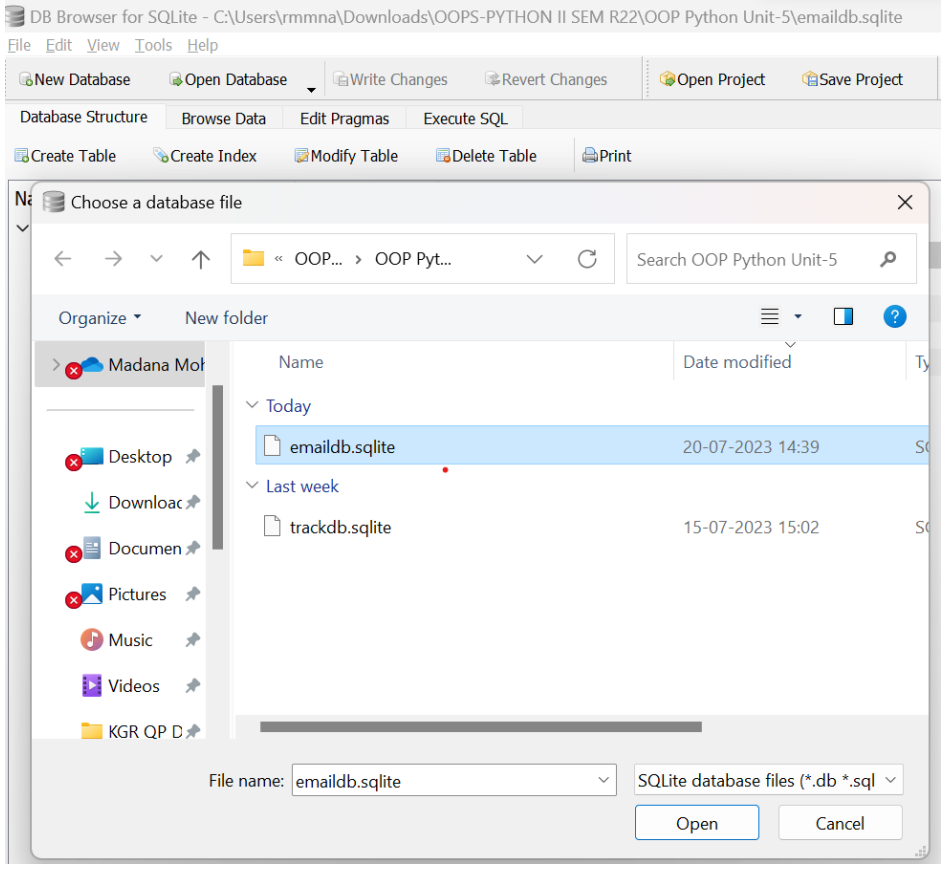
Single Table CRUD – Python Demo

Output:



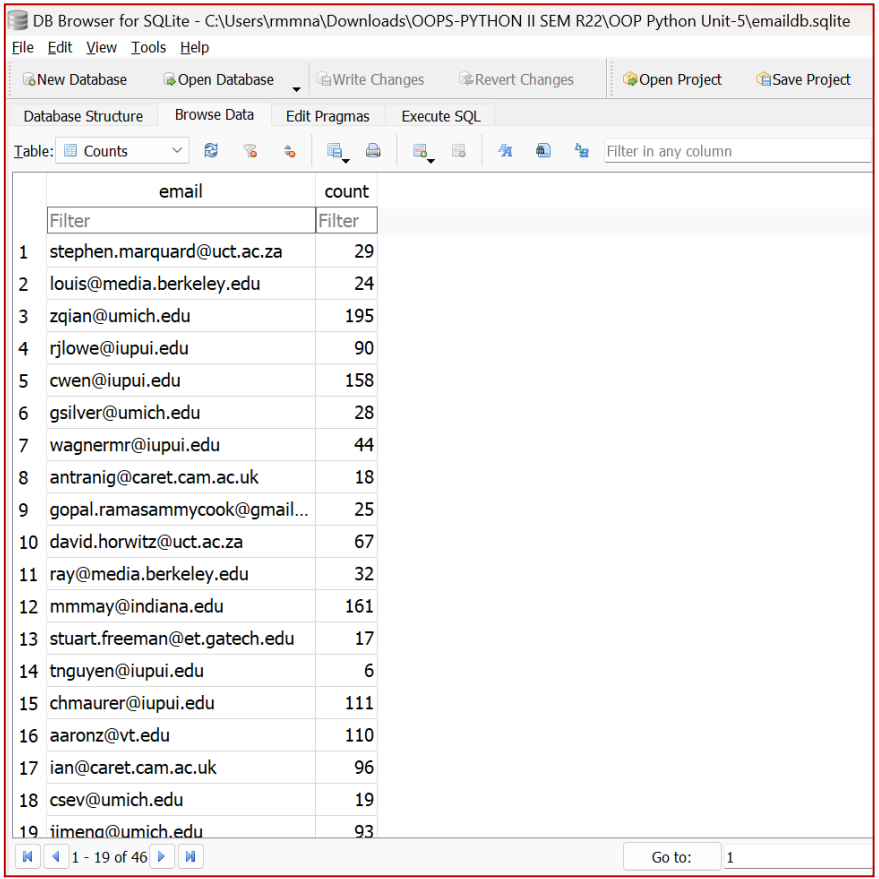
Single Table CRUD – Python Demo

Check the created database emaildb.sqlite in DB Browser for SQLIte:



Single Table CRUD – Python Demo

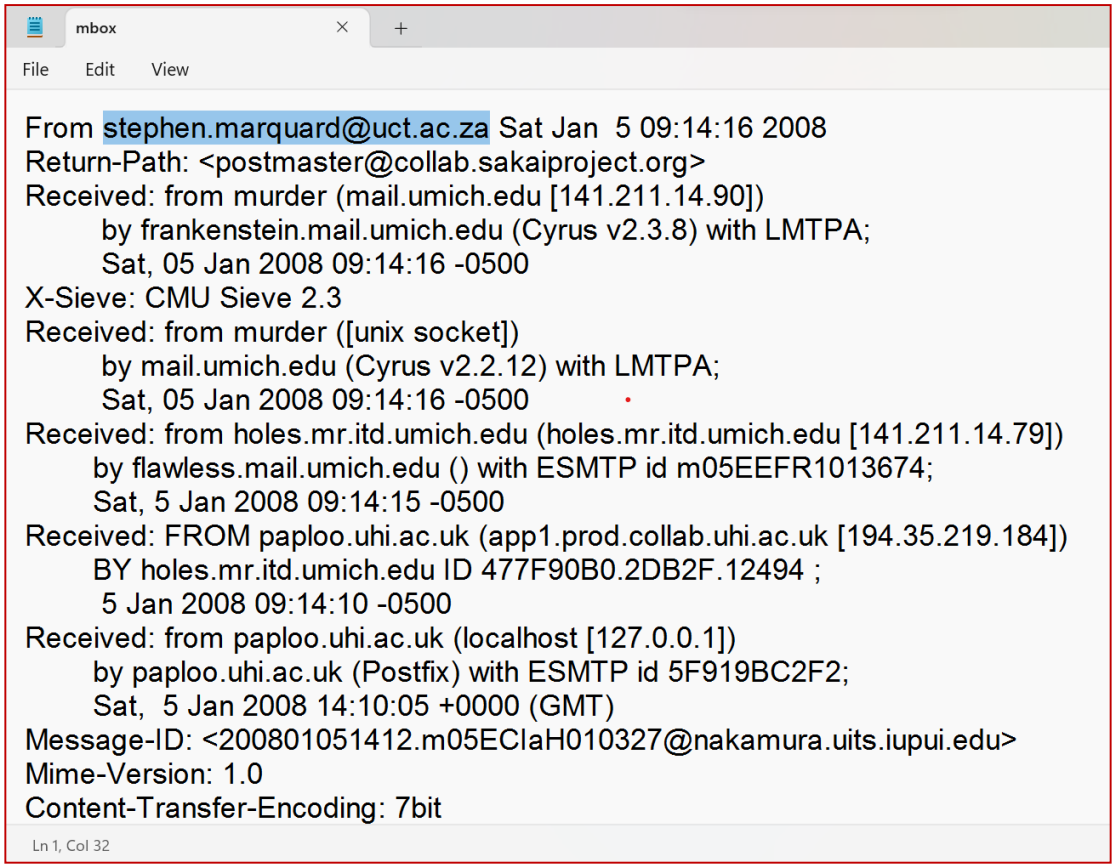
Check the created database emaildb.sqlite in DB Browser for SQLIte:



	email	count
	Filter	Filter
1	stephen.marquard@uct.ac.za	29
2	louis@media.berkeley.edu	24
3	zqian@umich.edu	195
4	rjlowe@iupui.edu	90
5	cwen@iupui.edu	158
6	gsilver@umich.edu	28
7	wagnermr@iupui.edu	44
8	antranig@caret.cam.ac.uk	18
9	gopal.ramasammycook@gmail...	25
10	david.horwitz@uct.ac.za	67
11	ray@media.berkeley.edu	32
12	mmmay@indiana.edu	161
13	stuart.freeman@et.gatech.edu	17
14	tnguyen@iupui.edu	6
15	chmaurer@iupui.edu	111
16	aaronz@vt.edu	110
17	ian@caret.cam.ac.uk	96
18	csev@umich.edu	19
19	jimeng@umich.edu	93

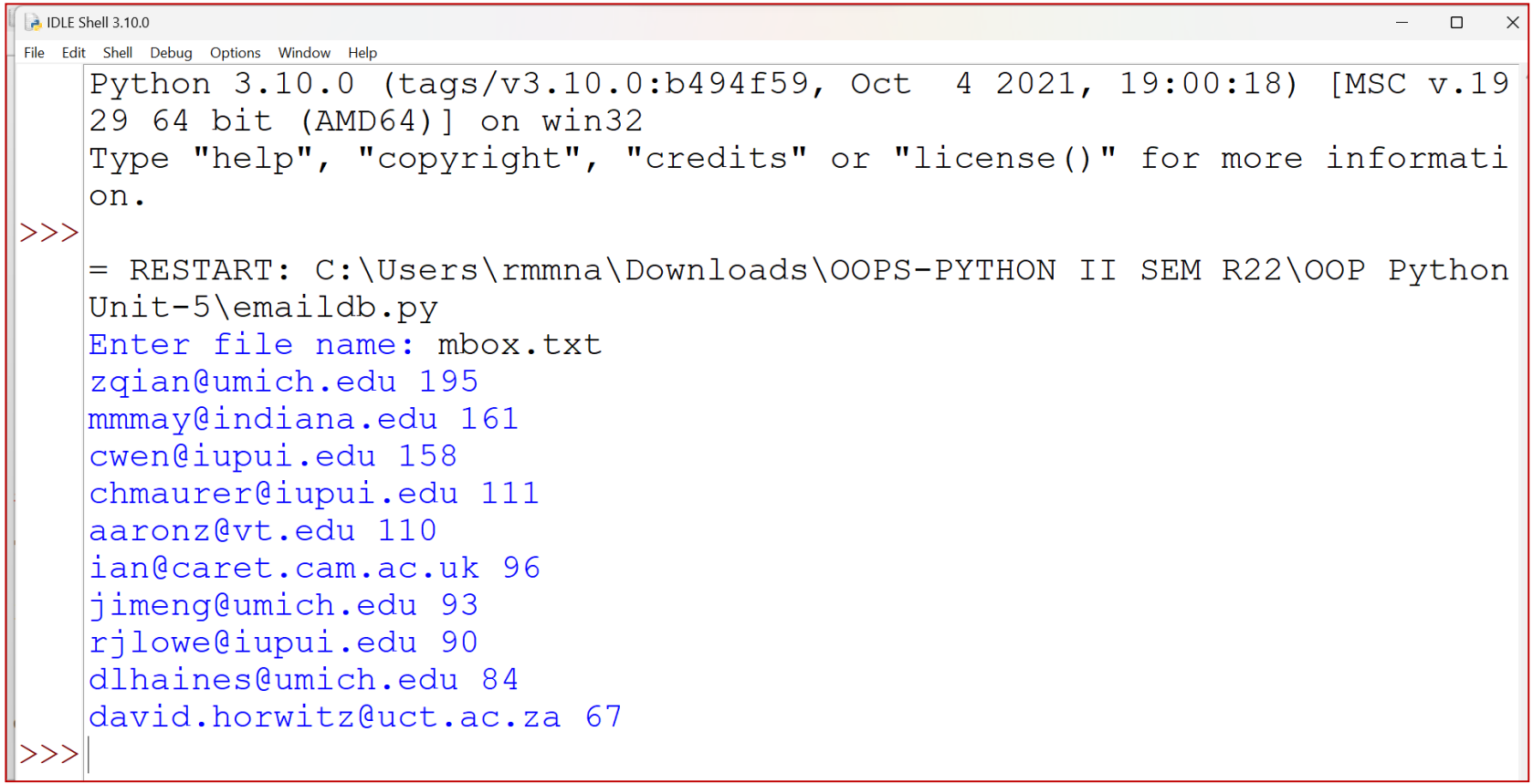
Single Table CRUD – Python Demo

Input File Name: `mbox.txt`



Single Table CRUD – Python Demo

Output:



Object Oriented Programming (Using Python)

UNIT- V

Using Databases with Python:

- Using Databases
- Single Table CRUD (Create, Read, Update, and Delete)
- Designing and representing a data model
- Reconstructing data with JOIN
- Many-to-many relationships

References

https://www.coursera.org/learn/python-databases/home/info

https://www.py4e.com/book

http://do1.dr-chuck.com/pythonlearn/EN_us/pythonlearn.pdf

https://www.py4e.com/code3/

https://www.py4e.com/lectures3/

https://www.youtube.com/playlist?list=PLlRFEj9H3Oj7Bp8-DfGpfAfDBiblRfl5p

http://sqlitebrowser.org/

Designing a Data Model

Database Model

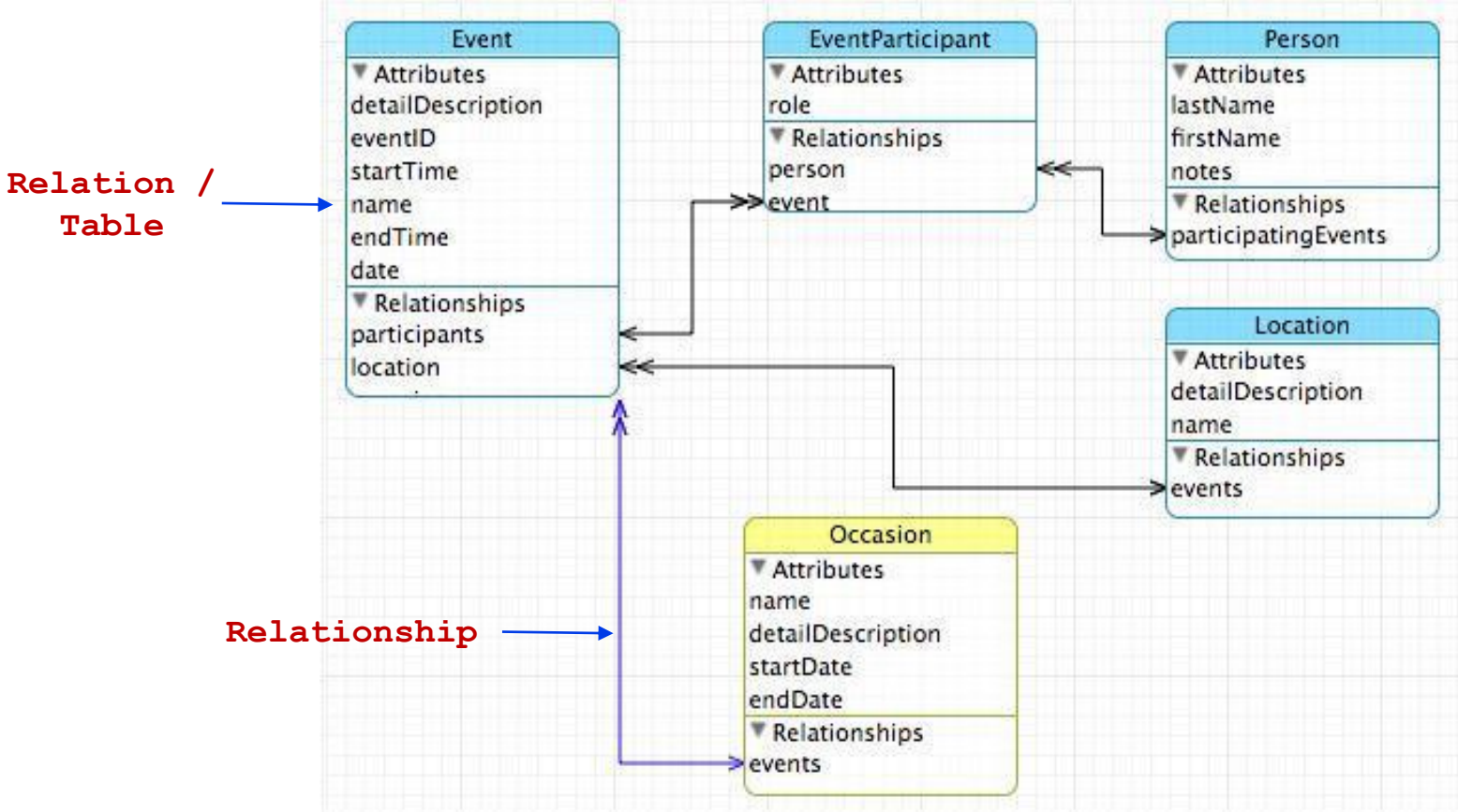
- A database model or database schema is the structure or format of a database, described in a formal language supported by the database management system.
- In other words, a "database model" is the application of a data model when used in conjunction with a database management system.

http://en.wikipedia.org/wiki/Database_model

Database Design

- Database design is an art form of its own with particular skills and experience
- Our goal is to avoid the really bad mistakes and design clean and easily understood databases
- Others may performance tune things later
- Database design starts with a picture...

Database Design



Building a Data Model

- Drawing a picture of the data objects for our application and then figuring out how to represent the objects and their relationships
- Basic Rule: Don’t put the same string data in twice - use a relationship instead
- When there is one thing in the “real world” there should be one copy of that thing in the database

Building a Data Model - Example

Track	Len	Artist	Album	Genre	Rating	Count
Hells Bells	5:13	AC/DC	Who Made Who	Rock	★★★★★	61
Shake Your Foundations	3:54	AC/DC	Who Made Who	Rock	★★★★★	70
Chase the Ace	3:01	AC/DC	Who Made Who	Rock		56
For Those About To Rock (We ...	5:54	AC/DC	Who Made Who	Rock	★★★★★	61
Dúlamán	3:43	Altan	Natural Wonders M...	New Age		31
Rode Across the Desert	4:10	America	Greatest Hits	Easy Listen...	★★★★★	23
Now You Are Gone	3:08	America	Greatest Hits	Easy Listen...	★★★★★	18
Tin Man	3:30	America	Greatest Hits	Easy Listen...	★★★★★	23
Sister Golden Hair	3:22	America	Greatest Hits	Easy Listen...	★★★★★	24
Track 01	4:22	Billy Price	Danger Zone	Blues/R&B	★★★★★	26
Track 02	2:45	Billy Price	Danger Zone	Blues/R&B	★★★★★	18
Track 03	3:26	Billy Price	Danger Zone	Blues/R&B	★★★★★	22
Track 04	4:17	Billy Price	Danger Zone	Blues/R&B	★★★★★	18
Track 05	3:50	Billy Price	Danger Zone	Blues/R&B	★★★★★	21
War Pigs/Luke's Wall	7:58	Black Sabbath	Paranoid	Metal	★★★★★	25
Paranoid	2:53	Black Sabbath	Paranoid	Metal	★★★★★	22
Planet Caravan	4:35	Black Sabbath	Paranoid	Metal	★★★★★	25
Iron Man	5:59	Black Sabbath	Paranoid	Metal	★★★★★	26
Electric Funeral	4:53	Black Sabbath	Paranoid	Metal	★★★★★	22
Hand of Doom	7:10	Black Sabbath	Paranoid	Metal	★★★★★	23
Rat Salad	2:30	Black Sabbath	Paranoid	Metal	★★★★★	31
Jack the Stripper/Fairies Wear ...	6:14	Black Sabbath	Paranoid	Metal	★★★★★	24
Bomb Squad (TECH)	3:28	Brent	Brent's Album			1
clay techno	4:36	Brent	Brent's Album			2
Heavy	3:08	Brent	Brent's Album			1
Hi metal man	4:20	Brent	Brent's Album			1
Mistro	2:58	Brent	Brent's Album			1

Building a Data Model - Example

For each "piece of info"…

- Is the column an object or an attribute of another object?
- Once we define objects, we need to define the relationships between objects

Len

Album

Genre

Artist

Rating

Track

Count

Hells Bells	5:13	AC/DC	Who Made Who	Rock	★★★★★	61
Shake Your Foundations	3:54	AC/DC	Who Made Who	Rock	★★★★★	70
Chase the Ace	3:01	AC/DC	Who Made Who	Rock		56
For Those About To Rock (We …	5:54	AC/DC	Who Made Who	Rock	★★★★★	61
Dúlamán	3:43	Altan	Natural Wonders M…	New Age		31
Rode Across the Desert	4:10	America	Greatest Hits	Easy Listen…	★★★★★	23
Now You Are Gone	3:08	America	Greatest Hits	Easy Listen…	★★★★★	18

Building a Data Model - Example

Track

Album

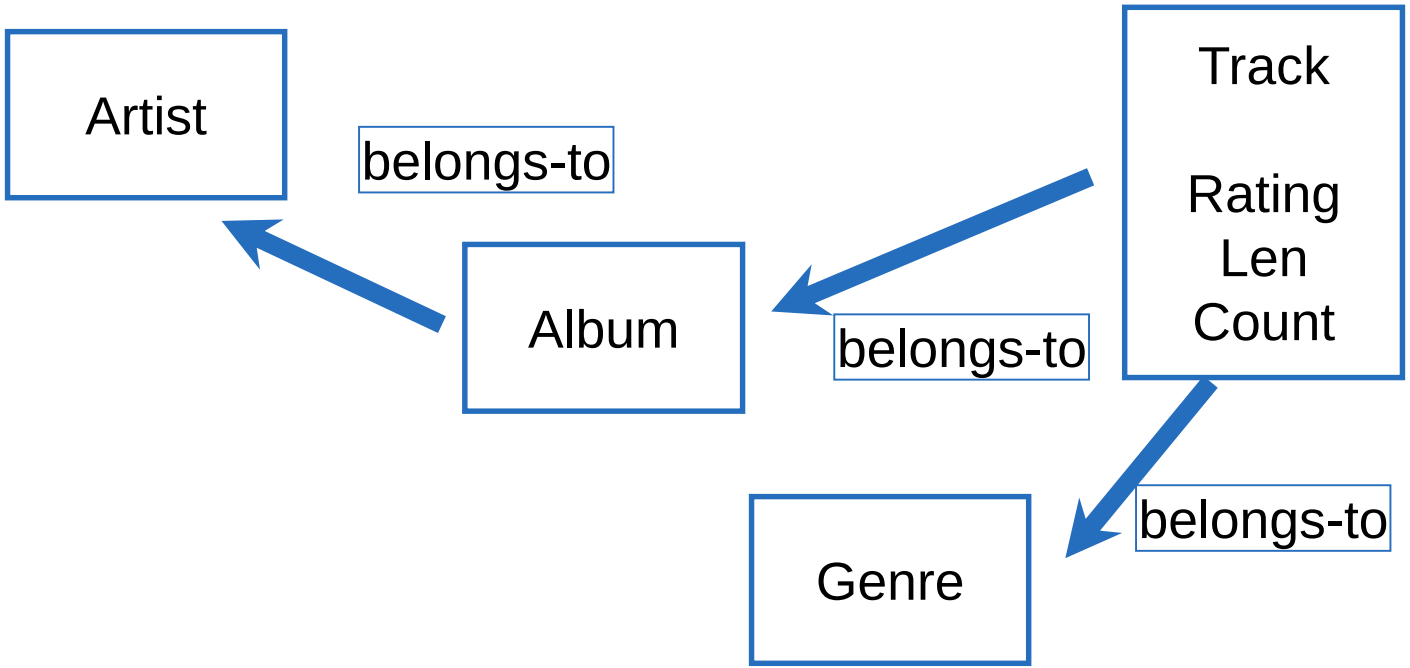
Artist

Genre

Rating

Len

Count



Hells Bells	5:13	AC/DC	Who Made Who	Rock	★★★★★	61
Shake Your Foundations	3:54	AC/DC	Who Made Who	Rock	★★★★★	70
Chase the Ace	3:01	AC/DC	Who Made Who	Rock		56
For Those About To Rock (We ...	5:54	AC/DC	Who Made Who	Rock	★★★★★	61
Dúlamán	3:43	Altan	Natural Wonders M...	New Age		31
Rode Across the Desert	4:10	America	Greatest Hits	Easy Listen...	★★★★★	23
Now You Are Gone	3:08	America	Greatest Hits	Easy Listen...	★★★★★	18

Representing Relationships in Database

Database Normalization (3NF)

- There is *tons* of database theory - way too much to understand without excessive predicate calculus
- Do not replicate data - reference data - point at data
- Use integers for keys and for references
- Add a special "key" column to each table which we will make references to. By convention, many programmers call this column "id"

http://en.wikipedia.org/wiki/Database_normalization

Database Normalization (3NF)

Hells Bells	5:13	AC/DC	Who Made Who	Rock	★★★★★	61
Shake Your Foundations	3:54	AC/DC	Who Made Who	Rock	★★★★★	70
Chase the Ace	3:01	AC/DC	Who Made Who	Rock		56
For Those About To Rock (We ...	5:54	AC/DC	Who Made Who	[illegible]	★★★★★	61
Dúlamán	3:43	Altan	Natural Wonders M...	New [illegible]ge		31
Rode Across the Desert	[illegible]	America	Greatest Hits	[illegible]	★★★★★	23
Now You Are Gone	3:08	America	Greatest Hits	Easy Listen...	★★★★★	18
Tin Man	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

We want to keep track of which band is the “creator” of each music track...

What album does this song “belong to”??

Which album is this song related to?

Integer Reference Pattern

We use integers to reference rows in another table

Artist

id	name
Filter	Filter
1	Led Zepplin
2	AC/DC

Album

id	artist_id	title
Filter	Filter	Filter
1	2	Who Made Who
2	1	IV

Three Kinds of Keys

Primary key - generally an integer auto-increment field

Logical key - What the outside world uses for lookup

Foreign key - generally an integer key pointing to a row in another table

Key Rules

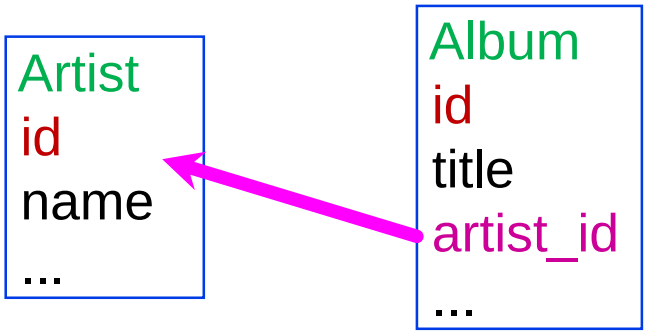
Best practices

- Never use your logical key as the primary key
- Logical keys can and do change, although slowly
- Relationships that are based on matching string fields are less efficient than integers

User
id
login
password
name
email
created_at
modified_at
login_at

Foreign Keys

- A foreign key is when a table has a column that contains a key which points to the primary key of another table.
- When all primary keys are integers, then all foreign keys are integers - this is good - very good



Representing a Data Model / Relationship Building (in Tables)

Track

Album

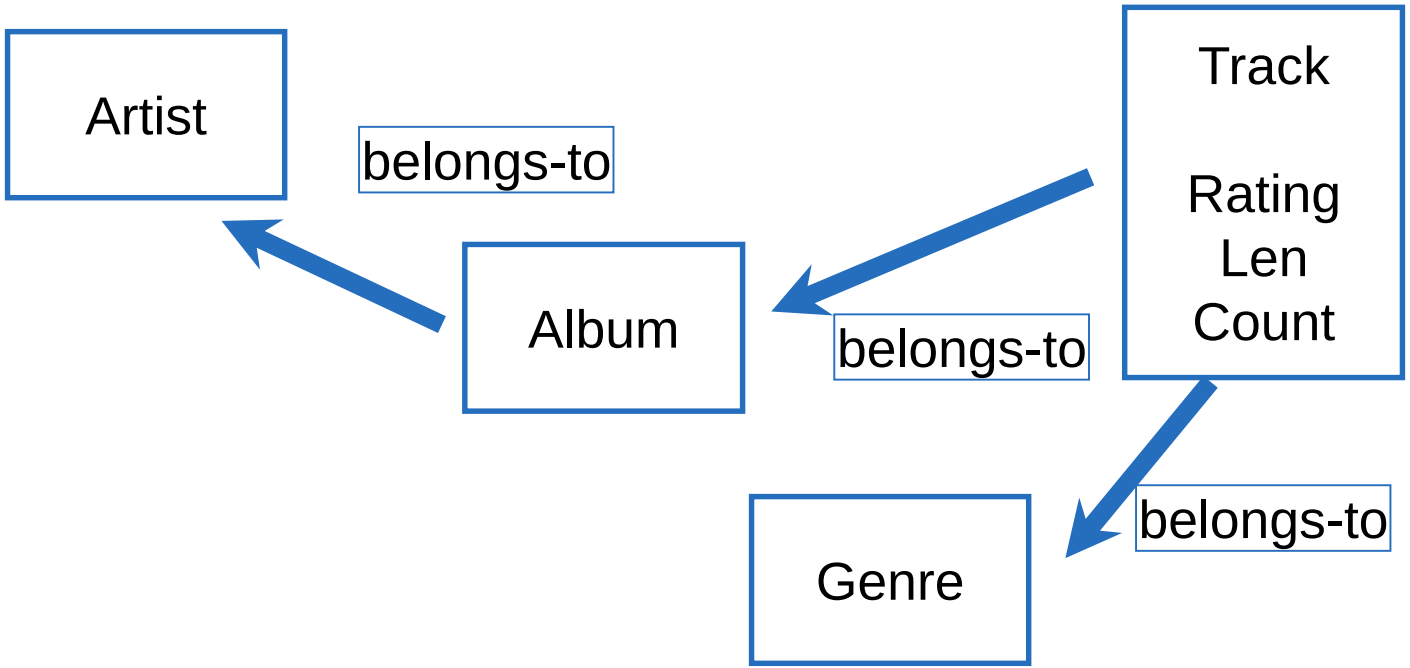
Artist

Genre

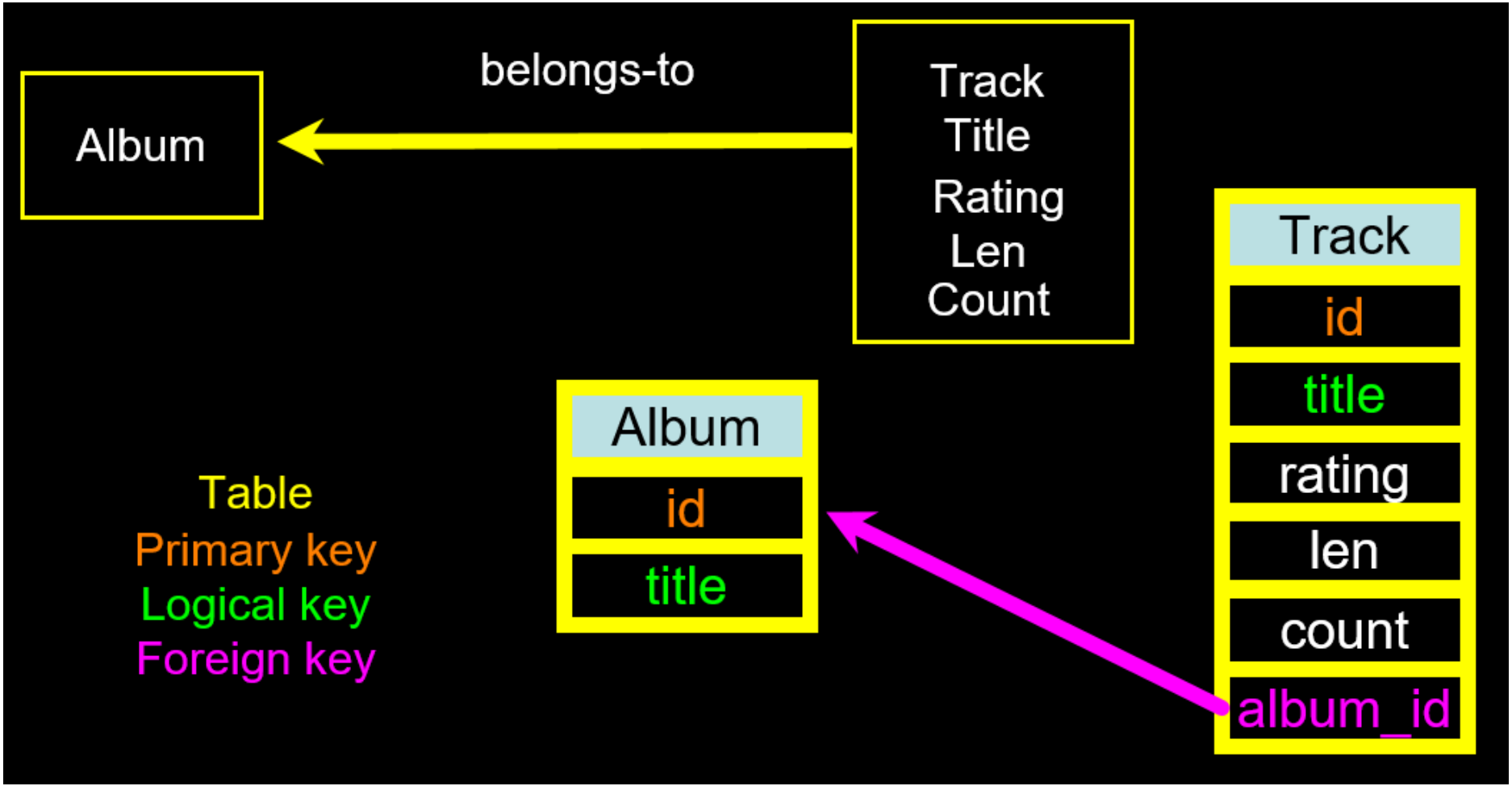
Rating

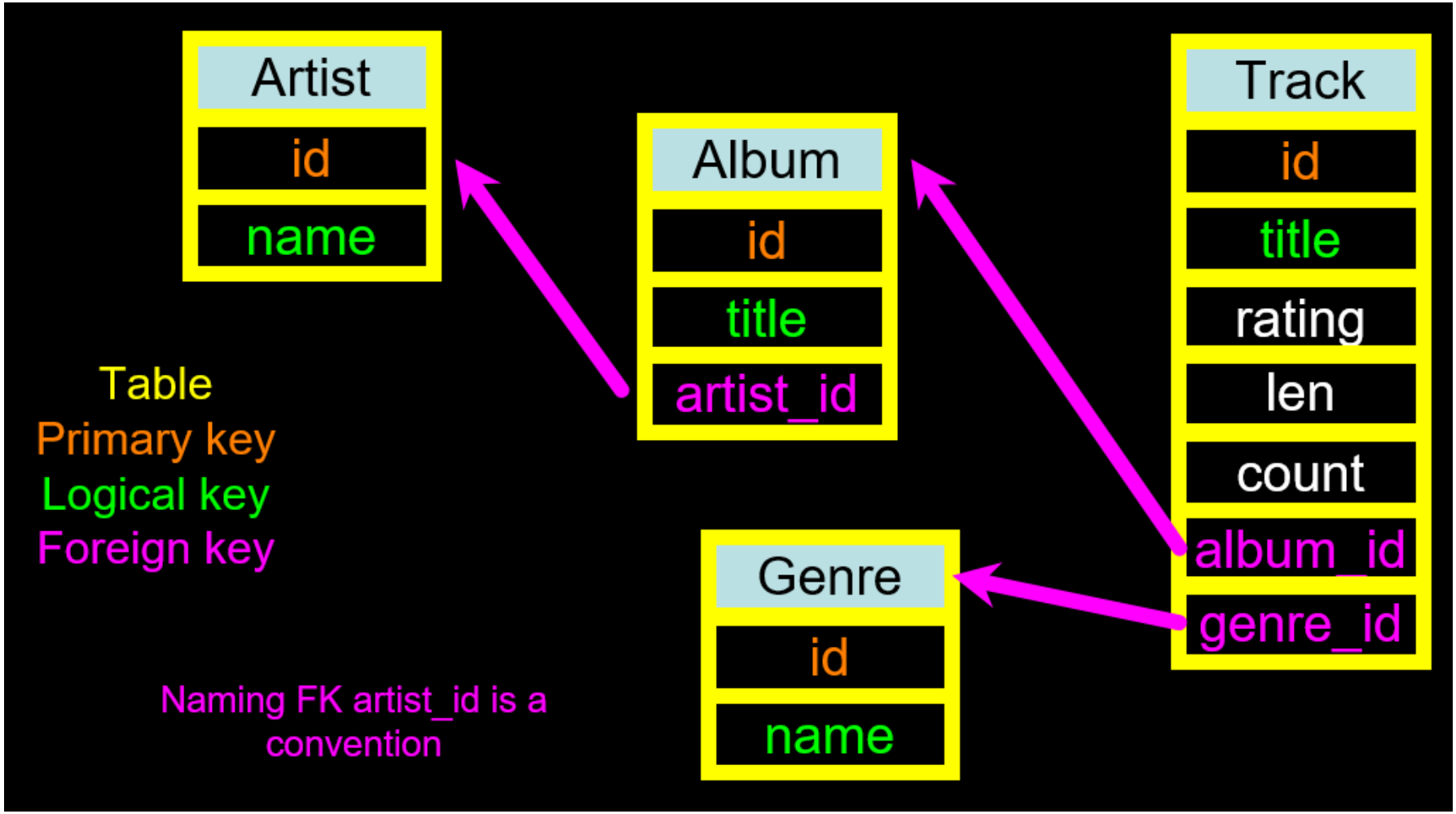
Len

Count



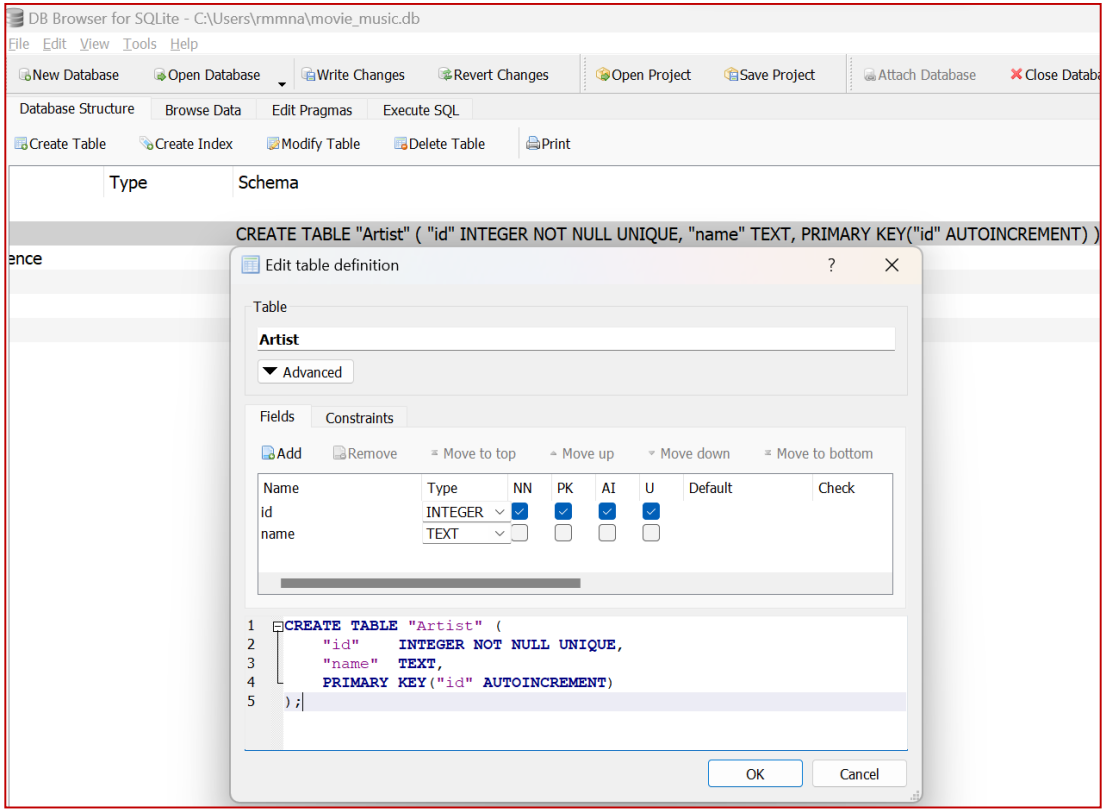
Hells Bells	5:13	AC/DC	Who Made Who	Rock	★★★★★	61
Shake Your Foundations	3:54	AC/DC	Who Made Who	Rock	★★★★★	70
Chase the Ace	3:01	AC/DC	Who Made Who	Rock		56
For Those About To Rock (We ...	5:54	AC/DC	Who Made Who	Rock	★★★★★	61
Dúlamán	3:43	Altan	Natural Wonders M...	New Age		31
Rode Across the Desert	4:10	America	Greatest Hits	Easy Listen...	★★★★★	23
Now You Are Gone	3:08	America	Greatest Hits	Easy Listen...	★★★★★	18


Album
belongs-to
Track
Title
Rating
Len
Count
Track
id
title
rating
len
count
album_id
Album
id
title
Table
Primary key
Logical key
Foreign key


Artist
id
name
Album
id
title
artist_id
Track
id
title
rating
len
count
album_id
genre_id
Genre
id
name
Table
Primary key
Logical key
Foreign key
Naming FK artist_id is a convention

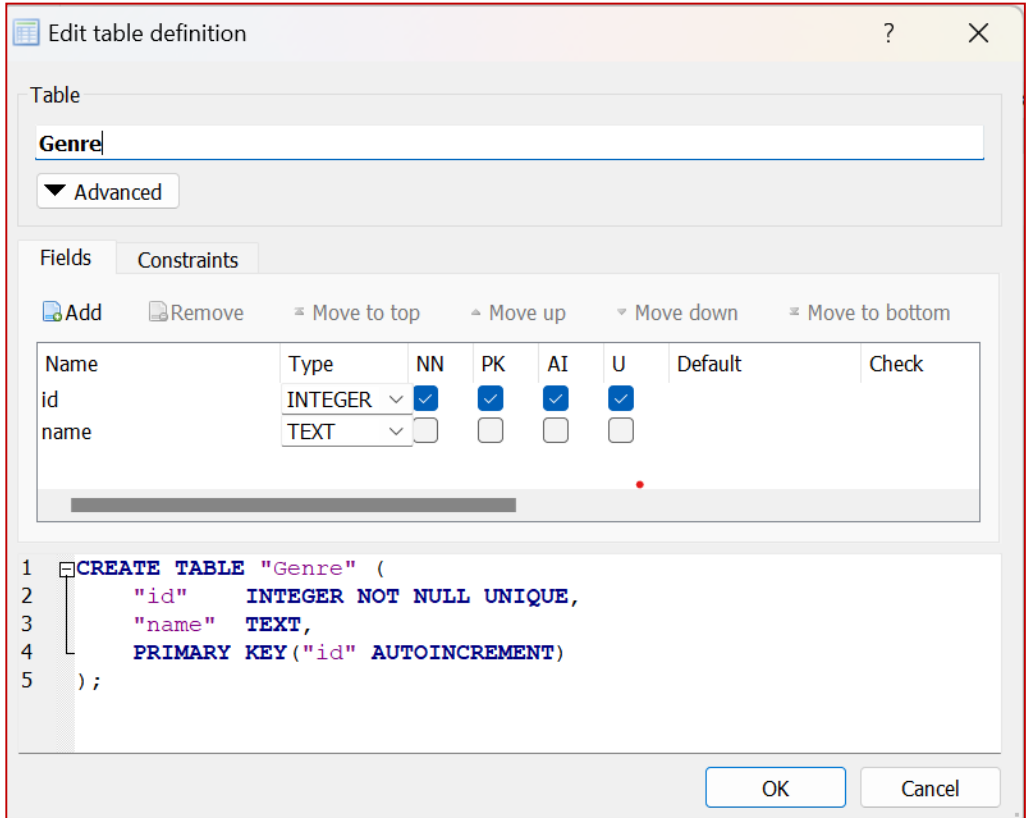
Creating Tables using SQLite

```
CREATE TABLE "Artist" ("id" INTEGER NOT NULL UNIQUE,
        "name" TEXT, PRIMARY KEY("id" AUTOINCREMENT));
```



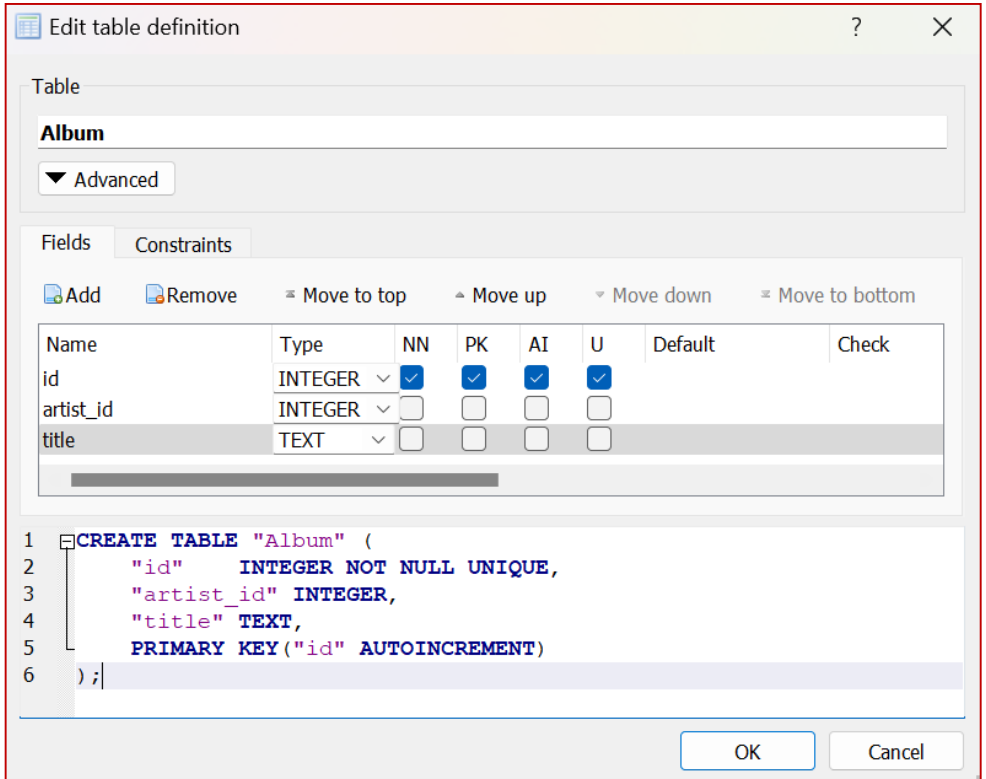
Creating Tables using SQLite

```
CREATE TABLE "Genre" ("id" INTEGER NOT NULL UNIQUE,
        "name" TEXT, PRIMARY KEY("id" AUTOINCREMENT));
```



Creating Tables using SQLite

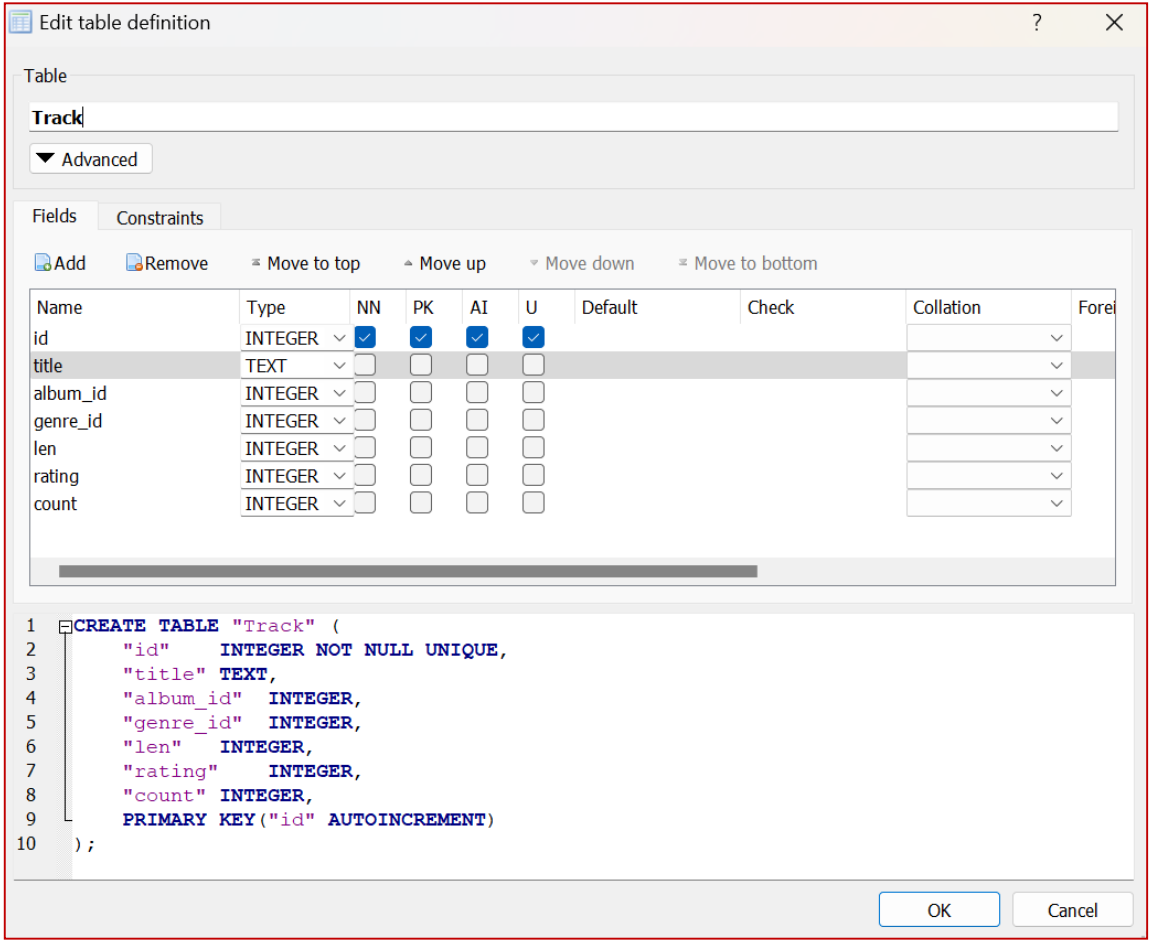
```
CREATE TABLE "Album" ("id" INTEGER NOT NULL UNIQUE,
      "artist_id" INTEGER, "title"  TEXT,PRIMARY KEY("id"
AUTOINCREMENT));
```



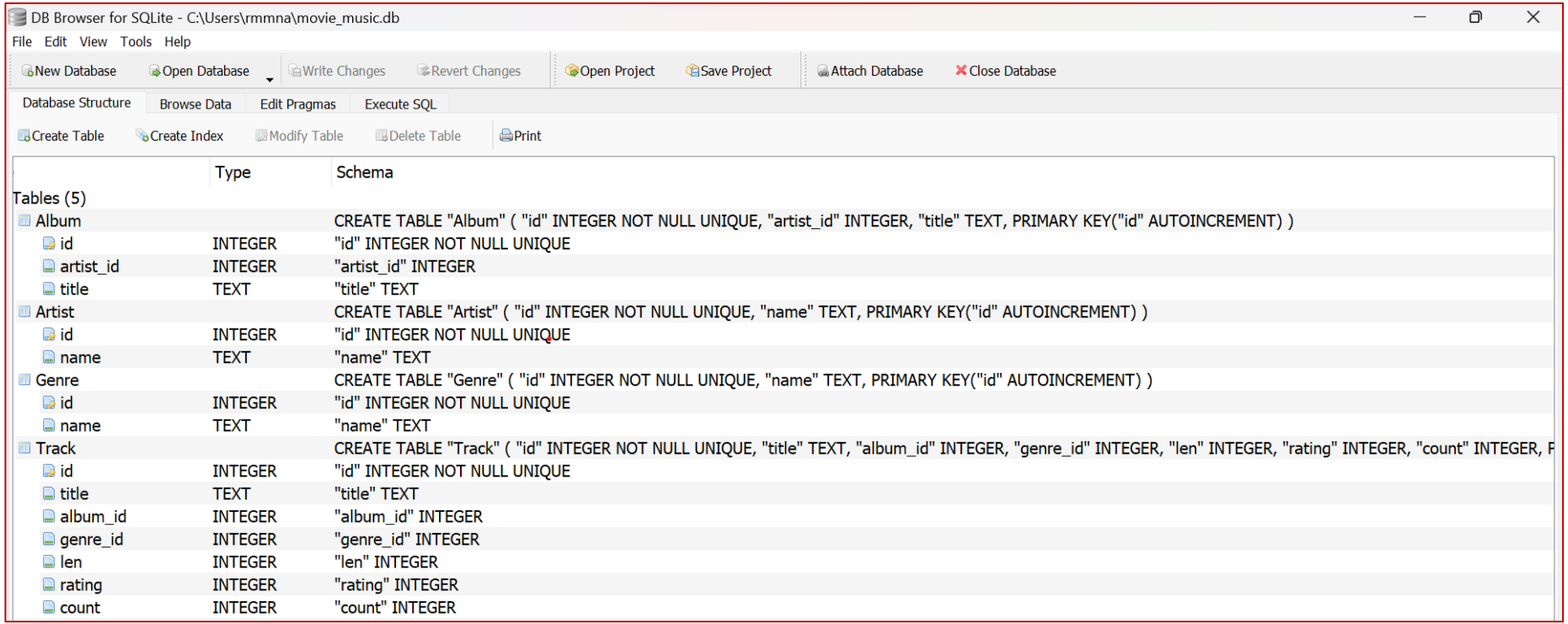
Creating Tables using SQLite

```
CREATE TABLE "Track" (
      "id"  INTEGER NOT NULL UNIQUE,
      "title"     TEXT,
      "album_id"  INTEGER,
      "genre_id"  INTEGER,
      "len" INTEGER,
      "rating"    INTEGER,
      "count"     INTEGER,
      PRIMARY KEY("id" AUTOINCREMENT)
);
```

Creating Tables using SQLite

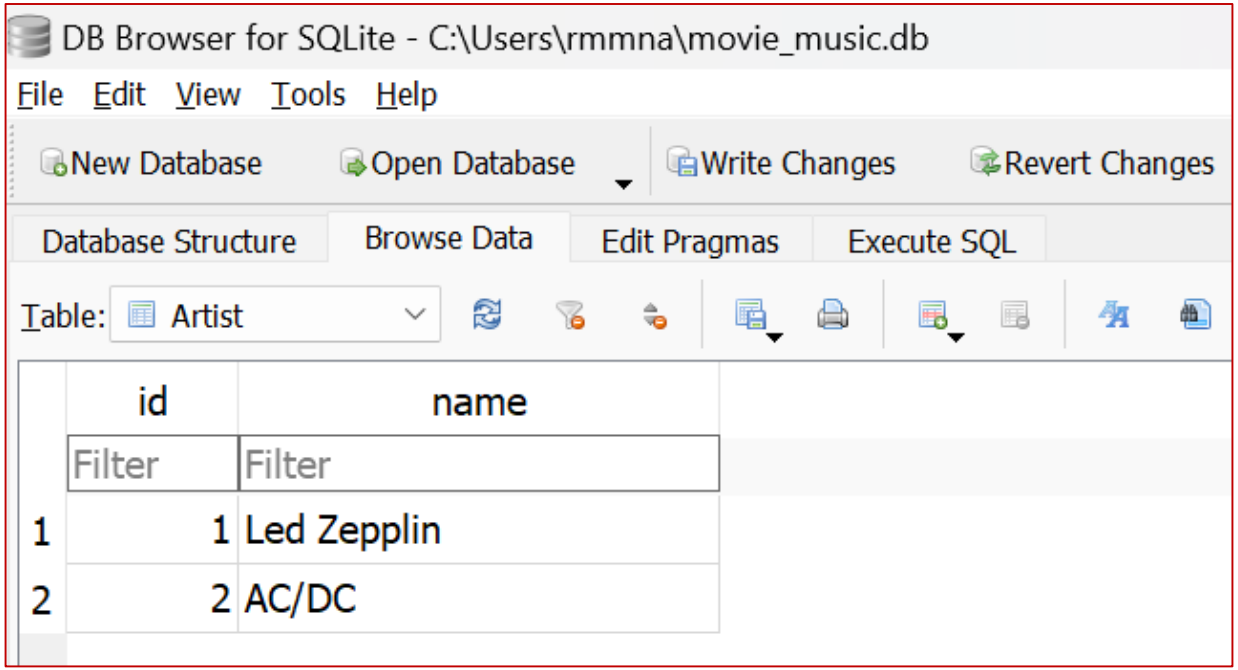


Inserting Relational Data



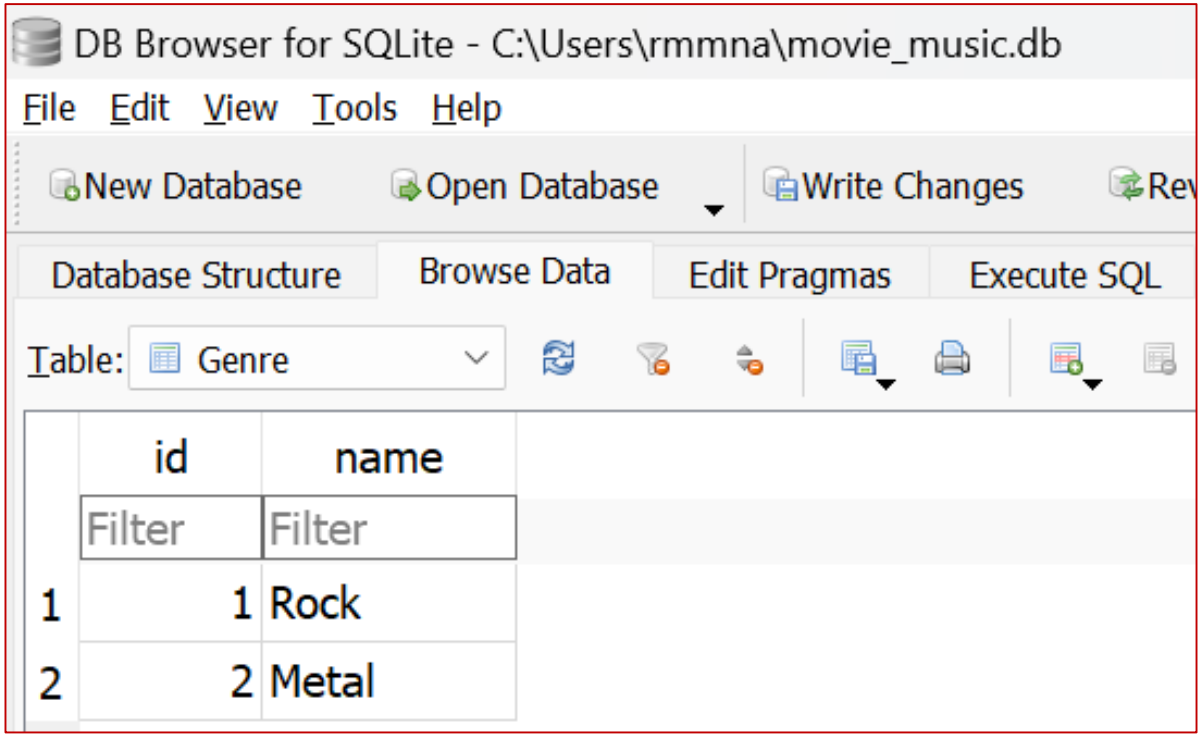
Inserting Relational Data

```
insert into Artist (name) values ('Led Zepplin')
insert into Artist (name) values ('AC/DC')
```



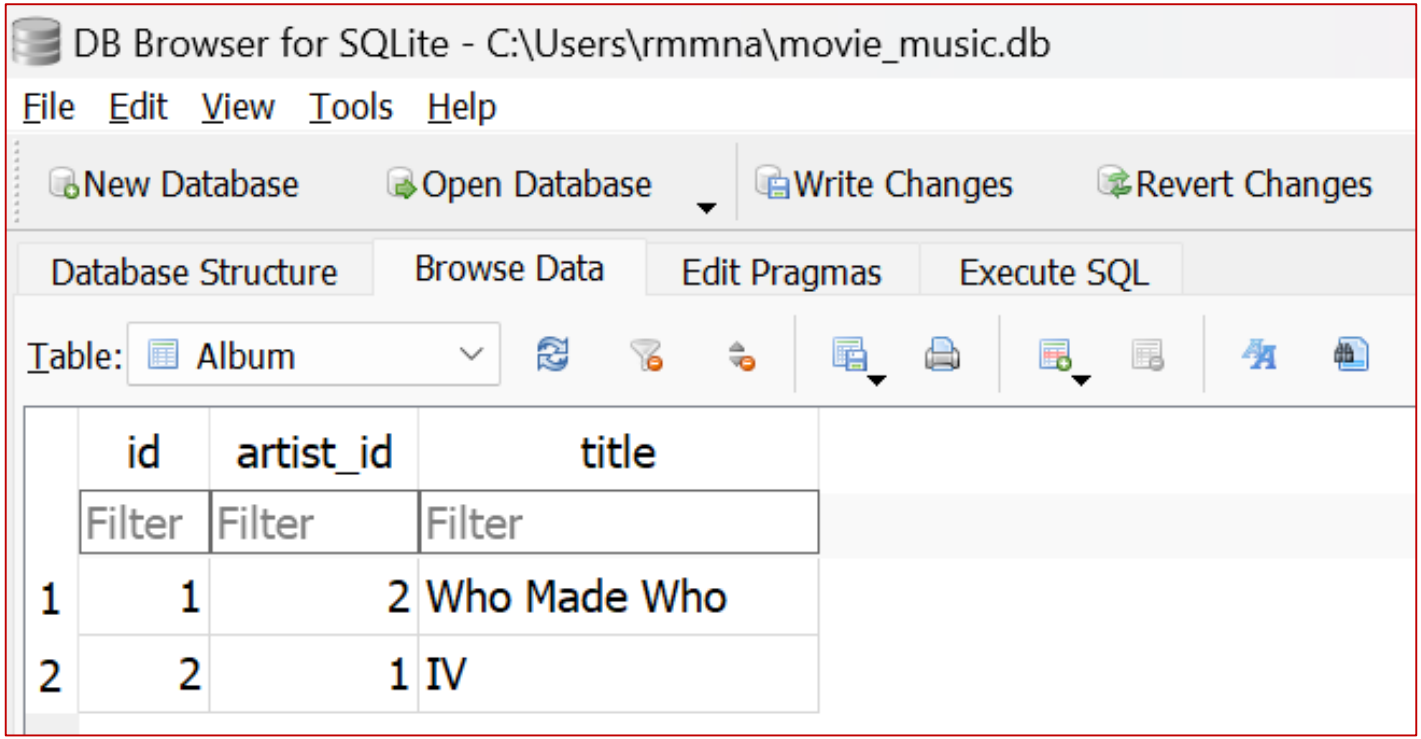
Inserting Relational Data

```
insert into Genre (name) values ('Rock')
insert into Genre (name) values ('Metal')
```



Inserting Relational Data

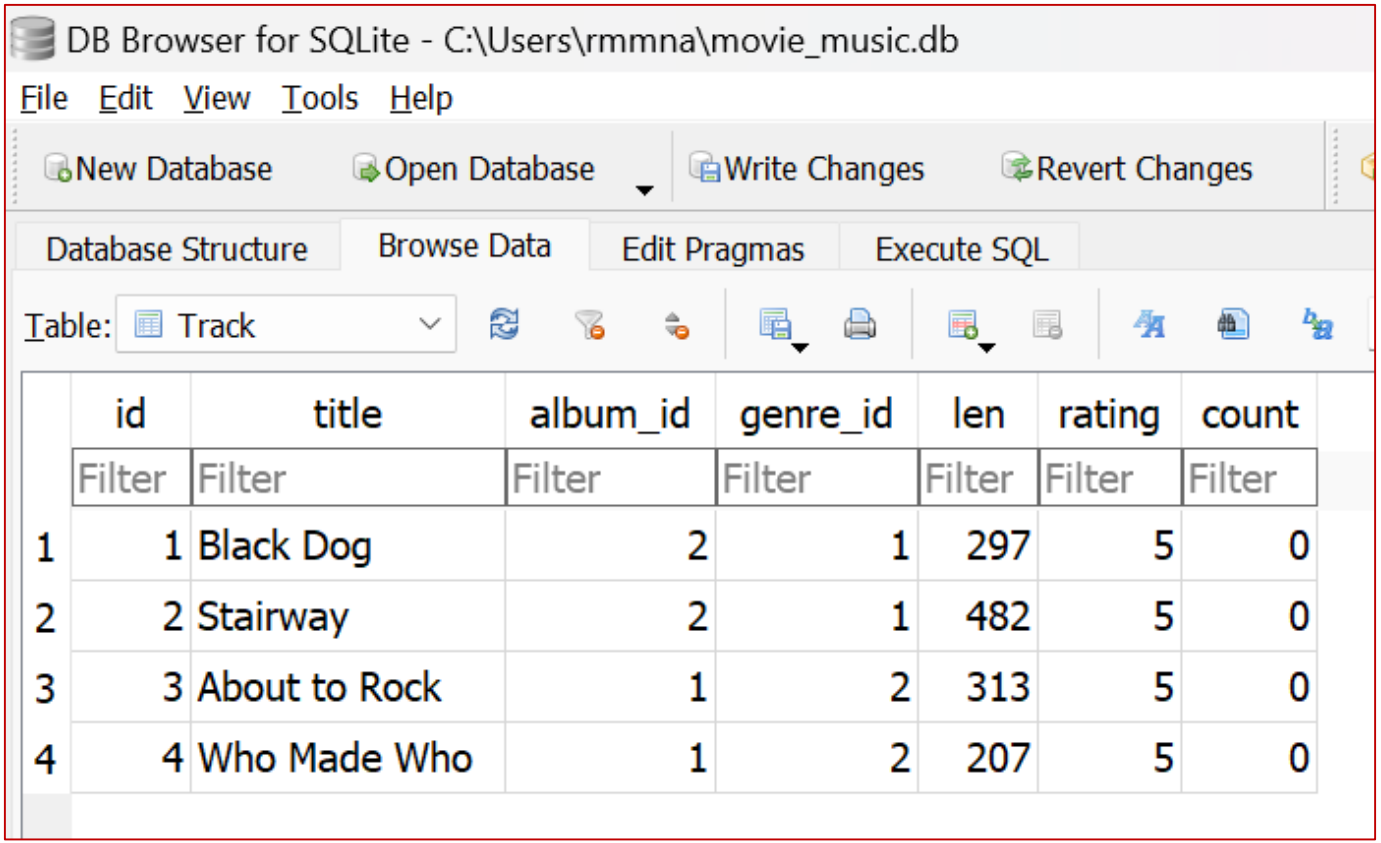
```
insert into Album (title, artist_id) values ('Who Made Who', 2)
insert into Album (title, artist_id) values ('IV', 1)
```



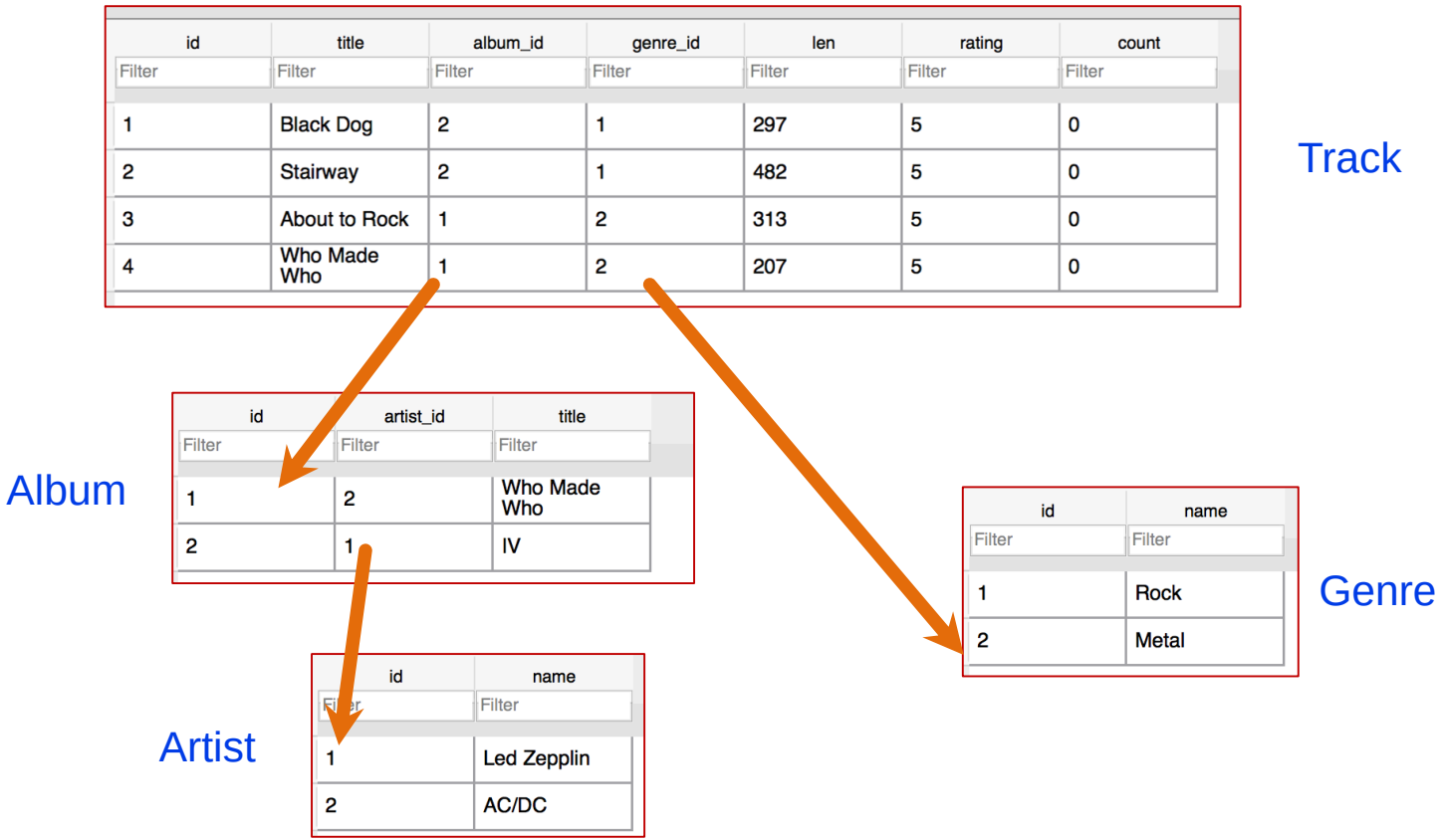
Inserting Relational Data

```
insert into Track (title, rating, len, count, album_id,
genre_id) values ('Black Dog', 5, 297, 0, 2, 1)

insert into Track (title, rating, len, count, album_id,
genre_id) values ('Stairway', 5, 482, 0, 2, 1)

insert into Track (title, rating, len, count, album_id,
genre_id) values ('About to Rock', 5, 313, 0, 1, 2)

insert into Track (title, rating, len, count, album_id,
genre_id) values (' Who Made Who ', 5, 313, 0, 1, 2)
```

Inserting Relational Data



	id	title	album_id	genre_id	len	rating	count
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	1	Black Dog	2	1	297	5	0
2	2	Stairway	2	1	482	5	0
3	3	About to Rock	1	2	313	5	0
4	4	Who Made Who	1	2	207	5	0


id
title
album_id
genre_id
len
rating
count
Filter
1
Black Dog
2
1
297
5
0
2
Stairway
2
1
482
5
0
3
About to Rock
1
2
313
5
0
4
Who Made Who
1
2
207
5
0
Track
Album
id
artist_id
title
Filter
1
2
Who Made Who
2
1
IV
Artist
id
name
Filter
1
Led Zepplin
2
AC/DC
Genre
id
name
Filter
1
Rock
2
Metal

Object Oriented Programming (Using Python)

UNIT- V

Using Databases with Python:

- Using Databases
- Single Table CRUD (Create, Read, Update, and Delete)
- Designing and representing a data model
- Reconstructing data with JOIN
- Many-to-many relationships

References

https://www.coursera.org/learn/python-databases/home/info

https://www.py4e.com/book

http://do1.dr-chuck.com/pythonlearn/EN_us/pythonlearn.pdf

https://www.py4e.com/code3/

https://www.py4e.com/lectures3/

https://www.youtube.com/playlist?list=PLlRFEj9H3Oj7Bp8-DfGpfAfDBiblRfl5p

http://sqlitebrowser.org/

Reconstructing data with JOIN

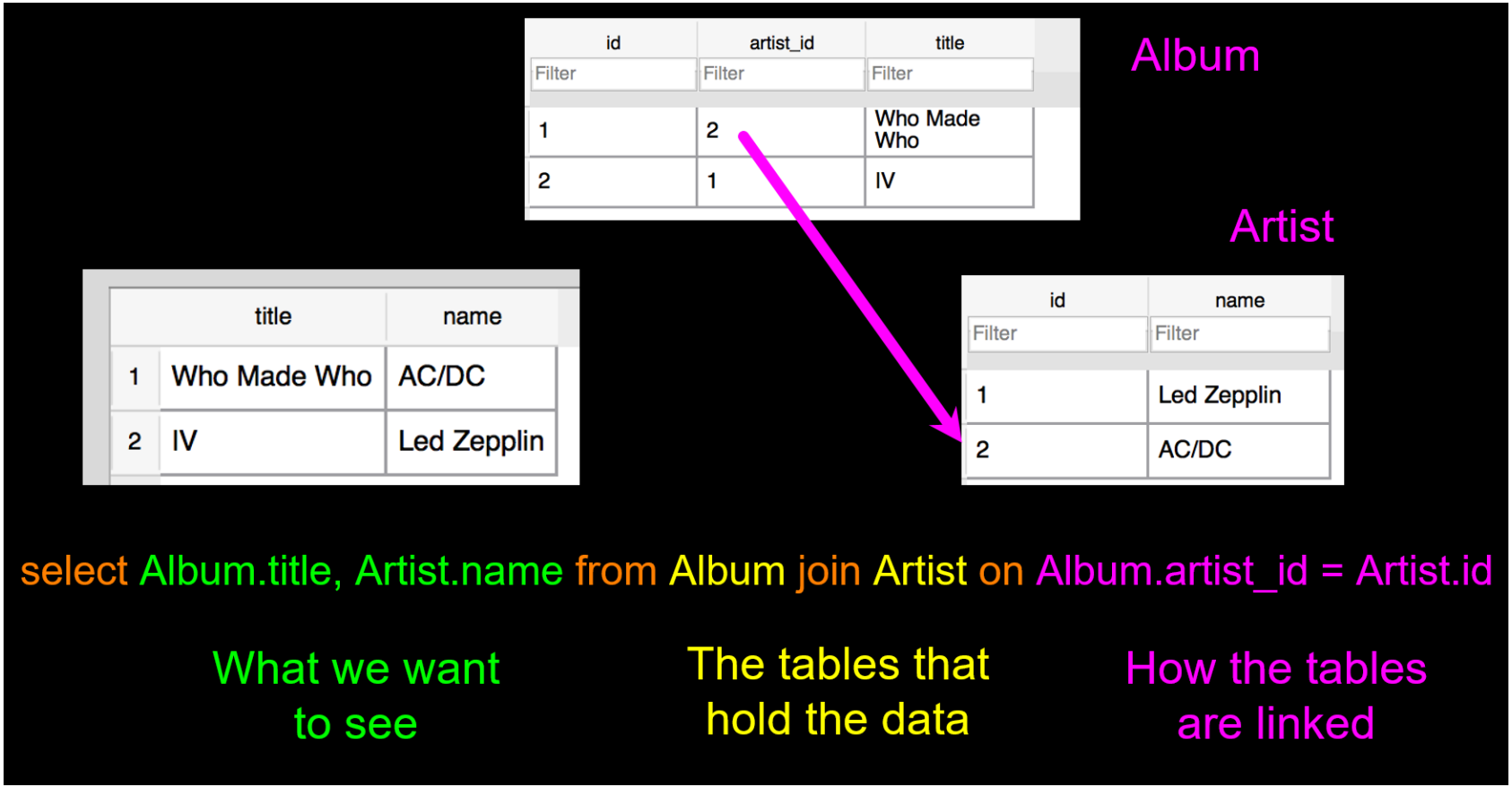
Relational Power

- By removing the replicated data and replacing it with references to a single copy of each bit of data we build a “web” of information that the relational database can read through very quickly - even for very large amounts of data
- Often when we want some data it comes from a number of tables linked by these foreign keys

The JOIN Operation

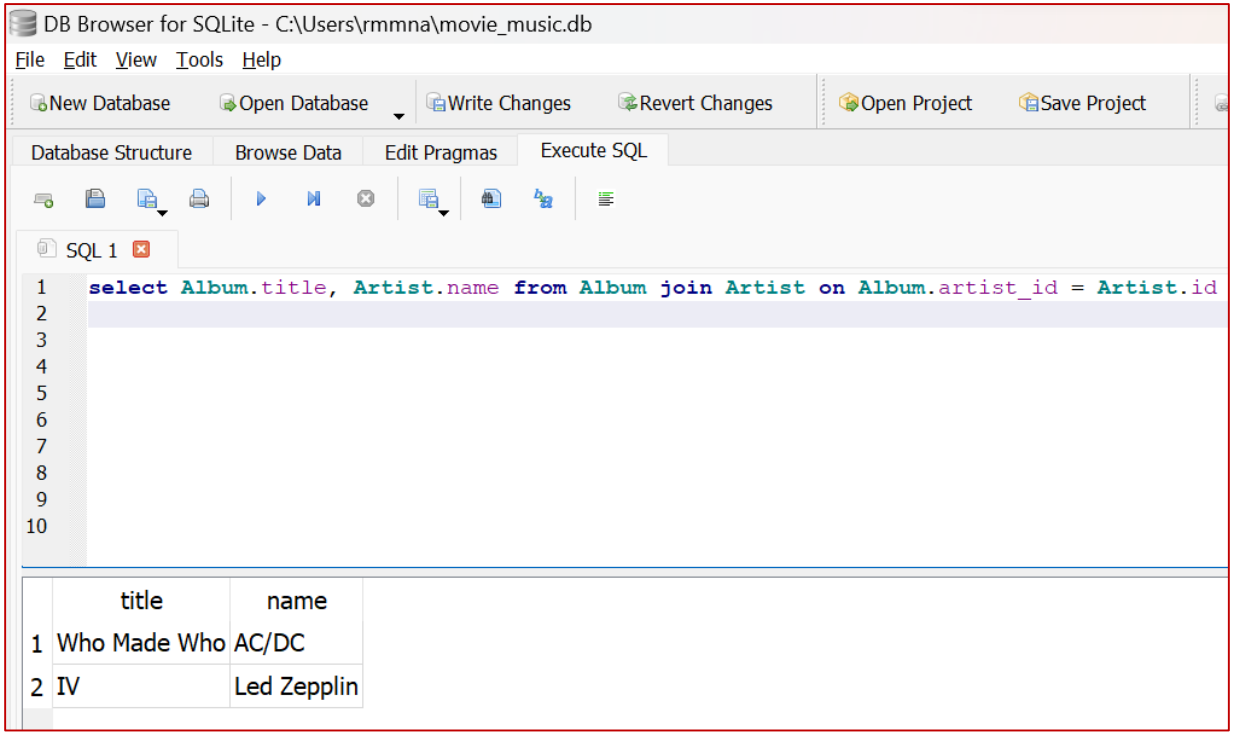
- The JOIN operation links across several tables as part of a SELECT operation
- We must tell the JOIN how to use the keys that make the connection between the tables using an ON clause

The JOIN Operation



The JOIN Operation

```
select Album.title, Artist.name from Album join Artist on
Album.artist_id = Artist.id
```



The JOIN Operation

id	artist_id	title
Filter	Filter	Filter
1	2	Who Made Who
2	1	IV

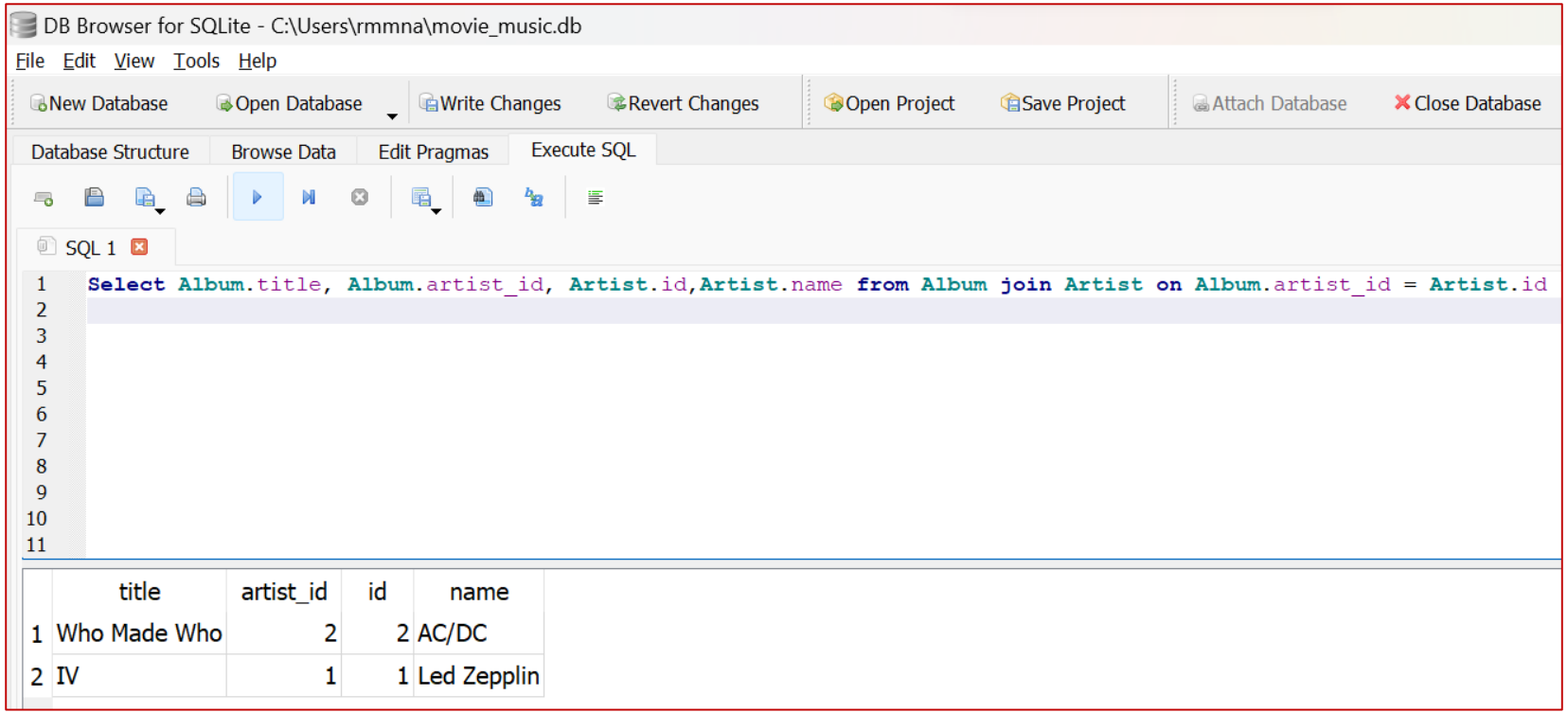
id	name
Filter	Filter
1	Led Zepplin
2	AC/DC

	title	artist_id	id	name
1	Who Made Who	2	2	AC/DC
2	IV	1	1	Led Zepplin

```
select Album.title, Album.artist_id, Artist.id,Artist.name
from Album join Artist on Album.artist_id = Artist.id
```

The JOIN Operation

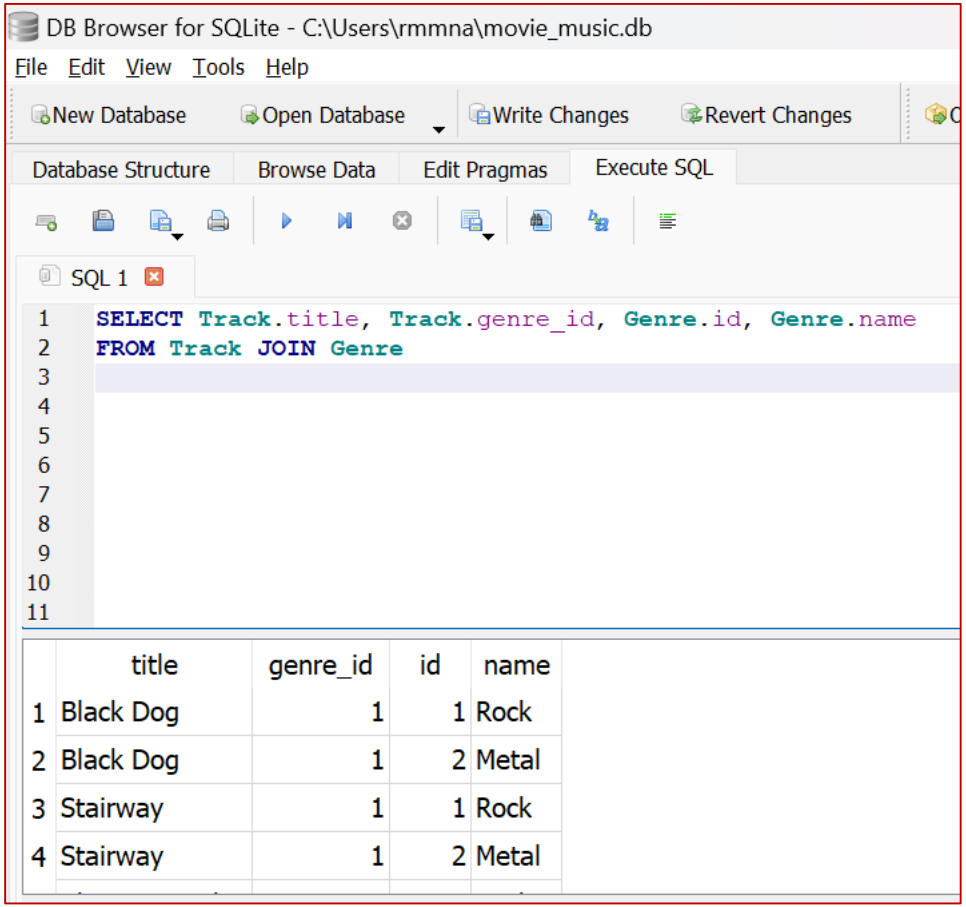
```
Select Album.title, Album.artist_id, Artist.id,Artist.name
from Album join Artist on Album.artist_id = Artist.id
```



The JOIN Operation

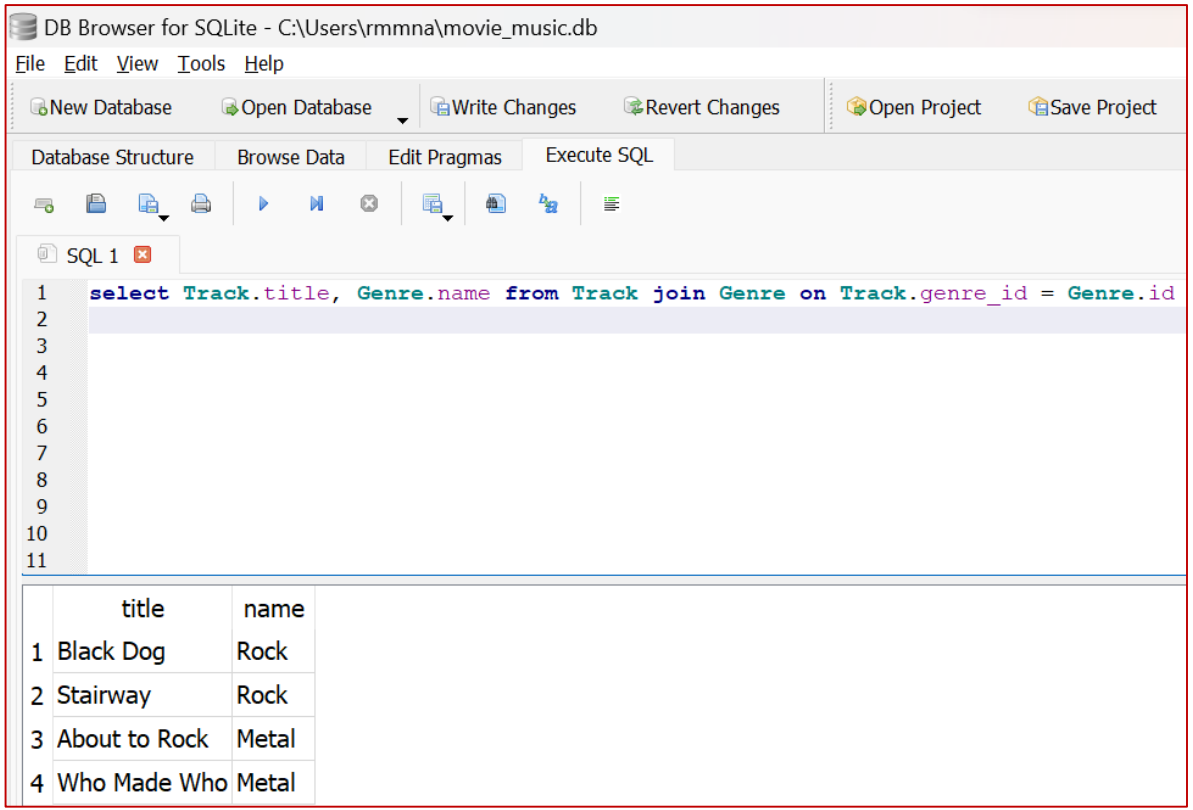
```
SELECT Track.title,
Track.genre_id, Genre.id,
Genre.name
FROM Track JOIN Genre
```

Joining two tables without an ON clause gives all possible combinations of rows.



The JOIN Operation

```
select Track.title, Genre.name from Track join Genre on
Track.genre_id = Genre.id
```



The JOIN Operation

	title	name
1	Black Dog	Rock
2	Stairway	Rock
3	About to Rock	Metal
4	Who Made Who	Metal

id	title	album_id	genre_id	len	rating	count
Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	Black Dog	2	1	297	5	0
2	Stairway	2	1	482	5	0
3	About to Rock	1	2	313	5	0
4	Who Made Who	1	2	207	5	0

id	name
Filter	Filter
1	Rock
2	Metal

```
select Track.title, Genre.name from Track join Genre on Track.genre_id = Genre.id
```

What we want to see

The tables that hold the data

How the tables are linked

The JOIN Operation

```
select Track.title, Artist.name, Album.title, Genre.name
from Track join Genre join Album join Artist on
Track.genre_id = Genre.id and Track.album_id = Album.id and
Album.artist_id = Artist.id
```

	title	name	title	name
1	Black Dog	Led Zepplin	IV	Rock
2	Stairway	Led Zepplin	IV	Rock
3	About to Rock	AC/DC	Who Made Who	Metal
4	Who Made Who	AC/DC	Who Made Who	Metal

The JOIN Operation

```
select Track.title, Artist.name, Album.title,
Genre.name from Track join Genre join Album join
Artist on Track.genre_id = Genre.id and
Track.album_id = Album.id and Album.artist_id =
Artist.id
```

	title	name	title	name
1	Black Dog	Led Zepplin	IV	Rock
2	Stairway	Led Zepplin	IV	Rock
3	About to Rock	AC/DC	Who Made Who	Metal
4	Who Made Who	AC/DC	Who Made Who	Metal

What we want to see

The tables which hold the data

How the tables are linked

The JOIN Operation

Hells Bells	5:13	AC/DC	Who Made Who	Rock	★★★★★	61
Shake Your Foundations	3:54	AC/DC	Who Made Who	Rock	★★★★★	70
Chase the Ace	3:01	AC/DC	Who Made Who	Rock		56
For Those About To Rock (We ...	5:54	AC/DC	Who Made Who	Rock	★★★★★	61
Dúlamán	3:43	Altan	Natural Wonders M...	New Age		31
Rode Across the Desert	4:10	America	Greatest Hits	Easy Listen...	★★★★★	23
Now You Are Gone	3:08	America	Greatest Hits	Easy Listen...	★★★★★	18
Tin Man	3:30	America	Greatest Hits	Easy Listen...	★★★★★	23
Sister Golden Hair	3:22	America	Greatest Hits	Easy Listen...	★★★★★	24
Track 01	4:22	Billy Price	Danger Zone	Blues/R&B	★★★★★	26
Track 02	2:45	Billy Price	Da			
Track 03	3:26	Billy Price	Da			
Track 04	4:17	Billy Price	Da			
Track 05	3:50	Billy Price	Da			
War Pigs/Luke's Wall	7:58	Black Sabbath	Pa			
Paranoid	2:53	Black Sabbath	Pa			
Planet Caravan	4:35	Black Sabbath	Pa			
Iron Man	5:59	Black Sabbath	Pa			
Electric Funeral	4:53	Black Sabbath	Pa			
Hand of Doom	7:10	Black Sabbath	Pa			
Rat Salad	2:30	Black Sabbath	Pa			
Jack the Stripper/Fairies Wear ...	6:14	Black Sabbath	Pa			
Bomb Squad (TECH)	3:28	Brent	Br			
clay techno	4:36	Brent	Br			
Heavy	3:08	Brent	Br			
Hi metal man	4:20	Brent	Brent's Album			1
Mistro	2:58	Brent	Brent's Album			1

	title	name	title	name
1	Black Dog	Led Zepplin	IV	Rock
2	Stairway	Led Zepplin	IV	Rock
3	About to Rock	AC/DC	Who Made Who	Metal
4	Who Made Who	AC/DC	Who Made Who	Metal

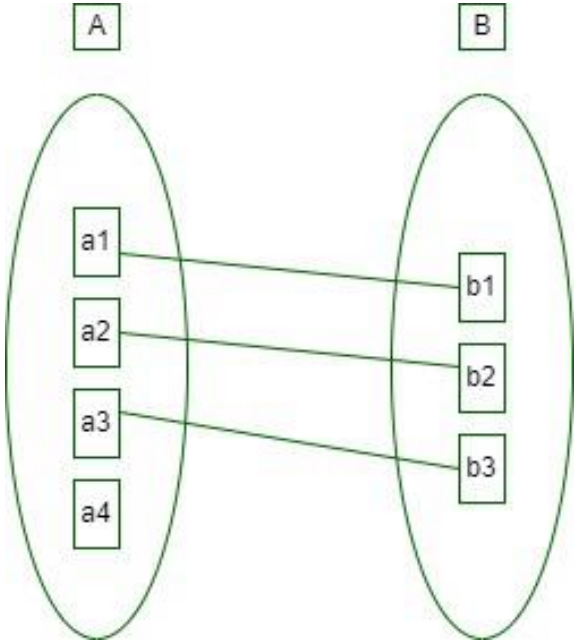
Many-to-Many Relationships

Cardinality in Relational Database Management System (RDBMS)

- Cardinality in RDBMS refers to the number of relationships between records in two related tables.
- It defines how many records from one table are associated with how many records from another table.
- There are four main types of cardinality in database relationships:
  - ✓ One-to-One (1:1),
  - ✓ One-to-Many (1:N),
  - ✓ Many-to-One (N:1), and
  - ✓ Many-to-Many (N:M)

One-to-One (1:1)

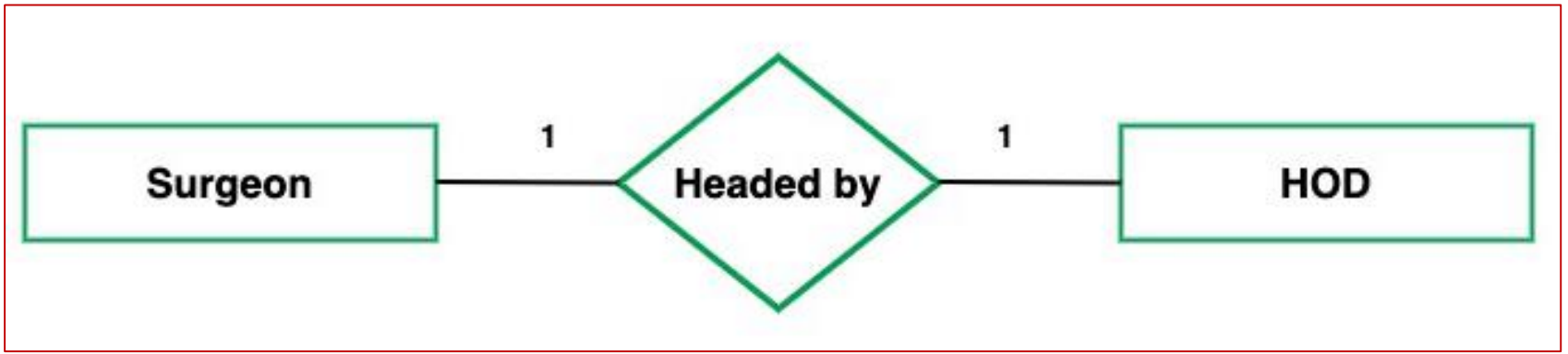
- In a one-to-one relationship, each record in one table is *associated* with only one record in another table, and vice versa. This type of relationship is relatively uncommon in databases.



One-to-One (1:1)

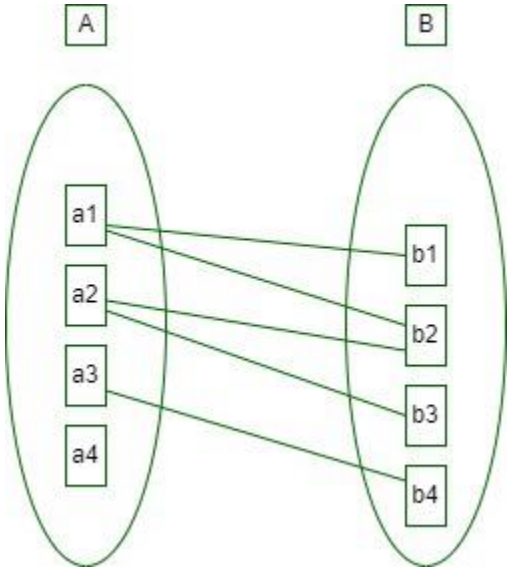
Example:

- In a particular hospital, the surgeon department has one head of department.
- They both serve one-to-one relationships.



One-to-Many (1:N)

- In a one-to-many relationship, each record in one table can be *associated* with multiple records in another table, but each record in the second table is *associated* with only one record in the first table. This is the most common type of relationship in databases



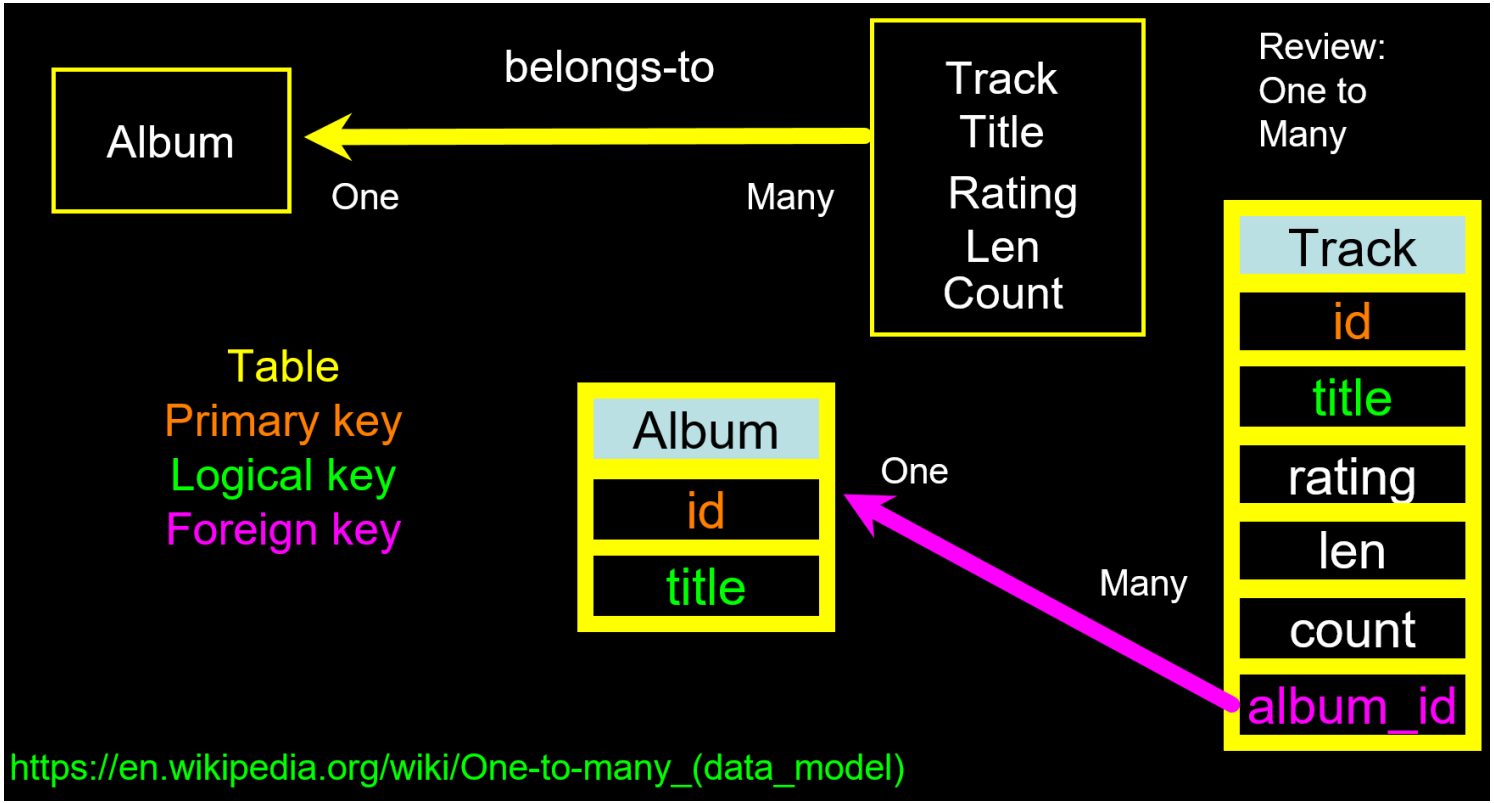
One-to-Many (1:N)

Example-1

- In a particular hospital, the surgeon department has multiple doctors. They serve one-to-many relationships.

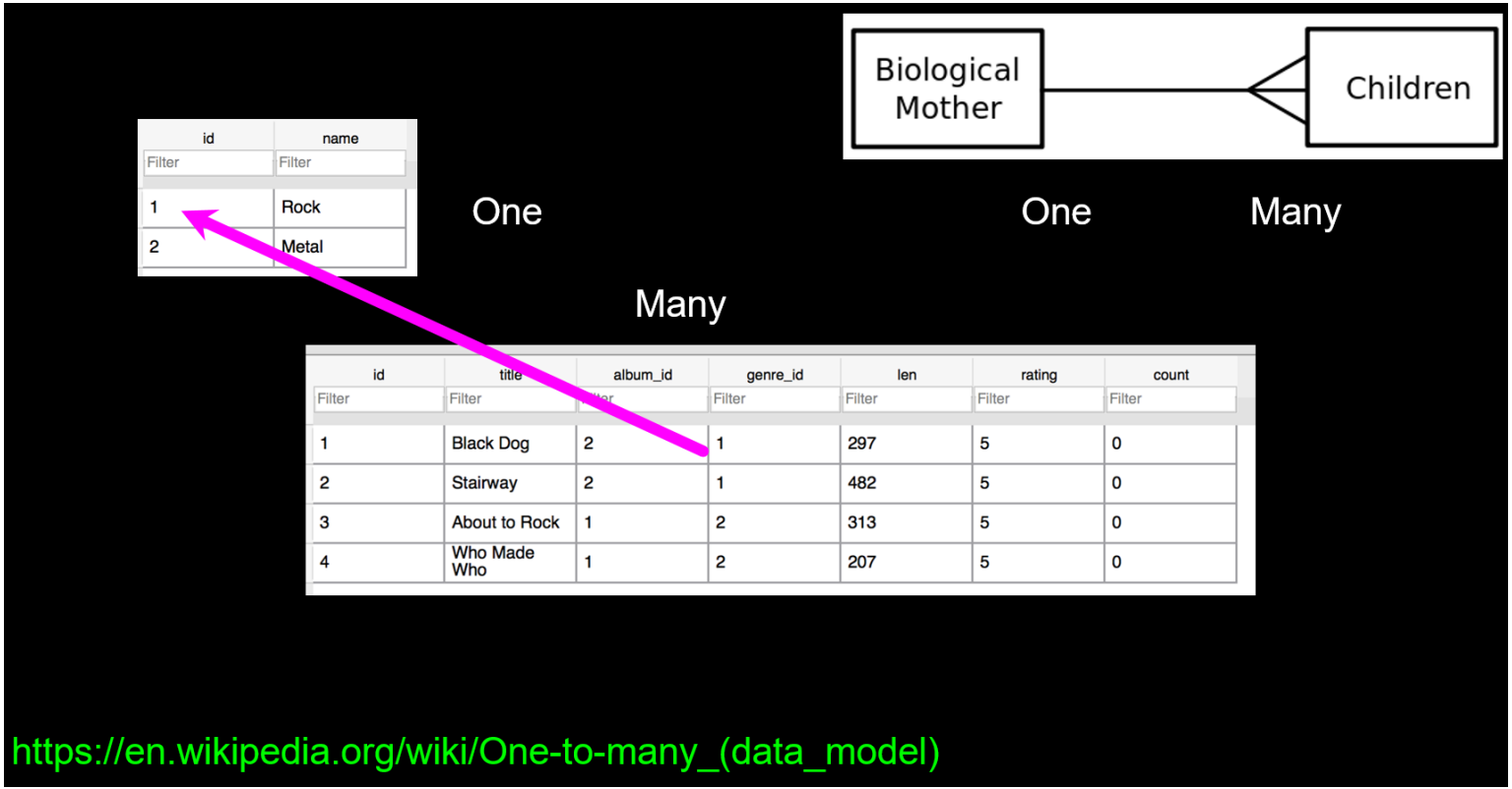
One-to-Many (1:N)

Example-2



One-to-Many (1:N)

Example-2 cont'd

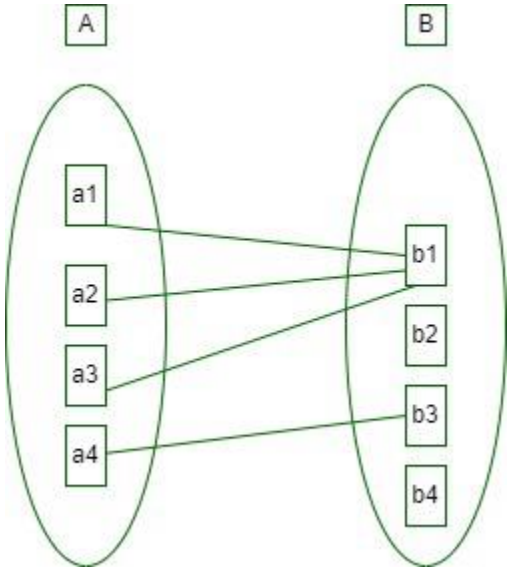


id	name
Filter	Filter
1	Rock
2	Metal

id	title	album_id	genre_id	len	rating	count
Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	Black Dog	2	1	297	5	0
2	Stairway	2	1	482	5	0
3	About to Rock	1	2	313	5	0
4	Who Made Who	1	2	207	5	0

Many-to-One (N:1)

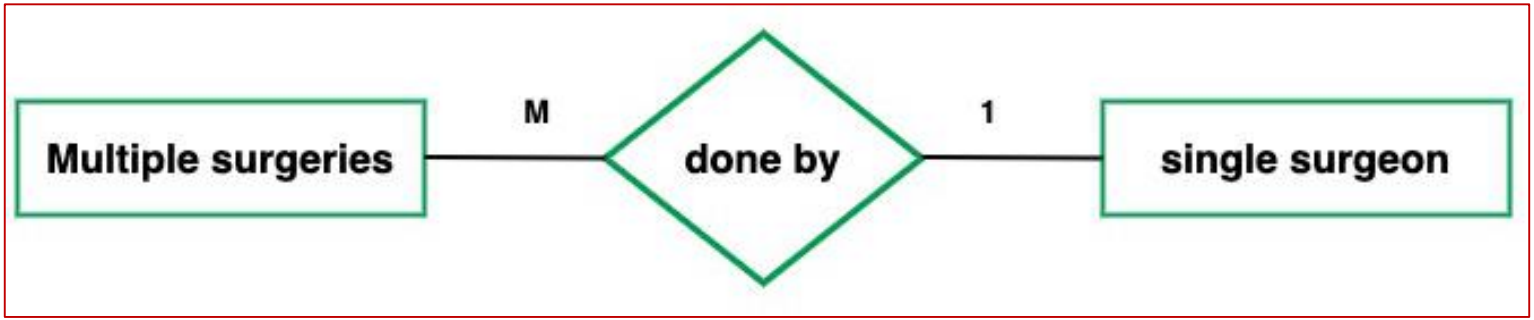
- In this type of cardinality mapping, an entity in A is *connected* to at most one entity in B.
- Or we can say a unit or item in B can be *associated* with any number (zero or more) of entities or items in A.



Many-to-One (N:1)

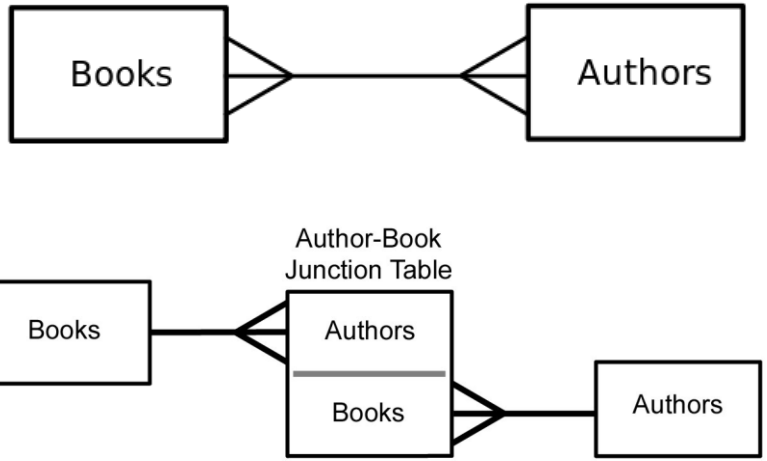
Example

- In a particular hospital, multiple surgeries are *done by* a single surgeon. Such a type of relationship is known as a many-to-one relationship.



Many-to-Many (N:M)

- Sometimes we need to model a relationship that is many-to-many
- We need to add a "connection" table with two foreign keys
- There is usually no separate primary key

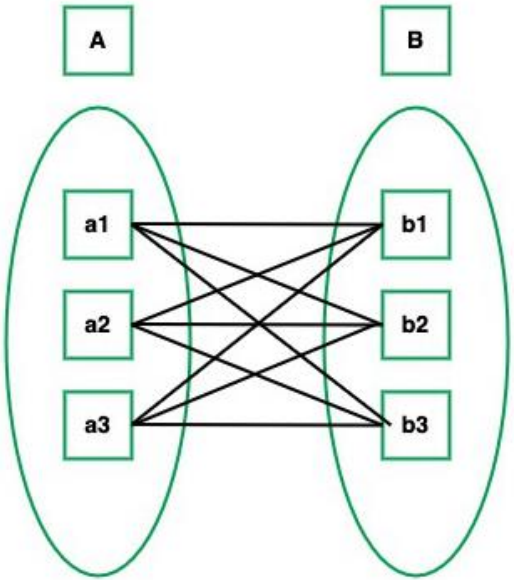


Many-to-Many (N:M)

- In a many-to-many relationship, each record in one table can be *associated* with multiple records in another table, and vice versa. This type of relationship requires a junction table to link the two main tables together.

Many-to-Many (N:M)

- In this type of cardinality mapping, an entity in A is *associated* with any number of entities in B, and an entity in B is *associated* with any number of entities in A.



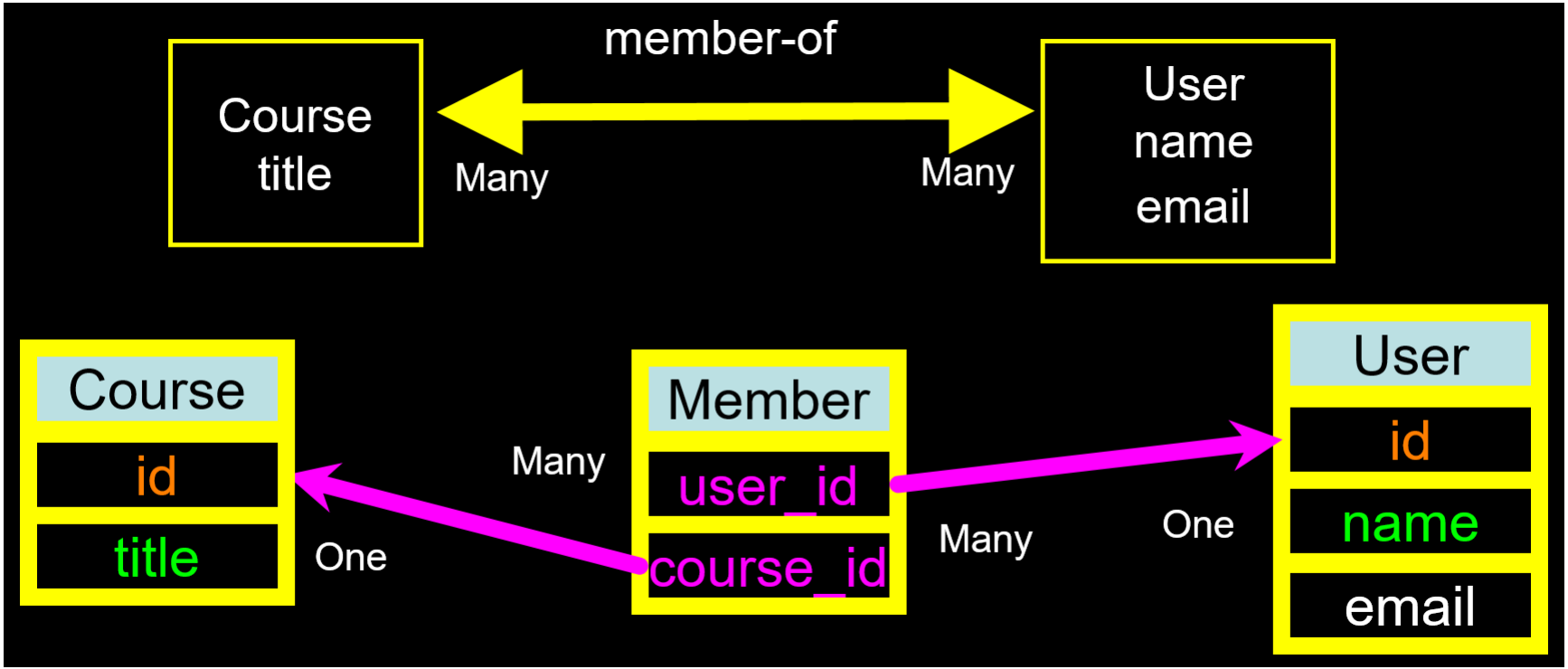
Many-to-Many (N:M)

Example-1

- In a particular company, multiple people work on multiple projects. They serve many-to-many relationships.

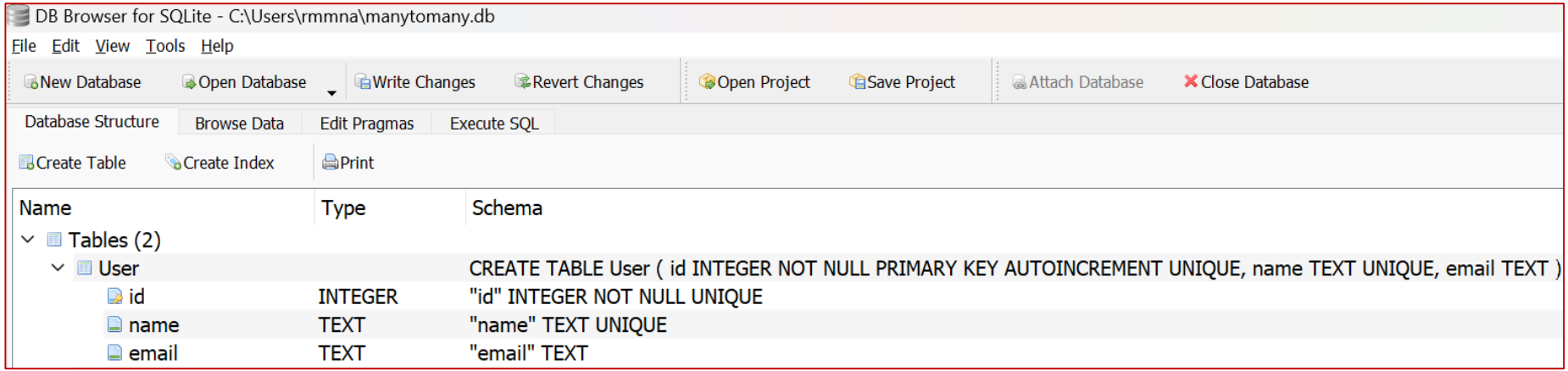
Many-to-Many (N:M)

Example-2



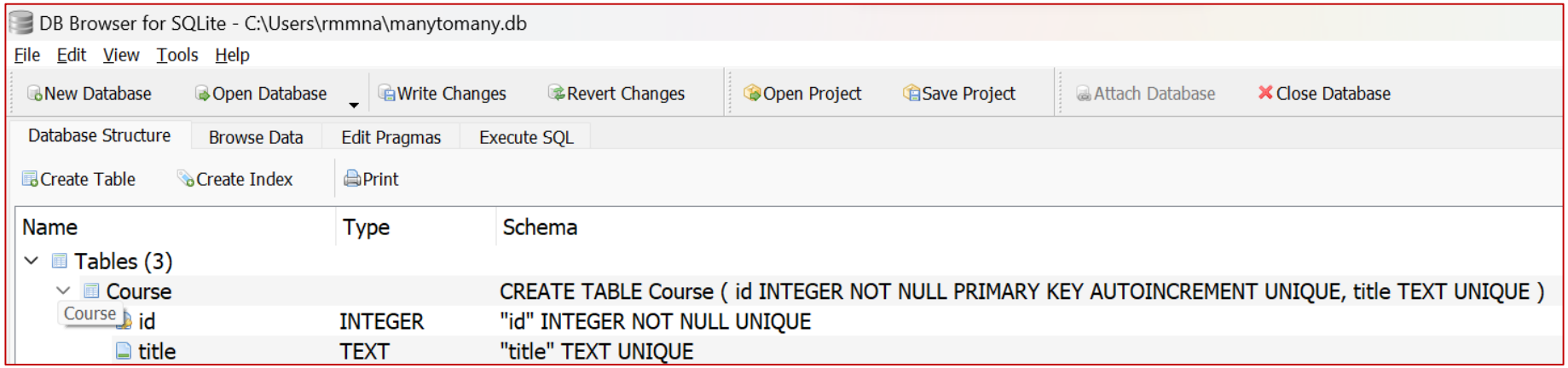
Many-to-Many (N:M): SQLite demo

```
CREATE TABLE User (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE,
    name    TEXT UNIQUE,
    email   TEXT
)
```



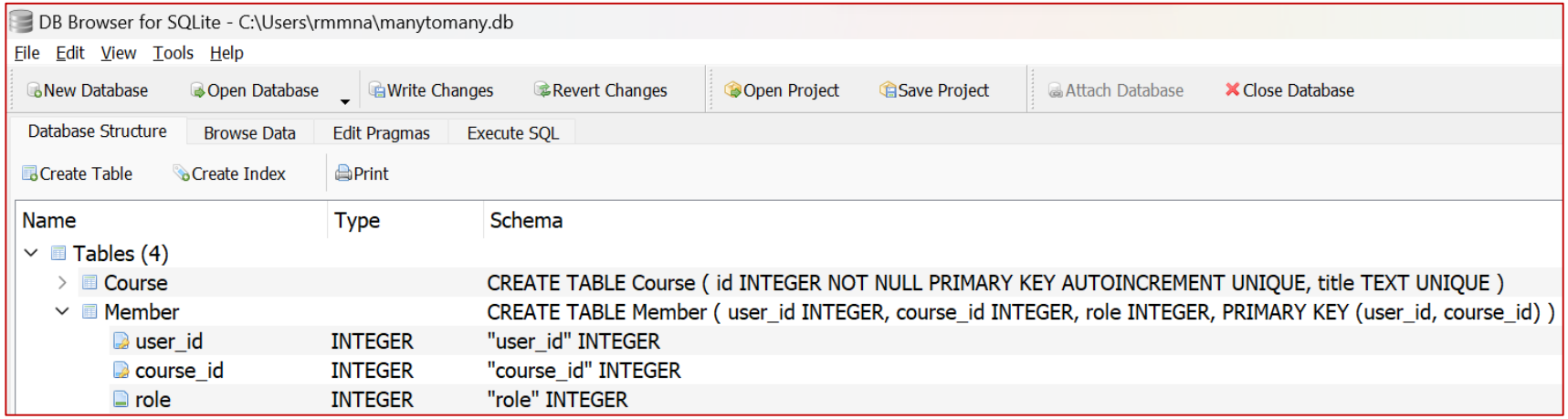
Many-to-Many (N:M): SQLite demo

```
CREATE TABLE Course (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE,
    title  TEXT UNIQUE
)
```

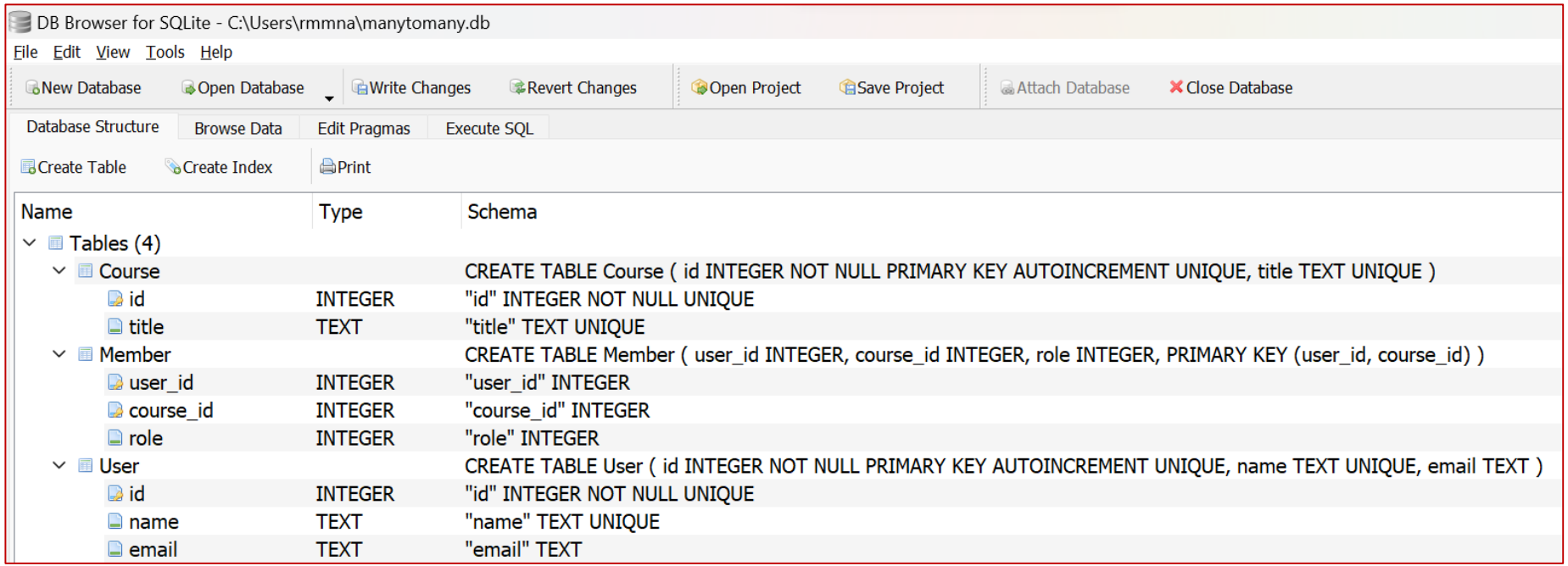


Many-to-Many (N:M): SQLite demo

```
CREATE TABLE Member (
    user_id      INTEGER,
    course_id    INTEGER,
    role         INTEGER,
    PRIMARY KEY (user_id, course_id)
)
```



Many-to-Many (N:M): SQLite demo



Many-to-Many (N:M): SQLite demo

```
INSERT INTO User (name, email) VALUES ('Prof R Madana
Mohana', 'rmmnaidu@gmail.com');

INSERT INTO User (name, email) VALUES ('Dr. R. Madana
Mohana', 'rmmnaidu@yahoo.co.in');

INSERT INTO User (name, email) VALUES ('R. Madana
Mohana', 'madanmohanar@gmail.com');
```

Many-to-Many (N:M): SQLite demo

```
Select * From User
```

	id	name	email
1	1	Prof R Madana Mohana	rmmnaidu@gmail.com
2	2	Dr. R. Madana Mohana	rmmnaidu@yahoo.co.in
3	3	R. Madana Mohana	madanmohanar@gmail.com

Many-to-Many (N:M): SQLite demo

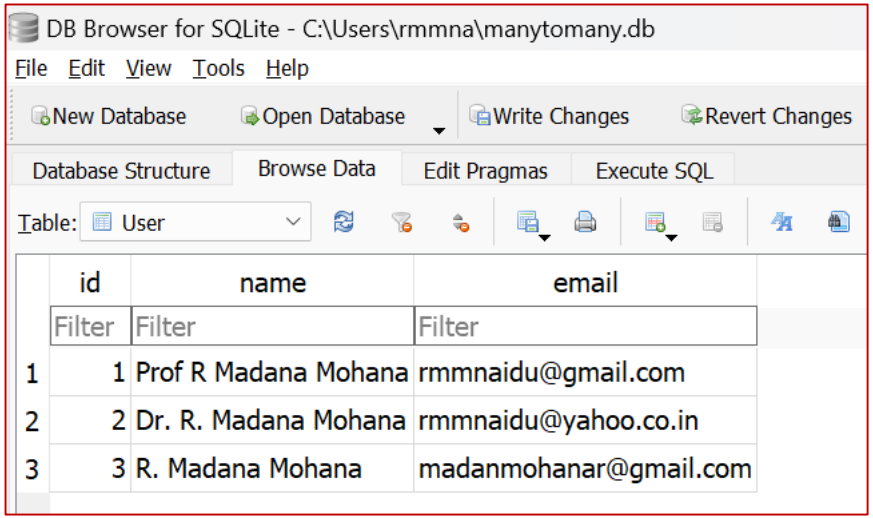
```
INSERT INTO Course (title) VALUES ('Python');

INSERT INTO Course (title) VALUES ('SQL');

INSERT INTO Course (title) VALUES ('PHP');
```

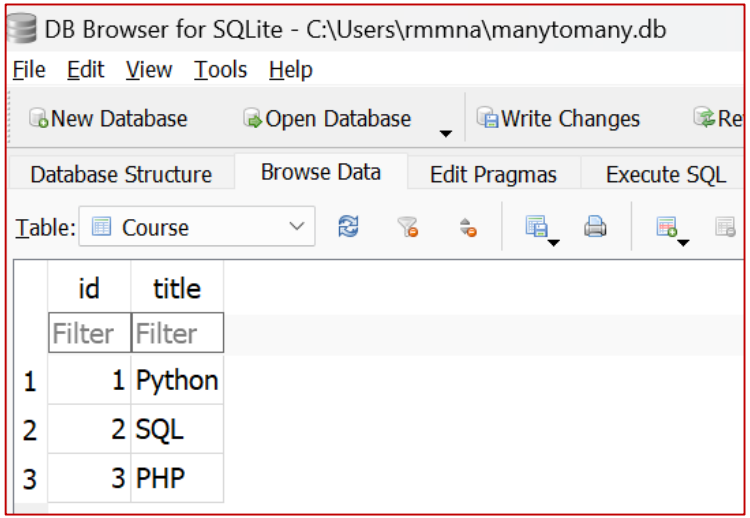
Many-to-Many (N:M): SQLite demo

```
Select * From Course
```

	id	title
1	1	Python
2	2	SQL
3	3	PHP

Many-to-Many (N:M): SQLite demo



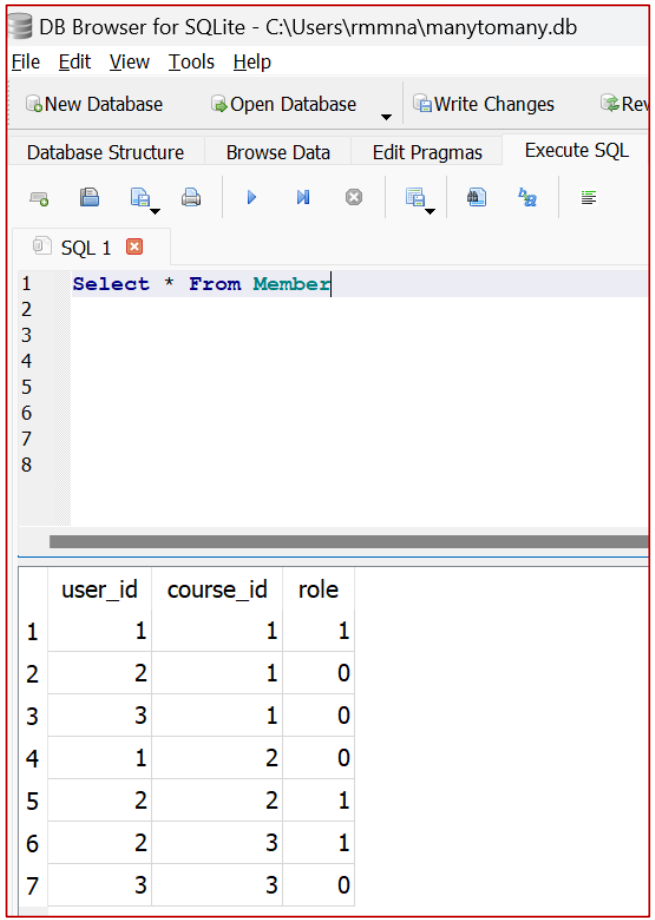


Many-to-Many (N:M): SQLite demo

```
INSERT INTO Member (user_id, course_id, role) VALUES (1, 1, 1);

INSERT INTO Member (user_id, course_id, role) VALUES (2, 1, 0);

INSERT INTO Member (user_id, course_id, role) VALUES (3, 1, 0);

INSERT INTO Member (user_id, course_id, role) VALUES (1, 2, 0);

INSERT INTO Member (user_id, course_id, role) VALUES (2, 2, 1);

INSERT INTO Member (user_id, course_id, role) VALUES (2, 3, 1);

INSERT INTO Member (user_id, course_id, role) VALUES (3, 3, 0);
```

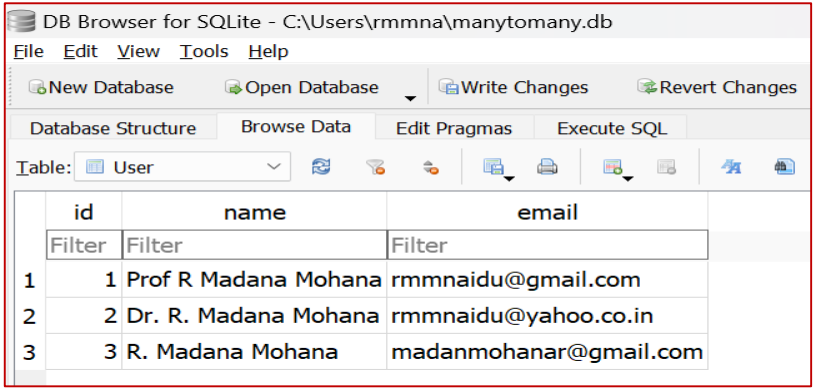
Many-to-Many (N:M): SQLite demo

```
Select * From Member;
```



	user_id	course_id	role
1	1	1	1
2	2	1	0
3	3	1	0
4	1	2	0
5	2	2	1
6	2	3	1
7	3	3	0

Many-to-Many (N:M): SQLite demo


DB Browser for SQLite - C:\Users\rmmna\manytomany.db

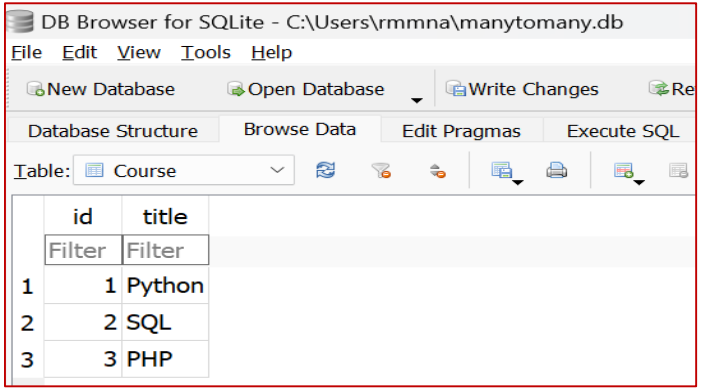
File Edit View Tools Help

New Database | Open Database | Write Changes | Revert Changes

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: User

	id	name	email
	Filter	Filter	Filter
1	1	Prof R Madana Mohana	rmmnaidu@gmail.com
2	2	Dr. R. Madana Mohana	rmmnaidu@yahoo.co.in
3	3	R. Madana Mohana	madanmohanar@gmail.com


DB Browser for SQLite - C:\Users\rmmna\manytomany.db

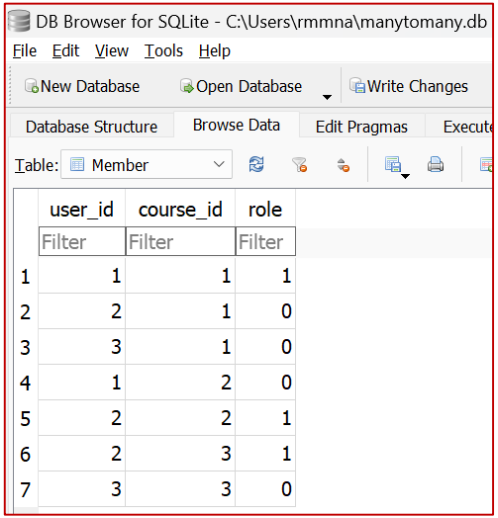
File Edit View Tools Help

New Database | Open Database | Write Changes | Re

Database Structure | Browse Data | Edit Pragmas | Execute SQL

Table: Course

	id	title
	Filter	Filter
1	1	Python
2	2	SQL
3	3	PHP


DB Browser for SQLite - C:\Users\rmmna\manytomany.db

File Edit View Tools Help

New Database | Open Database | Write Changes

Database Structure | Browse Data | Edit Pragmas | Execute

Table: Member

	user_id	course_id	role
	Filter	Filter	Filter
1	1	1	1
2	2	1	0
3	3	1	0
4	1	2	0
5	2	2	1
6	2	3	1
7	3	3	0

Many-to-Many (N:M): SQLite demo

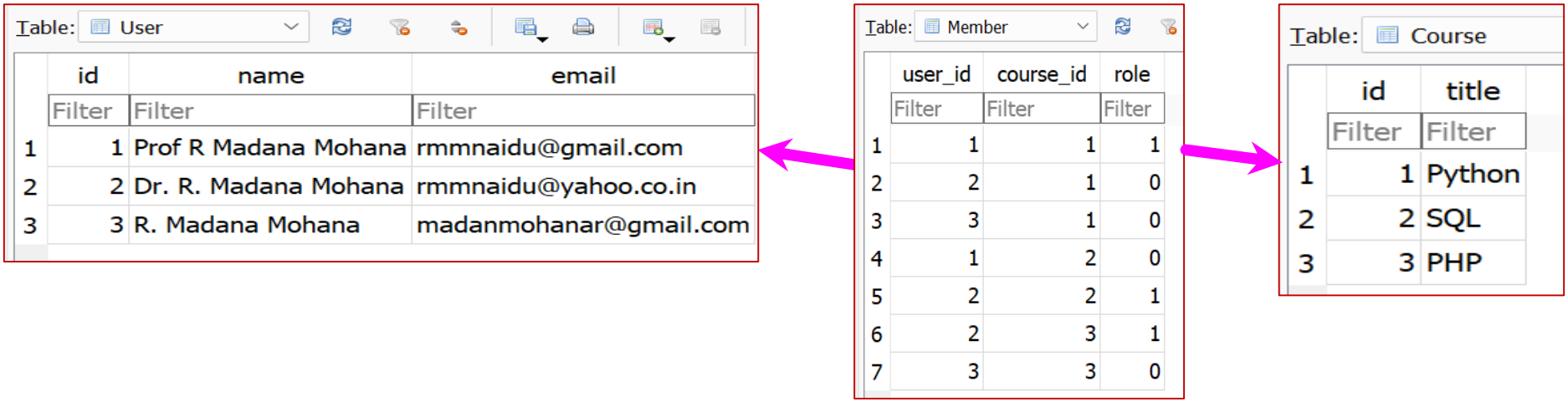


Table: User

	id	name	email
	Filter	Filter	Filter
1	1	Prof R Madana Mohana	rmmnaidu@gmail.com
2	2	Dr. R. Madana Mohana	rmmnaidu@yahoo.co.in
3	3	R. Madana Mohana	madanmohanar@gmail.com

Table: Member

	user_id	course_id	role
	Filter	Filter	Filter
1	1	1	1
2	2	1	0
3	3	1	0
4	1	2	0
5	2	2	1
6	2	3	1
7	3	3	0

Table: Course

	id	title
	Filter	Filter
1	1	Python
2	2	SQL
3	3	PHP

Many-to-Many (N:M): SQLite demo

```
SELECT User.name, Member.role, Course.title
FROM User JOIN Member JOIN Course
ON Member.user_id = User.id AND
Member.course_id = Course.id
ORDER BY Course.title, Member.role DESC, User.name
```

	name	role	title
1	Dr. R. Madana Mohana	1	PHP
2	R. Madana Mohana	0	PHP
3	Prof R Madana Mohana	1	Python
4	Dr. R. Madana Mohana	0	Python
5	R. Madana Mohana	0	Python
6	Dr. R. Madana Mohana	1	SQL
7	Prof R Madana Mohana	0	SQL

Many-to-Many (N:M) using SQLite JOIN

Python Demo

Creation of Relations

File Name: sqlitejoin_create.py

sqlitejoin_create.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/sqlitejoin_create.py (3.10.0)

File Edit Format Run Options Window Help

```
import sqlite3
conn = sqlite3.connect('sqlitemanytomany.db')
print('Opened database successfully');
conn.execute('''CREATE TABLE User (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE,
    name   TEXT UNIQUE,
    email  TEXT
)''')
conn.execute('''CREATE TABLE Course (
    id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT UNIQUE,
    title  TEXT UNIQUE
)''')
conn.execute('''CREATE TABLE Member (
    user_id     INTEGER,
    course_id   INTEGER,
    role        INTEGER,
    PRIMARY KEY (user_id, course_id)
)''')
print('Tables created successfully')
conn.close()
```

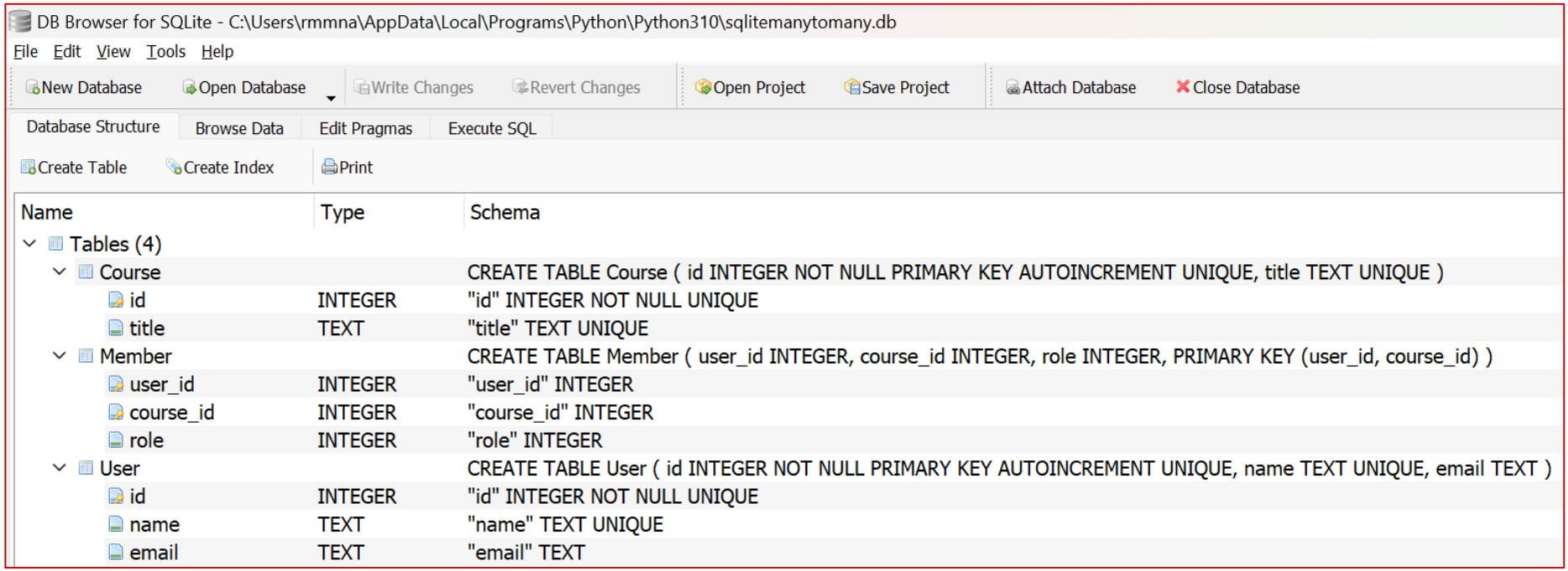
IDLE Shell 3.10.0

File Edit Shell Debug Options Window Help

```
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win3
2
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rmmna/AppData/Local/Programs/Python/Python310/sqlitejoin_create.py
Opened database successfully
Tables created successfully
>>>
```

Creation of Relations

Database Name: sqlitemanytomany.db



Insertion of Data

File Name: sqlitejoin_insert.py

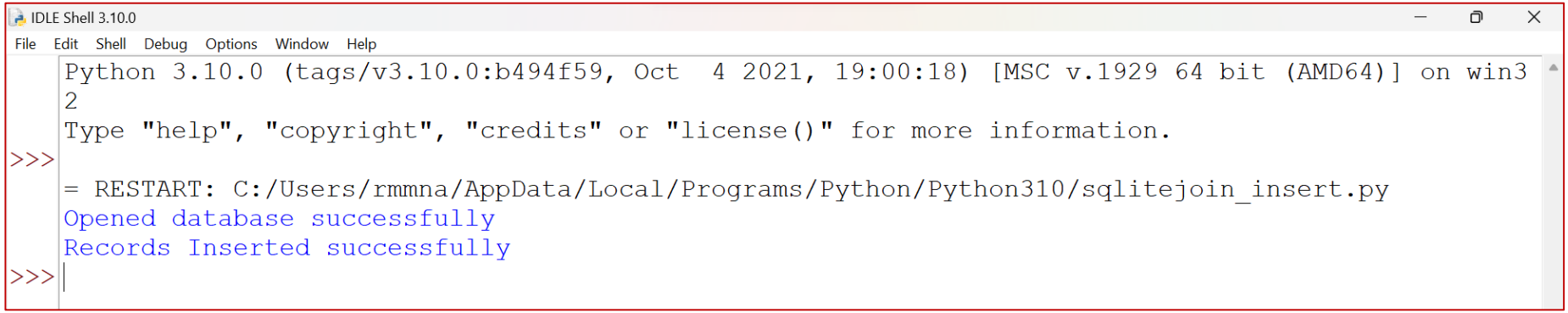
sqlitejoin_insert.py - C:/Users/rmmna/AppData/Local/Programs/Python/Python310/sqlitejoin_insert.py (3.10.0)

File Edit Format Run Options Window Help

```
import sqlite3
conn = sqlite3.connect('sqlitemanytomany.db')
print('Opened database successfully');
# Inserting data inot User relation
conn.execute("INSERT INTO User (name, email) VALUES ('Prof R Madana Mohana', 'rmmnaidu@gmail.com')"
conn.execute("INSERT INTO User (name, email) VALUES ('Dr. R. Madana Mohana', 'rmmnaidu@yahoo.co.in'
conn.execute("INSERT INTO User (name, email) VALUES ('R. Madana Mohana', 'madanmohanar@gmail.com')"
# Inserting data inot Course relation
conn.execute("INSERT INTO Course (title) VALUES ('Python')")
conn.execute("INSERT INTO Course (title) VALUES ('SQL')")
conn.execute("INSERT INTO Course (title) VALUES ('PHP')")
# Inserting data inot Member relation
conn.execute("INSERT INTO Member (user_id, course_id, role) VALUES (1, 1, 1)")
conn.execute("INSERT INTO Member (user_id, course_id, role) VALUES (2, 1, 0)")
conn.execute("INSERT INTO Member (user_id, course_id, role) VALUES (3, 1, 0)")
conn.execute("INSERT INTO Member (user_id, course_id, role) VALUES (1, 2, 0)")
conn.execute("INSERT INTO Member (user_id, course_id, role) VALUES (2, 2, 1)")
conn.execute("INSERT INTO Member (user_id, course_id, role) VALUES (2, 3, 1)")
conn.execute("INSERT INTO Member (user_id, course_id, role) VALUES (3, 3, 0)")
conn.commit()
print('Records Inserted successfully')
conn.close()
```

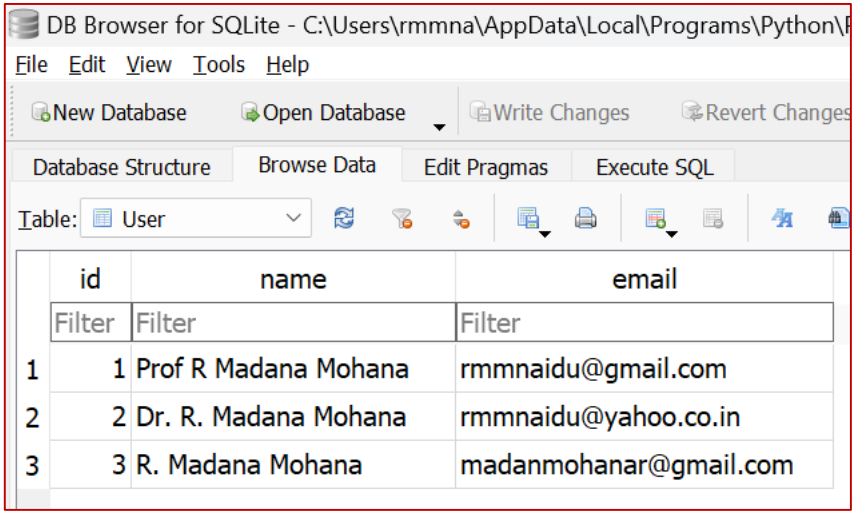
Insertion of Data

File Name: sqlitejoin_insert.py

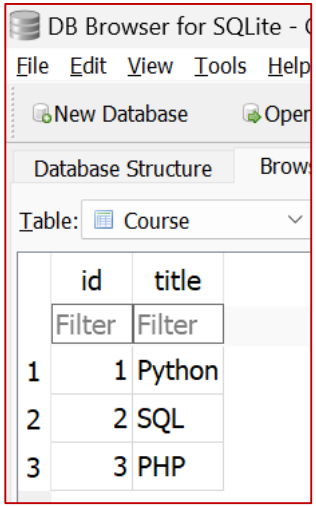


Insertion of Data

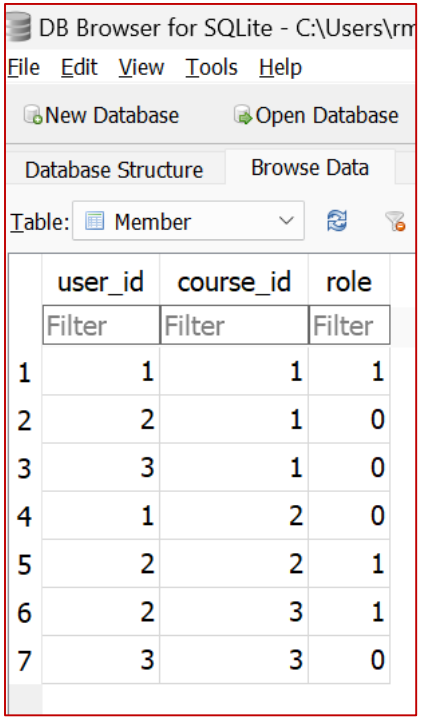
File Name: sqlitejoin_insert.py



	id	name	email
	Filter	Filter	Filter
1	1	Prof R Madana Mohana	rmmnaidu@gmail.com
2	2	Dr. R. Madana Mohana	rmmnaidu@yahoo.co.in
3	3	R. Madana Mohana	madanmohanar@gmail.com



	id	title
	Filter	Filter
1	1	Python
2	2	SQL
3	3	PHP



	user_id	course_id	role
	Filter	Filter	Filter
1	1	1	1
2	2	1	0
3	3	1	0
4	1	2	0
5	2	2	1
6	2	3	1
7	3	3	0

Join Operation

File Name: sqlitejoin_select.py

```
import sqlite3
conn = sqlite3.connect('sqlitemanytomany.db')
print('Opened database successfully');
#Join Operation
cursor=conn.execute("SELECT User.name, Member.role, Course.title
FROM User JOIN Member JOIN Course ON Member.user_id = User.id AND
Member.course_id = Course.id ORDER BY Course.title, Member.role
DESC, User.name")
print('JOIN Operation Results Retrieval:')
for row in cursor:
    print("Name = ", row[0])
    print("Role = ", row[1])
    print("Title = ", row[2], "\n")
print('Records Retrieved successfully')
conn.close()
```

Join Operation

File Name: sqlitejoin_select.py